

Backend Code Reference

Package, module, class, and function reference for app/.

Eurskem AI · Agentic Workflow Platform

Generated 2026-08-24

Code revision 8bd16ca4a2a3

Derived from the live repository

Source of truth. Inventories and symbol tables are generated from the current checkout. Narrative explains observed structures and does not replace executable contracts.

Backend package inventory

Package / area	Files	Symbols	Representative responsibility
app	3	18	Python module.
app/api	29	382	Python module.
app/db	3	25	Python module.
app/evaluation	6	29	Evaluation package: LLM-as-a-Judge harness.
app/evidence	6	54	Proposal-grade evidence gathering and citation verification.
app/ingestion	14	129	Python module.
app/integrations	21	186	Python module.
app/knowledge	6	90	Knowledge Studio control plane: stable resources, profiles and indexes.
app/llm	18	183	LLM gateway package.
app/mcp	28	268	Real, live MySQL-backed business-records MCP server.
app/nodes	62	603	Built-in workflow-node discovery.
app/observability	5	17	Python module.
app/proposal_graph	9	68	Proposal workspace graph: domain state and workspace store.
app/rag	3	7	Saved RAG Agent resources and grounded generation.
app/research	3	30	Controlled adapters for external scientific research assets.
app/retrieval	16	69	Hybrid RAG retrieval pipeline.
app/runtime	21	272	Phase 4 runtime: YAML to LangGraph compiler and executor.
app/security	17	94	Python module.
app/storage	2	12	Python module.
app/tools	14	104	Character-count helper for the proposal-size guardrails.
app/workflow	16	170	Durable, file-backed state used by the Workflow Builder.

Backend symbols

Symbol	Kind	Location	Signature / summary
get_session_audit	async function	app/api/audit.py:15	<code>session_id: str, request: Request, run_id: str None=None, user: CurrentUser=Depends(require_consultant)</code> Return the session audit.
Token	class	app/api/auth.py:27	

Symbol	Kind	Location	Signature / summary
			<code>BaseModel</code> Pydantic model defining the Token shape.
Identity	class	<code>app/api/auth.py:42</code>	<code>BaseModel</code> Pydantic model defining the Identity shape.
_set_auth_cookie	function	<code>app/api/auth.py:55</code>	<code>response: Response, token: str</code> Attach the JWT as an HttpOnly cookie.
login	async function	<code>app/api/auth.py:76</code>	<code>request: Request, response: Response, form: OAuth2PasswordRequestForm=Depends()</code> Compute the login.
me	async function	<code>app/api/auth.py:134</code>	<code>user: CurrentUser=Depends(require_consultant)</code> Rehydrate identity from the auth cookie after a page refresh.
logout	async function	<code>app/api/auth.py:148</code>	<code>response: Response</code> Clear the auth cookie.
_services	function	<code>app/api/builder.py:76</code>	<code>request: Request</code> Internal helper for the services step.
_friendly_validation_message	function	<code>app/api/builder.py:88</code>	<code>error: ValidationError</code> One line per Pydantic error, without the "N validation errors for ModelName" header or the errors.pydantic.dev URL — those name an internal class the Builder never shows the author, and would otherwise leak straight into the Test tab's error banner verbatim.
operators	function	<code>app/api/builder.py:112</code>	<code>user: CurrentUser=Depends(require_consultant)</code> Which operators apply to which field type, and how to render each one.
ContractRequest	class	<code>app/api/builder.py:144</code>	<code>BaseModel</code> Pydantic model defining the ContractRequest shape.
ContractField	class	<code>app/api/builder.py:159</code>	<code>BaseModel</code> Pydantic model defining the ContractField shape.
ContractNode	class	<code>app/api/builder.py:183</code>	<code>BaseModel</code> Pydantic model defining the ContractNode shape.
output_contract	function	<code>app/api/builder.py:203</code>	<code>body: ContractRequest, user: CurrentUser=Depends(require_consultant)</code> What each step guarantees, as typed dotted paths (§40).
SchemaPreviewRequest	class	<code>app/api/builder.py:284</code>	<code>BaseModel</code> Pydantic model defining the SchemaPreviewRequest shape.
schema_preview	function	<code>app/api/builder.py:296</code>	<code>body: SchemaPreviewRequest, user: CurrentUser=Depends(require_consultant)</code> Compile visual schema rows and show what the model will be held to.
NodeTestRequest	class	<code>app/api/builder.py:327</code>	<code>BaseModel</code> Pydantic model defining the NodeTestRequest shape.
node_test	async function	<code>app/api/builder.py:358</code>	<code>body: NodeTestRequest, request: Request, user:</code>

Symbol	Kind	Location	Signature / summary
			<code>CurrentUser=Depends(require_consultant)</code> Run one node against sample data (§21).
SimulateRequest	class	<code>app/api/builder.py:485</code>	<code>BaseModel</code> Pydantic model defining the SimulateRequest shape.
simulate	async function	<code>app/api/builder.py:507</code>	<code>body: SimulateRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Run a workflow in memory and return a per-step trace (§23, §24, §47).
_interrupt_values	function	<code>app/api/builder.py:620</code>	<code>interrupt: Any</code> Pull the real payloads out of LangGraph's Interrupt objects.
_waiting_steps	function	<code>app/api/builder.py:639</code>	<code>spec: WorkflowSpec, interrupt: Any</code> One step per gate the run is parked at.
_timeline	function	<code>app/api/builder.py:676</code>	<code>spec: WorkflowSpec, state: dict[str, Any], stubbed: set[str]</code> Per-step trace in execution order, with each step's own explanation.
_executed_path	function	<code>app/api/builder.py:722</code>	<code>state: dict[str, Any]</code> Internal helper for the executed path step.
_explain	function	<code>app/api/builder.py:739</code>	<code>type_name: str, output: dict[str, Any]</code> Turn a node's output into the "why did this happen?" view (§24, §47).
SchemaAssistRequest	class	<code>app/api/builder.py:861</code>	<code>BaseModel</code> Pydantic model defining the SchemaAssistRequest shape.
ProposedField	class	<code>app/api/builder.py:878</code>	<code>BaseModel</code> One suggested row.
ProposedSchema	class	<code>app/api/builder.py:899</code>	<code>BaseModel</code> Pydantic model defining the ProposedSchema shape.
assist_schema	async function	<code>app/api/builder.py:913</code>	<code>body: SchemaAssistRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Propose a structured-output schema from a description (§37).
RuleAssistRequest	class	<code>app/api/builder.py:1001</code>	<code>BaseModel</code> Pydantic model defining the RuleAssistRequest shape.
ProposedRules	class	<code>app/api/builder.py:1018</code>	<code>BaseModel</code> Pydantic model defining the ProposedRules shape.
assist_rules	async function	<code>app/api/builder.py:1032</code>	<code>body: RuleAssistRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Turn a described business rule into deterministic configuration (§36).
mcp_servers	function	<code>app/api/builder.py:1124</code>	<code>request: Request, user: CurrentUser=Depends(require_consultant)</code> Configured MCP servers for the Builder's connection panel.
mcp_tools	async function	<code>app/api/builder.py:1142</code>	<code>server_id: str, request: Request, refresh: bool=False, user:</code>

Symbol	Kind	Location	Signature / summary
			<code>CurrentUser=Depends(require_consultant)</code> Discover what a server can do (§5).
<code>mcp_health</code>	async function	<code>app/api/builder.py:1176</code>	<code>server_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> Live connection check, for the panel's Test Connection button.
<code>MCPToolTestRequest</code>	class	<code>app/api/builder.py:1191</code>	<code>BaseModel</code> Pydantic model defining the MCPToolTestRequest shape.
<code>mcp_test_tool</code>	async function	<code>app/api/builder.py:1207</code>	<code>body: MCPToolTestRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Run one MCP tool with literal arguments (§21).
<code>_require_mcp</code>	function	<code>app/api/builder.py:1281</code>	<code>request: Request</code> Internal helper for the require mcp step.
<code>email_connections</code>	function	<code>app/api/builder.py:1300</code>	<code>request: Request, user: CurrentUser=Depends(require_consultant)</code> Configured mailboxes for the Email node's connection picker.
<code>_contract_sort_key</code>	function	<code>app/api/builder.py:1329</code>	<code>field: Any</code> Order a step's contract so the author's own fields come first.
<code>_parse_workflow</code>	function	<code>app/api/builder.py:1342</code>	<code>yaml_text: str</code> Parse the workflow.
<code>_require_llm</code>	function	<code>app/api/builder.py:1359</code>	<code>request: Request</code> Internal helper for the require llm step.
<code>_label_of</code>	function	<code>app/api/builder.py:1377</code>	<code>node: Any</code> The business label if the author set one, else the technical id (§17).
<code>_execution_kind</code>	function	<code>app/api/builder.py:1384</code>	<code>type_name: str</code> Internal helper for the execution kind step.
<code>_is_external_write</code>	function	<code>app/api/builder.py:1409</code>	<code>type_name: str, config: dict[str, Any]</code> Return whether external write.
<code>_ancestors</code>	function	<code>app/api/builder.py:1426</code>	<code>spec: WorkflowSpec, node_id: str</code> Internal helper for the ancestors step.
<code>_slice_through</code>	function	<code>app/api/builder.py:1458</code>	<code>spec: WorkflowSpec, node_id: str</code> The smallest valid upstream slice ending at `node_id` (§22).
<code>_short</code>	function	<code>app/api/builder.py:1510</code>	<code>value: Any, limit: int=80</code> Internal helper for the short step.
<code>_previewable</code>	function	<code>app/api/builder.py:1524</code>	<code>value: Any, depth: int=0</code> Trim large values before they cross the wire.
<code>_acquired_document_from_run</code>	function	<code>app/api/candidates.py:53</code>	<code>run: dict[str, Any], document_id: str</code> Find an acquired document only in recorded acquisition-node output.
<code>download_acquired_research_document</code>	async function	<code>app/api/candidates.py:70</code>	<code>run_id: str, document_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code>

Symbol	Kind	Location	Signature / summary
			Authorize against owned run provenance, then issue a short-lived PDF URL.
<code>_discovery_candidate_url</code>	function	<code>app/api/candidates.py:97</code>	<code>candidate: dict[str, Any]</code> Internal helper for the discovery candidate url step.
<code>_discovered_candidates_from_run</code>	function	<code>app/api/candidates.py:115</code>	<code>run: dict[str, Any]</code> Every discovery/Deep Research/prior-project candidate this run ever produced.
<code>_structured_dataset_query_url</code>	function	<code>app/api/candidates.py:154</code>	<code>record: dict[str, Any]</code> Reconstruct the exact bounded Eurostat query that produced this row.
<code>_structured_dataset_candidates_from_run</code>	function	<code>app/api/candidates.py:175</code>	<code>run: dict[str, Any]</code> StructuredDatasetRetrieverAgent rows, reshaped into the same discovered-candidate table shape as web-search-based discovery.
<code>_internal_evidence_from_run</code>	function	<code>app/api/candidates.py:224</code>	<code>run: dict[str, Any]</code> InternalProjectEvidenceRetrieverAgent facts: no URL exists for these, only a source name and an exact quoted passage — the UI offers a secondary "verify this claim" action for them instead of a link.
<code>VerifyClaimRequest</code>	class	<code>app/api/candidates.py:258</code>	<code>BaseModel</code> Pydantic model defining the VerifyClaimRequest shape.
<code>list_candidates</code>	async function	<code>app/api/candidates.py:268</code>	<code>run_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> Every acquired evidence PDF for a run, plus discovered candidates.
<code>verify_claim_endpoint</code>	async function	<code>app/api/candidates.py:341</code>	<code>run_id: str, body: VerifyClaimRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Run the gpt-5.6-sol secondary check for one internal-evidence record.
<code>_scope</code>	function	<code>app/api/chat_conversations.py:23</code>	<code>user: CurrentUser</code> No summary declared.
<code>_store</code>	function	<code>app/api/chat_conversations.py:27</code>	<code>request: Request</code> No summary declared.
<code>ResolveConversationBody</code>	class	<code>app/api/chat_conversations.py:37</code>	<code>BaseModel</code> No summary declared.
<code>MessageBody</code>	class	<code>app/api/chat_conversations.py:42</code>	<code>BaseModel</code> No summary declared.
<code>MessageBody.content_is_bounded</code>	method	<code>app/api/chat_conversations.py:50</code>	<code>cls, value: dict[str, Any]</code> No summary declared.
<code>ReplaceMessageBody</code>	class	<code>app/api/chat_conversations.py:60</code>	<code>BaseModel</code> No summary declared.
<code>resolve_conversation</code>	async function	<code>app/api/chat_conversations.py:71</code>	<code>body: ResolveConversationBody, request: Request, user: CurrentUser=Depends(require_consultant)</code> No summary declared.
<code>get_conversation</code>	async function	<code>app/api/chat_conversations.py:90</code>	<code>conversation_id: str, request: Request, user:</code>

Symbol	Kind	Location	Signature / summary
			<code>CurrentUser=Depends(require_consultant)</code> No summary declared.
<code>append_message</code>	async function	<code>app/api/ chat_conversations.py:108</code>	<code>conversation_id: str, body: MessageBody, request: Request, user: CurrentUser=Depends(require_consultant)</code> No summary declared.
<code>replace_message</code>	async function	<code>app/api/ chat_conversations.py:124</code>	<code>conversation_id: str, message_id: str, body: ReplaceMessageBody, request: Request, user: CurrentUser=Depends(require_consultant)</code> No summary declared.
<code>build_deep_research_chat _workflow</code>	function	<code>app/api/ chat_workflows.py:267</code>	<code>rag_agent_id: str None=None</code> Add a saved RAG Agent lane without exposing collection/index filters.
<code>_scope</code>	function	<code>app/api/ chat_workflows.py:309</code>	<code>user: CurrentUser</code> No summary declared.
<code>_store</code>	function	<code>app/api/ chat_workflows.py:313</code>	<code>request: Request</code> No summary declared.
<code>_slug</code>	function	<code>app/api/ chat_workflows.py:323</code>	<code>value: str</code> No summary declared.
<code>_private_yaml</code>	function	<code>app/api/ chat_workflows.py:333</code>	<code>yaml_text: str</code> Force private lifecycle metadata without changing executable semantics.
<code>_validate</code>	function	<code>app/api/ chat_workflows.py:349</code>	<code>yaml_text: str</code> No summary declared.
<code>_summary</code>	function	<code>app/api/ chat_workflows.py:362</code>	<code>record: ChatWorkflowRecord</code> No summary declared.
<code>_repair_workspace_existin g_adapter</code>	async function	<code>app/api/ chat_workflows.py:366</code>	<code>record: ChatWorkflowRecord, store: ChatWorkflowStore</code> Refresh planner-created saved-workflow wrappers after adapter fixes.
<code>ImportChatWorkflowRequ est</code>	class	<code>app/api/ chat_workflows.py:399</code>	<code>BaseModel</code> No summary declared.
<code>CopyChatWorkflowReques t</code>	class	<code>app/api/ chat_workflows.py:405</code>	<code>BaseModel</code> No summary declared.
<code>GenerateChatWorkflowRe quest</code>	class	<code>app/api/ chat_workflows.py:411</code>	<code>BaseModel</code> No summary declared.
<code>_save</code>	async function	<code>app/api/ chat_workflows.py:421</code>	<code>request: Request, user: CurrentUser, *, slug: str, display_name: str, yaml_text: str, source: Literal['generated', 'imported', 'existing'], source_workflow_name: str None=None, managed: bool=False, adapter_key: str None=None, adapter_fingerprint: str None=None</code> No summary declared.
<code>ensure_managed_adapter</code>	async function	<code>app/api/ chat_workflows.py:464</code>	<code>request: Request, user: CurrentUser, *, adapter_key: str, display_name: str, yaml_text: str, source: Literal['generated', 'imported', 'existing'], source_workflow_name: str </code>

Symbol	Kind	Location	Signature / summary
			None=None, preferred_slug: str None=None Reuse one owner-scoped canonical adapter and upgrade it in place.
list_private_chat_workflows	async function	app/api/chat_workflows.py:547	request: Request, user: CurrentUser=Depends(require_consultant), include_managed: bool=False No summary declared.
ensure_deep_research_chat_workflow	async function	app/api/chat_workflows.py:562	request: Request, user: CurrentUser=Depends(require_consultant), rag_agent_id: str None=None Return the owner's vetted Deep Research workflow for one Knowledge binding.
ensure_general_chat_workflow	async function	app/api/chat_workflows.py:585	request: Request, user: CurrentUser=Depends(require_consultant) Return one stable owner-scoped workflow for general Chat requests.
get_builder_chat_execution_adapter	async function	app/api/chat_workflows.py:602	workflow_name: str, user: CurrentUser=Depends(require_consultant) Build a universal, execution-only Chat wrapper around a saved workflow.
get_private_chat_workflow	async function	app/api/chat_workflows.py:637	chat_workflow_id: str, request: Request, user: CurrentUser=Depends(require_consultant) No summary declared.
import_private_chat_workflow	async function	app/api/chat_workflows.py:652	body: ImportChatWorkflowRequest, request: Request, user: CurrentUser=Depends(require_consultant) No summary declared.
copy_existing_chat_workflow	async function	app/api/chat_workflows.py:667	body: CopyChatWorkflowRequest, request: Request, user: CurrentUser=Depends(require_consultant) No summary declared.
generate_private_chat_workflow	async function	app/api/chat_workflows.py:688	body: GenerateChatWorkflowRequest, request: Request, user: CurrentUser=Depends(require_consultant) No summary declared.
archive_private_chat_workflow	async function	app/api/chat_workflows.py:726	chat_workflow_id: str, request: Request, user: CurrentUser=Depends(require_consultant) No summary declared.
request_private_workflow_publication	async function	app/api/chat_workflows.py:738	chat_workflow_id: str, request: Request, user: CurrentUser=Depends(require_consultant) No summary declared.
_managed_adapter_key	function	app/api/chat_workspace.py:32	body: PlanWorkspaceRequest, plan Stable owner-local identity for execution-equivalent Chat adapters.
PlanWorkspaceRequest	class	app/api/chat_workspace.py:62	BaseModel No summary declared.
WorkflowLabelInput	class	app/api/chat_workspace.py:79	BaseModel No summary declared.

Symbol	Kind	Location	Signature / summary
WorkflowLabelsRequest	class	app/api/chat_workspace.py:86	<code>BaseModel</code> No summary declared.
GeneratedWorkflowLabel	class	app/api/chat_workspace.py:90	<code>BaseModel</code> No summary declared.
GeneratedWorkflowLabels	class	app/api/chat_workspace.py:95	<code>BaseModel</code> No summary declared.
_humanize_workflow_name	function	app/api/chat_workspace.py:99	<code>value: str</code> No summary declared.
_weak_workflow_label	function	app/api/chat_workspace.py:105	<code>item: WorkflowLabelInput</code> No summary declared.
_workflow_label_fingerprint	function	app/api/chat_workspace.py:116	<code>item: WorkflowLabelInput</code> No summary declared.
_clean_workflow_label	function	app/api/chat_workspace.py:121	<code>value: str, fallback: str</code> No summary declared.
list_workspace_experiences	async function	app/api/chat_workspace.py:135	<code>user:</code> <code>CurrentUser=Depends(require_consultant)</code> No summary declared.
chat_workflow_labels	async function	app/api/chat_workspace.py:143	<code>body: WorkflowLabelsRequest, request: Request, user:</code> <code>CurrentUser=Depends(require_consultant)</code> Return Chat-only business labels without renaming saved workflows.
plan_chat_workspace	async function	app/api/chat_workspace.py:211	<code>body: PlanWorkspaceRequest, user:</code> <code>CurrentUser=Depends(require_consultant)</code> No summary declared.
prepare_chat_workspace	async function	app/api/chat_workspace.py:220	<code>body: PlanWorkspaceRequest, request: Request, user:</code> <code>CurrentUser=Depends(require_consultant)</code> No summary declared.
run_cost	async function	app/api/cost.py:14	<code>run_id: str, request: Request, user:</code> <code>CurrentUser=Depends(get_current_user)</code> Run the cost.
session_cost	async function	app/api/cost.py:34	<code>session_id: str, request: Request, user:</code> <code>CurrentUser=Depends(get_current_user)</code> Compute the session cost.
_db	function	app/api/cost_admin.py:33	<code>request: Request</code> Internal helper for the db step.
_cost_ledger_entries	function	app/api/cost_admin.py:42	<code>db, *, since: datetime None=None</code> Internal helper for the cost ledger entries step.
PricingOverrideRequest	class	app/api/cost_admin.py:65	<code>BaseModel</code> Pydantic model defining the PricingOverrideRequest shape.
list_pricing	async function	app/api/cost_admin.py:77	<code>request: Request, openrouter_q: str None=Query(default=None, max_length=200), openrouter_limit: int=Query(default=25, ge=1, le=200), user:</code> <code>CurrentUser=Depends(require_admin)</code> Direct/local catalog pricing (editable overrides layered on MODEL_PRICING defaults) plus a live, read-only

Symbol	Kind	Location	Signature / summary
			slice of OpenRouter's own per-model pricing — OpenRouter's ~400-500 models are priced by OpenRouter itself and reported per-call via usage.cost (LedgerEntry.cost_source == "provider_reported"),
set_pricing_override	async function	app/api/cost_admin.py:125	model: str, body: PricingOverrideRequest, request: Request, user: CurrentUser=Depends(require_admin) Set the pricing override.
clear_pricing_override	async function	app/api/cost_admin.py:165	model: str, request: Request, user: CurrentUser=Depends(require_admin) Clear the pricing override.
InfraAllocationRequest	class	app/api/cost_admin.py:194	BaseModel Pydantic model defining the InfraAllocationRequest shape.
_effective_per_call_cost	function	app/api/cost_admin.py:207	allocation: dict[str, Any] Internal helper for the effective per call cost step.
list_infra_allocations	async function	app/api/cost_admin.py:225	request: Request, user: CurrentUser=Depends(require_admin) List the infra allocations.
set_infra_allocation	async function	app/api/cost_admin.py:256	model: str, body: InfraAllocationRequest, request: Request, user: CurrentUser=Depends(require_admin) Set the infra allocation.
_allocated_infra_cost	function	app/api/cost_admin.py:300	entries: list[dict], allocations: dict[str, dict] Internal helper for the allocated infra cost step.
_cache_savings_usd	function	app/api/cost_admin.py:327	entry: dict Mirrors CostLedger.calculate()'s own cache multipliers — what THIS entry's cached tokens would have cost at full input price, minus what they actually cost.
cache_summary	async function	app/api/cost_admin.py:342	request: Request, since_days: int=Query(default=30, ge=1, le=365), user: CurrentUser=Depends(require_admin) Compute the cache summary.
BudgetRequest	class	app/api/cost_admin.py:397	BaseModel Pydantic model defining the BudgetRequest shape.
_today_start	function	app/api/cost_admin.py:406	Internal helper for the today start step.
get_budgets	async function	app/api/cost_admin.py:417	request: Request, user: CurrentUser=Depends(require_admin) Return the budgets.
set_global_budget	async function	app/api/cost_admin.py:467	body: BudgetRequest, request: Request, user: CurrentUser=Depends(require_admin) Set the global budget.
set_session_budget	async function	app/api/cost_admin.py:494	session_id: str, body: BudgetRequest, request: Request, user: CurrentUser=Depends(require_admin) Set the session budget.

Symbol	Kind	Location	Signature / summary
<code>cost_overview</code>	async function	<code>app/api/cost_admin.py:531</code>	<code>request: Request, days: int=Query(default=30, ge=1, le=365), user: CurrentUser=Depends(require_admin)</code> Compute the cost overview.
<code>ensure_indexes</code>	async function	<code>app/api/email_oauth.py:55</code>	<code>db: Any</code> TTL-expires an abandoned connect attempt's pending state — a user who starts connecting a mailbox and never finishes leaves one small, non-secret document (no token, no code) that ages out on its own.
<code>_services</code>	function	<code>app/api/email_oauth.py:62</code>	<code>request: Request</code> Internal helper for the services step.
<code>_require_db</code>	function	<code>app/api/email_oauth.py:74</code>	<code>request: Request</code> Internal helper for the require db step.
<code>_require_email_service</code>	function	<code>app/api/email_oauth.py:92</code>	<code>request: Request</code> Internal helper for the require email service step.
<code>_connection_id</code>	function	<code>app/api/email_oauth.py:107</code>	<code>provider: Provider, address: str</code> Internal helper for the connection id step.
<code>_html_result</code>	function	<code>app/api/email_oauth.py:124</code>	<code>*, ok: bool, message: str</code> A minimal, self-contained page for the OAuth popup window — no external assets (this is a plain HTTP response, not a Builder route), just enough to tell the user what happened and let the opener refresh its connection list before the window closes.
<code>connect</code>	async function	<code>app/api/email_oauth.py:146</code>	<code>provider: Provider, request: Request, user: CurrentUser=Depends(require_consultant)</code> Starts the authorization-code flow: stash a one-time state (+ PKCE verifier for Gmail), redirect the user's browser to the provider.
<code>oauth_callback</code>	async function	<code>app/api/email_oauth.py:178</code>	<code>provider: Provider, request: Request, code: str None=None, state: str None=None, error: str None=None, error_description: str None=None</code> The provider redirects the user's browser back here after consent.
<code>AllowSendUpdate</code>	class	<code>app/api/email_oauth.py:262</code>	<code>BaseModel</code> Pydantic model defining the AllowSendUpdate shape.
<code>update_connection</code>	async function	<code>app/api/email_oauth.py:274</code>	<code>connection_id: str, body: AllowSendUpdate, request: Request, user: CurrentUser=Depends(require_consultant)</code> The only lever to actually let a connection send — every connection (OAuth-established or static) starts refused; this is the explicit second step, same reasoning as every other write-capable connection in this platform (MCP, F&O).
<code>disconnect</code>	async function	<code>app/api/email_oauth.py:303</code>	<code>connection_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> Compute the disconnect.
<code>_scope</code>	function	<code>app/api/entity_registry.py:25</code>	<code>user: CurrentUser</code> Internal helper for the scope step.

Symbol	Kind	Location	Signature / summary
<code>_service</code>	function	<code>app/api/entity_registry.py:37</code>	<code>request: Request</code> Internal helper for the service step.
<code>RegisterEntityRequest</code>	class	<code>app/api/entity_registry.py:49</code>	<code>BaseModel</code> Pydantic model defining the RegisterEntityRequest shape.
<code>DeleteEntityRequest</code>	class	<code>app/api/entity_registry.py:62</code>	<code>BaseModel</code> Pydantic model defining the DeleteEntityRequest shape.
<code>list_entity_types</code>	function	<code>app/api/entity_registry.py:76</code>	 List the entity types.
<code>list_entities</code>	async function	<code>app/api/entity_registry.py:86</code>	<code>request: Request, collection_id: str='default', user: CurrentUser=Depends(require_consultant)</code> List the entities.
<code>register_entity</code>	async function	<code>app/api/entity_registry.py:112</code>	<code>req: RegisterEntityRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Register the entity.
<code>delete_entity</code>	async function	<code>app/api/entity_registry.py:144</code>	<code>req: DeleteEntityRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Delete the entity.
<code>RunEvalRequest</code>	class	<code>app/api/eval.py:26</code>	<code>BaseModel</code> Pydantic model defining the RunEvalRequest shape.
<code>WorkflowCompareRequest</code>	class	<code>app/api/eval.py:36</code>	<code>BaseModel</code> Pydantic model defining the WorkflowCompareRequest shape.
<code>ScoreOutputRequest</code>	class	<code>app/api/eval.py:46</code>	<code>BaseModel</code> Pydantic model defining the ScoreOutputRequest shape.
<code>_load_reference</code>	function	<code>app/api/eval.py:62</code>	 Extract text from the stored reference proposal (gitignored, private).
<code>score_output</code>	async function	<code>app/api/eval.py:71</code>	<code>req: ScoreOutputRequest, request: Request, user: CurrentUser=Depends(require_permission('eval:run'))</code> Score the output.
<code>list_golden_set</code>	async function	<code>app/api/eval.py:116</code>	<code>name: str='document_qa', _user: CurrentUser=Depends(require_permission('eval:run'))</code> List the golden set.
<code>run</code>	async function	<code>app/api/eval.py:138</code>	<code>req: RunEvalRequest, request: Request, user: CurrentUser=Depends(require_permission('eval:run'))</code> Run the result.
<code>history</code>	async function	<code>app/api/eval.py:211</code>	<code>request: Request, limit: int=20, _user: CurrentUser=Depends(require_permission('eval:run'))</code> Compute the history.

Symbol	Kind	Location	Signature / summary
<code>list_workflow_golden_set</code>	async function	<code>app/api/eval.py:234</code>	<pre>name: str='verder_customer_triage', _user: CurrentUser=Depends(require_permission('eval:run'))</pre> List the workflow golden set.
<code>workflow_compare</code>	async function	<code>app/api/eval.py:255</code>	<pre>req: WorkflowCompareRequest, request: Request, user: CurrentUser=Depends(require_permission('eval:run'))</pre> Run the same golden set of real customer messages through the flagship triage workflow once per candidate model, and compare which one actually reaches the right business outcome — not a judged quality score, since there's a deterministic right answer (intent/complexity/route/review) to check against
<code>_call_probe</code>	async function	<code>app/api/health.py:20</code>	<pre>probe: Probe</pre> Internal helper for the call probe step.
<code>_probe</code>	async function	<code>app/api/health.py:35</code>	<pre>name: str, probe: Probe None</pre> Probe the result.
<code>_mongo_probe</code>	function	<code>app/api/health.py:78</code>	<pre>services: dict[str, Any]</pre> Internal helper for the mongo probe step.
<code>_weaviate_probe</code>	function	<code>app/api/health.py:93</code>	<pre>services: dict[str, Any]</pre> Internal helper for the weaviate probe step.
<code>_minio_probe</code>	function	<code>app/api/health.py:108</code>	<pre>services: dict[str, Any]</pre> Internal helper for the minio probe step.
<code>_redis_probe</code>	function	<code>app/api/health.py:126</code>	<pre>services: dict[str, Any]</pre> Internal helper for the redis probe step.
<code>_checkpointer_probe</code>	function	<code>app/api/health.py:141</code>	<pre>services: dict[str, Any]</pre> Internal helper for the checkpointer probe step.
<code>probe_services</code>	async function	<code>app/api/health.py:163</code>	<pre>services: dict[str, Any]</pre> Probe every configured dependency concurrently.
<code>_health_payload</code>	async function	<code>app/api/health.py:207</code>	<pre>request: Request</pre> Internal helper for the health payload step.
<code>health</code>	async function	<code>app/api/health.py:264</code>	<pre>request: Request</pre> Return live dependency status without failing the liveness request.
<code>ready</code>	async function	<code>app/api/health.py:272</code>	<pre>request: Request</pre> Return 503 until every required runtime dependency is reachable.
<code>ChunkView</code>	class	<code>app/api/inspect.py:40</code>	<pre>BaseModel</pre> Pydantic model defining the ChunkView shape.
<code>SourceGroup</code>	class	<code>app/api/inspect.py:64</code>	<pre>BaseModel</pre> Pydantic model defining the SourceGroup shape.
<code>ChunksResponse</code>	class	<code>app/api/inspect.py:83</code>	<pre>BaseModel</pre> Pydantic model defining the ChunksResponse shape.
<code>RetrieveRequest</code>	class	<code>app/api/inspect.py:100</code>	<pre>BaseModel</pre> Pydantic model defining the RetrieveRequest shape.
<code>RetrievedView</code>	class	<code>app/api/inspect.py:117</code>	<pre>BaseModel</pre> Pydantic model defining the RetrievedView shape.

Symbol	Kind	Location	Signature / summary
RetrieveResponse	class	app/api/inspect.py:140	<code>BaseModel</code> Pydantic model defining the RetrieveResponse shape.
_get_weaviate	function	app/api/inspect.py:160	<code>request: Request</code> Return the WeaviateClient wrapper.
_native_collection	function	app/api/inspect.py:172	<code>wv</code> Return the v4 native collection handle for DocumentChunk.
list_chunks	function	app/api/inspect.py:192	<code>request: Request, collection_id: Optional[str]=Query(None), session_id: Optional[str]=Query(None), limit: int=Query(2000, le=10000), user: CurrentUser=Depends(require_permission('workflow:read'))</code> List ingested chunks grouped by source document.
_run_retrieval	async function	app/api/inspect.py:282	<code>request: Request, req: RetrieveRequest, user: CurrentUser</code> Run the query through the canonical retrieval path.
retrieve_preview	async function	app/api/inspect.py:345	<code>request: Request, req: RetrieveRequest, user: CurrentUser=Depends(require_permission('workflow:read'))</code> Run a query through hybrid search + rerank and show the ranked chunks.
ensure_indexes	async function	app/api/integration_oauth.py:52	<code>db: Any</code> TTL-expires an abandoned connect attempt's pending state.
_services	function	app/api/integration_oauth.py:57	<code>request: Request</code> Internal helper for the services step.
_require_db	function	app/api/integration_oauth.py:69	<code>request: Request</code> Internal helper for the require db step.
_require_integration_service	function	app/api/integration_oauth.py:87	<code>request: Request</code> Internal helper for the require integration service step.
_connection_id	function	app/api/integration_oauth.py:102	<code>provider: Provider, address: str</code> Internal helper for the connection id step.
_html_result	function	app/api/integration_oauth.py:118	<code>*, ok: bool, message: str</code> A minimal, self-contained page for the OAuth popup window.
connect	async function	app/api/integration_oauth.py:137	<code>provider: Provider, request: Request, user: CurrentUser=Depends(require_consultant)</code> Starts the authorization-code flow: stash a one-time state (+ PKCE verifier for Google), redirect the user's browser to the provider.
oauth_callback	async function	app/api/integration_oauth.py:169	<code>provider: Provider, request: Request, code: str None=None, state: str None=None, error: str None=None, error_description: str None=None</code> The provider redirects the user's browser back here after consent.
list_connections	async function	app/api/integration_oauth.py:249	

Symbol	Kind	Location	Signature / summary
			<code>request: Request, user: CurrentUser=Depends(require_consultant)</code> List the connections.
disconnect	async function	<code>app/api/integration_oauth.py:270</code>	<code>connection_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> Compute the disconnect.
browse_files	async function	<code>app/api/integration_oauth.py:295</code>	<code>connection_id: str, request: Request, folder_id: str None=None, query: str='', page_size: int=25, page_token: str None=None, user: CurrentUser=Depends(require_consultant)</code> Live folder/search browsing for the node config panel's file picker — not workflow execution, just a thin proxy through the service for whichever operation the presence of `query` selects.
browse_path	async function	<code>app/api/integration_oauth.py:327</code>	<code>connection_id: str, file_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> Breadcrumb chain for the file browser: walk `parent_id` up from `file_id` via repeated lookups, bounded so a cyclic/misbehaving provider response can never hang this request.
download_file	async function	<code>app/api/integration_oauth.py:359</code>	<code>connection_id: str, file_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> Streams one file's real bytes straight to the browser — a plain navigation (an ` <a href="">` click, not a fetch), authenticated by the same HttpOnly cookie the OAuth popup flow relies on.
_scope	function	<code>app/api/knowledge.py:40</code>	<code>user: CurrentUser</code> Internal helper for the scope step.
_service	function	<code>app/api/knowledge.py:52</code>	<code>request: Request</code> Internal helper for the service step.
_repository	function	<code>app/api/knowledge.py:64</code>	<code>request: Request</code> Internal helper for the repository step.
CollectionCreate	class	<code>app/api/knowledge.py:76</code>	<code>BaseModel</code> Pydantic model defining the CollectionCreate shape.
create_collection	async function	<code>app/api/knowledge.py:92</code>	<code>payload: CollectionCreate, request: Request, user: CurrentUser=Depends(require_permission('knowledge:write'))</code> Create the collection.
list_collections	async function	<code>app/api/knowledge.py:110</code>	<code>request: Request, user: CurrentUser=Depends(require_permission('knowledge:read'))</code> List the collections.
get_collection	async function	<code>app/api/knowledge.py:124</code>	<code>collection_id: str, request: Request, user: CurrentUser=Depends(require_permission('knowledge:read'))</code> Return the collection.
ProfileCreate	class	<code>app/api/knowledge.py:142</code>	

Symbol	Kind	Location	Signature / summary
			<code>BaseModel</code> Pydantic model defining the ProfileCreate shape.
<code>create_profile</code>	async function	<code>app/api/knowledge.py:164</code>	<code>payload: ProfileCreate, request: Request, user: CurrentUser=Depends(require_permission('knowledge:write'))</code> Create the profile.
<code>list_profiles</code>	async function	<code>app/api/knowledge.py:185</code>	<code>request: Request, profile_type: ProfileType None=Query(None), user: CurrentUser=Depends(require_permission('knowledge:read'))</code> List the profiles.
<code>get_profile</code>	async function	<code>app/api/knowledge.py:201</code>	<code>profile_id: str, request: Request, version: int None=Query(None, ge=1), user: CurrentUser=Depends(require_permission('knowledge:read'))</code> Return the profile.
<code>create_default_profiles</code>	async function	<code>app/api/knowledge.py:222</code>	<code>request: Request, user: CurrentUser=Depends(require_permission('knowledge:write'))</code> Create the default profiles.
<code>ingestion_presets</code>	async function	<code>app/api/knowledge.py:264</code>	<code>user: CurrentUser=Depends(require_permission('knowledge:read'))</code> Compute the ingestion presets.
<code>doc_types</code>	async function	<code>app/api/knowledge.py:276</code>	<code>user: CurrentUser=Depends(require_permission('knowledge:read'))</code> Document types a Collection may declare.
<code>embedding_models</code>	async function	<code>app/api/knowledge.py:288</code>	<code>user: CurrentUser=Depends(require_permission('knowledge:read'))</code> Embedding models selectable when building an Index.
<code>IndexCreate</code>	class	<code>app/api/knowledge.py:303</code>	<code>BaseModel</code> Pydantic model defining the IndexCreate shape.
<code>create_index</code>	async function	<code>app/api/knowledge.py:323</code>	<code>collection_id: str, payload: IndexCreate, request: Request, user: CurrentUser=Depends(require_permission('knowledge:write'))</code> Create the index.
<code>list_indexes</code>	async function	<code>app/api/knowledge.py:346</code>	<code>collection_id: str, request: Request, user: CurrentUser=Depends(require_permission('knowledge:read'))</code> List the indexes.
<code>activate_index</code>	async function	<code>app/api/knowledge.py:363</code>	<code>collection_id: str, index_id: str, request: Request, user: CurrentUser=Depends(require_permission('knowledge:write'))</code> Compute the activate index.
<code>start_ingestion</code>	async function	<code>app/api/knowledge.py:386</code>	<code>collection_id: str, request: Request, files: list[UploadFile]=File(...),</code>

Symbol	Kind	Location	Signature / summary
			<pre> parser_profile_id: str None=Form(None), parser_profile_version: int None=Form(None), chunking_profile_id: str None=Form(None), chunking_profile_version: int None=Form(None), embedding_profile_id: str None=Form(None), embedding_profile_version: int None=Form(None), embedding_model: str None=Form(None), metadata_json: str=Form('{}'), user: CurrentUser=Depends(require_permission(' knowledge:write')) Start the ingestion. </pre>
list_ingestions	async function	app/api/knowledge.py:606	<pre> request: Request, collection_id: str None=Query(None), user: CurrentUser=Depends(require_permission(' knowledge:read')) List the ingestions. </pre>
get_ingestion	async function	app/api/knowledge.py:622	<pre> ingestion_job_id: str, request: Request, user: CurrentUser=Depends(require_permission(' knowledge:read')) Return the ingestion. </pre>
cancel_ingestion	async function	app/api/knowledge.py:641	<pre> ingestion_job_id: str, request: Request, user: CurrentUser=Depends(require_permission(' knowledge:write')) Cancel the ingestion. </pre>
list_documents	async function	app/api/knowledge.py:669	<pre> collection_id: str, request: Request, user: CurrentUser=Depends(require_permission(' knowledge:read')) List the documents. </pre>
get_document	async function	app/api/knowledge.py:686	<pre> document_id: str, request: Request, user: CurrentUser=Depends(require_permission(' knowledge:read')) Return the document. </pre>
document_source_url	async function	app/api/knowledge.py:705	<pre> document_id: str, request: Request, user: CurrentUser=Depends(require_permission(' knowledge:read')) Compute the document source url. </pre>
_configured	function	app/api/llm_providers.py:22	<pre> model: str, provider: str Internal helper for the configured step. </pre>
list_models	async function	app/api/llm_providers.py:44	<pre> user: CurrentUser=Depends(require_consultant) Return public capability metadata without URLs or credentials. </pre>
list_openrouter_models	async function	app/api/llm_providers.py:90	<pre> q: str None=Query(default=None, max_length=200), limit: int=Query(default=50, ge=1, le=200), user: CurrentUser=Depends(require_consultant) </pre>

Symbol	Kind	Location	Signature / summary
			Live OpenRouter model catalog (via OpenRouter's own `GET /v1/models`), cached with a short TTL — see app/llm/openrouter_catalog.py.
probe_model	async function	app/api/llm_providers.py:112	<code>model: str, user: CurrentUser=Depends(require_admin)</code> Probe an enabled local endpoint; never return its private address.
_schema_field_names	function	app/api/node_types_chat.py:49	<code>schema: dict[str, Any] None</code> Internal helper for the schema field names step.
_build_node_type_catalog	function	app/api/node_types_chat.py:63	<code></code> Always reads NodeRegistry.manifest() fresh — the same live source GET /api/node-types serves — so a node type added to the codebase shows up here automatically with no other change needed.
_manifest_entry	function	app/api/node_types_chat.py:81	<code>type_name: str</code> Live lookup — same freshness guarantee as _build_node_type_catalog.
ChatMessage	class	app/api/node_types_chat.py:89	<code>BaseModel</code> Pydantic model defining the ChatMessage shape.
AskContext	class	app/api/node_types_chat.py:100	<code>BaseModel</code> Compact, structured context for a scoped Ask AI question — what the user actually clicked, not the whole workflow.
AskAboutNodeTypesRequest	class	app/api/node_types_chat.py:119	<code>BaseModel</code> Pydantic model defining the AskAboutNodeTypesRequest shape.
_compact_catalog	function	app/api/node_types_chat.py:134	<code>type_names: list[str]</code> Same one-line-per-type shape as _build_node_type_catalog(), but for an explicit shortlist rather than the whole registry — what a node- or field-scoped question actually needs.
ask_about_node_types	async function	app/api/node_types_chat.py:153	<code>req: AskAboutNodeTypesRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Compute the ask about node types.
DraftPromptRequest	class	app/api/node_types_chat.py:367	<code>BaseModel</code> Pydantic model defining the DraftPromptRequest shape.
draft_prompt	async function	app/api/node_types_chat.py:383	<code>req: DraftPromptRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Draft the prompt.
DraftInstructionsFieldSpec	class	app/api/node_types_chat.py:456	<code>BaseModel</code> Pydantic model defining the DraftInstructionsFieldSpec shape.
DraftInstructionsRequest	class	app/api/node_types_chat.py:471	<code>BaseModel</code> Pydantic model defining the DraftInstructionsRequest shape.
_describe_fields	function	app/api/node_types_chat.py:484	<code>fields: list[DraftInstructionsFieldSpec]</code> Internal helper for the describe fields step.

Symbol	Kind	Location	Signature / summary
draft_instructions	async function	app/api/node_types_chat.py:507	<code>req: DraftInstructionsRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Draft the instructions.
DraftCodeRequest	class	app/api/node_types_chat.py:567	<code>BaseModel</code> Pydantic model defining the DraftCodeRequest shape.
draft_code	async function	app/api/node_types_chat.py:592	<code>req: DraftCodeRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Draft the code.
_scope	function	app/api/prompt_templates.py:16	<code>user: CurrentUser</code> No summary declared.
_store	function	app/api/prompt_templates.py:20	<code>request: Request</code> No summary declared.
TemplateBody	class	app/api/prompt_templates.py:27	<code>BaseModel</code> No summary declared.
FavoriteBody	class	app/api/prompt_templates.py:34	<code>BaseModel</code> No summary declared.
list_prompt_templates	async function	app/api/prompt_templates.py:39	<code>request: Request, user: CurrentUser=Depends(require_consultant)</code> No summary declared.
create_prompt_template	async function	app/api/prompt_templates.py:45	<code>body: TemplateBody, request: Request, user: CurrentUser=Depends(require_consultant)</code> No summary declared.
update_prompt_template	async function	app/api/prompt_templates.py:50	<code>template_id: str, body: TemplateBody, request: Request, user: CurrentUser=Depends(require_consultant)</code> No summary declared.
duplicate_prompt_template	async function	app/api/prompt_templates.py:60	<code>template_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> No summary declared.
favorite_prompt_template	async function	app/api/prompt_templates.py:75	<code>template_id: str, body: FavoriteBody, request: Request, user: CurrentUser=Depends(require_consultant)</code> No summary declared.
delete_prompt_template	async function	app/api/prompt_templates.py:85	<code>template_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> No summary declared.
_scope	function	app/api/proposals.py:36	<code>user: CurrentUser</code> Internal helper for the scope step.
_services	function	app/api/proposals.py:48	<code>request: Request</code> Internal helper for the services step.
_store	function	app/api/proposals.py:60	<code>request: Request, *, require_objects: bool=False</code> Internal helper for the store step.
_graph_from_checkpoint	function	app/api/proposals.py:83	<code>checkpoint: dict[str, Any]</code> Internal helper for the graph from checkpoint step.

Symbol	Kind	Location	Signature / summary
GraphRequest	class	app/api/proposals.py:104	<code>BaseModel</code> Pydantic model defining the GraphRequest shape.
coverage	async function	app/api/proposals.py:114	<code>body: GraphRequest, user: CurrentUser=Depends(require_consultant)</code> Compute the coverage.
review_run	async function	app/api/proposals.py:129	<code>run_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> Compute the review run.
SourceVersionRequest	class	app/api/proposals.py:170	<code>BaseModel</code> Pydantic model defining the SourceVersionRequest shape.
register_source_version	async function	app/api/proposals.py:188	<code>proposal_id: str, source_id: str, body: SourceVersionRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Register the source version.
list_source_versions	async function	app/api/proposals.py:225	<code>proposal_id: str, request: Request, source_id: str None=None, user: CurrentUser=Depends(require_consultant)</code> List the source versions.
VerifyClaimsRequest	class	app/api/proposals.py:247	<code>BaseModel</code> Pydantic model defining the VerifyClaimsRequest shape.
verify_claims	async function	app/api/proposals.py:261	<code>proposal_id: str, body: VerifyClaimsRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Verify the claims.
ConceptRequest	class	app/api/proposals.py:327	<code>BaseModel</code> Pydantic model defining the ConceptRequest shape.
concept_alternatives	async function	app/api/proposals.py:341	<code>proposal_id: str, body: ConceptRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Compute the concept alternatives.
ApprovalRequest	class	app/api/proposals.py:381	<code>BaseModel</code> Pydantic model defining the ApprovalRequest shape.
request_approval	async function	app/api/proposals.py:395	<code>proposal_id: str, body: ApprovalRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Compute the request approval.
ApprovalDecisionRequest	class	app/api/proposals.py:423	<code>BaseModel</code> Pydantic model defining the ApprovalDecisionRequest shape.
decide_approval	async function	app/api/proposals.py:435	<code>proposal_id: str, approval_id: str, body: ApprovalDecisionRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Compute the decide approval.
list_approvals	async function	app/api/proposals.py:466	<code>proposal_id: str, request: Request, user:</code>

Symbol	Kind	Location	Signature / summary
			<code>CurrentUser=Depends(require_consultant)</code> List the approvals.
HorizonEvaluationRequest	class	<code>app/api/proposals.py:485</code>	<code>BaseModel</code> Pydantic model defining the HorizonEvaluationRequest shape.
horizon_evaluation	async function	<code>app/api/proposals.py:507</code>	<code>proposal_id: str, body: HorizonEvaluationRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Compute the horizon evaluation.
ProposalEvidenceFactoryRequest	class	<code>app/api/proposals.py:547</code>	<code>BaseModel</code> Pydantic model defining the ProposalEvidenceFactoryRequest shape.
run_proposal_evidence_factory	async function	<code>app/api/proposals.py:569</code>	<code>proposal_id: str, body: ProposalEvidenceFactoryRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Verify already-fetched full text and return a citable evidence package.
RenderProposalRequest	class	<code>app/api/proposals.py:619</code>	<code>BaseModel</code> Pydantic model defining the RenderProposalRequest shape.
RenderDOCXProposalRequest	class	<code>app/api/proposals.py:645</code>	<code>RenderProposalRequest</code> Provides the RenderDOCXProposalRequest behaviour.
render_proposal	async function	<code>app/api/proposals.py:659</code>	<code>proposal_id: str, body: RenderProposalRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Render a proposal and persist both sanitised HTML and final PDF.
render_proposal_docx	async function	<code>app/api/proposals.py:696</code>	<code>proposal_id: str, body: RenderDOCXProposalRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Render a proposal as editable DOCX and persist its sanitised source.
_scope	function	<code>app/api/rag_agents.py:16</code>	<code>user: CurrentUser</code> Internal helper for the scope step.
RAGAgentCreate	class	<code>app/api/rag_agents.py:28</code>	<code>BaseModel</code> Pydantic model defining the RAGAgentCreate shape.
create_agent	async function	<code>app/api/rag_agents.py:48</code>	<code>payload: RAGAgentCreate, request: Request, user: CurrentUser=Depends(require_permission('rag:write'))</code> Create the agent.
list_agents	async function	<code>app/api/rag_agents.py:69</code>	<code>request: Request, search: str None=Query(default=None), user: CurrentUser=Depends(require_permission('knowledge:read'))</code> List the agents.
get_agent	async function	<code>app/api/rag_agents.py:87</code>	<code>rag_agent_id: str, request: Request, user: CurrentUser=Depends(require_permission('</code>

Symbol	Kind	Location	Signature / summary
			<code>knowledge: read'))</code> Return the agent.
RAGQueryRequest	class	<code>app/api/rag_agents.py:107</code>	<code>BaseModel</code> Pydantic model defining the RAGQueryRequest shape.
query_agent	async function	<code>app/api/rag_agents.py:121</code>	<code>rag_agent_id: str, payload: RAGQueryRequest, request: Request, user: CurrentUser=Depends(require_permission('rag:query'))</code> Query the agent.
list_scientific_skills	async function	<code>app/api/research.py:10</code>	<code>request: Request, user: CurrentUser=Depends(require_consultant)</code> List the scientific skills.
presets	async function	<code>app/api/retrieval.py:20</code>	<code>user: CurrentUser=Depends(require_permission('knowledge:read'))</code> Return editable starting configurations; saving creates a profile.
_scope	function	<code>app/api/retrieval.py:28</code>	<code>user: CurrentUser</code> Internal helper for the scope step.
_service	function	<code>app/api/retrieval.py:40</code>	<code>request: Request</code> Internal helper for the service step.
_llm	function	<code>app/api/retrieval.py:52</code>	<code>request: Request, scope: str, node_id: str, collection_id: str None=None</code> Internal helper for the llm step.
RetrievalSearchRequest	class	<code>app/api/retrieval.py:75</code>	<code>BaseModel</code> Pydantic model defining the RetrievalSearchRequest shape.
RetrievalSearchRequest.to_query	method	<code>app/api/retrieval.py:110</code>	<code>self, owner_scope_id: str</code> Compute the to query.
_safe_query	function	<code>app/api/retrieval.py:142</code>	<code>value: str</code> Internal helper for the safe query step.
search	async function	<code>app/api/retrieval.py:158</code>	<code>payload: RetrievalSearchRequest, request: Request, user: CurrentUser=Depends(require_permission('rag:query'))</code> Search the result.
RetrievalCompareRequest	class	<code>app/api/retrieval.py:181</code>	<code>BaseModel</code> Pydantic model defining the RetrievalCompareRequest shape.
compare	async function	<code>app/api/retrieval.py:191</code>	<code>payload: RetrievalCompareRequest, request: Request, user: CurrentUser=Depends(require_permission('rag:query'))</code> Compare the result.
list_traces	async function	<code>app/api/retrieval.py:239</code>	<code>request: Request, limit: int=Query(default=100, ge=1, le=500), user: CurrentUser=Depends(require_permission('knowledge:read'))</code> List the traces.
get_trace		<code>app/api/retrieval.py:255</code>	

Symbol	Kind	Location	Signature / summary
	async function		<code>retrieval_request_id: str, request: Request, user: CurrentUser=Depends(require_permission('knowledge:read'))</code> Return the trace.
<code>get_trace_chunk_context</code>	async function	<code>app/api/retrieval.py:276</code>	<code>retrieval_request_id: str, chunk_id: str, request: Request, user: CurrentUser=Depends(require_permission('knowledge:read'))</code> Return cited and adjacent same-document passages from one scoped trace.
<code>starter_questions</code>	function	<code>app/api/run_chat.py:27</code>	<code>run_id: str</code> Compute the starter questions.
<code>_summarize_value</code>	function	<code>app/api/run_chat.py:59</code>	<code>value: Any, limit: int=800</code> Summarize the value.
<code>_node_configs_from_workflow_yaml</code>	function	<code>app/api/run_chat.py:85</code>	<code>workflow_yaml: str None</code> Internal helper for the node configs from workflow yaml step.
<code>_node_type_catalog</code>	function	<code>app/api/run_chat.py:107</code>	<code></code> Internal helper for the node type catalog step.
<code>_build_run_context</code>	function	<code>app/api/run_chat.py:116</code>	<code>run: dict[str, Any]</code> Build the run context.
<code>ChatMessage</code>	class	<code>app/api/run_chat.py:192</code>	<code>BaseModel</code> Pydantic model defining the ChatMessage shape.
<code>AskAboutRunRequest</code>	class	<code>app/api/run_chat.py:203</code>	<code>BaseModel</code> Pydantic model defining the AskAboutRunRequest shape.
<code>_scope</code>	function	<code>app/api/run_chat.py:214</code>	<code>user: CurrentUser</code> Internal helper for the scope step.
<code>get_run_chat</code>	async function	<code>app/api/run_chat.py:227</code>	<code>run_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> Return the run chat.
<code>ask_about_run</code>	async function	<code>app/api/run_chat.py:248</code>	<code>run_id: str, req: AskAboutRunRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Compute the ask about run.
<code>_scope</code>	function	<code>app/api/runs.py:37</code>	<code>user: CurrentUser</code> Internal helper for the scope step.
<code>my_runs</code>	async function	<code>app/api/runs.py:50</code>	<code>request: Request, limit: Annotated[int, Query(ge=1, le=200)]=50, user: CurrentUser=Depends(require_consultant)</code> Return a bounded, newest-first list of the caller's global workflow runs.
<code>my_run_detail</code>	async function	<code>app/api/runs.py:67</code>	<code>run_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> Compute the my run detail.
<code>pending_gate</code>	async function	<code>app/api/runs.py:101</code>	<code>run_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> The human-review gate a paused run is currently waiting on, if any.

Symbol	Kind	Location	Signature / summary
<code>_resolve_pending_gate</code>	async function	<code>app/api/runs.py:123</code>	<code>db: Any, scope: str, run_id: str, *, parent_run_id: str, visited: set[str] None=None</code> Follow subprocess waits to the deepest actionable human gate.
<code>_hitl_config_fallback</code>	function	<code>app/api/runs.py:204</code>	<code>checkpoint: dict[str, Any], node_id: str None</code> Best-effort HITL gate fields straight from the workflow's own node config, for a checkpoint whose captured interrupt payload is missing or unusable.
<code>_hitl_display_name</code>	function	<code>app/api/runs.py:232</code>	<code>checkpoint: dict[str, Any], node_id: str None</code> No summary declared.
<code>RetryRunRequest</code>	class	<code>app/api/runs.py:243</code>	<code>BaseModel</code> Pydantic model defining the RetryRunRequest shape.
<code>retry_failed_run</code>	async function	<code>app/api/runs.py:253</code>	<code>source_run_id: str, req: RetryRunRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Create a new attempt that reuses every completed node checkpoint.
<code>pause_run</code>	async function	<code>app/api/runs.py:454</code>	<code>run_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> Ask a running workflow to pause at its next node boundary.
<code>resume_paused_run</code>	async function	<code>app/api/runs.py:500</code>	<code>run_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> Resume a run paused by the run-history "pause" action.
<code>restart_run</code>	async function	<code>app/api/runs.py:580</code>	<code>source_run_id: str, req: RetryRunRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Create a brand-new run of the same workflow and original inputs.
<code>delete_run_endpoint</code>	async function	<code>app/api/runs.py:758</code>	<code>run_id: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> Delete the run endpoint.
<code>_scope</code>	function	<code>app/api/workflow_files.py:40</code>	<code>user: CurrentUser</code> Internal helper for the scope step.
<code>capabilities</code>	async function	<code>app/api/workflow_files.py:53</code>	<code>_user: CurrentUser=Depends(require_consultant)</code> Return the authoritative UI picker limits and accepted extensions.
<code>upload_workflow_input_files</code>	async function	<code>app/api/workflow_files.py:72</code>	<code>files: Annotated[list[UploadFile], File(...)], request: Request, user: CurrentUser=Depends(require_consultant)</code> Store files and return stable references suitable for workflow state.
<code>ExtractWorkflowFileRequest</code>	class	<code>app/api/workflow_files.py:136</code>	<code>BaseModel</code> Pydantic model defining the ExtractWorkflowFileRequest shape.
<code>extract_workflow_input_file</code>	async function	<code>app/api/workflow_files.py:148</code>	<code>body: ExtractWorkflowFileRequest, request: Request, user:</code>

Symbol	Kind	Location	Signature / summary
			<code>CurrentUser=Depends(require_consultant)</code> Convert one uploaded document into plain text for the HITL editor.
<code>download_workflow_input_file</code>	async function	<code>app/api/workflow_files.py:171</code>	<code>key: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> Download one uploaded input after enforcing the authenticated scope.
<code>_field_type_label</code>	function	<code>app/api/workflow_generation.py:410</code>	<code>schema: dict</code> A short type descriptor for one JSON-schema property, e.g.
<code>_node_type_catalog</code>	function	<code>app/api/workflow_generation.py:446</code>	<code>type_names: list[str] None=None</code> The catalog text embedded in the generation system prompt.
<code>_real_usage_snippet</code>	function	<code>app/api/workflow_generation.py:495</code>	<code>type_name: str, manifest_entry: dict</code> Step 2's grounding for one node type: its own config, exactly as filled in by a real, already-working workflow on this platform, if one exists.
<code>_real_usage_examples</code>	function	<code>app/api/workflow_generation.py:531</code>	<code>type_names: list[str], manifest: list[dict]</code> Step 2's text block: one real config snippet per candidate type that has one (see <code>`_real_usage_snippet`</code>).
<code>_reference_workflow_examples</code>	function	<code>app/api/workflow_generation.py:555</code>	<code>type_names: list[str], *, limit: int=3</code> Build a bounded, type-ranked context slice from the hidden corpus.
<code>_strip_code_fence</code>	function	<code>app/api/workflow_generation.py:576</code>	<code>text: str</code> Internal helper for the strip code fence step.
<code>_synthesize_sample_inputs</code>	function	<code>app/api/workflow_generation.py:597</code>	<code>specs: dict[str, WorkflowInputSpec], provided: dict[str, Any] None</code> Fill in placeholder values for a workflow's declared inputs so it can actually be run.
<code>GenerationAttempt</code>	class	<code>app/api/workflow_generation.py:624</code>	 Provides the GenerationAttempt behaviour.
<code>GenerationResult</code>	class	<code>app/api/workflow_generation.py:640</code>	 Provides the GenerationResult behaviour.
<code>run_generation_pipeline</code>	async function	<code>app/api/workflow_generation.py:659</code>	<code>*, prompt: str, sample_inputs: dict[str, Any] None, generate_yaml: Callable[[str, str None, str None], Awaitable[str]], static_check: Callable[[str], Awaitable[tuple[WorkflowPreflightReport, dict[str, WorkflowInputSpec]]]], execute: Callable[[str, dict[str, Any]], Awaitable[dict[str, Any]]]</code> The repair-loop control flow, dependency-injected so it's testable without a real LLM/Mongo/executor (see <code>tests/test_workflow_generation.py</code>).
<code>GenerateWorkflowRequest</code>	class	<code>app/api/workflow_generation.py:771</code>	<code>BaseModel</code> Pydantic model defining the <code>GenerateWorkflowRequest</code> shape.
<code>build_llm_yaml_generator</code>	function	<code>app/api/workflow_generation.py:782</code>	<code>llm: Any, services: dict[str, Any], scope: str, *, mode: str='generate'</code>

Symbol	Kind	Location	Signature / summary
			Build a `generate_yaml(base_prompt, prior_yaml, feedback) -> yaml text` closure over a live LLM client.
generate_workflow_endpoint	async function	app/api/workflow_generation.py:940	<code>req: GenerateWorkflowRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Generate the workflow endpoint.
_reserve_run_id	async function	app/api/workflows.py:70	<code>services: dict[str, Any], *, run_id: str, session_id: str</code> Claim a client-supplied run_id for one tenant, idempotently.
_scope	function	app/api/workflows.py:127	<code>user: CurrentUser, body_session: str None</code> The value a run is scoped to for storage + retrieval.
list_workflow_root	async function	app/api/workflows.py:146	<code></code> List the workflows.
list_node_types	function	app/api/workflows.py:152	<code>user: CurrentUser=Depends(require_admin)</code> The Workflow Builder UI calls this to populate the palette and forms.
allowed_models	async function	app/api/workflows.py:158	<code>node_type: str</code> Returns the default allowed model list for a node type.
RunRequest	class	app/api/workflows.py:168	<code>BaseModel</code> Pydantic model defining the RunRequest shape.
ValidateWorkflowRequest	class	app/api/workflows.py:199	<code>BaseModel</code> Pydantic model defining the ValidateWorkflowRequest shape.
_preflight_http_detail	function	app/api/workflows.py:212	<code>report: WorkflowPreflightReport</code> Internal helper for the preflight http detail step.
validate_workflow	async function	app/api/workflows.py:231	<code>req: ValidateWorkflowRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Validate a workflow without running a node or consuming LLM tokens.
AutofixWorkflowRequest	class	app/api/workflows.py:309	<code>BaseModel</code> Pydantic model defining the AutofixWorkflowRequest shape.
AutofixWorkflowResponse	class	app/api/workflows.py:322	<code>BaseModel</code> Pydantic model defining the AutofixWorkflowResponse shape.
AutofixNodeRequest	class	app/api/workflows.py:339	<code>BaseModel</code> No summary declared.
AutofixNodeResponse	class	app/api/workflows.py:345	<code>BaseModel</code> No summary declared.
_node_autofix_scope_valid	function	app/api/workflows.py:358	<code>original_yaml: str, candidate_yaml: str, node_id: str</code> Only the selected node's configuration/model controls may change.
_node_issues	function	app/api/workflows.py:394	<code>report: WorkflowPreflightReport, node_id: str</code> No summary declared.

Symbol	Kind	Location	Signature / summary
<code>_run_preflight</code>	async function	<code>app/api/workflows.py:398</code>	<code>yaml_text: str, req: AutofixWorkflowRequest, services: dict[str, Any]</code> Same branching <code>`validate_workflow`</code> uses, so Auto-fix and Preflight agree on what "valid" means for identical input.
<code>autofix_workflow</code>	async function	<code>app/api/workflows.py:420</code>	<code>req: AutofixWorkflowRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Best-effort repair of a workflow's preflight errors: deterministic patches first (free, predictable), then an LLM repair loop for whatever remains.
<code>autofix_node</code>	async function	<code>app/api/workflows.py:501</code>	<code>req: AutofixNodeRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Repair one selected node while rejecting every unrelated YAML change.
<code>get_preflight_stats</code>	async function	<code>app/api/workflows.py:620</code>	<code>request: Request, since_hours: int None=None, user: CurrentUser=Depends(require_admin)</code> Platform-wide preflight reliability rollup across every / validate, /generate, and /autofix call — success rate, and which error codes recur most before any fix is attempted.
<code>run</code>	async function	<code>app/api/workflows.py:640</code>	<code>req: RunRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Run the result.
<code>ResumeRequest</code>	class	<code>app/api/workflows.py:752</code>	<code>BaseModel</code> Pydantic model defining the ResumeRequest shape.
<code>resume</code>	async function	<code>app/api/workflows.py:764</code>	<code>run_id: str, req: ResumeRequest, request: Request, user: CurrentUser=Depends(require_consultant)</code> Resume doesn't need services for execution — the cached compiled graph in <code>_PAUSED_GRAPHS</code> already has services bound into every node instance.
<code>SubprocessCallbackRequest</code>	class	<code>app/api/workflows.py:842</code>	<code>BaseModel</code> Pydantic model defining the SubprocessCallbackRequest shape.
<code>subprocess_callback</code>	async function	<code>app/api/workflows.py:858</code>	<code>token: str, req: SubprocessCallbackRequest, request: Request</code> Deliver a Subprocess child's terminal result to its waiting parent.
<code>list_workflows</code>	function	<code>app/api/workflows.py:901</code>	 Library view in the UI calls this to render the workflow collection.
<code>list_chat_workflows</code>	function	<code>app/api/workflows.py:951</code>	 Return the small published-workflow catalog used by Business Chat.
<code>get_workflow</code>	function	<code>app/api/workflows.py:984</code>	<code>name: str</code> Builder calls this to load an existing workflow for editing.
<code>get_workflow_detail</code>	function	<code>app/api/workflows.py:997</code>	

Symbol	Kind	Location	Signature / summary
			<code>name: str</code> Workflow Library details panel calls this to refresh 'library' and 'readiness' for one workflow on demand (e.g.
<code>get_workflow_stats</code>	async function	<code>app/api/workflows.py:1032</code>	<code>name: str, request: Request, user: CurrentUser=Depends(require_consultant)</code> Library "Runs and performance" tab.
<code>SaveWorkflowRequest</code>	class	<code>app/api/workflows.py:1049</code>	<code>BaseModel</code> Pydantic model defining the SaveWorkflowRequest shape.
<code>save_workflow</code>	function	<code>app/api/workflows.py:1061</code>	<code>req: SaveWorkflowRequest</code> Builder calls this on Save.
<code>delete_workflow</code>	function	<code>app/api/workflows.py:1082</code>	<code>name: str, user: CurrentUser=Depends(require_consultant)</code> Permanently removes a workflow's YAML together with its autosave draft and full version history — there is no undo.
<code>SaveDraftRequest</code>	class	<code>app/api/workflows.py:1098</code>	<code>BaseModel</code> Pydantic model defining the SaveDraftRequest shape.
<code>_builder_name</code>	function	<code>app/api/workflows.py:1109</code>	<code>name: str</code> Internal helper for the builder name step.
<code>_builder_version_id</code>	function	<code>app/api/workflows.py:1124</code>	<code>version_id: str</code> Internal helper for the builder version id step.
<code>_version_compatibility</code>	function	<code>app/api/workflows.py:1140</code>	<code>yaml_text: str</code> Return cached structural compatibility for immutable version content.
<code>save_workflow_draft</code>	function	<code>app/api/workflows.py:1159</code>	<code>name: str, req: SaveDraftRequest</code> Autosave a recoverable draft.
<code>get_workflow_draft</code>	function	<code>app/api/workflows.py:1170</code>	<code>name: str</code> Builder calls this on load to offer 'Resume draft' for a newer autosave.
<code>delete_workflow_draft</code>	function	<code>app/api/workflows.py:1180</code>	<code>name: str</code> Delete the workflow draft.
<code>list_workflow_versions</code>	function	<code>app/api/workflows.py:1192</code>	<code>name: str</code> List immutable versions without rewriting or preloading their YAML.
<code>get_workflow_version</code>	function	<code>app/api/workflows.py:1204</code>	<code>name: str, version_id: str</code> Return the workflow version.
<code>restore_workflow_version</code>	function	<code>app/api/workflows.py:1224</code>	<code>name: str, version_id: str</code> Restoring an old version is Save-equivalent: it must pass the same zero-token preflight gate before it can become the live workflow again.
<code>_sse_message</code>	function	<code>app/api/workflows.py:1247</code>	<code>*, event: str, data: dict[str, Any], event_id: int None=None</code> Internal helper for the sse message step.
<code>stream_run_events</code>	async function	<code>app/api/workflows.py:1275</code>	<code>run_id: str, request: Request, last_event_id: int None=Header(default=None, alias='Last-Event-ID'), user: CurrentUser=Depends(require_consultant)</code>

Symbol	Kind	Location	Signature / summary
			Authenticated Server-Sent Events stream for one workflow run.
<code>get_file</code>	function	<code>app/api/workflows.py:1345</code>	<code>key: str, request: Request, download: bool=False</code> Stream a file from object storage by key (used by the Output Viewer).
<code>Settings</code>	class	<code>app/config.py:15</code>	<code>BaseSettings</code> Provides the Settings behaviour.
<code>Settings.workflow_file_max_bytes</code>	method	<code>app/config.py:72</code>	<code>self</code> The workflow file max bytes.
<code>Settings.workflow_file_max_total_bytes</code>	method	<code>app/config.py:77</code>	<code>self</code> The workflow file max total bytes.
<code>Settings.max_request_body_bytes</code>	method	<code>app/config.py:82</code>	<code>self</code> The max request body bytes.
<code>Settings.validate_production_security</code>	method	<code>app/config.py:442</code>	<code>self</code> Refuse to boot production with development security settings.
<code>Settings.moonshot_api_key</code>	method	<code>app/config.py:560</code>	<code>self</code> Kimi web search and vision (<code>app/tools/web_io.py</code> , <code>vision_io.py</code>) authenticate as the same Moonshot account as the LLM-gateway route — one credential, <code>LOCAL_KIMI_API_KEY</code> , not a second parallel setting.
<code>Settings.kimi_api_base_url</code>	method	<code>app/config.py:567</code>	<code>self</code> The kimi api base url.
<code>Settings.required_readiness_services</code>	method	<code>app/config.py:572</code>	<code>self</code> The required readiness services.
<code>Settings.resolved_paper_search_mcp_path</code>	method	<code>app/config.py:581</code>	<code>self</code> The resolved paper search mcp path.
<code>Settings.resolved_scientific_skills_path</code>	method	<code>app/config.py:587</code>	<code>self</code> The resolved scientific skills path.
<code>Settings.allowed_scientific_skills</code>	method	<code>app/config.py:592</code>	<code>self</code> The allowed scientific skills.
<code>Settings.allowed_cors_origins</code>	method	<code>app/config.py:597</code>	<code>self</code> The allowed cors origins.
<code>Settings.allowed_hosts</code>	method	<code>app/config.py:602</code>	<code>self</code> The allowed hosts.
<code>_csv_values</code>	function	<code>app/config.py:607</code>	<code>value: str</code> Internal helper for the csv values step.
<code>_redis_url_has_password</code>	function	<code>app/config.py:619</code>	<code>url: str</code> Return whether a Redis URL contains a non-empty password component.
<code>_valid_http_url</code>	function	<code>app/config.py:630</code>	<code>value: str</code> Internal helper for the valid http url step.
<code>_is_placeholder</code>	function	<code>app/config.py:653</code>	<code>value: str</code> Return whether placeholder.
<code>MigrationError</code>	class	<code>app/db/migrations.py:29</code>	<code>RuntimeError</code> Raised when the database cannot be brought to the required schema.

Symbol	Kind	Location	Signature / summary
Migration	class	app/db/migrations.py:34	 Provides the Migration behaviour.
_set_missing	async function	app/db/migrations.py:47	<code>collection: Any, field: str, value: Any</code> Set the missing.
_backfill_run_documents_v1	async function	app/db/migrations.py:61	<code>db: Any</code> Give legacy run and checkpoint documents the explicit v1 shape.
_backfill_knowledge_resources_v2	async function	app/db/migrations.py:88	<code>db: Any</code> Expand legacy "collections" (CollectionConfig) records with the lifecycle fields Knowledge Studio needs.
_remove_business_and_pipeline_persistence_v3	async function	app/db/migrations.py:131	<code>db: Any</code> Remove persistence owned exclusively by the retired product concepts.
_acquire_lock	async function	app/db/migrations.py:166	<code>collection: Any, *, owner: str, wait_timeout_seconds: float</code> Acquire the lock.
_renew_lock	async function	app/db/migrations.py:212	<code>collection: Any, *, owner: str</code> Renew the lock.
run_migrations	async function	app/db/migrations.py:233	<code>db: Any, *, owner: str None=None, wait_timeout_seconds: float=60.0</code> Run every pending migration and return the applied migration ids.
MongoClient	class	app/db/mongo.py:53	 Async Mongo wrapper.
MongoClient.ensure_client	method	app/db/mongo.py:69	<code>self</code> Ensure the client.
MongoClient.manifests	method	app/db/mongo.py:88	<code>self</code> The manifests collection, opened lazily.
MongoClient.close	method	app/db/mongo.py:92	<code>self</code> Close the result.
MongoClient.scorecards	method	app/db/mongo.py:100	<code>self</code> The eval scorecards collection, opened lazily.
MongoClient.collections	method	app/db/mongo.py:105	<code>self</code> The collections.
MongoClient.ensure_indexes	method	app/db/mongo.py:111	<code>self</code> Create the indexes the pipeline relies on.
MongoClient.upsert_manifest	method	app/db/mongo.py:127	<code>self, manifest: dict[str, Any]</code> Insert or replace a manifest by _id.
MongoClient.get_manifest	method	app/db/mongo.py:145	<code>self, manifest_id: str</code> Fetch a single manifest by _id.
MongoClient.list_manifests	method	app/db/mongo.py:149	<code>self, status: str None=None, limit: int=100</code> List manifests, optionally filtered by status, newest first.
MongoClient.mark_status	method	app/db/mongo.py:165	<code>self, manifest_id: str, status: str</code> Update only the status field of an existing manifest.
MongoClient.delete_manifest	method	app/db/mongo.py:173	<code>self, manifest_id: str</code> Delete a manifest.

Symbol	Kind	Location	Signature / summary
MongoClient.save_scorecard	method	app/db/mongo.py:183	<code>self, scorecard: dict[str, Any]</code> Insert one eval scorecard.
MongoClient.list_scorecards	method	app/db/mongo.py:196	<code>self, workflow_name: str None=None, limit: int=20</code> List scorecards newest-first, optionally for one workflow.
build_manifest	function	app/db/mongo.py:217	<code>minio_key: str, original_filename: str, source_format: str, byte_size: int, chunk_ids: list[str], metadata: dict[str, Any] None=None, status: str='completed'</code> Construct a manifest dict in the canonical shape.
get_mongo_client	function	app/db/mongo.py:252	<code></code> Lazily-constructed module-level client.
GoldenExample	class	app/evaluation/golden_set.py:18	<code>BaseModel</code> Pydantic model defining the GoldenExample shape.
load_golden_set	function	app/evaluation/golden_set.py:33	<code>path: str Path</code> Load a JSONL golden set.
_JudgeVerdict	class	app/evaluation/judge.py:73	<code>BaseModel</code> Provider-facing schema for one criterion.
LLMJudge	class	app/evaluation/judge.py:84	<code></code> Scores an answer on the four RAG criteria via the LLM gateway.
LLMJudge._score_one	method	app/evaluation/judge.py:101	<code>self, *, criterion: str, question: str, answer: str, context: str, reference: str</code> Score the one.
LLMJudge.score_one	method	app/evaluation/judge.py:151	<code>self, criterion: str, *, question: str, answer: str, context: str, reference: str=''</code> Public single-criterion API used by focused evaluation runs.
LLMJudge.score_all	method	app/evaluation/judge.py:174	<code>self, *, question: str, answer: str, context: str, reference: str=''</code> Score one answer on all four criteria, one independent call each.
_now_iso	function	app/evaluation/models.py:34	<code></code> Internal helper for the now iso step.
CriterionScore	class	app/evaluation/models.py:43	<code>BaseModel</code> One judged criterion for one answer.
ExampleResult	class	app/evaluation/models.py:50	<code>BaseModel</code> All criterion scores for a single golden example.
ExampleResult.mean	method	app/evaluation/models.py:58	<code>self</code> The mean.
Scorecard	class	app/evaluation/models.py:65	<code>BaseModel</code> Aggregate result of evaluating one workflow over one golden set.
Scorecard.per_criterion_mean	method	app/evaluation/models.py:84	<code>self</code> Backward-compatible read alias for earlier API/UI consumers.

Symbol	Kind	Location	Signature / summary
<code>_eval_one</code>	async function	<code>app/evaluation/runner.py:30</code>	<code>ex: GoldenExample, judge: LLMJudge, produce_answer: ProduceAnswer</code> Internal helper for the eval one step.
<code>run_eval</code>	async function	<code>app/evaluation/runner.py:60</code>	<code>*, workflow_name: str, examples: list[GoldenExample], judge: LLMJudge, produce_answer: ProduceAnswer</code> Run the whole golden set through produce_answer + judge, aggregate.
<code>WorkflowGoldenCase</code>	class	<code>app/evaluation/workflow_golden.py:31</code>	<code>BaseModel</code> Pydantic model defining the WorkflowGoldenCase shape.
<code>load_workflow_golden_set</code>	function	<code>app/evaluation/workflow_golden.py:50</code>	<code>path: str Path</code> Load the workflow golden set.
<code>FieldCheck</code>	class	<code>app/evaluation/workflow_golden.py:66</code>	<code>BaseModel</code> Pydantic model defining the FieldCheck shape.
<code>WorkflowCaseResult</code>	class	<code>app/evaluation/workflow_golden.py:81</code>	<code>BaseModel</code> Pydantic model defining the WorkflowCaseResult shape.
<code>ModelComparisonResult</code>	class	<code>app/evaluation/workflow_golden.py:104</code>	<code>BaseModel</code> Pydantic model defining the ModelComparisonResult shape.
<code>ModelOverrideGateway</code>	class	<code>app/evaluation/workflow_golden.py:125</code>	 Forces every LLM call a run makes onto one candidate model, regardless of what the node/YAML asked for (including AUTO_MODEL routing).
<code>ModelOverrideGateway.with_context</code>	method	<code>app/evaluation/workflow_golden.py:144</code>	<code>self, **kwargs: Any</code> Compute the with context.
<code>ModelOverrideGateway.complete</code>	method	<code>app/evaluation/workflow_golden.py:155</code>	<code>self, *, model: str None=None, **kwargs: Any</code> Complete the result.
<code>ModelOverrideGateway.complete_structured</code>	method	<code>app/evaluation/workflow_golden.py:165</code>	<code>self, *, model: str None=None, **kwargs: Any</code> Complete the structured.
<code>ModelOverrideGateway.chat_with_tools</code>	method	<code>app/evaluation/workflow_golden.py:175</code>	<code>self, *, model: str None=None, **kwargs: Any</code> Compute the chat with tools.
<code>_actual_value</code>	function	<code>app/evaluation/workflow_golden.py:186</code>	<code>result: dict[str, Any], field: str</code> Internal helper for the actual value step.
<code>run_workflow_golden_case</code>	async function	<code>app/evaluation/workflow_golden.py:205</code>	<code>case: WorkflowGoldenCase, *, model: str, services: dict[str, Any], run_id: str</code> Run the workflow golden case.
<code>run_golden_set_with_model</code>	async function	<code>app/evaluation/workflow_golden.py:265</code>	<code>cases: list[WorkflowGoldenCase], *, model: str, services: dict[str, Any], run_id_prefix: str</code> Run the golden set with model.
<code>recommend_model</code>	function	<code>app/evaluation/workflow_golden.py:303</code>	<code>comparisons: list[ModelComparisonResult]</code> Highest pass rate wins; ties broken by lower cost, then lower latency.
<code>ClaimVerificationResult</code>	class		

Symbol	Kind	Location	Signature / summary
		app/evidence/ claim_verification.py:21	<code>BaseModel</code> Pydantic model defining the ClaimVerificationResult shape.
<code>_format_results</code>	function	app/evidence/ claim_verification.py:42	<code>results: list[dict[str, Any]]</code> Format the results.
<code>verify_claim</code>	async function	app/evidence/ claim_verification.py:59	<code>content: str, *, source_name: str, llm: Any, web_search: Any</code> Verify the claim.
<code>StructuredDatasetQuery</code>	class	app/evidence/ database_models.py:20	<code>BaseModel</code> One bounded, reproducible public-database retrieval contract.
<code>StructuredDatasetQuery.safe_identifier</code>	method	app/evidence/ database_models.py:40	<code>cls, value: str</code> Internal helper for the safe identifier step.
<code>StructuredDatasetQuery.safe_period</code>	method	app/evidence/ database_models.py:59	<code>cls, value: str None</code> Internal helper for the safe period step.
<code>StructuredDatasetQuery.safe_filters</code>	method	app/evidence/ database_models.py:79	<code>cls, value: dict[str, list[str]]</code> Internal helper for the safe filters step.
<code>CountReconciliation</code>	class	app/evidence/ database_models.py:111	<code>BaseModel</code> Pydantic model defining the CountReconciliation shape.
<code>StructuredDataEvidenceRecord</code>	class	app/evidence/ database_models.py:130	<code>BaseModel</code> One value decoded from a versioned official-database response.
<code>DatasetRetrievalAudit</code>	class	app/evidence/ database_models.py:159	<code>BaseModel</code> Pydantic model defining the DatasetRetrievalAudit shape.
<code>InternalEvidenceRecord</code>	class	app/evidence/ database_models.py:186	<code>BaseModel</code> Exact-passage-matched fact from an internal or partner source.
<code>InternalEvidenceReviewItem</code>	class	app/evidence/ database_models.py:210	<code>BaseModel</code> Pydantic model defining the InternalEvidenceReviewItem shape.
<code>ResolvedWork</code>	class	app/evidence/ identifiers.py:52	<code></code> Canonical identity for one scholarly work, as far as it is known.
<code>ResolvedWork.as_dict</code>	method	app/evidence/ identifiers.py:76	<code>self</code> Only the populated identifier fields, for storage on a candidate.
<code>ResolvedWork.has_strong_identifier</code>	method	app/evidence/ identifiers.py:93	<code>self</code> The has strong identifier.
<code>ResolvedWork.is_peer_reviewed</code>	method	app/evidence/ identifiers.py:104	<code>self</code> True ONLY when resolved metadata positively supports it.
<code>normalise_doi_value</code>	function	app/evidence/ identifiers.py:139	<code>value: Any</code> Lowercased bare DOI from anything that might contain one.
<code>_arxiv_from</code>	function	app/evidence/ identifiers.py:155	<code>text: str</code> Internal helper for the arxiv from step.
<code>extract_identifiers</code>	function		

Symbol	Kind	Location	Signature / summary
		app/evidence/identifiers.py:173	<code>url: str None=None, *, metadata: dict[str, Any] None=None, text: str None=None</code> Best-effort offline identifier extraction.
normalise_title	function	app/evidence/identifiers.py:281	<code>title: str None</code> Punctuation/whitespace-insensitive title key.
work_identity	function	app/evidence/identifiers.py:290	<code>work: ResolvedWork None, *, title: str None=None, fallback: str None=None</code> The single cross-lane dedup key for one work.
classify_authority	function	app/evidence/identifiers.py:331	<code>url: str None, work: ResolvedWork</code> “(source_label, authority)” from what is actually known.
EvidencePolicy	class	app/evidence/models.py:46	<code>BaseModel</code> Hard evidence rules applied before drafting may use a citation.
SearchAuditRecord	class	app/evidence/models.py:78	<code>BaseModel</code> Pydantic model defining the SearchAuditRecord shape.
CandidateSource	class	app/evidence/models.py:109	<code>BaseModel</code> A discovered record.
FullTextDocument	class	app/evidence/models.py:155	<code>BaseModel</code> Immutable, fetched source version available for passage retrieval.
RetrievedPassage	class	app/evidence/models.py:185	<code>BaseModel</code> Pydantic model defining the RetrievedPassage shape.
ClaimEvidenceLink	class	app/evidence/models.py:208	<code>BaseModel</code> Pydantic model defining the ClaimEvidenceLink shape.
CitationRegistryEntry	class	app/evidence/models.py:240	<code>BaseModel</code> Pydantic model defining the CitationRegistryEntry shape.
VerifiedClaim	class	app/evidence/models.py:273	<code>BaseModel</code> Pydantic model defining the VerifiedClaim shape.
RejectedCandidate	class	app/evidence/models.py:304	<code>BaseModel</code> Pydantic model defining the RejectedCandidate shape.
EvidenceGap	class	app/evidence/models.py:321	<code>BaseModel</code> Pydantic model defining the EvidenceGap shape.
EvidenceQAReport	class	app/evidence/models.py:347	<code>BaseModel</code> Pydantic model defining the EvidenceQAReport shape.
ProposalEvidencePackage	class	app/evidence/models.py:383	<code>BaseModel</code> Pydantic model defining the ProposalEvidencePackage shape.
coerce_typed_list_field	function	app/evidence/models.py:415	<code>value: Any, model: type[_ModelT], field_name: str</code> Accept a node's records as a list, JSON text, or empty text.
utc_now	function	app/evidence/retrieval.py:43	<code></code> Compute the utc now.
stable_id	function	app/evidence/retrieval.py:52	<code>prefix: str, *parts: Any, length: int=16</code> Compute the stable id.

Symbol	Kind	Location	Signature / summary
<code>normalise_doi</code>	function	<code>app/evidence/retrieval.py:68</code>	<code>value: Any</code> Compute the normalise doi.
<code>parse_authors</code>	function	<code>app/evidence/retrieval.py:84</code>	<code>value: Any</code> Parse the authors.
<code>parse_year</code>	function	<code>app/evidence/retrieval.py:111</code>	<code>value: Any</code> Parse the year.
<code>first_value</code>	function	<code>app/evidence/retrieval.py:124</code>	<code>data: dict[str, Any], *keys: str</code> Compute the first value.
<code>parse_mcp_payload</code>	function	<code>app/evidence/retrieval.py:141</code>	<code>raw: Any</code> Parse the mcp payload.
<code>papers_from_payload</code>	function	<code>app/evidence/retrieval.py:163</code>	<code>raw: Any</code> Compute the papers from payload.
<code>source_errors_from_payload</code>	function	<code>app/evidence/retrieval.py:185</code>	<code>raw: Any</code> Per-source failures from a multi-source search payload (e.g.
<code>candidate_from_paper</code>	function	<code>app/evidence/retrieval.py:202</code>	<code>paper: dict[str, Any], *, claim_id: str, query: str, purpose: str, source_hint: str None=None, discovery_lane: str None=None</code> Compute the candidate from paper.
<code>_normalise_title_for_identity</code>	function	<code>app/evidence/retrieval.py:330</code>	<code>title: str</code> Collapse cosmetic title differences that otherwise defeat dedup.
<code>candidate_work_identity</code>	function	<code>app/evidence/retrieval.py:343</code>	<code>candidate: CandidateSource</code> Cross-lane work identity for one candidate.
<code>deduplicate_candidates</code>	function	<code>app/evidence/retrieval.py:373</code>	<code>candidates: list[CandidateSource]</code> Deduplicate the same work for one claim, across queries AND lanes.
<code>formatted_citation</code>	function	<code>app/evidence/retrieval.py:476</code>	<code>candidate: CandidateSource</code> Compute the formatted citation.
<code>download_path_from_payload</code>	function	<code>app/evidence/retrieval.py:499</code>	<code>raw: Any</code> Download the path from payload.
<code>validate_downloaded_pdf</code>	function	<code>app/evidence/retrieval.py:519</code>	<code>raw_path: str None, *, allowed_root: Path, max_bytes: int</code> Validate the downloaded pdf.
<code>_tokens</code>	function	<code>app/evidence/retrieval.py:552</code>	<code>text: str</code> Internal helper for the tokens step.
<code>_page_windows</code>	function	<code>app/evidence/retrieval.py:568</code>	<code>text: str, size: int=1800, overlap: int=250</code> Internal helper for the page windows step.
<code>retrieve_passages</code>	function	<code>app/evidence/retrieval.py:599</code>	<code>claim: str, pages: list[dict[str, Any]], *, document_id: str, limit: int=8</code> Page-aware lexical retrieval before semantic stance verification.
<code>Chunk</code>	class	<code>app/ingestion/chunker.py:34</code>	<code></code> One unit of text ready for embedding.
<code>Chunk.embedding_content</code>	method	<code>app/ingestion/chunker.py:59</code>	

Symbol	Kind	Location	Signature / summary
			<code>self</code> The text that should actually be embedded for this chunk.
ChunkConfig	class	<code>app/ingestion/chunker.py:73</code>	Tunable chunking parameters.
<code>_get_tokenizer</code>	function	<code>app/ingestion/chunker.py:96</code>	<code>name: str</code> Return the tokenizer.
<code>_split_sections</code>	function	<code>app/ingestion/chunker.py:121</code>	<code>text: str</code> Split on top-level section headers.
<code>_split_paragraphs</code>	function	<code>app/ingestion/chunker.py:137</code>	<code>text: str</code> Split on blank lines.
<code>_split_sentences</code>	function	<code>app/ingestion/chunker.py:143</code>	<code>text: str</code> Split on sentence boundaries.
<code>_hard_split</code>	function	<code>app/ingestion/chunker.py:152</code>	<code>tokens: list[int], target: int, max_size: int, encoding: tiktoken.Encoding</code> Last-resort: split a token list at fixed boundaries.
<code>_pack_pieces</code>	function	<code>app/ingestion/chunker.py:169</code>	<code>pieces: list[str], encoding: tiktoken.Encoding, config: ChunkConfig</code> Greedy packer.
<code>_split_oversized</code>	function	<code>app/ingestion/chunker.py:220</code>	<code>text: str, encoding: tiktoken.Encoding, config: ChunkConfig</code> Recursive descent: paragraphs → sentences → hard cut.
<code>_apply_overlap</code>	function	<code>app/ingestion/chunker.py:237</code>	<code>chunks: list[str], encoding: tiktoken.Encoding, overlap_tokens: int</code> Prepend the last `overlap_tokens` of chunk N-1 onto the start of chunk N.
<code>chunk_text</code>	function	<code>app/ingestion/chunker.py:261</code>	<code>text: str, config: ChunkConfig None=None</code> Chunk a raw text string.
<code>chunk_document</code>	function	<code>app/ingestion/chunker.py:282</code>	<code>doc: ExtractedDocument, config: ChunkConfig None=None, chunk_id_prefix: str None=None</code> Chunk a full ExtractedDocument into Chunk objects with metadata.
<code>main</code>	function	<code>app/ingestion/chunker.py:368</code>	Quick CLI: <code>`python -m app.ingestion.chunker <path>`</code> .
<code>_parse_metadata</code>	function	<code>app/ingestion/cli.py:18</code>	<code>raw: list[str]</code> Parse repeated <code>--meta key=value</code> flags into a dict.
<code>_amain</code>	async function	<code>app/ingestion/cli.py:29</code>	<code>args: argparse.Namespace</code> Internal helper for the <code>amain</code> step.
<code>main</code>	function	<code>app/ingestion/cli.py:64</code>	Compute the main.
CollectionConfig	class	<code>app/ingestion/collections.py:21</code>	<code>BaseModel</code> Declares one use case's document taxonomy.
<code>CollectionConfig.validate_doc_type</code>	method	<code>app/ingestion/collections.py:35</code>	<code>self, dt: str</code> Raise if <code>'dt'</code> is not in this collection's declared vocabulary.

Symbol	Kind	Location	Signature / summary
CollectionConfig.validate_doc_types	method	app/ingestion/collections.py:47	<code>self, dts: list[str]</code> Validate a list (retrieval filters carry a list of doc_types).
CollectionRegistry	class	app/ingestion/collections.py:53	<code></code> Loads/saves CollectionConfig via the MongoClient's 'collections' accessor — never touches the raw db, matching the project's convention that callers get named collections.
CollectionRegistry.get	method	app/ingestion/collections.py:67	<code>self, collection_id: str</code> Return the result.
CollectionRegistry.upsert	method	app/ingestion/collections.py:90	<code>self, cfg: CollectionConfig</code> Upsert the result.
CollectionRegistry.list_all	method	app/ingestion/collections.py:104	<code>self</code> List the all.
DocumentParser	class	app/ingestion/contracts.py:17	<code>Protocol</code> Provides the DocumentParser behaviour.
DocumentParser.parse	method	app/ingestion/contracts.py:19	<code>self, path: Path, *, config: dict[str, Any]</code> No summary declared.
ChunkingStrategy	class	app/ingestion/contracts.py:22	<code>Protocol</code> Provides the ChunkingStrategy behaviour.
ChunkingStrategy.chunk	method	app/ingestion/contracts.py:24	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str</code> No summary declared.
ChunkEnricher	class	app/ingestion/contracts.py:29	<code>Protocol</code> Provides the ChunkEnricher behaviour.
ChunkEnricher.enrich	method	app/ingestion/contracts.py:31	<code>self, chunks: list[Chunk], *, document: ExtractedDocument, config: dict[str, Any]</code> No summary declared.
EmbeddingProvider	class	app/ingestion/contracts.py:36	<code>Protocol</code> Provides the EmbeddingProvider behaviour.
EmbeddingProvider.embed	method	app/ingestion/contracts.py:38	<code>self, texts: list[str]</code> No summary declared.
SearchIndex	class	app/ingestion/contracts.py:41	<code>Protocol</code> Provides the SearchIndex behaviour.
SearchIndex.index	method	app/ingestion/contracts.py:43	<code>self, objects: list[dict[str, Any]], vectors: list[list[float]]</code> Write "objects" (with their parallel "vectors") and return the number of objects actually inserted, so the caller can detect a partial write.
UnknownStrategyError	class	app/ingestion/contracts.py:50	<code>LookupError</code> Exception raised for the UnknownStrategyError case.
StageRegistry	class	app/ingestion/contracts.py:55	<code></code> Name -> strategy lookup for every ingestion stage kind.
StageRegistry.parser	method	app/ingestion/contracts.py:69	<code>self, name: str, instance: DocumentParser</code> Compute the parser.

Symbol	Kind	Location	Signature / summary
StageRegistry.chunker	method	app/ingestion/ contracts.py:78	<code>self, name: str, instance: ChunkingStrategy</code> Compute the chunker.
StageRegistry.enricher	method	app/ingestion/ contracts.py:87	<code>self, name: str, instance: ChunkEnricher</code> Compute the enricher.
StageRegistry.get_parser	method	app/ingestion/ contracts.py:96	<code>self, name: str</code> Return the parser.
StageRegistry.get_chunker	method	app/ingestion/ contracts.py:110	<code>self, name: str</code> Return the chunker.
IngestionCoordinator	class	app/ingestion/ coordinator.py:26	 Provides the IngestionCoordinator behaviour.
IngestionCoordinator.submit	method	app/ingestion/ coordinator.py:53	<code>self, job: IngestionJob</code> Schedule ``job`` to run on the event loop.
IngestionCoordinator.resubmit	method	app/ingestion/ coordinator.py:60	<code>self</code> Resubmit every job left non-terminal by an interrupted process.
IngestionCoordinator.materialize_sources	method	app/ingestion/ coordinator.py:75	<code>self, job: IngestionJob</code> Download this job's already-uploaded sources to a local temp dir.
IngestionCoordinator.run_safely	method	app/ingestion/ coordinator.py:93	<code>self, job: IngestionJob</code> Run the safely.
EmbedderConfig	class	app/ingestion/ embedder.py:31	 Tunable embedder parameters.
Embedder	class	app/ingestion/ embedder.py:45	 Async OpenAI embedding client.
Embedder.embed	method	app/ingestion/ embedder.py:86	<code>self, texts: list[str]</code> Embed a list of texts.
Embedder.embed_batch_with_retry	method	app/ingestion/ embedder.py:116	<code>self, batch: list[str]</code> Single batch with tenacity-managed retry.
get_embedder	function	app/ingestion/ embedder.py:146	 Lazily-constructed module-level embedder.
EmbeddingModelChoice	class	app/ingestion/ embedding_catalog.py:20	 Provides the EmbeddingModelChoice behaviour.
embedding_model_catalog	function	app/ingestion/ embedding_catalog.py:115	 Compute the embedding model catalog.
_looks_multilingual	function	app/ingestion/ embedding_catalog.py:126	<code>sample: str</code> Internal helper for the looks multilingual step.
select_embedding_model	function	app/ingestion/ embedding_catalog.py:141	<code>*, doc_types: list[str] None=None, document_count: int=0, total_bytes: int=0, sample_text: str=''</code> Deterministically pick an embedding model for a corpus.
ExtractedUnit	class	app/ingestion/ extractor.py:35	 One logical unit of an extracted document.
ExtractedDocument	class	app/ingestion/ extractor.py:50	

Symbol	Kind	Location	Signature / summary
			A whole document, broken into ordered units, with metadata.
ExtractedDocument.full_text	method	<code>app/ingestion/extractor.py:60</code>	<code>self</code> All units joined with double newlines.
Extractor	class	<code>app/ingestion/extractor.py:69</code>	<code>Protocol</code> Anything callable with <code>.extract(path)</code> returning <code>ExtractedDocument</code> .
Extractor.extract	method	<code>app/ingestion/extractor.py:72</code>	<code>self, path: Path</code> No summary declared.
PdfExtractor	class	<code>app/ingestion/extractor.py:78</code>	Extract text from a PDF, one <code>ExtractedUnit</code> per page.
PdfExtractor.extract	method	<code>app/ingestion/extractor.py:81</code>	<code>self, path: Path</code> Extract the result.
XlsxExtractor	class	<code>app/ingestion/extractor.py:145</code>	Extract data from an Excel workbook, one <code>ExtractedUnit</code> per worksheet.
XlsxExtractor.extract	method	<code>app/ingestion/extractor.py:148</code>	<code>self, path: Path</code> Extract the result.
DocxExtractor	class	<code>app/ingestion/extractor.py:182</code>	Extract text from a Word document.
DocxExtractor.extract	method	<code>app/ingestion/extractor.py:190</code>	<code>self, path: Path</code> Extract the result.
PptxExtractor	class	<code>app/ingestion/extractor.py:249</code>	Extract text from a PowerPoint deck, one <code>ExtractedUnit</code> per slide.
PptxExtractor.extract	method	<code>app/ingestion/extractor.py:256</code>	<code>self, path: Path</code> Extract the result.
PlainTextExtractor	class	<code>app/ingestion/extractor.py:307</code>	Extract any plain-text file.
PlainTextExtractor.extract	method	<code>app/ingestion/extractor.py:314</code>	<code>self, path: Path</code> Extract the result.
MarkdownExtractor	class	<code>app/ingestion/extractor.py:347</code>	Extract text from a Markdown file.
MarkdownExtractor.extract	method	<code>app/ingestion/extractor.py:357</code>	<code>self, path: Path</code> Extract the result.
DelimitedTextExtractor	class	<code>app/ingestion/extractor.py:393</code>	Extract rows from a CSV/TSV file, one <code>ExtractedUnit</code> per row block.
DelimitedTextExtractor.extract	method	<code>app/ingestion/extractor.py:414</code>	<code>self, path: Path</code> Extract the result.
_is_number	function	<code>app/ingestion/extractor.py:489</code>	<code>value: str</code> Return whether number.
UnsupportedFormatError	class	<code>app/ingestion/extractor.py:569</code>	<code>ValueError</code> Raised when no extractor is registered for a given file extension.
get_extractor	function		

Symbol	Kind	Location	Signature / summary
		<code>app/ingestion/extractor.py:573</code>	<code>path: Path</code> Return the right extractor for the file's extension.
<code>supported_extensions</code>	function	<code>app/ingestion/extractor.py:583</code>	 Sorted list of supported file extensions.
<code>main</code>	function	<code>app/ingestion/extractor.py:591</code>	 Quick CLI: <code>`python -m app.ingestion.extractor path/to/file.<ext>`</code> .
<code>WeaviateSearchIndex</code>	class	<code>app/ingestion/indexes.py:17</code>	 Satisfies <code>:class:`app.ingestion.contracts.SearchIndex`</code> for one physical Weaviate collection.
<code>WeaviateSearchIndex.ensure_schema</code>	method	<code>app/ingestion/indexes.py:36</code>	<code>self</code> Ensure the schema.
<code>WeaviateSearchIndex.index</code>	method	<code>app/ingestion/indexes.py:43</code>	<code>self, objects: list[dict[str, Any]], vectors: list[list[float]]</code> Compute the index.
<code>IngestionCancelled</code>	class	<code>app/ingestion/jobs.py:31</code>	<code>Exception</code> Exception raised for the IngestionCancelled case.
<code>IngestionJobRunner</code>	class	<code>app/ingestion/jobs.py:36</code>	 Runs without a second queue platform; progress lives in Mongo.
<code>IngestionJobRunner.embedding_for</code>	method	<code>app/ingestion/jobs.py:69</code>	<code>self, embedding_profile: Any</code> Embedder matching an Embedding Profile, cached per model+dimensions.
<code>IngestionJobRunner.raise_if_cancelled</code>	method	<code>app/ingestion/jobs.py:101</code>	<code>self, job: IngestionJob</code> Raise the if cancelled.
<code>IngestionJobRunner.run</code>	method	<code>app/ingestion/jobs.py:115</code>	<code>self, job: IngestionJob, paths: list[Path], *, metadata: dict[str, Any] None=None</code> Run the result.
<code>IngestionResult</code>	class	<code>app/ingestion/pipeline.py:46</code>	 What <code>`ingest_file`</code> returns.
<code>ingest_file</code>	async function	<code>app/ingestion/pipeline.py:59</code>	<code>path: Path, metadata: dict[str, Any] None=None, chunk_config: ChunkConfig None=None, object_store: ObjectStore None=None, embedder: Embedder None=None, weaviate: WeaviateClient None=None, mongo: MongoClient None=None, collection_config: CollectionConfig None=None</code> Ingest one document end-to-end.
<code>StandardDocumentParser</code>	class	<code>app/ingestion/strategies.py:16</code>	 Provides the StandardDocumentParser behaviour.
<code>StandardDocumentParser.parse</code>	method	<code>app/ingestion/strategies.py:18</code>	<code>self, path: Path, *, config: dict[str, Any]</code> Parse the result.
<code>LayoutAwareDocumentParser</code>	class	<code>app/ingestion/strategies.py:31</code>	<code>StandardDocumentParser</code> Use reliable native page/slide/sheet/heading structure when available.
<code>StructureAwareDocumentParser</code>	class	<code>app/ingestion/strategies.py:39</code>	

Symbol	Kind	Location	Signature / summary
			LayoutAwareDocumentParser Provides the StructureAwareDocumentParser behaviour.
StructureAwareDocumentParser.parse	method	app/ingestion/strategies.py:41	<code>self, path: Path, *, config: dict[str, Any]</code> Parse the result.
OcrFallbackDocumentParser	class	app/ingestion/strategies.py:58	StandardDocumentParser Fall back to reading the page as an image when there is no text layer.
OcrFallbackDocumentParser.parse	method	app/ingestion/strategies.py:79	<code>self, path: Path, *, config: dict[str, Any]</code> Parse the result.
VisionAugmentedDocumentParser	class	app/ingestion/strategies.py:140	StandardDocumentParser Add vision-model transcriptions of figures, charts and image tables.
VisionAugmentedDocumentParser.parse	method	app/ingestion/strategies.py:159	<code>self, path: Path, *, config: dict[str, Any]</code> Parse the result.
_chunk_config	function	app/ingestion/strategies.py:213	<code>config: dict[str, Any]</code> Chunk the config.
RecursiveChunkingStrategy	class	app/ingestion/strategies.py:226	 Provides the RecursiveChunkingStrategy behaviour.
RecursiveChunkingStrategy.chunk	method	app/ingestion/strategies.py:228	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str</code> Chunk the result.
FixedTokenChunkingStrategy	class	app/ingestion/strategies.py:248	 Provides the FixedTokenChunkingStrategy behaviour.
FixedTokenChunkingStrategy.chunk	method	app/ingestion/strategies.py:250	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str</code> Chunk the result.
StructureAwareChunkingStrategy	class	app/ingestion/strategies.py:295	RecursiveChunkingStrategy Provides the StructureAwareChunkingStrategy behaviour.
StructureAwareChunkingStrategy.chunk	method	app/ingestion/strategies.py:297	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str</code> Chunk the result.
ParentChildChunkingStrategy	class	app/ingestion/strategies.py:316	RecursiveChunkingStrategy Provides the ParentChildChunkingStrategy behaviour.
ParentChildChunkingStrategy.chunk	method	app/ingestion/strategies.py:318	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str</code> Chunk the result.
ContextualChunkingStrategy	class	app/ingestion/strategies.py:365	StructureAwareChunkingStrategy Provides the ContextualChunkingStrategy behaviour.
ContextualChunkingStrategy.chunk	method	app/ingestion/strategies.py:367	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix:</code>

Symbol	Kind	Location	Signature / summary
			<code>str</code> Chunk the result.
SentenceWindowChunkingStrategy	class	<code>app/ingestion/strategies.py:389</code>	<code></code> Provides the SentenceWindowChunkingStrategy behaviour.
SentenceWindowChunkingStrategy.chunk	method	<code>app/ingestion/strategies.py:391</code>	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str</code> Chunk the result.
_terms	function	<code>app/ingestion/strategies.py:430</code>	<code>text: str</code> Internal helper for the terms step.
SemanticChunkingStrategy	class	<code>app/ingestion/strategies.py:442</code>	<code></code> Lightweight semantic-boundary strategy using paragraph topic overlap.
SemanticChunkingStrategy.chunk	method	<code>app/ingestion/strategies.py:450</code>	<code>self, document: ExtractedDocument, *, config: dict[str, Any], chunk_id_prefix: str</code> Chunk the result.
MetadataContextEnricher	class	<code>app/ingestion/strategies.py:490</code>	<code></code> Provides the MetadataContextEnricher behaviour.
MetadataContextEnricher.enrich	method	<code>app/ingestion/strategies.py:492</code>	<code>self, chunks: list[Chunk], *, document: ExtractedDocument, config: dict[str, Any]</code> Compute the enrich.
build_stage_registry	function	<code>app/ingestion/strategies.py:512</code>	<code></code> Build the stage registry.
PageDescription	class	<code>app/ingestion/vision.py:39</code>	<code></code> Provides the PageDescription behaviour.
PdfPageVisionDescriber	class	<code>app/ingestion/vision.py:56</code>	<code></code> Render PDF pages and describe their visual content with a vision model.
PdfPageVisionDescriber.provider	method	<code>app/ingestion/vision.py:76</code>	<code>self</code> The provider.
PdfPageVisionDescriber.model	method	<code>app/ingestion/vision.py:81</code>	<code>self</code> The model.
PdfPageVisionDescriber.available	method	<code>app/ingestion/vision.py:90</code>	<code>self</code> Compute the available.
PdfPageVisionDescriber.unavailable_reason	method	<code>app/ingestion/vision.py:104</code>	<code>self</code> Compute the unavailable reason.
PdfPageVisionDescriber.describe_pages	method	<code>app/ingestion/vision.py:120</code>	<code>self, path: Path, page_indexes: list[int], *, prompt: str None=None</code> Describe the given pages.
PdfPageVisionDescriber.describe	method	<code>app/ingestion/vision.py:176</code>	<code>self, index: int, png: bytes, prompt: str</code> Internal helper for the describe step.
PdfPageVisionDescriber.build_client	method	<code>app/ingestion/vision.py:212</code>	<code>self</code> Build the client.
_can_render	function	<code>app/ingestion/vision.py:239</code>	

Symbol	Kind	Location	Signature / summary
			Return whether render.
<code>_render_pages</code>	function	<code>app/ingestion/vision.py:252</code>	<code>path: Path, indexes: list[int], scale: float</code> Render the given 0-based page indexes to PNG bytes.
<code>pages_with_visual_content</code>	function	<code>app/ingestion/vision.py:273</code>	<code>path: Path, *, min_images: int=1</code> 0-based indexes of PDF pages that carry figures, images or table rules.
<code>load_connections</code>	function	<code>app/integrations/email/__init__.py:73</code>	<code>raw: str None=None</code> Parse configured connections and resolve their credentials.
<code>_resolve_credentials</code>	function	<code>app/integrations/email/__init__.py:115</code>	<code>raw: dict[str, Any]</code> Turn <code>*_env`</code> indirections into their values.
<code>build_email_service</code>	function	<code>app/integrations/email/__init__.py:133</code>	<code>*, db: Any=None, connections: dict[str, EmailConnection] None=None, include_memory_adapter: bool=True</code> Assemble the service with every adapter registered.
<code>EmailAddress</code>	class	<code>app/integrations/email/base.py:42</code>	<code>BaseModel</code> Pydantic model defining the EmailAddress shape.
<code>EmailAddress.render</code>	method	<code>app/integrations/email/base.py:54</code>	<code>self</code> Render the result.
<code>EmailAttachmentRef</code>	class	<code>app/integrations/email/base.py:63</code>	<code>BaseModel</code> Attachment metadata only.
<code>EmailMessage</code>	class	<code>app/integrations/email/base.py:76</code>	<code>BaseModel</code> One message, in the shape every adapter maps onto.
<code>EmailSearchCriteria</code>	class	<code>app/integrations/email/base.py:98</code>	<code>BaseModel</code> Search terms, expressed once and translated per provider.
<code>EmailDraft</code>	class	<code>app/integrations/email/base.py:113</code>	<code>BaseModel</code> An outbound message, before it is sent or saved as a draft.
<code>EmailResult</code>	class	<code>app/integrations/email/base.py:129</code>	<code>BaseModel</code> What an operation returns.
<code>EmailConnection</code>	class	<code>app/integrations/email/base.py:148</code>	<code>BaseModel</code> A named, credential-bearing connection.
<code>EmailAdapterError</code>	class	<code>app/integrations/email/base.py:169</code>	<code>RuntimeError</code> A provider call failed in a way the caller can describe to a user.
<code>EmailAmbiguousFailure</code>	class	<code>app/integrations/email/base.py:173</code>	<code>EmailAdapterError</code> A side-effect call failed without a definitive outcome — a timeout, a dropped connection, a 5xx after the request was accepted.
<code>EmailAdapter</code>	class	<code>app/integrations/email/base.py:184</code>	<code>ABC</code> What every provider implementation must offer.
<code>EmailAdapter.search</code>	method	<code>app/integrations/email/base.py:194</code>	<code>self, connection: EmailConnection, criteria: EmailSearchCriteria</code> Search the result.
<code>EmailAdapter.read</code>	method		

Symbol	Kind	Location	Signature / summary
		app/integrations/email/base.py:209	<code>self, connection: EmailConnection, message_id: str</code> Read the result.
EmailAdapter.create_draft	method	app/integrations/email/base.py:224	<code>self, connection: EmailConnection, draft: EmailDraft</code> Returns the provider's draft id.
EmailAdapter.send	method	app/integrations/email/base.py:230	<code>self, connection: EmailConnection, draft: EmailDraft</code> Returns the provider's sent-message id.
EmailAdapter.reply	method	app/integrations/email/base.py:235	<code>self, connection: EmailConnection, draft: EmailDraft</code> Reply defaults to a threaded send.
save_connection_record	async function	app/integrations/email/connections_store.py:23	<code>db: Any, *, connection_id: str, provider: str, display_name: str, address: str</code> Save the connection record.
delete_connection_record	async function	app/integrations/email/connections_store.py:55	<code>db: Any, connection_id: str</code> Delete the connection record.
set_allow_send	async function	app/integrations/email/connections_store.py:66	<code>db: Any, connection_id: str, allow_send: bool</code> Returns True if a record was actually updated (the connection exists).
load_dynamic_connections	async function	app/integrations/email/connections_store.py:74	<code>db: Any</code> Every OAuth-established connection, with its tokens attached.
GmailAdapter	class	app/integrations/email/gmail.py:37	<code>EmailAdapter</code> Provides the GmailAdapter behaviour.
GmailAdapter._headers	method	app/integrations/email/gmail.py:53	<code>self, connection: EmailConnection</code> Internal helper for the headers step.
GmailAdapter._request	method	app/integrations/email/gmail.py:71	<code>self, connection: EmailConnection, method: str, path: str, *, side_effect: bool=False, **kwargs: Any</code> Internal helper for the request step.
GmailAdapter.search	method	app/integrations/email/gmail.py:130	<code>self, connection: EmailConnection, criteria: EmailSearchCriteria</code> Search the result.
GmailAdapter.read	method	app/integrations/email/gmail.py:158	<code>self, connection: EmailConnection, message_id: str</code> Read the result.
GmailAdapter.create_draft	method	app/integrations/email/gmail.py:178	<code>self, connection: EmailConnection, draft: EmailDraft</code> Create the draft.
GmailAdapter.send	method	app/integrations/email/gmail.py:202	<code>self, connection: EmailConnection, draft: EmailDraft</code> Send the result.
_gmail_query	function	app/integrations/email/gmail.py:225	<code>criteria: EmailSearchCriteria</code> Translate neutral criteria into Gmail's `q` grammar.
_encode	function	app/integrations/email/gmail.py:247	<code>draft: EmailDraft, connection: EmailConnection</code>

Symbol	Kind	Location	Signature / summary
			Build a base64url RFC-2822 message, which is what Gmail accepts.
<code>_to_message</code>	function	<code>app/integrations/email/gmail.py:270</code>	<code>raw: dict[str, Any]</code> Internal helper for the to message step.
<code>_extract_bodies</code>	function	<code>app/integrations/email/gmail.py:305</code>	<code>payload: dict[str, Any]</code> Walk the MIME parts tree for text and HTML bodies.
<code>_extract_attachments</code>	function	<code>app/integrations/email/gmail.py:337</code>	<code>payload: dict[str, Any]</code> Extract the attachments.
<code>_decode_b64</code>	function	<code>app/integrations/email/gmail.py:372</code>	<code>data: str</code> Decode the b64.
<code>_parse_address</code>	function	<code>app/integrations/email/gmail.py:390</code>	<code>value: str None</code> Parse the address.
<code>_parse_addresses</code>	function	<code>app/integrations/email/gmail.py:407</code>	<code>value: str None</code> Parse the addresses.
<code>_parse_internal_date</code>	function	<code>app/integrations/email/gmail.py:422</code>	<code>value: Any</code> Parse the internal date.
<code>InMemoryEmailAdapter</code>	class	<code>app/integrations/email/memory.py:29</code>	<code>EmailAdapter</code> Provides the InMemoryEmailAdapter behaviour.
<code>InMemoryEmailAdapter.add</code>	method	<code>app/integrations/email/memory.py:45</code>	<code>self, message: EmailMessage</code> Add the result.
<code>InMemoryEmailAdapter.search</code>	method	<code>app/integrations/email/memory.py:57</code>	<code>self, connection: EmailConnection, criteria: EmailSearchCriteria</code> Search the result.
<code>InMemoryEmailAdapter.read</code>	method	<code>app/integrations/email/memory.py:106</code>	<code>self, connection: EmailConnection, message_id: str</code> Read the result.
<code>InMemoryEmailAdapter.create_draft</code>	method	<code>app/integrations/email/memory.py:124</code>	<code>self, connection: EmailConnection, draft: EmailDraft</code> Create the draft.
<code>InMemoryEmailAdapter.send</code>	method	<code>app/integrations/email/memory.py:141</code>	<code>self, connection: EmailConnection, draft: EmailDraft</code> Send the result.
<code>MicrosoftGraphAdapter</code>	class	<code>app/integrations/email/msgraph.py:41</code>	<code>EmailAdapter</code> Provides the MicrosoftGraphAdapter behaviour.
<code>MicrosoftGraphAdapter.headers</code>	method	<code>app/integrations/email/msgraph.py:55</code>	<code>self, connection: EmailConnection</code> Internal helper for the headers step.
<code>MicrosoftGraphAdapter.mailbox</code>	method	<code>app/integrations/email/msgraph.py:76</code>	<code>self, connection: EmailConnection</code> Graph addresses either the signed-in user or a specific mailbox.
<code>MicrosoftGraphAdapter.request</code>	method	<code>app/integrations/email/msgraph.py:86</code>	<code>self, connection: EmailConnection, method: str, path: str, *, side_effect: bool=False, **kwargs: Any</code> Internal helper for the request step.
<code>MicrosoftGraphAdapter.search</code>	method	<code>app/integrations/email/msgraph.py:141</code>	<code>self, connection: EmailConnection, criteria: EmailSearchCriteria</code> Search the result.
<code>MicrosoftGraphAdapter.read</code>	method	<code>app/integrations/email/msgraph.py:176</code>	

Symbol	Kind	Location	Signature / summary
			<code>self, connection: EmailConnection, message_id: str</code> Read the result.
MicrosoftGraphAdapter.create_draft	method	<code>app/integrations/email/msgraph.py:214</code>	<code>self, connection: EmailConnection, draft: EmailDraft</code> Create the draft.
MicrosoftGraphAdapter.send	method	<code>app/integrations/email/msgraph.py:235</code>	<code>self, connection: EmailConnection, draft: EmailDraft</code> Send the result.
MicrosoftGraphAdapter.reply	method	<code>app/integrations/email/msgraph.py:257</code>	<code>self, connection: EmailConnection, draft: EmailDraft</code> Use Graph's own reply endpoint so it quotes and threads correctly.
_graph_filters	function	<code>app/integrations/email/msgraph.py:284</code>	<code>criteria: EmailSearchCriteria</code> Internal helper for the graph filters step.
_recipient	function	<code>app/integrations/email/msgraph.py:313</code>	<code>address: EmailAddress</code> Internal helper for the recipient step.
_draft_payload	function	<code>app/integrations/email/msgraph.py:328</code>	<code>draft: EmailDraft</code> Draft the payload.
_address	function	<code>app/integrations/email/msgraph.py:356</code>	<code>raw: dict[str, Any] None</code> Internal helper for the address step.
_addresses	function	<code>app/integrations/email/msgraph.py:374</code>	<code>raw: list[dict[str, Any]] None</code> Internal helper for the addresses step.
_to_message	function	<code>app/integrations/email/msgraph.py:387</code>	<code>raw: dict[str, Any]</code> Internal helper for the to message step.
_parse_timestamp	function	<code>app/integrations/email/msgraph.py:418</code>	<code>value: Any</code> Parse the timestamp.
OAuthConfigurationError	class	<code>app/integrations/email/oauth.py:44</code>	<code>RuntimeError</code> This provider has no client id/secret configured for this deployment.
OAuthExchangeError	class	<code>app/integrations/email/oauth.py:48</code>	<code>RuntimeError</code> The provider rejected the code/refresh-token exchange.
TokenSet	class	<code>app/integrations/email/oauth.py:53</code>	Provides the TokenSet behaviour.
Profile	class	<code>app/integrations/email/oauth.py:67</code>	Provides the Profile behaviour.
_redirect_uri	function	<code>app/integrations/email/oauth.py:76</code>	<code>provider: Provider</code> Internal helper for the redirect uri step.
generate_pkce_pair	function	<code>app/integrations/email/oauth.py:92</code>	 (code_verifier, code_challenge) for Google's PKCE requirement.
authorize_url	function	<code>app/integrations/email/oauth.py:100</code>	<code>provider: Provider, *, state: str, code_challenge: str None=None</code> Authorize the url.
exchange_code	async function	<code>app/integrations/email/oauth.py:149</code>	<code>provider: Provider, *, code: str, code_verifier: str None=None, client:</code>

Symbol	Kind	Location	Signature / summary
			<code>httpx.AsyncClient</code> <code>None=None</code> Compute the exchange code.
<code>refresh_access_token</code>	async function	<code>app/integrations/email/ oauth.py:200</code>	<code>provider: Provider, *, refresh_token: str, client: httpx.AsyncClient</code> <code>None=None</code> Refresh the access token.
<code>_parse_token_response</code>	function	<code>app/integrations/email/ oauth.py:252</code>	<code>provider: Provider, response: httpx.Response</code> Parse the token response.
<code>fetch_profile</code>	async function	<code>app/integrations/email/ oauth.py:278</code>	<code>provider: Provider, *, access_token: str, client: httpx.AsyncClient</code> <code>None=None</code> Fetch the profile.
<code>_now</code>	function	<code>app/integrations/email/ service.py:56</code>	 Internal helper for the now step.
<code>EmailConnectionError</code>	class	<code>app/integrations/email/ service.py:65</code>	<code>EmailAdapterError</code> The referenced connection does not exist, or is not allowed to do this.
<code>EmailOperationInFlight</code>	class	<code>app/integrations/email/ service.py:69</code>	<code>EmailAdapterError, OperationInFlight</code> An identical side effect is already in flight or ended ambiguously.
<code>EmailOperationLedger</code>	class	<code>app/integrations/email/ service.py:78</code>	<code>ExternalOperationLedger</code> Email's view of the shared external-operation ledger.
<code>EmailService</code>	class	<code>app/integrations/email/ service.py:100</code>	 One capability, many providers, one place that owns the safety rules.
<code>EmailService.connection</code>	method	<code>app/integrations/email/ service.py:128</code>	<code>self, connection_id: str</code> Compute the connection.
<code>EmailService.add_connecti on</code>	method	<code>app/integrations/email/ service.py:145</code>	<code>self, connection: EmailConnection</code> Adds or replaces a connection at runtime — how a newly completed OAuth connect (<code>app/api/ email_oauth.py</code>) becomes usable immediately, with no restart needed.
<code>EmailService.remove_conn ection</code>	method	<code>app/integrations/email/ service.py:151</code>	<code>self, connection_id: str</code> Remove the connection.
<code>EmailService._refreshed</code>	method	<code>app/integrations/email/ service.py:159</code>	<code>self, connection: EmailConnection</code> Refresh this connection's access token if it's near/past expiry.
<code>EmailService.adapter</code>	method	<code>app/integrations/email/ service.py:218</code>	<code>self, connection: EmailConnection</code> Compute the adapter.
<code>EmailService.describe_con nections</code>	method	<code>app/integrations/email/ service.py:235</code>	<code>self</code> Connection list for the Builder's picker.
<code>EmailService.execute</code>	method	<code>app/integrations/email/ service.py:251</code>	<code>self, *, connection_id: str, operation: EmailOperation, criteria: EmailSearchCriteria</code> <code>None=None</code> , <code>message_id: str</code> <code>None=None</code> , <code>draft: EmailDraft</code> <code>None=None</code> , <code>idempotency_scope: str</code> <code>None=None</code> Execute the result.
	method		

Symbol	Kind	Location	Signature / summary
EmailService.perform_write		app/integrations/email/service.py:320	<code>self, *, adapter: EmailAdapter, connection: EmailConnection, operation: EmailOperation, draft: EmailDraft, idempotency_scope: str None</code> Internal helper for the perform write step.
idempotency_key	function	app/integrations/email/service.py:425	<code>*, connection_id: str, operation: str, draft: EmailDraft, scope: str None=None</code> Stable fingerprint of "this exact email side effect".
TokenVaultUnavailableError	class	app/integrations/email/token_vault.py:31	<code>RuntimeError</code> The token vault's storage (Mongo) could not be reached.
_load_kek	function	app/integrations/email/token_vault.py:35	<code></code> Derive the vault's Key Encryption Key from settings, failing closed.
_seal	function	app/integrations/email/token_vault.py:62	<code>payload: dict[str, Any], *, connection_id: str</code> Internal helper for the seal step.
_unseal	function	app/integrations/email/token_vault.py:89	<code>doc: dict[str, Any], *, connection_id: str</code> Internal helper for the unseal step.
TokenVault	class	app/integrations/email/token_vault.py:109	<code></code> Mongo-backed envelope-encrypted store for one connection's OAuth tokens.
TokenVault.tokens	method	app/integrations/email/token_vault.py:121	<code>self</code> The tokens.
TokenVault.store	method	app/integrations/email/token_vault.py:125	<code>self, connection_id: str, *, access_token: str, refresh_token: str None, expires_in_seconds: int</code> Compute the store.
TokenVault.load	method	app/integrations/email/token_vault.py:159	<code>self, connection_id: str</code> Returns `{access_token, refresh_token, expires_at}` (expires_at a real datetime), or None if this connection has no stored tokens.
TokenVault.forget	method	app/integrations/email/token_vault.py:172	<code>self, connection_id: str</code> Compute the forget.
ExternalActionError	class	app/integrations/external_action.py:40	<code>RuntimeError</code> A call failed in a way worth reporting to a workflow author.
ExternalActionService	class	app/integrations/external_action.py:62	<code></code> Provides the ExternalActionService behaviour.
ExternalActionService.default_url_validator	method	app/integrations/external_action.py:90	<code>url: str</code> Apply the SSRF guard, translating violations into node errors.
ExternalActionService.call	method	app/integrations/external_action.py:101	<code>self, *, run_id: str, node_id: str, safety_class: str, method: str, url: str, headers: dict[str, str], body: Any, timeout_seconds: float, approval_satisfied: bool</code> Compute the call.
	method		

Symbol	Kind	Location	Signature / summary
ExternalActionService._replay_or_refuse		app/integrations/external_action.py:248	<code>self, existing: dict[str, Any], key: str</code> Internal helper for the replay or refuse step.
_parse_body	function	app/integrations/external_action.py:277	<code>response: httpx.Response</code> Parse the body.
get_external_action_service	function	app/integrations/external_action.py:300	<code></code> Return the external action service.
load_connections	function	app/integrations/files/__init__.py:68	<code>raw: str None=None</code> Parse configured connections and resolve their credentials.
_resolve_credentials	function	app/integrations/files/__init__.py:106	<code>raw: dict[str, Any]</code> Turn `*_env` indirections into their values.
build_integration_service	function	app/integrations/files/__init__.py:124	<code>*, db: Any=None, connections: dict[str, IntegrationConnection] None=None</code> Assemble the service with every provider adapter registered.
CloudFileMeta	class	app/integrations/files/base.py:32	<code>BaseModel</code> Metadata only — never bytes.
DownloadedFile	class	app/integrations/files/base.py:50	<code>BaseModel</code> Bytes, held only long enough to hand off to object storage.
Page	class	app/integrations/files/base.py:65	<code>BaseModel</code> One page of a folder listing or search result.
IntegrationConnection	class	app/integrations/files/base.py:74	<code>BaseModel</code> A named, credential-bearing connection.
IntegrationAdapterError	class	app/integrations/files/base.py:101	<code>RuntimeError</code> A provider call failed in a way the caller can describe to a user.
IntegrationAuthError	class	app/integrations/files/base.py:105	<code>IntegrationAdapterError</code> The access token was rejected — revoked or expired, reauth needed.
IntegrationNotFoundError	class	app/integrations/files/base.py:109	<code>IntegrationAdapterError</code> The referenced file or folder does not exist (or is not accessible).
IntegrationProvider	class	app/integrations/files/base.py:113	<code>ABC</code> What every cloud-storage adapter must offer.
IntegrationProvider.list_folder	method	app/integrations/files/base.py:124	<code>self, connection: IntegrationConnection, *, folder_id: str None, page_size: int, page_token: str None</code> List the folder.
IntegrationProvider.search_files	method	app/integrations/files/base.py:146	<code>self, connection: IntegrationConnection, *, query: str, folder_id: str None, page_size: int, page_token: str None</code> Search the files.
IntegrationProvider.get_file_meta	method	app/integrations/files/base.py:170	<code>self, connection: IntegrationConnection, *, file_id: str</code> Return the file meta.
IntegrationProvider.download_file	method	app/integrations/files/base.py:185	

Symbol	Kind	Location	Signature / summary
			<code>self, connection: IntegrationConnection, *, file_id: str</code> Download the file.
<code>save_connection_record</code>	async function	<code>app/integrations/files/connections_store.py:23</code>	<code>db: Any, *, connection_id: str, provider: str, display_name: str, address: str</code> Save the connection record.
<code>delete_connection_record</code>	async function	<code>app/integrations/files/connections_store.py:58</code>	<code>db: Any, connection_id: str</code> Delete the connection record.
<code>set_needs_reauth</code>	async function	<code>app/integrations/files/connections_store.py:69</code>	<code>db: Any, connection_id: str, needs_reauth: bool</code> Persist the reauth flag so the Builder can show it without a live provider call — set on a failed refresh, cleared on any success.
<code>load_dynamic_connections</code>	async function	<code>app/integrations/files/connections_store.py:77</code>	<code>db: Any</code> Every OAuth-established connection, with its tokens attached.
<code>GoogleDriveProvider</code>	class	<code>app/integrations/files/google_drive.py:49</code>	<code>IntegrationProvider</code> Provides the GoogleDriveProvider behaviour.
<code>GoogleDriveProvider.headers</code>	method	<code>app/integrations/files/google_drive.py:63</code>	<code>self, connection: IntegrationConnection</code> Internal helper for the headers step.
<code>GoogleDriveProvider.request</code>	method	<code>app/integrations/files/google_drive.py:81</code>	<code>self, connection: IntegrationConnection, method: str, url: str, **kwargs: Any</code> Internal helper for the request step.
<code>GoogleDriveProvider.list_folders</code>	method	<code>app/integrations/files/google_drive.py:130</code>	<code>self, connection: IntegrationConnection, *, folder_id: str None, page_size: int, page_token: str None</code> List the folder.
<code>GoogleDriveProvider.search_files</code>	method	<code>app/integrations/files/google_drive.py:164</code>	<code>self, connection: IntegrationConnection, *, query: str, folder_id: str None, page_size: int, page_token: str None</code> Search the files.
<code>GoogleDriveProvider.get_file_meta</code>	method	<code>app/integrations/files/google_drive.py:203</code>	<code>self, connection: IntegrationConnection, *, file_id: str</code> Return the file meta.
<code>GoogleDriveProvider.download_file</code>	method	<code>app/integrations/files/google_drive.py:225</code>	<code>self, connection: IntegrationConnection, *, file_id: str</code> Download the file.
<code>_to_meta</code>	function	<code>app/integrations/files/google_drive.py:261</code>	<code>raw: dict[str, Any], *, parent_id: str None=None</code> Internal helper for the to meta step.
<code>_to_page</code>	function	<code>app/integrations/files/google_drive.py:286</code>	<code>raw: dict[str, Any], *, parent_id: str None</code> Internal helper for the to page step.
<code>_parse_timestamp</code>	function	<code>app/integrations/files/google_drive.py:300</code>	<code>value: Any</code> Parse the timestamp.
<code>OAuthConfigurationError</code>	class	<code>app/integrations/files/oauth.py:42</code>	<code>RuntimeError</code> This provider has no client id/secret configured for this deployment.
<code>OAuthExchangeError</code>	class		

Symbol	Kind	Location	Signature / summary
		app/integrations/files/oauth.py:46	<code>RuntimeError</code> The provider rejected the code/refresh-token exchange.
TokenSet	class	app/integrations/files/oauth.py:51	Provides the TokenSet behaviour.
Profile	class	app/integrations/files/oauth.py:65	Provides the Profile behaviour.
_redirect_uri	function	app/integrations/files/oauth.py:74	<code>provider: Provider</code> Internal helper for the redirect uri step.
generate_pkce_pair	function	app/integrations/files/oauth.py:90	(code_verifier, code_challenge) for Google's PKCE requirement.
authorize_url	function	app/integrations/files/oauth.py:98	<code>provider: Provider, *, state: str, code_challenge: str None=None</code> Authorize the url.
exchange_code	async function	app/integrations/files/oauth.py:147	<code>provider: Provider, *, code: str, code_verifier: str None=None, client: httpx.AsyncClient None=None</code> Compute the exchange code.
refresh_access_token	async function	app/integrations/files/oauth.py:198	<code>provider: Provider, *, refresh_token: str, client: httpx.AsyncClient None=None</code> Refresh the access token.
_parse_token_response	function	app/integrations/files/oauth.py:250	<code>provider: Provider, response: httpx.Response</code> Parse the token response.
fetch_profile	async function	app/integrations/files/oauth.py:276	<code>provider: Provider, *, access_token: str, client: httpx.AsyncClient None=None</code> Fetch the profile.
OneDriveProvider	class	app/integrations/files/onedrive.py:36	<code>IntegrationProvider</code> Provides the OneDriveProvider behaviour.
OneDriveProvider.headers	method	app/integrations/files/onedrive.py:48	<code>self, connection: IntegrationConnection</code> Internal helper for the headers step.
OneDriveProvider.request	method	app/integrations/files/onedrive.py:66	<code>self, connection: IntegrationConnection, method: str, url: str, *, follow_redirects: bool=False, **kwargs: Any</code> Internal helper for the request step.
OneDriveProvider.list_folders	method	app/integrations/files/onedrive.py:120	<code>self, connection: IntegrationConnection, *, folder_id: str None, page_size: int, page_token: str None</code> List the folder.
OneDriveProvider.search_files	method	app/integrations/files/onedrive.py:152	<code>self, connection: IntegrationConnection, *, query: str, folder_id: str None, page_size: int, page_token: str None</code> Search the files.
OneDriveProvider.get_file_meta	method	app/integrations/files/onedrive.py:187	<code>self, connection: IntegrationConnection, *, file_id: str</code> Return the file meta.

Symbol	Kind	Location	Signature / summary
OneDriveProvider.download_file	method	app/integrations/files/onedrive.py:207	<code>self, connection: IntegrationConnection, *, file_id: str</code> Download the file.
_to_meta	function	app/integrations/files/onedrive.py:233	<code>raw: dict[str, Any]</code> Internal helper for the to meta step.
_to_page	function	app/integrations/files/onedrive.py:258	<code>raw: dict[str, Any]</code> Internal helper for the to page step.
_parse_timestamp	function	app/integrations/files/onedrive.py:271	<code>value: Any</code> Parse the timestamp.
_now	function	app/integrations/files/service.py:44	<code></code> Internal helper for the now step.
IntegrationConnectionError	class	app/integrations/files/service.py:53	<code>IntegrationAdapterError</code> The referenced connection does not exist, or has no adapter registered.
_as_id_list	function	app/integrations/files/service.py:57	<code>value: str list[str] None</code> Normalize a config field that may hold one id, several, or none.
IntegrationResult	class	app/integrations/files/service.py:71	<code>BaseModel</code> Pydantic model defining the IntegrationResult shape.
IntegrationService	class	app/integrations/files/service.py:105	<code></code> One capability, many providers, one place that owns token refresh.
IntegrationService.connect	method	app/integrations/files/service.py:130	<code>self, connection_id: str</code> Compute the connection.
IntegrationService.add_connection	method	app/integrations/files/service.py:147	<code>self, connection: IntegrationConnection</code> Adds or replaces a connection at runtime — how a newly completed OAuth connect (app/api/integration_oauth.py) becomes usable immediately, with no restart needed.
IntegrationService.remove_connection	method	app/integrations/files/service.py:153	<code>self, connection_id: str</code> Remove the connection.
IntegrationService.refresh	method	app/integrations/files/service.py:161	<code>self, connection: IntegrationConnection</code> Refresh this connection's access token if it's near/past expiry.
IntegrationService.provider_for	method	app/integrations/files/service.py:230	<code>self, connection: IntegrationConnection</code> Compute the provider for.
IntegrationService.describe_connections	method	app/integrations/files/service.py:246	<code>self</code> Connection list for the Builder's picker.
IntegrationService.execute	method	app/integrations/files/service.py:262	<code>self, *, connection_id: str, operation: IntegrationOperation, folder_id: str list[str] None=None, file_id: str list[str] None=None, query: str='', page_size: int=25, page_token: str None=None</code> Execute the result.
TokenVaultUnavailableError	class	app/integrations/files/token_vault.py:31	<code>RuntimeError</code> The token vault's storage (Mongo) could not be reached.
_load_kek	function		

Symbol	Kind	Location	Signature / summary
		app/integrations/files/token_vault.py:35	Derive the vault's Key Encryption Key from settings, failing closed.
<code>_seal</code>	function	app/integrations/files/token_vault.py:68	<code>payload: dict[str, Any], *, connection_id: str</code> Internal helper for the seal step.
<code>_unseal</code>	function	app/integrations/files/token_vault.py:95	<code>doc: dict[str, Any], *, connection_id: str</code> Internal helper for the unseal step.
<code>TokenVault</code>	class	app/integrations/files/token_vault.py:115	Mongo-backed envelope-encrypted store for one connection's OAuth tokens.
<code>TokenVault._tokens</code>	method	app/integrations/files/token_vault.py:127	<code>self</code> The tokens.
<code>TokenVault.store</code>	method	app/integrations/files/token_vault.py:131	<code>self, connection_id: str, *, access_token: str, refresh_token: str None, expires_in_seconds: int</code> Compute the store.
<code>TokenVault.load</code>	method	app/integrations/files/token_vault.py:165	<code>self, connection_id: str</code> Returns `{access_token, refresh_token, expires_at}` (expires_at a real datetime), or None if this connection has no stored tokens.
<code>TokenVault.forget</code>	method	app/integrations/files/token_vault.py:178	<code>self, connection_id: str</code> Compute the forget.
<code>OperationInFlight</code>	class	app/integrations/operations.py:33	<code>RuntimeError</code> An identical side effect is already in flight, or ended ambiguously.
<code>AmbiguousOperationFailure</code>	class	app/integrations/operations.py:42	<code>RuntimeError</code> An external call failed without a definitive outcome.
<code>_now</code>	function	app/integrations/operations.py:50	Internal helper for the now step.
<code>ExternalOperationLedger</code>	class	app/integrations/operations.py:59	Provides the ExternalOperationLedger behaviour.
<code>ExternalOperationLedger.find</code>	method	app/integrations/operations.py:72	<code>self, key: str</code> Find the result.
<code>ExternalOperationLedger.reserve</code>	method	app/integrations/operations.py:85	<code>self, key: str, record: dict[str, Any]</code> Claim the key.
<code>ExternalOperationLedger.complete</code>	method	app/integrations/operations.py:109	<code>self, key: str, outcome: dict[str, Any]</code> Complete the result.
<code>ExternalOperationLedger.mark_ambiguous</code>	method	app/integrations/operations.py:122	<code>self, key: str, error: str</code> Mark the ambiguous.
<code>ExternalOperationLedger.release</code>	method	app/integrations/operations.py:135	<code>self, key: str, error: str</code> Drop the reservation after a *definitive* failure.
<code>operation_key</code>	function	app/integrations/operations.py:149	<code>*, scope: str, target: str, payload: Any</code> Stable fingerprint of "this exact side effect".
<code>ExternalActionUrlError</code>	class	app/integrations/url_guard.py:27	

Symbol	Kind	Location	Signature / summary
			<code>ValueError</code> A candidate External Action URL cannot be used safely.
<code>_is_forbidden_address</code>	function	<code>app/integrations/url_guard.py:31</code>	<code>ip: ipaddress.IPv4Address ipaddress.IPv6Address</code> No summary declared.
<code>validate_external_action_url</code>	function	<code>app/integrations/url_guard.py:42</code>	<code>url: str, *, allow_http: bool=False, resolve: Callable[..., list]=socket.getaddrinfo</code> Validate an outbound External Action URL and return it unchanged.
<code>DocTypeChoice</code>	class	<code>app/knowledge/doc_types.py:23</code>	<code></code> Provides the DocTypeChoice behaviour.
<code>DocTypeChoice.precision_sensitive</code>	method	<code>app/knowledge/doc_types.py:36</code>	<code>self</code> The precision sensitive.
<code>doc_type_catalog</code>	function	<code>app/knowledge/doc_types.py:67</code>	<code></code> Compute the doc type catalog.
<code>_encode</code>	function	<code>app/knowledge/ids.py:49</code>	<code>value: int, length: int</code> Encode the result.
<code>prefix_for</code>	function	<code>app/knowledge/ids.py:66</code>	<code>kind: str</code> Compute the prefix for.
<code>new_resource_id</code>	function	<code>app/knowledge/ids.py:81</code>	<code>kind: str</code> Mint a fresh opaque identifier for "kind".
<code>scoped_legacy_id</code>	function	<code>app/knowledge/ids.py:90</code>	<code>kind: str, owner_scope_id: str, legacy_key: str</code> Derive a stable identifier for a pre-existing record.
<code>kind_of</code>	function	<code>app/knowledge/ids.py:108</code>	<code>resource_id: str</code> Return the resource kind for "resource_id", or "None" if unknown.
<code>is_kind</code>	function	<code>app/knowledge/ids.py:121</code>	<code>resource_id: str, kind: str</code> Return whether kind.
<code>utcnow</code>	function	<code>app/knowledge/models.py:13</code>	<code></code> Compute the utcnow.
<code>ResourceStatus</code>	class	<code>app/knowledge/models.py:22</code>	<code>str, Enum</code> Enumeration of ResourceStatus values.
<code>ProfileType</code>	class	<code>app/knowledge/models.py:33</code>	<code>str, Enum</code> Enumeration of ProfileType values.
<code>IngestionJobStatus</code>	class	<code>app/knowledge/models.py:46</code>	<code>str, Enum</code> Enumeration of IngestionJobStatus values.
<code>ScopedResource</code>	class	<code>app/knowledge/models.py:61</code>	<code>BaseModel</code> Pydantic model defining the ScopedResource shape.
<code>CollectionResource</code>	class	<code>app/knowledge/models.py:78</code>	<code>ScopedResource</code> Provides the CollectionResource behaviour.
<code>CollectionResource.display_name</code>	method	<code>app/knowledge/models.py:105</code>	<code>self</code> The display name.
<code>DocumentResource</code>	class	<code>app/knowledge/models.py:110</code>	<code>ScopedResource</code> Provides the DocumentResource behaviour.

Symbol	Kind	Location	Signature / summary
SourceVersionResource	class	app/knowledge/models.py:136	ScopedResource Provides the SourceVersionResource behaviour.
ParserProfileConfig	class	app/knowledge/models.py:164	BaseModel Pydantic model defining the ParserProfileConfig shape.
ChunkingProfileConfig	class	app/knowledge/models.py:189	BaseModel Pydantic model defining the ChunkingProfileConfig shape.
ChunkingProfileConfig.validate_token_windows	method	app/knowledge/models.py:218	self Validate the token windows.
EmbeddingProfileConfig	class	app/knowledge/models.py:231	BaseModel Pydantic model defining the EmbeddingProfileConfig shape.
RetrievalProfileConfig	class	app/knowledge/models.py:248	BaseModel Pydantic model defining the RetrievalProfileConfig shape.
RetrievalProfileConfig.validate_counts	method	app/knowledge/models.py:273	self Validate the counts.
RetrievalRoute	class	app/knowledge/models.py:284	BaseModel Pydantic model defining the RetrievalRoute shape.
RetrievalRoutingProfileConfig	class	app/knowledge/models.py:305	BaseModel Pydantic model defining the RetrievalRoutingProfileConfig shape.
RetrievalRoutingProfileConfig.validate_routes	method	app/knowledge/models.py:318	self Validate the routes.
GenerationProfileConfig	class	app/knowledge/models.py:332	BaseModel Pydantic model defining the GenerationProfileConfig shape.
ProfileVersion	class	app/knowledge/models.py:352	ScopedResource Provides the ProfileVersion behaviour.
IndexVersion	class	app/knowledge/models.py:376	ScopedResource Provides the IndexVersion behaviour.
IngestionSourceInput	class	app/knowledge/models.py:413	BaseModel Pydantic model defining the IngestionSourceInput shape.
IngestionJob	class	app/knowledge/models.py:430	ScopedResource Provides the IngestionJob behaviour.
RAGAgentDefinition	class	app/knowledge/models.py:466	ScopedResource Provides the RAGAgentDefinition behaviour.
RetrievalTrace	class	app/knowledge/models.py:492	ScopedResource Provides the RetrievalTrace behaviour.
ResourceNotFoundError	class	app/knowledge/repository.py:41	LookupError A scoped resource lookup found nothing for this owner.
ResourceConflictError	class	app/knowledge/repository.py:45	ValueError A resource exists but is not in a state the caller may act on.
_scope	function		

Symbol	Kind	Location	Signature / summary
		app/knowledge/ repository.py:49	<code>owner_scope_id: str, **extra: Any</code> Internal helper for the scope step.
<code>_strip</code>	function	app/knowledge/ repository.py:62	<code>document: dict[str, Any]</code> Internal helper for the strip step.
<code>KnowledgeRepository</code>	class	app/knowledge/ repository.py:75	<code></code> One instance shared across the app via `services["knowledge_repository"]`.
<code>KnowledgeRepository._collections</code>	method	app/knowledge/ repository.py:88	<code>self</code> The collections.
<code>KnowledgeRepository._documents</code>	method	app/knowledge/ repository.py:93	<code>self</code> The documents.
<code>KnowledgeRepository._source_versions</code>	method	app/knowledge/ repository.py:98	<code>self</code> The source versions.
<code>KnowledgeRepository._profiles</code>	method	app/knowledge/ repository.py:103	<code>self</code> The profiles.
<code>KnowledgeRepository._indexes</code>	method	app/knowledge/ repository.py:108	<code>self</code> The indexes.
<code>KnowledgeRepository._index_documents</code>	method	app/knowledge/ repository.py:113	<code>self</code> The index documents.
<code>KnowledgeRepository._ingestion_jobs</code>	method	app/knowledge/ repository.py:118	<code>self</code> The ingestion jobs.
<code>KnowledgeRepository._rag_agents</code>	method	app/knowledge/ repository.py:123	<code>self</code> The rag agents.
<code>KnowledgeRepository._traces</code>	method	app/knowledge/ repository.py:128	<code>self</code> The traces.
<code>KnowledgeRepository.ensure_indexes</code>	method	app/knowledge/ repository.py:132	<code>self</code> Ensure the indexes.
<code>KnowledgeRepository.create_collection</code>	method	app/knowledge/ repository.py:152	<code>self, resource: CollectionResource</code> Create the collection.
<code>KnowledgeRepository.save_collection</code>	method	app/knowledge/ repository.py:164	<code>self, resource: CollectionResource</code> Save the collection.
<code>KnowledgeRepository.get_collection</code>	method	app/knowledge/ repository.py:181	<code>self, owner_scope_id: str, collection_id: str</code> Return the collection.
<code>KnowledgeRepository.list_collections</code>	method	app/knowledge/ repository.py:196	<code>self, owner_scope_id: str</code> List the collections.
<code>KnowledgeRepository.save_document</code>	method	app/knowledge/ repository.py:209	<code>self, resource: DocumentResource</code> Save the document.
<code>KnowledgeRepository.get_document</code>	method	app/knowledge/ repository.py:226	<code>self, owner_scope_id: str, document_id: str</code> Return the document.
<code>KnowledgeRepository.find_document_by_hash</code>	method	app/knowledge/ repository.py:241	<code>self, owner_scope_id: str, collection_id: str, content_hash: str</code> Find the document by hash.
<code>KnowledgeRepository.list_documents</code>	method	app/knowledge/ repository.py:259	<code>self, owner_scope_id: str, collection_id: str</code> List the documents.

Symbol	Kind	Location	Signature / summary
KnowledgeRepository.save_source_version	method	app/knowledge/repository.py:272	<code>self, resource: SourceVersionResource</code> Save the source version.
KnowledgeRepository.get_source_version	method	app/knowledge/repository.py:288	<code>self, owner_scope_id: str, source_version_id: str</code> Return the source version.
KnowledgeRepository.next_profile_version	method	app/knowledge/repository.py:308	<code>self, owner_scope_id: str, profile_id: str</code> Compute the next profile version.
KnowledgeRepository.save_profile	method	app/knowledge/repository.py:323	<code>self, profile: ProfileVersion</code> Save the profile.
KnowledgeRepository.get_profile	method	app/knowledge/repository.py:335	<code>self, owner_scope_id: str, profile_id: str, version: int None=None, expected_type: ProfileType None=None</code> Return the profile.
KnowledgeRepository.list_profiles	method	app/knowledge/repository.py:367	<code>self, owner_scope_id: str, profile_type: ProfileType None=None</code> List the profiles.
KnowledgeRepository.save_index	method	app/knowledge/repository.py:389	<code>self, index: IndexVersion</code> Save the index.
KnowledgeRepository.get_index	method	app/knowledge/repository.py:406	<code>self, owner_scope_id: str, index_id: str</code> Return the index.
KnowledgeRepository.list_indexes	method	app/knowledge/repository.py:421	<code>self, owner_scope_id: str, collection_id: str</code> List the indexes.
KnowledgeRepository.save_index_document	method	app/knowledge/repository.py:435	<code>self, *, owner_scope_id: str, index_id: str, document_id: str, source_version_id: str, ingestion_job_id: str, chunk_ids: list[str]</code> Save the index document.
KnowledgeRepository.get_index_document	method	app/knowledge/repository.py:470	<code>self, owner_scope_id: str, index_id: str, document_id: str</code> Return the index document.
KnowledgeRepository.index_document_stats	method	app/knowledge/repository.py:488	<code>self, owner_scope_id: str, index_id: str</code> Return ``(document_count, chunk_count)`` from authoritative mappings.
KnowledgeRepository.save_ingestion_job	method	app/knowledge/repository.py:503	<code>self, job: IngestionJob</code> Save the ingestion job.
KnowledgeRepository.get_ingestion_job	method	app/knowledge/repository.py:520	<code>self, owner_scope_id: str, ingestion_job_id: str</code> Return the ingestion job.
KnowledgeRepository.list_ingestion_jobs	method	app/knowledge/repository.py:537	<code>self, owner_scope_id: str, collection_id: str None=None</code> List the ingestion jobs.
KnowledgeRepository.list_recoverable_jobs	method	app/knowledge/repository.py:555	<code>self</code> Non-terminal jobs across every owner, for startup recovery only.
	method		

Symbol	Kind	Location	Signature / summary
KnowledgeRepository.save_rag_agent		app/knowledge/repository.py:568	<code>self, agent: RAGAgentDefinition</code> Save the rag agent.
KnowledgeRepository.get_rag_agent	method	app/knowledge/repository.py:585	<code>self, owner_scope_id: str, rag_agent_id: str</code> Return the rag agent.
KnowledgeRepository.list_rag_agents	method	app/knowledge/repository.py:600	<code>self, owner_scope_id: str, search: str None=None</code> List the rag agents.
KnowledgeRepository.save_trace	method	app/knowledge/repository.py:620	<code>self, trace: RetrievalTrace</code> Save the trace.
KnowledgeRepository.get_trace	method	app/knowledge/repository.py:636	<code>self, owner_scope_id: str, retrieval_request_id: str</code> Return the trace.
KnowledgeRepository.list_traces	method	app/knowledge/repository.py:653	<code>self, owner_scope_id: str, limit: int=100</code> List the traces.
KnowledgeRepository.reco rd_manifest_index_history	method	app/knowledge/repository.py:667	<code>self, *, storage_key: str, manifest: dict[str, Any], index_history: dict[str, Any]</code> Upsert the legacy manifest while appending this index's history.
validate_metadata_schema	function	app/knowledge/service.py:26	<code>schema: dict[str, Any]</code> Validate the metadata schema.
workspace_for_scope	function	app/knowledge/service.py:71	<code>owner_scope_id: str</code> Compute the workspace for scope.
KnowledgeService	class	app/knowledge/service.py:83	<code></code> Provides the KnowledgeService behaviour.
KnowledgeService.create_collection	method	app/knowledge/service.py:93	<code>self, *, owner_scope_id: str, name: str, description: str='', metadata_schema: dict[str, Any] None=None, doc_types: list[str] None=None</code> Create the collection.
KnowledgeService.create_profile_version	method	app/knowledge/service.py:125	<code>self, *, owner_scope_id: str, profile_type: ProfileType, name: str, strategy: str, config: dict[str, Any], profile_id: str None=None, description: str='', based_on_preset: str None=None</code> Create the profile version.
KnowledgeService.validate_profile_config	method	app/knowledge/service.py:181	<code>profile_type: ProfileType, config: dict[str, Any]</code> Validate the profile config.
KnowledgeService.create_index	method	app/knowledge/service.py:201	<code>self, *, owner_scope_id: str, collection_id: str, parser_profile_id: str, parser_profile_version: int, chunking_profile_id: str, chunking_profile_version: int, embedding_profile_id: str, embedding_profile_version: int</code> Create the index.
KnowledgeService.activate_index	method		

Symbol	Kind	Location	Signature / summary
		app/knowledge/service.py:259	<code>self, *, owner_scope_id: str, collection_id: str, index_id: str</code> Compute the activate index.
KnowledgeService.create_rag_agent	method	app/knowledge/service.py:289	<code>self, *, owner_scope_id: str, name: str, collection_id: str, retrieval_profile_id: str, generation_profile_id: str, routing_profile_id: str None=None, description: str=''</code> Create the rag agent.
KnowledgeService.ensure_default_profiles	method	app/knowledge/service.py:347	<code>self, owner_scope_id: str</code> Ensure the default profiles.
_tool_name_for	function	app/llm/anthropic_batch.py:28	<code>response_model: Type[BaseModel]</code> Internal helper for the tool name for step.
_build_params	function	app/llm/anthropic_batch.py:40	<code>item: BatchRequestItem</code> Build one MessageCreateParamsNonStreaming-shaped dict for a batch row.
AnthropicBatchService	class	app/llm/anthropic_batch.py:61	<code></code> Submit, poll, and read back an Anthropic Message Batch.
AnthropicBatchService.submit	method	app/llm/anthropic_batch.py:76	<code>self, items: list[BatchRequestItem]</code> Submit a batch and return its batch_id.
AnthropicBatchService.status	method	app/llm/anthropic_batch.py:85	<code>self, batch_id: str</code> Compute the status.
AnthropicBatchService.cancel	method	app/llm/anthropic_batch.py:109	<code>self, batch_id: str</code> Request cancellation.
AnthropicBatchService.results	method	app/llm/anthropic_batch.py:114	<code>self, batch_id: str, *, response_models: dict[str, Type[BaseModel]] None=None</code> Read back results for an ended batch.
_supports_temperature	function	app/llm/anthropic_gw.py:52	<code>model: str</code> Return True if the model accepts a non-default temperature.
_cacheable_system	function	app/llm/anthropic_gw.py:57	<code>system: str</code> Turn a plain system string into a cache-marked content-block list.
_mark_last_block_cacheable	function	app/llm/anthropic_gw.py:78	<code>messages: list[dict]</code> Add a cache breakpoint to the last content block of the last message.
_cache_usage_fields	function	app/llm/anthropic_gw.py:102	<code>usage</code> Extract cache_creation/cache_read counts off a Message.usage object.
AnthropicGateway	class	app/llm/anthropic_gw.py:114	<code>LLMGateway</code> Live Anthropic Claude gateway.
AnthropicGateway.probe_model_access	method	app/llm/anthropic_gw.py:129	<code>self, model: str</code> Verify workspace access through model metadata, with no generation.
AnthropicGateway.create	method	app/llm/anthropic_gw.py:138	<code>self, *, on_token: Callable[[str], Awaitable[None]] None=None, **kwargs</code> Issue a request via streaming and return the assembled Message.

Symbol	Kind	Location	Signature / summary
AnthropicGateway.complete	method	app/llm/anthropic_gw.py:156	<pre>self, *, model: str, system: str, user: str, temperature: float=0.0, max_tokens: int=1024, reasoning_effort: str None=None, on_token: Callable[[str], Awaitable[None]] None=None</pre> Complete the result.
AnthropicGateway.complete_structured	method	app/llm/anthropic_gw.py:204	<pre>self, *, model: str, system: str, user: str, response_model: Type[T], temperature: float=0.0, max_tokens: int=4096, reasoning_effort: str None=None</pre> Complete the structured.
AnthropicGateway.chat_with_tools	method	app/llm/anthropic_gw.py:287	<pre>self, *, model: str, system: str, messages: list[dict], tools: list[dict], temperature: float=0.0, max_tokens: int=4096</pre> Compute the chat with tools.
LLMResponse	class	app/llm/base.py:17	BaseModel Plain text response from an LLM call.
ToolCall	class	app/llm/base.py:40	BaseModel One tool invocation the model wants to make.
LLMToolUseResponse	class	app/llm/base.py:47	BaseModel Response from a multi-turn chat-with-tools call.
LLMGateway	class	app/llm/base.py:67	ABC Abstract base for all LLM providers.
LLMGateway.complete	method	app/llm/base.py:77	<pre>self, *, model: str, system: str, user: str, temperature: float=0.0, max_tokens: int=1024, reasoning_effort: str None=None</pre> Plain text completion.
LLMGateway.complete_structured	method	app/llm/base.py:97	<pre>self, *, model: str, system: str, user: str, response_model: Type[T], temperature: float=0.0, max_tokens: int=1024, reasoning_effort: str None=None</pre> Structured completion.
LLMGateway.chat_with_tools	method	app/llm/base.py:118	<pre>self, *, model: str, system: str, messages: list[dict], tools: list[dict], temperature: float=0.0, max_tokens: int=4096</pre> Multi-turn chat with tool calling.
BatchRequestItem	class	app/llm/batch_types.py:22	 One request in a batch submission.
BatchResultItem	class	app/llm/batch_types.py:42	 One result row, keyed by the custom_id it was submitted with.
BatchStatus	class	app/llm/batch_types.py:61	 Point-in-time status of a submitted batch.
ModelDefinition	class	app/llm/catalog.py:13	 Provides the ModelDefinition behaviour.
	method	app/llm/catalog.py:41	

Symbol	Kind	Location	Signature / summary
ModelDefinition.permits_data_class			<code>self, data_class: str None</code> Compute the permits data class.
ModelDefinition.as_dict	method	<code>app/llm/catalog.py:59</code>	<code>self</code> Compute the as dict.
model_definition	function	<code>app/llm/catalog.py:148</code>	<code>model: str</code> Compute the model definition.
provider_for_model	function	<code>app/llm/catalog.py:163</code>	<code>model: str</code> Compute the provider for model.
is_local_model	function	<code>app/llm/catalog.py:175</code>	<code>model: str</code> Return whether local model.
local_service_name	function	<code>app/llm/catalog.py:188</code>	<code>model: str</code> Compute the local service name.
StructuredOutputError	class	<code>app/llm/errors.py:6</code>	<code>RuntimeError</code> The provider responded, but its output violated the required schema.
LLMInputLimitError	class	<code>app/llm/errors.py:14</code>	<code>ValueError</code> The resolved prompt exceeds a configured token/cost safety boundary.
LLMProviderUnavailableError	class	<code>app/llm/errors.py:18</code>	<code>RuntimeError</code> No configured provider can serve the requested model/fallback chain.
BatchTimeoutError	class	<code>app/llm/errors.py:22</code>	<code>RuntimeError</code> A submitted batch did not reach a terminal state within the deadline.
LLMPolicyDeniedError	class	<code>app/llm/errors.py:30</code>	<code>RuntimeError</code> The data-classification/OPA policy check (<code>app/security/llm_policy.py</code>) denied this call — e.g.
LocalModelProfile	class	<code>app/llm/local_openai_gw.py:25</code>	<code></code> Provides the LocalModelProfile behaviour.
normalize_openai_base_url	function	<code>app/llm/local_openai_gw.py:50</code>	<code>value: str</code> Validate and normalize a deployment-owned OpenAI-compatible URL.
_reasoning_content	function	<code>app/llm/local_openai_gw.py:77</code>	<code>message: Any</code> Internal helper for the reasoning content step.
_usage	function	<code>app/llm/local_openai_gw.py:94</code>	<code>response: Any</code> Internal helper for the usage step.
LocalOpenAICompatibleGateway	class	<code>app/llm/local_openai_gw.py:110</code>	<code>LLMGateway</code> Gateway used by one configured local/private model endpoint.
LocalOpenAICompatibleGateway.generation_options	method	<code>app/llm/local_openai_gw.py:138</code>	<code>self</code> Internal helper for the generation options step.
LocalOpenAICompatibleGateway.fixed_sampling_provider	method	<code>app/llm/local_openai_gw.py:154</code>	<code>self</code> Internal helper for the fixed sampling provider step.
LocalOpenAICompatibleGateway.max_tokens_field	method	<code>app/llm/local_openai_gw.py:165</code>	<code>self</code> Internal helper for the max tokens field step.
	method		

Symbol	Kind	Location	Signature / summary
LocalOpenAICompatibleGateway.complete		app/llm/local_openai_gw.py:178	<code>self, *, model: str, system: str, user: str, temperature: float=0.0, max_tokens: int=1024, reasoning_effort: str None=None, on_token: object None=None</code> Complete the result.
LocalOpenAICompatibleGateway.complete_structured	method	app/llm/local_openai_gw.py:229	<code>self, *, model: str, system: str, user: str, response_model: Type[T], temperature: float=0.0, max_tokens: int=4096, reasoning_effort: str None=None</code> Complete the structured.
LocalOpenAICompatibleGateway.chat_with_tools	method	app/llm/local_openai_gw.py:294	<code>self, *, model: str, system: str, messages: list[dict], tools: list[dict], temperature: float=0.0, max_tokens: int=4096</code> Compute the chat with tools.
LocalOpenAICompatibleGateway.probe_details	method	app/llm/local_openai_gw.py:405	<code>self</code> Probe the details.
LocalOpenAICompatibleGateway.probe	method	app/llm/local_openai_gw.py:429	<code>self</code> Probe the result.
KimiK3LocalGateway	class	app/llm/local_openai_gw.py:439	<code>LocalOpenAICompatibleGateway</code> Distinct class keeps Kimi and GLM instances separate in the registry.
GLM5LocalGateway	class	app/llm/local_openai_gw.py:443	<code>LocalOpenAICompatibleGateway</code> Distinct class keeps Kimi and GLM instances separate in the registry.
ModelProfile	class	app/llm/model_catalog.py:28	<code></code> Provides the ModelProfile behaviour.
estimate_model_cost	function	app/llm/model_catalog.py:151	<code>model: str, input_tokens: int, output_tokens: int</code> Estimate request cost using the platform's configured price catalog.
ModelRoutingError	class	app/llm/model_router.py:109	<code>RuntimeError</code> Raised when an automatic selection cannot satisfy its constraints.
ModelSelection	class	app/llm/model_router.py:114	<code></code> Provides the ModelSelection behaviour.
ModelSelection.to_event	method	app/llm/model_router.py:138	<code>self, *, actual_model: str, call_id: int, fallback: bool=False, cache_hit: bool=False</code> Compute the to event.
ModelRouter	class	app/llm/model_router.py:175	<code></code> Choose the most suitable configured model without spending tokens.
ModelRouter.select	method	app/llm/model_router.py:178	<code>self, *, method_name: str, kwargs: Mapping[str, Any], input_tokens: int, allowed_models: Sequence[str], is_available: Callable[[str], bool], node_type: str None=None, policy: Mapping[str, Any] None=None</code> Select the result.
	method	app/llm/model_router.py:317	

Symbol	Kind	Location	Signature / summary
ModelRouter.describe_manual			<pre>self, *, requested_model: str, selected_model: str, method_name: str, kwargs: Mapping[str, Any], input_tokens: int, node_type: str None=None, mode: Literal['manual', 'cost_protection']='manual'</pre> Compute the describe manual.
infer_task_kind	function	app/llm/model_router.py:373	<pre>method_name: str, kwargs: Mapping[str, Any], *, node_type: str None=None</pre> Compute the infer task kind.
infer_complexity	function	app/llm/model_router.py:410	<pre>method_name: str, kwargs: Mapping[str, Any], *, input_tokens: int, node_type: str None=None</pre> Compute the infer complexity.
_request_text	function	app/llm/model_router.py:462	<pre>kwargs: Mapping[str, Any], *, node_type: str None</pre> Internal helper for the request text step.
_accuracy_priority	function	app/llm/model_router.py:498	<pre>value: Any</pre> Internal helper for the accuracy priority step.
_hosted_bonus	function	app/llm/model_router.py:514	<pre>profile: ModelProfile, priority: AccuracyPriority</pre> Break same-score ties toward hosted providers under max accuracy.
_target_tier	function	app/llm/model_router.py:528	<pre>complexity: Complexity, priority: AccuracyPriority</pre> Internal helper for the target tier step.
_score_profile	function	app/llm/model_router.py:549	<pre>profile: ModelProfile, *, task: TaskKind, complexity: Complexity, priority: AccuracyPriority, estimated_cost: float, max_cost: float, prefer_low_latency: bool, observed_quality: float None</pre> Score the profile.
_selection_reason	function	app/llm/model_router.py:610	<pre>profile: ModelProfile, *, task: TaskKind, complexity: Complexity, priority: AccuracyPriority, source: Literal['policy', 'evaluation'], estimated_cost: float, ceiling: float None</pre> Internal helper for the selection reason step.
_build_body	function	app/llm/openai_batch.py:35	<pre>item: BatchRequestItem</pre> Build one /v1/chat/completions request body for a batch JSONL row.
OpenAIBatchService	class	app/llm/openai_batch.py:61	<pre></pre> Submit, poll, and read back an OpenAI Batch.
OpenAIBatchService.submit	method	app/llm/openai_batch.py:76	<pre>self, items: list[BatchRequestItem]</pre> Upload the JSONL request file, create the batch, return its id.
OpenAIBatchService.status	method	app/llm/openai_batch.py:99	<pre>self, batch_id: str</pre> Compute the status.
OpenAIBatchService.cancel	method	app/llm/openai_batch.py:121	<pre>self, batch_id: str</pre> Request cancellation.

Symbol	Kind	Location	Signature / summary
OpenAIBatchService.results	method	app/llm/openai_batch.py:126	<code>self, batch_id: str, *, response_models: dict[str, Type[BaseModel]] None=None</code> Read back results for an ended batch.
_parse_jsonl	function	app/llm/openai_batch.py:162	<code>text: str</code> Parse the jsonl.
_result_from_output_row	function	app/llm/openai_batch.py:174	<code>row: dict, response_models: dict[str, Type[BaseModel]]</code> Internal helper for the result from output row step.
StructuredResult	class	app/llm/openai_gw.py:35	<code></code> Carries a parsed Pydantic model plus token usage, so the registry wrapper can record cost.
_supports_custom_temperature	function	app/llm/openai_gw.py:58	<code>model: str</code> Return True if the model accepts a non-default temperature.
_completion_tokens_for	function	app/llm/openai_gw.py:63	<code>model: str, requested: int</code> Apply provider-specific minimums to max_completion_tokens.
_cached_tokens_from_usage	function	app/llm/openai_gw.py:74	<code>usage: Any</code> Read OpenAI's automatic-prompt-cache hit count off a usage payload.
_prompt_cache_key	function	app/llm/openai_gw.py:90	<code>model: str, cacheable_prefix: str</code> Stable prompt_cache_key so requests sharing a static prefix (same model + system prompt) route to the same cache-holding backend -- OpenAI's own recommendation for boosting cache hit rates.
_prompt_cache_retention_kwargs	function	app/llm/openai_gw.py:107	<code>model: str</code> Extended (24h) cache retention for pre-GPT-5.6 models, at the same cost as the default in-memory (5-10 min) window per OpenAI's prompt caching guide -- a free win for workflows with gaps between calls.
_system_content	function	app/llm/openai_gw.py:120	<code>text: str, model: str None</code> Render system-prompt text as a bare string, or (gpt-5.6+ only) as a text content-part carrying an explicit prompt_cache_breakpoint.
_system_messages	function	app/llm/openai_gw.py:138	<code>system: str, *, model: str None=None</code> Build the leading system message, omitting it when empty.
_chat_tool_reasoning_effort	function	app/llm/openai_gw.py:150	<code>model: str</code> Return the safe Chat Completions effort for function-tool calls.
OpenAIGateway	class	app/llm/openai_gw.py:165	<code>LLMGateway</code> OpenAI-backed gateway.
OpenAIGateway.probe_model_access	method	app/llm/openai_gw.py:180	<code>self, model: str</code> Verify project access through model metadata, with no generation.
OpenAIGateway.complete	method	app/llm/openai_gw.py:189	<code>self, *, model: str, system: str, user: str, temperature: float=0.0, max_tokens: int=1024, reasoning_effort: str None=None, on_token: Callable[[str], Awaitable[None]] None=None</code> Complete the result.

Symbol	Kind	Location	Signature / summary
OpenAIGateway.complete_structured	method	app/llm/openai_gw.py:274	<code>self, *, model: str, system: str, user: str, response_model: Type[T], temperature: float=0.0, max_tokens: int=4096, reasoning_effort: str None=None</code> Complete the structured.
OpenAIGateway.chat_with_tools	method	app/llm/openai_gw.py:347	<code>self, *, model: str, system: str, messages: list[dict], tools: list[dict], temperature: float=0.0, max_tokens: int=4096</code> Compute the chat with tools.
OpenAIModelDefinition	class	app/llm/openai_registry.py:24	<code></code> Provides the OpenAIModelDefinition behaviour.
OpenAIModelDefinition.as_dict	method	app/llm/openai_registry.py:52	<code>self</code> Compute the as dict.
models_for_kind	function	app/llm/openai_registry.py:256	<code>kind: OpenAIModelKind</code> Compute the models for kind.
openai_model	function	app/llm/openai_registry.py:336	<code>model: str</code> Compute the openai model.
require_openai_model_kind	function	app/llm/openai_registry.py:353	<code>model: str, kind: OpenAIModelKind</code> Compute the require openai model kind.
OpenRouterModelInfo	class	app/llm/openrouter_catalog.py:34	<code></code> Provides the OpenRouterModelInfo behaviour.
OpenRouterModelInfo.as_dict	method	app/llm/openrouter_catalog.py:57	<code>self</code> Compute the as dict.
is_openrouter_model_id	function	app/llm/openrouter_catalog.py:77	<code>value: str</code> Structural check only (no live lookup) — used by preflight, which must stay synchronous and network-free.
_price_per_million	function	app/llm/openrouter_catalog.py:84	<code>raw: Any</code> OpenRouter reports pricing as a per-token USD string (e.g.
_to_model_info	function	app/llm/openrouter_catalog.py:94	<code>entry: dict[str, Any]</code> Internal helper for the to model info step.
_fetch_raw_catalog	async function	app/llm/openrouter_catalog.py:125	<code>client: httpx.AsyncClient None</code> Fetch the raw catalog.
OpenRouterCatalogCache	class	app/llm/openrouter_catalog.py:150	<code></code> Process-wide TTL cache.
OpenRouterCatalogCache._is_stale	method	app/llm/openrouter_catalog.py:164	<code>self</code> Return whether stale.
OpenRouterCatalogCache.get_models	method	app/llm/openrouter_catalog.py:172	<code>self, *, client: httpx.AsyncClient None=None, force_refresh: bool=False</code> Return the models.
OpenRouterCatalogCache.search	method	app/llm/openrouter_catalog.py:191	<code>self, query: str None=None, *, limit: int=50, client: httpx.AsyncClient None=None</code> Search the result.
get_default_cache	function	app/llm/openrouter_catalog.py:222	<code></code> Return the default cache.

Symbol	Kind	Location	Signature / summary
StructuredResult	class	app/llm/openrouter_gw.py:47	<code></code> Mirrors app/llm/openai_gw.py's StructuredResult — carries usage/cost alongside the parsed model so the registry wrapper can record both, then unwraps <code>`parsed`</code> .
strip_openrouter_prefix	function	app/llm/openrouter_gw.py:59	<code>model: str</code> Compute the strip openrouter prefix.
unfence_json	function	app/llm/openrouter_gw.py:71	<code>content: str</code> Return the JSON body of a Markdown-fenced payload, or <code>`content`</code> unchanged.
_parse_structured	function	app/llm/openrouter_gw.py:83	<code>content: str, response_model: Type[T]</code> Parse a structured response, retrying once on the unfenced body.
_usage_fields	function	app/llm/openrouter_gw.py:102	<code>usage: dict[str, Any] None</code> Internal helper for the usage fields step.
OpenRouterGateway	class	app/llm/openrouter_gw.py:124	<code>LLMGateway</code> OpenRouter-backed gateway.
OpenRouterGateway.probe_model_access	method	app/llm/openrouter_gw.py:140	<code>self, model: str</code> Cheap existence check via the catalog — OpenRouter has no per-model equivalent of OpenAI's GET /models/{id}.
OpenRouterGateway._post	method	app/llm/openrouter_gw.py:151	<code>self, body: dict[str, Any]</code> Internal helper for the post step.
OpenRouterGateway.complete	method	app/llm/openrouter_gw.py:164	<code>self, *, model: str, system: str, user: str, temperature: float=0.0, max_tokens: int=1024, reasoning_effort: str None=None</code> Complete the result.
OpenRouterGateway.complete_structured	method	app/llm/openrouter_gw.py:210	<code>self, *, model: str, system: str, user: str, response_model: Type[T], temperature: float=0.0, max_tokens: int=1024, reasoning_effort: str None=None</code> Complete the structured.
OpenRouterGateway.chat_with_tools	method	app/llm/openrouter_gw.py:271	<code>self, *, model: str, system: str, messages: list[dict], tools: list[dict], temperature: float=0.0, max_tokens: int=4096</code> Compute the chat with tools.
_safe_json_loads	function	app/llm/openrouter_gw.py:363	<code>value: str</code> Internal helper for the safe json loads step.
_cost_tier_for_priority	function	app/llm/openrouter_ranking.py:42	<code>accuracy_priority: str</code> Internal helper for the cost tier for priority step.
build_auto_router_plugin	function	app/llm/openrouter_ranking.py:56	<code>*, accuracy_priority: str='balanced', max_estimated_cost_usd: float None=None, quality_scores: Mapping[str, float] None=None, faithfulness_scores: Mapping[str, float] None=None</code> Builds one <code>`plugins[]`</code> entry for OpenRouter's auto-router from Eurskem's ModelRoutingPolicy fields.
	function		

Symbol	Kind	Location	Signature / summary
<code>is_openrouter_router_model</code>		<code>app/llm/openrouter_ranking.py:90</code>	<code>value: str</code> True for OpenRouter's own router meta-models (openrouter/auto, openrouter/free, ...).
<code>_promote_tier_peers</code>	function	<code>app/llm/registry.py:130</code>	<code>requested: list[str]</code> Move each model's documented cross-provider peer to directly follow it.
<code>RetryPolicy</code>	class	<code>app/llm/registry.py:167</code>	 One explicit retry policy shared by all LLM providers.
<code>ModelAccessResult</code>	class	<code>app/llm/registry.py:192</code>	 Result of a model-metadata probe that uses no generation tokens.
<code>_status_code</code>	function	<code>app/llm/registry.py:201</code>	<code>exc: BaseException</code> Internal helper for the status code step.
<code>_anthropic_error_type</code>	function	<code>app/llm/registry.py:218</code>	<code>exc: BaseException</code> Anthropic streams some errors (e.g.
<code>_is_retryable_error</code>	function	<code>app/llm/registry.py:234</code>	<code>exc: BaseException</code> Return whether another generation may succeed.
<code>_provider_error_code</code>	function	<code>app/llm/registry.py:263</code>	<code>exc: BaseException</code> Extract a provider error code without depending on one SDK's shape.
<code>_is_model_unavailable_error</code>	function	<code>app/llm/registry.py:303</code>	<code>exc: BaseException</code> Return whether this candidate cannot serve the requested model.
<code>_retry_after_seconds</code>	function	<code>app/llm/registry.py:320</code>	<code>exc: BaseException</code> Retry the after seconds.
<code>resolve_model</code>	function	<code>app/llm/registry.py:351</code>	<code>intended: str</code> Map an intended model name to the model name we'll actually call.
<code>get_gateway</code>	function	<code>app/llm/registry.py:372</code>	<code>model_name: str</code> Return (gateway, resolved_model_name).
<code>_gateway_class_for</code>	function	<code>app/llm/registry.py:385</code>	<code>model_name: str</code> Internal helper for the gateway class for step.
<code>_construct</code>	function	<code>app/llm/registry.py:410</code>	<code>gw_cls: type[LLMGateway]</code> Internal helper for the construct step.
<code>_kimi_profile</code>	function	<code>app/llm/registry.py:432</code>	 Internal helper for the kimi profile step.
<code>_glm_profile</code>	function	<code>app/llm/registry.py:451</code>	 Internal helper for the glm profile step.
<code>_provider_name_for</code>	function	<code>app/llm/registry.py:482</code>	<code>model: str</code> Internal helper for the provider name for step.
<code>_safe_task_kind</code>	function	<code>app/llm/registry.py:497</code>	<code>method_name: str, kwargs: dict[str, Any], node_type: str None</code> <code>infer_task_kind()</code> , but cost telemetry must never break a call.
<code>local_model_enabled</code>	function	<code>app/llm/registry.py:507</code>	<code>model: str</code> Compute the local model enabled.
<code>configured_local_model_profiles</code>	function	<code>app/llm/registry.py:523</code>	

Symbol	Kind	Location	Signature / summary
			Return health probes for enabled local providers without exposing URLs.
<code>probe_local_model</code>	async function	<code>app/llm/registry.py:536</code>	<code>model: str</code> Probe the local model.
<code>_record_usage</code>	function	<code>app/llm/registry.py:553</code>	<code>intended: str, resolved: str, resp</code> Record token metrics.
<code>_estimate_input_tokens</code>	function	<code>app/llm/registry.py:573</code>	<code>kwargs: dict[str, Any]</code> Cheap pre-call token estimate for deterministic routing.
<code>_is_model_available</code>	function	<code>app/llm/registry.py:593</code>	<code>model: str</code> Return whether auto-routing may safely select "model".
<code>RegistryLLMGateway</code>	class	<code>app/llm/registry.py:622</code>	<code>LLMGateway</code> LLMGateway adapter that dispatches every call through the registry.
<code>RegistryLLMGateway.with_context</code>	method	<code>app/llm/registry.py:695</code>	<code>self, run_id: str, session_id: str, node_id: str, ledger=None, *, event_bus=None, node_type: str None=None, allowed_models: list[str] None=None, routing_policy: dict[str, Any] None=None, semantic_cache=None, use_cache: bool=False, entity_tokenizer=None, collection_id: str='default', processing_mode: str None=None, workflow_name: str None=None</code> Return a context-bound copy for per-node cost tracking and routing.
<code>RegistryLLMGateway.ensure_routed</code>	method	<code>app/llm/registry.py:743</code>	<code>self, *, node_type: str None=None</code> Return a gateway guaranteed to resolve AUTO_MODEL via ModelRouter.
<code>RegistryLLMGateway.with_collection_id</code>	method	<code>app/llm/registry.py:770</code>	<code>self, collection_id: str</code> Return a clone with only collection_id overridden.
<code>RegistryLLMGateway.tokenize_text</code>	method	<code>app/llm/registry.py:793</code>	<code>self, text: str None</code> Tokenize the text.
<code>RegistryLLMGateway.tokenize_value</code>	method	<code>app/llm/registry.py:812</code>	<code>self, value: Any</code> Tokenize the value.
<code>RegistryLLMGateway.tokenize_messages</code>	method	<code>app/llm/registry.py:832</code>	<code>self, messages: list[dict]</code> Tokenize the messages.
<code>RegistryLLMGateway.detokenize_value</code>	method	<code>app/llm/registry.py:861</code>	<code>self, value: Any</code> Detokenize the value.
<code>RegistryLLMGateway.cached_model_access</code>	method	<code>app/llm/registry.py:879</code>	<code>self, model: str</code> Internal helper for the cached model access step.
<code>RegistryLLMGateway.available_for_auto</code>	method	<code>app/llm/registry.py:905</code>	<code>self, model: str</code> Combine configured-provider checks with verified project access.
<code>RegistryLLMGateway.probe_model_access</code>	method	<code>app/llm/registry.py:913</code>	<code>self, models: list[str] tuple[str, ...] set[str]</code> Probe concrete models without creating a completion.
	method	<code>app/llm/registry.py:1017</code>	

Symbol	Kind	Location	Signature / summary
RegistryLLMGateway.selection_history			<code>self</code> Per-call model-selection events recorded on this bound gateway.
RegistryLLMGateway.next_call_id	method	<code>app/llm/registry.py:1025</code>	<code>self</code> Internal helper for the next call id step.
RegistryLLMGateway.select_model	method	<code>app/llm/registry.py:1034</code>	<code>self, *, method_name: str, requested: str, kwargs: dict[str, Any]</code> Deterministic, zero-token model choice for one call.
RegistryLLMGateway.publish	method	<code>app/llm/registry.py:1069</code>	<code>self, evt: RunEvent</code> Publish the result.
RegistryLLMGateway.record_cost	method	<code>app/llm/registry.py:1078</code>	<code>self, intended: str, resolved: str, resp, *, task_type: str='unknown', latency_ms: float None=None, fallback_reason: str None=None, stage: str None=None</code> Write a LedgerEntry if we have context + a ledger.
RegistryLLMGateway.models_for_call	method	<code>app/llm/registry.py:1141</code>	<code>self, intended: str, candidate_models: tuple[str, ...] list[str] None=None</code> Return the permitted, ordered runtime fallback chain.
RegistryLLMGateway.auto_candidates	method	<code>app/llm/registry.py:1206</code>	<code>self</code> Concrete models "auto" may resolve to, best first.
RegistryLLMGateway.delay_for	method	<code>app/llm/registry.py:1212</code>	<code>self, attempt: int, exc: BaseException</code> Internal helper for the delay for step.
RegistryLLMGateway.call_resilient	method	<code>app/llm/registry.py:1240</code>	<code>self, method_name: str, *, intended: str, kwargs: dict[str, Any], candidate_models: tuple[str, ...] list[str] None=None</code> Returns (response, resolved_model, latency_ms, fallback_reason).
RegistryLLMGateway.complete	method	<code>app/llm/registry.py:1381</code>	<code>self, *, model: str, stage: str None=None, **kwargs</code> Public entry point — tokenizes system/user, delegates to the unchanged implementation, then detokenizes the response.
RegistryLLMGateway.complete_impl	method	<code>app/llm/registry.py:1399</code>	<code>self, *, model: str, stage: str None=None, **kwargs</code> Complete the impl.
RegistryLLMGateway.complete_structured	method	<code>app/llm/registry.py:1536</code>	<code>self, *, model: str, stage: str None=None, **kwargs</code> Public entry point — tokenizes system/user, delegates, then detokenizes every string field of the parsed structured result by walking its <code>model_dump()</code> and re-validating.
RegistryLLMGateway.complete_structured_impl	method	<code>app/llm/registry.py:1551</code>	<code>self, *, model: str, stage: str None=None, **kwargs</code> Complete the structured impl.
RegistryLLMGateway.chat_with_tools	method	<code>app/llm/registry.py:1625</code>	<code>self, *, model: str, **kwargs</code> Public entry point — tokenizes system + every message's content/ <code>tool_calls[]</code> .arguments, delegates, then detokenizes text/ reasoning_content/ <code>tool_calls[]</code> .arguments on the response.
	method	<code>app/llm/registry.py:1654</code>	

Symbol	Kind	Location	Signature / summary
RegistryLLMGateway._chat_with_tools_impl			<code>self, *, model: str, **kwargs</code> Internal helper for the chat with tools impl step.
get_llm_gateway	function	app/llm/registry.py:1727	<code></code> Module-level singleton used by main.py's lifespan.
CacheLookup	class	app/llm/semantic_cache.py:29	<code></code> Provides the CacheLookup behaviour.
SemanticLLMCache	class	app/llm/semantic_cache.py:44	<code></code> Small-scope semantic cache backed by Redis.
SemanticLLMCache.probe	method	app/llm/semantic_cache.py:63	<code>self</code> Probe the result.
SemanticLLMCache.get	method	app/llm/semantic_cache.py:71	<code>self, *, session_id: str, model: str, system: str, user: str, temperature: float, max_tokens: int</code> Return the result.
SemanticLLMCache.put	method	app/llm/semantic_cache.py:158	<code>self, *, session_id: str, model: str, system: str, user: str, temperature: float, max_tokens: int, response: dict[str, Any], query_embedding: list[float] None</code> Store the result.
SemanticLLMCache.prune	method	app/llm/semantic_cache.py:216	<code>self, scope: str</code> Prune the result.
SemanticLLMCache.scope	method	app/llm/semantic_cache.py:235	<code>*, session_id: str, model: str, system: str, temperature: float, max_tokens: int</code> Internal helper for the scope step.
SemanticLLMCache._entry_key	method	app/llm/semantic_cache.py:267	<code>scope: str, user: str</code> Internal helper for the entry key step.
SemanticLLMCache._index_key	method	app/llm/semantic_cache.py:281	<code>scope: str</code> Internal helper for the index key step.
_cosine	function	app/llm/semantic_cache.py:293	<code>left: list[float], right: list[float]</code> Internal helper for the cosine step.
lifespan	async function	app/main.py:104	<code>app: FastAPI</code> Compute the lifespan.
connect	function	app/mcp/business_records/db.py:21	<code>app_settings: Settings=settings</code> Compute the connect.
connect_readonly	function	app/mcp/business_records/db.py:47	<code>app_settings: Settings=settings</code> A genuinely lower-privileged connection for query_readonly (app/nodes/sql_query.py) — GRANT SELECT only (see seed.py's ensure_readonly_user).
execute	function	app/mcp/business_records/db.py:66	<code>conn: MySQLConnectionAbstract, query: str, params: tuple[Any, ...]=()</code> Run a parameterized SELECT and return every row as a dict.
execute_write	function	app/mcp/business_records/db.py:78	<code>conn: MySQLConnectionAbstract, query: str, params: tuple[Any, ...]=()</code> Run a parameterized INSERT/UPDATE, commit, and return the affected/ inserted row count (or 'lastrowid' for an auto-increment insert).

Symbol	Kind	Location	Signature / summary
<code>_collection</code>	function	<code>app/mcp/business_records/handlers.py:19</code>	<code>rows: list[dict[str, Any]], key: str, limit: int</code> Internal helper for the collection step.
<code>_limit</code>	function	<code>app/mcp/business_records/handlers.py:34</code>	<code>arguments: dict[str, Any], default: int, maximum: int</code> Internal helper for the limit step.
<code>customer_search</code>	async function	<code>app/mcp/business_records/handlers.py:55</code>	<code>conn: MySQLConnectionAbstract, arguments: dict[str, Any]</code> Compute the customer search.
<code>order_search</code>	async function	<code>app/mcp/business_records/handlers.py:99</code>	<code>conn: MySQLConnectionAbstract, arguments: dict[str, Any]</code> Compute the order search.
<code>inventory_check</code>	async function	<code>app/mcp/business_records/handlers.py:155</code>	<code>conn: MySQLConnectionAbstract, arguments: dict[str, Any]</code> Compute the inventory check.
<code>product_search</code>	async function	<code>app/mcp/business_records/handlers.py:185</code>	<code>conn: MySQLConnectionAbstract, arguments: dict[str, Any]</code> Compute the product search.
<code>query_readonly</code>	async function	<code>app/mcp/business_records/handlers.py:244</code>	<code>conn: MySQLConnectionAbstract, arguments: dict[str, Any]</code> Deliberately ignores `conn` — the shared module-level connection (<code>server.py</code> 's <code>backend()</code>) authenticates as the full-privilege app user.
<code>create_case</code>	async function	<code>app/mcp/business_records/handlers.py:269</code>	<code>conn: MySQLConnectionAbstract, arguments: dict[str, Any]</code> Create the case.
<code>create_opportunity</code>	async function	<code>app/mcp/business_records/handlers.py:298</code>	<code>conn: MySQLConnectionAbstract, arguments: dict[str, Any]</code> Create the opportunity.
<code>create_order</code>	async function	<code>app/mcp/business_records/handlers.py:326</code>	<code>conn: MySQLConnectionAbstract, arguments: dict[str, Any]</code> Create the order.
<code>_order_view</code>	function	<code>app/mcp/business_records/handlers.py:355</code>	<code>row: dict[str, Any]</code> Normalizes a raw `crm_salesorders` row to <code>create_order</code> 's field names — a workflow mapping `order_number`/`account_id` after an update must not have to switch to `accountid` because this tool happened to read the row back from SQL instead of building it in Python.
<code>update_order</code>	async function	<code>app/mcp/business_records/handlers.py:371</code>	<code>conn: MySQLConnectionAbstract, arguments: dict[str, Any]</code> Update the order.
<code>_case_view</code>	function	<code>app/mcp/business_records/handlers.py:406</code>	<code>row: dict[str, Any]</code> Normalizes a raw `crm_service_cases` row to <code>create_case</code> 's field names — see <code>_order_view</code> for why this matters.
<code>update_case</code>	async function	<code>app/mcp/business_records/handlers.py:420</code>	<code>conn: MySQLConnectionAbstract, arguments: dict[str, Any]</code> Update the case.
<code>_next_suffix</code>	function	<code>app/mcp/business_records/handlers.py:455</code>	<code>conn: MySQLConnectionAbstract, table: str, column: str, prefix: str</code> A small counter derived from how many rows already

Symbol	Kind	Location	Signature / summary
			start with `prefix` — good enough for demo-scale seeded data; a real deployment would use the source system's own id generation instead of this.
<code>_load</code>	function	<code>app/mcp/business_records/seed.py:27</code>	<code>path: Path</code> Load the result.
<code>_dt</code>	function	<code>app/mcp/business_records/seed.py:39</code>	<code>value: str None</code> ISO 8601 ("...T...Z") -> MySQL DATETIME literal ("...
<code>_upsert</code>	function	<code>app/mcp/business_records/seed.py:51</code>	<code>conn, table: str, row: dict[str, Any]</code> Upsert the result.
<code>ensure_readonly_user</code>	function	<code>app/mcp/business_records/seed.py:69</code>	<code>app_settings: Any=settings</code> Create (idempotently) the genuinely lower-privileged credential query_readonly runs under (app/mcp/business_records/db.py's connect_readonly) — GRANT SELECT only, no INSERT/UPDATE/DELETE/DDL.
<code>apply_schema</code>	function	<code>app/mcp/business_records/seed.py:112</code>	<code>conn</code> Apply the schema.
<code>seed_d365_finance</code>	function	<code>app/mcp/business_records/seed.py:138</code>	<code>conn</code> Seed the d365 finance.
<code>seed_dynamics_crm</code>	function	<code>app/mcp/business_records/seed.py:278</code>	<code>conn</code> Seed the dynamics crm.
<code>seed_all</code>	function	<code>app/mcp/business_records/seed.py:449</code>	<code>app_settings=settings</code> Seed the all.
<code>backend</code>	function	<code>app/mcp/business_records/server.py:45</code>	<code></code> Compute the backend.
<code>list_tools</code>	async function	<code>app/mcp/business_records/server.py:59</code>	<code></code> List the tools.
<code>call_tool</code>	async function	<code>app/mcp/business_records/server.py:91</code>	<code>name: str, arguments: dict[str, Any] None</code> Compute the call tool.
<code>_error_result</code>	function	<code>app/mcp/business_records/server.py:165</code>	<code>error: dict[str, Any]</code> Internal helper for the error result step.
<code>main</code>	async function	<code>app/mcp/business_records/server.py:182</code>	<code></code> Compute the main.
<code>SQLGuardError</code>	class	<code>app/mcp/business_records/sql_guard.py:50</code>	<code>RuntimeError</code> Exception raised for the SQLGuardError case.
<code>classify_sql_verb</code>	function	<code>app/mcp/business_records/sql_guard.py:63</code>	<code>sql: str</code> "read" or "write", from the statement's first keyword alone.
<code>run_readonly_query</code>	async function	<code>app/mcp/business_records/sql_guard.py:80</code>	<code>sql: str, params: dict[str, Any], *, connect: Any, max_rows: int, timeout_seconds: float</code> Run one read-only query with every layer above applied.
<code>_kill_query</code>	async function	<code>app/mcp/business_records/sql_guard.py:146</code>	<code>connect: Any, connection_id: int</code> A second, short-lived connection to cancel the first's running query — verified this actually stops a running SELECT SLEEP() and leaves the original connection usable (which is then closed anyway by

Symbol	Kind	Location	Signature / summary
			run_readonly_query's own finally, since its result is now meaningless).
<code>_string</code>	function	<code>app/mcp/business_records/tools.py:18</code>	<code>description: str, **extra: Any</code> Internal helper for the string step.
<code>_collection</code>	function	<code>app/mcp/business_records/tools.py:31</code>	<code>key: str, item_schema: dict[str, Any]</code> Every collection-returning tool shares this envelope.
<code>_customer_item</code>	function	<code>app/mcp/business_records/tools.py:55</code>	<code></code> Internal helper for the customer item step.
<code>_order_item</code>	function	<code>app/mcp/business_records/tools.py:85</code>	<code></code> Internal helper for the order item step.
<code>_inventory_item</code>	function	<code>app/mcp/business_records/tools.py:104</code>	<code></code> Internal helper for the inventory item step.
<code>_product_item</code>	function	<code>app/mcp/business_records/tools.py:124</code>	<code></code> Internal helper for the product item step.
<code>_case_record</code>	function	<code>app/mcp/business_records/tools.py:147</code>	<code></code> Internal helper for the case record step.
<code>_opportunity_record</code>	function	<code>app/mcp/business_records/tools.py:171</code>	<code></code> Internal helper for the opportunity record step.
<code>_order_record</code>	function	<code>app/mcp/business_records/tools.py:192</code>	<code></code> Internal helper for the order record step.
<code>_paper_search_mcp_env</code>	function	<code>app/mcp/client.py:43</code>	<code>app_settings: Settings</code> Research credentials for the paper-search-mcp subprocess.
<code>_dynamics_env</code>	function	<code>app/mcp/client.py:79</code>	<code>app_settings: Settings</code> Environment for the Dynamics 365 MCP subprocess.
<code>_finance_scm_spec</code>	function	<code>app/mcp/client.py:102</code>	<code>app_settings: Settings</code> Launch spec for the Dynamics 365 Finance & Operations MCP server.
<code>_business_records_env</code>	function	<code>app/mcp/client.py:142</code>	<code>app_settings: Settings</code> Environment for the Business Records MCP subprocess.
<code>build_server_specs</code>	function	<code>app/mcp/client.py:159</code>	<code>app_settings: Settings=settings</code> Build portable MCP launch specs from application configuration.
<code>MCPClient</code>	class	<code>app/mcp/client.py:224</code>	<code></code> Multi-server MCP wrapper.
<code>MCPClient.configured_servers</code>	method	<code>app/mcp/client.py:259</code>	<code>self</code> The configured servers.
<code>MCPClient.running_servers</code>	method	<code>app/mcp/client.py:264</code>	<code>self</code> The running servers.
<code>MCPClient.start</code>	method	<code>app/mcp/client.py:268</code>	<code>self, servers: list[str] None=None</code> Launch the requested servers (default: all declared).
<code>MCPClient.stop</code>	method	<code>app/mcp/client.py:309</code>	<code>self</code> Stop the result.
<code>MCPClient._require</code>	method	<code>app/mcp/client.py:328</code>	<code>self, server: str</code> Internal helper for the require step.

Symbol	Kind	Location	Signature / summary
MCPClient.has_server	method	app/mcp/client.py:345	<code>self, server: str</code> Return whether server.
MCPClient.list_tools	method	app/mcp/client.py:356	<code>self, server: str=DEFAULT_SERVER</code> List the tools.
MCPClient.probe	method	app/mcp/client.py:371	<code>self, server: str=DEFAULT_SERVER</code> Perform a live MCP request instead of trusting a cached session.
MCPClient.call_tool	method	app/mcp/client.py:379	<code>self, name: str, arguments: dict, server: str=DEFAULT_SERVER</code> Call a tool on `server` (default: primary).
MCPClient.call_tool_raw	method	app/mcp/client.py:390	<code>self, name: str, arguments: dict, *, server: str=DEFAULT_SERVER, timeout_seconds: float None=None</code> Call a tool and return the full CallToolResult.
launch_mcp_session	async function	app/mcp/client.py:416	<code>servers: list[str] None=None</code> Convenience used by app/main.py at FastAPI startup.
_dynamics_tool_policies	function	app/mcp/connections.py:54	<code></code> State each Dynamics tool's class as deployment policy.
dynamics_connection	function	app/mcp/connections.py:71	<code>app_settings: Settings=settings</code> Compute the dynamics connection.
read_only_dynamics_connection	function	app/mcp/connections.py:113	<code>app_settings: Settings=settings</code> A least-privilege variant, for deployments that want CRM context only.
finance_scm_connection	function	app/mcp/connections.py:134	<code>app_settings: Settings=settings</code> The Dynamics 365 Finance & Operations connector.
business_records_connection	function	app/mcp/connections.py:211	<code>app_settings: Settings=settings</code> A real, persistent MySQL database — genuine Lookup/Create/Update tools.
build_registry	function	app/mcp/connections.py:252	<code>app_settings: Settings=settings</code> Assemble every configured connection.
build_mcp_service	function	app/mcp/connections.py:306	<code>*, client: Any, db: Any=None, app_settings: Settings=settings</code> Build the mcp service.
_collection	function	app/mcp/d365_finance/handlers.py:19	<code>rows: list[dict[str, Any]], key: str, limit: int</code> Internal helper for the collection step.
_limit	function	app/mcp/d365_finance/handlers.py:34	<code>arguments: dict[str, Any], default: int, maximum: int</code> Internal helper for the limit step.
_any_of_present	function	app/mcp/d365_finance/handlers.py:53	<code>*filters: str None</code> Internal helper for the any of present step.
_priority_filter	function	app/mcp/d365_finance/handlers.py:66	<code>*fields: tuple[str, str None]</code> Search by the first-given reference only.
_customer	function	app/mcp/d365_finance/handlers.py:90	<code>row: dict[str, Any]</code> Internal helper for the customer step.
find_customer	async function	app/mcp/d365_finance/handlers.py:108	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Search customers by name — a customer writing

Symbol	Kind	Location	Signature / summary
			'Meridian' should find 'Meridian Process Systems', same contains-or-exact convention as the CRM connector's find_account.
find_account_ownership	async function	app/mcp/d365_finance/handlers.py:133	backend: DynamicsBackend, arguments: dict[str, Any] Resolve account ownership to people who can actually be assigned today.
find_credit_status	async function	app/mcp/d365_finance/handlers.py:186	backend: DynamicsBackend, arguments: dict[str, Any] Find the credit status.
_quote	function	app/mcp/d365_finance/handlers.py:211	row: dict[str, Any], *, account_id: str, purchase_order_number: str Internal helper for the quote step.
find_quote	async function	app/mcp/d365_finance/handlers.py:246	backend: DynamicsBackend, arguments: dict[str, Any] Find a quotation by quotation number or purchase-order number.
_order_filter	function	app/mcp/d365_finance/handlers.py:281	arguments: dict[str, Any] Customers quote their own purchase-order number at least as often as our sales-order number, and a workflow may only have one ambiguous reference field to give us — search whichever value(s) were supplied against *both* columns, since which kind of reference it is often is not knowable in advance.
_sales_order	function	app/mcp/d365_finance/handlers.py:303	row: dict[str, Any] Internal helper for the sales order step.
find_sales_order	async function	app/mcp/d365_finance/handlers.py:323	backend: DynamicsBackend, arguments: dict[str, Any] Find the sales order.
find_order_fulfilment_statuses	async function	app/mcp/d365_finance/handlers.py:344	backend: DynamicsBackend, arguments: dict[str, Any] Ask why an order is where it is — the customer's wording is identical for a material shortage and a quality hold; this is what tells them apart.
_availability	function	app/mcp/d365_finance/handlers.py:369	row: dict[str, Any] Internal helper for the availability step.
_resolve_pump_models	async function	app/mcp/d365_finance/handlers.py:385	backend: DynamicsBackend, arguments: dict[str, Any] product_name/product_family describe a catalogue product, not an inventory record directly — resolve to the pump model(s) inventory is actually keyed by before querying availability.
find_inventory_availability	async function	app/mcp/d365_finance/handlers.py:406	backend: DynamicsBackend, arguments: dict[str, Any] Find the inventory availability.
find_installed_unit	async function	app/mcp/d365_finance/handlers.py:446	backend: DynamicsBackend, arguments: dict[str, Any] Resolve installed equipment by serial, model, manufacturer or site.
_shipment	function	app/mcp/d365_finance/handlers.py:499	row: dict[str, Any] Internal helper for the shipment step.
find_shipment			

Symbol	Kind	Location	Signature / summary
	async function	app/mcp/d365_finance/handlers.py:518	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Find the shipment.
<code>_invoice</code>	function	app/mcp/d365_finance/handlers.py:549	<code>row: dict[str, Any]</code> Internal helper for the invoice step.
<code>find_invoice</code>	async function	app/mcp/d365_finance/handlers.py:568	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Find the invoice.
<code>_contract</code>	function	app/mcp/d365_finance/handlers.py:598	<code>row: dict[str, Any]</code> Internal helper for the contract step.
<code>find_contract</code>	async function	app/mcp/d365_finance/handlers.py:616	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Find the contract.
<code>_product</code>	function	app/mcp/d365_finance/handlers.py:644	<code>row: dict[str, Any]</code> Internal helper for the product step.
<code>find_products</code>	async function	app/mcp/d365_finance/handlers.py:666	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Find the products.
<code>backend</code>	function	app/mcp/d365_finance/server.py:53	<code></code> Compute the backend.
<code>list_tools</code>	async function	app/mcp/d365_finance/server.py:67	<code></code> List the tools.
<code>call_tool</code>	async function	app/mcp/d365_finance/server.py:99	<code>name: str, arguments: dict[str, Any] None</code> Compute the call tool.
<code>_error_result</code>	function	app/mcp/d365_finance/server.py:149	<code>error: dict[str, Any]</code> Internal helper for the error result step.
<code>main</code>	async function	app/mcp/d365_finance/server.py:166	<code></code> Compute the main.
<code>_string</code>	function	app/mcp/d365_finance/tools.py:12	<code>description: str, **extra: Any</code> Internal helper for the string step.
<code>_collection</code>	function	app/mcp/d365_finance/tools.py:25	<code>key: str</code> Internal helper for the collection step.
<code>DynamicsError</code>	class	app/mcp/dynamics/client.py:45	<code>RuntimeError</code> A CRM call failed in a way worth describing to a workflow author.
<code>DynamicsError.as_payload</code>	method	app/mcp/dynamics/client.py:69	<code>self</code> Compute the as payload.
<code>DynamicsAuthError</code>	class	app/mcp/dynamics/client.py:83	<code>DynamicsError</code> Exception raised for the DynamicsAuthError case.
<code>DynamicsBackend</code>	class	app/mcp/dynamics/client.py:102	<code>ABC</code> What the MCP server needs from a CRM backend.
<code>DynamicsBackend.whoami</code>	method	app/mcp/dynamics/client.py:108	<code>self</code> Compute the whoami.
<code>DynamicsBackend.query</code>	method	app/mcp/dynamics/client.py:117	<code>self, entity_set: str, *, select: list[str], filter_expression: str None=None, order_by: str None=None, top: int None=None, expand: str </code>

Symbol	Kind	Location	Signature / summary
			<code>None=None</code> Query the result.
DynamicsBackend.get	method	<code>app/mcp/dynamics/client.py:143</code>	<code>self, entity_set: str, record_id: str, *, select: list[str]</code> Return the result.
DynamicsBackend.create	method	<code>app/mcp/dynamics/client.py:159</code>	<code>self, entity_set: str, payload: dict[str, Any]</code> Create the result.
DynamicsBackend.update	method	<code>app/mcp/dynamics/client.py:172</code>	<code>self, entity_set: str, record_id: str, payload: dict[str, Any]</code> Update the result.
DynamicsBackend.close	method	<code>app/mcp/dynamics/client.py:184</code>	<code>self</code> Close the result.
DataverseClient	class	<code>app/mcp/dynamics/client.py:189</code>	<code>DynamicsBackend</code> Live Dynamics 365 over the Dataverse Web API.
DataverseClient._http	method	<code>app/mcp/dynamics/client.py:219</code>	<code>self</code> Internal helper for the http step.
DataverseClient.close	method	<code>app/mcp/dynamics/client.py:229</code>	<code>self</code> Close the result.
DataverseClient._access_token	method	<code>app/mcp/dynamics/client.py:237</code>	<code>self</code> Client-credentials token for the organisation's `.default` scope.
DataverseClient._headers	method	<code>app/mcp/dynamics/client.py:292</code>	<code>self, *, write: bool=False</code> Internal helper for the headers step.
DataverseClient._url	method	<code>app/mcp/dynamics/client.py:316</code>	<code>self, path: str, params: dict[str, str] None=None</code> Internal helper for the url step.
DataverseClient._request	method	<code>app/mcp/dynamics/client.py:331</code>	<code>self, method: str, path: str, *, params: dict[str, str] None=None, json_body: dict[str, Any] None=None, write: bool=False</code> Internal helper for the request step.
DataverseClient.whoami	method	<code>app/mcp/dynamics/client.py:398</code>	<code>self</code> Compute the whoami.
DataverseClient.query	method	<code>app/mcp/dynamics/client.py:420</code>	<code>self, entity_set: str, *, select: list[str], filter_expression: str None=None, order_by: str None=None, top: int None=None, expand: str None=None</code> Query the result.
DataverseClient.get	method	<code>app/mcp/dynamics/client.py:455</code>	<code>self, entity_set: str, record_id: str, *, select: list[str]</code> Return the result.
DataverseClient.create	method	<code>app/mcp/dynamics/client.py:479</code>	<code>self, entity_set: str, payload: dict[str, Any]</code> Create the result.
DataverseClient.update	method	<code>app/mcp/dynamics/client.py:508</code>	<code>self, entity_set: str, record_id: str, payload: dict[str, Any]</code> Update the result.

Symbol	Kind	Location	Signature / summary
<code>_safe_error_text</code>	function	<code>app/mcp/dynamics/client.py:526</code>	<code>response: httpx.Response</code> Extract Dataverse's error message without echoing an entire payload.
<code>FixtureBackend</code>	class	<code>app/mcp/dynamics/client.py:538</code>	<code>DynamicsBackend</code> The same contracts, served from a fixture file.
<code>FixtureBackend.from_file</code>	method	<code>app/mcp/dynamics/client.py:564</code>	<code>cls, path: str Path</code> Compute the from file.
<code>FixtureBackend._next_id</code>	method	<code>app/mcp/dynamics/client.py:576</code>	<code>self, prefix: str</code> Internal helper for the next id step.
<code>FixtureBackend.whoami</code>	method	<code>app/mcp/dynamics/client.py:590</code>	<code>self</code> Compute the whoami.
<code>FixtureBackend.query</code>	method	<code>app/mcp/dynamics/client.py:603</code>	<code>self, entity_set: str, *, select: list[str], filter_expression: str None=None, order_by: str None=None, top: int None=None, expand: str None=None</code> Query the result.
<code>FixtureBackend.get</code>	method	<code>app/mcp/dynamics/client.py:640</code>	<code>self, entity_set: str, record_id: str, *, select: list[str]</code> Return the result.
<code>FixtureBackend.create</code>	method	<code>app/mcp/dynamics/client.py:659</code>	<code>self, entity_set: str, payload: dict[str, Any]</code> Create the result.
<code>FixtureBackend.update</code>	method	<code>app/mcp/dynamics/client.py:678</code>	<code>self, entity_set: str, record_id: str, payload: dict[str, Any]</code> Update the result.
<code>_project</code>	function	<code>app/mcp/dynamics/client.py:709</code>	<code>row: dict[str, Any], select: list[str]</code> Mimic '\$select': a fixture row returns only the requested columns.
<code>_fixture_matches</code>	function	<code>app/mcp/dynamics/client.py:716</code>	<code>row: dict[str, Any], expression: str</code> Interpret the small OData subset this package generates.
<code>_split_top_level</code>	function	<code>app/mcp/dynamics/client.py:775</code>	<code>expression: str, joiner: str</code> Split on a joiner that is not inside parentheses or a string literal.
<code>_account</code>	function	<code>app/mcp/dynamics/handlers.py:106</code>	<code>row: dict[str, Any]</code> Internal helper for the account step.
<code>_contact</code>	function	<code>app/mcp/dynamics/handlers.py:128</code>	<code>row: dict[str, Any]</code> Internal helper for the contact step.
<code>_opportunity</code>	function	<code>app/mcp/dynamics/handlers.py:147</code>	<code>row: dict[str, Any]</code> Internal helper for the opportunity step.
<code>_order</code>	function	<code>app/mcp/dynamics/handlers.py:167</code>	<code>row: dict[str, Any]</code> Internal helper for the order step.
<code>_quote</code>	function	<code>app/mcp/dynamics/handlers.py:196</code>	<code>row: dict[str, Any]</code> Map a Dataverse quotation row onto business field names.
<code>_product</code>	function	<code>app/mcp/dynamics/handlers.py:213</code>	<code>row: dict[str, Any]</code> Internal helper for the product step.

Symbol	Kind	Location	Signature / summary
<code>_activity</code>	function	<code>app/mcp/dynamics/handlers.py:230</code>	<code>row: dict[str, Any]</code> Internal helper for the activity step.
<code>_shipment</code>	function	<code>app/mcp/dynamics/handlers.py:248</code>	<code>row: dict[str, Any]</code> Internal helper for the shipment step.
<code>_service_case</code>	function	<code>app/mcp/dynamics/handlers.py:268</code>	<code>row: dict[str, Any]</code> Internal helper for the service case step.
<code>_collection</code>	function	<code>app/mcp/dynamics/handlers.py:292</code>	<code>rows: list[dict[str, Any]], key: str, limit: int</code> Wrap results in the shared envelope.
<code>_limit</code>	function	<code>app/mcp/dynamics/handlers.py:306</code>	<code>arguments: dict[str, Any], default: int, maximum: int</code> Internal helper for the limit step.
<code>get_current_user</code>	async function	<code>app/mcp/dynamics/handlers.py:329</code>	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Return the current user.
<code>find_account</code>	async function	<code>app/mcp/dynamics/handlers.py:345</code>	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Find the account.
<code>get_account</code>	async function	<code>app/mcp/dynamics/handlers.py:375</code>	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Return the account.
<code>find_contact</code>	async function	<code>app/mcp/dynamics/handlers.py:399</code>	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Find the contact.
<code>get_contacts_for_account</code>	async function	<code>app/mcp/dynamics/handlers.py:436</code>	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Return the contacts for account.
<code>get_open_opportunities</code>	async function	<code>app/mcp/dynamics/handlers.py:461</code>	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Return the open opportunities.
<code>get_quotations_for_account</code>	async function	<code>app/mcp/dynamics/handlers.py:489</code>	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> List the stored quotations belonging to one CRM account.
<code>find_quotation</code>	async function	<code>app/mcp/dynamics/handlers.py:511</code>	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Find one quotation by its customer-facing number.
<code>find_previous_orders</code>	async function	<code>app/mcp/dynamics/handlers.py:542</code>	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Find the previous orders.
<code>find_order</code>	async function	<code>app/mcp/dynamics/handlers.py:567</code>	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Find one sales order by order number or equipment serial number.
<code>get_shipments_for_account</code>	async function	<code>app/mcp/dynamics/handlers.py:612</code>	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> List the stored shipment records belonging to one CRM account.
<code>find_product</code>	async function	<code>app/mcp/dynamics/handlers.py:629</code>	

Symbol	Kind	Location	Signature / summary
			<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Find the product.
find_shipment	async function	app/mcp/dynamics/handlers.py:656	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Find the shipment.
find_service_case	async function	app/mcp/dynamics/handlers.py:697	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Find the service case.
get_service_cases_for_account	async function	app/mcp/dynamics/handlers.py:740	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Return the service cases for account.
get_recent_activities	async function	app/mcp/dynamics/handlers.py:765	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Return the recent activities.
_text	function	app/mcp/dynamics/handlers.py:794	<code>arguments: dict[str, Any], key: str, limit: int</code> Read an optional write field, bounded.
create_lead	async function	app/mcp/dynamics/handlers.py:817	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Create the lead.
create_followup_activity	async function	app/mcp/dynamics/handlers.py:853	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Create the followup activity.
update_account_contact_details	async function	app/mcp/dynamics/handlers.py:901	<code>backend: DynamicsBackend, arguments: dict[str, Any]</code> Update the account contact details.
ODataValueError	class	app/mcp/dynamics/odata.py:43	<code>ValueError</code> A caller-supplied value cannot be used safely in a query.
guid	function	app/mcp/dynamics/odata.py:47	<code>value: Any, *, field: str='id'</code> Validate an identifier as a GUID and return its canonical form.
escape_literal	function	app/mcp/dynamics/odata.py:62	<code>value: Any, *, field: str='value'</code> Escape a string for use inside an OData string literal.
string_filter	function	app/mcp/dynamics/odata.py:84	<code>field: str, value: str</code> `field eq 'value'` with the value escaped.
contains_filter	function	app/mcp/dynamics/odata.py:89	<code>field: str, value: str</code> `contains(field,'value')` with the value escaped.
lookup_filter	function	app/mcp/dynamics/odata.py:94	<code>field: str, record_id: str</code> `_lookup_value eq <guid>` — the case the reference implementation gets wrong.
any_of	function	app/mcp/dynamics/odata.py:101	<code>*filters: str</code> OR the given filters, parenthesised so precedence cannot surprise.
all_of	function	app/mcp/dynamics/odata.py:111	<code>*filters: str</code> Compute the all of.
entity_path	function	app/mcp/dynamics/odata.py:128	

Symbol	Kind	Location	Signature / summary
			<code>entity_set: str, record_id: str None=None</code> Build an entity path, validating the key predicate.
build_query	function	<code>app/mcp/dynamics/odata.py:142</code>	<code>*, select: list[str] None=None, filter_expression: str None=None, order_by: str None=None, top: int None=None, expand: str None=None</code> Assemble OData system query options.
_column	function	<code>app/mcp/dynamics/odata.py:176</code>	<code>name: str</code> Internal helper for the column step.
_order_by	function	<code>app/mcp/dynamics/odata.py:190</code>	<code>clause: str</code> Internal helper for the order by step.
encode_params	function	<code>app/mcp/dynamics/odata.py:208</code>	<code>params: dict[str, str]</code> Percent-encode query values while leaving OData's own syntax readable.
build_backend	function	<code>app/mcp/dynamics/server.py:72</code>	 Choose a backend from the environment.
backend	function	<code>app/mcp/dynamics/server.py:101</code>	 Compute the backend.
_annotations	function	<code>app/mcp/dynamics/server.py:113</code>	<code>definition: dict[str, Any]</code> Declare the tool's nature honestly.
list_tools	async function	<code>app/mcp/dynamics/server.py:136</code>	 List the tools.
call_tool	async function	<code>app/mcp/dynamics/server.py:164</code>	<code>name: str, arguments: dict[str, Any] None</code> Execute one tool.
_error_result	function	<code>app/mcp/dynamics/server.py:236</code>	<code>error: dict[str, Any]</code> Internal helper for the error result step.
main	async function	<code>app/mcp/dynamics/server.py:257</code>	 Compute the main.
_string	function	<code>app/mcp/dynamics/tools.py:55</code>	<code>description: str, **extra: Any</code> Internal helper for the string step.
_account_summary	function	<code>app/mcp/dynamics/tools.py:68</code>	 Internal helper for the account summary step.
_contact_summary	function	<code>app/mcp/dynamics/tools.py:106</code>	 Internal helper for the contact summary step.
_opportunity_summary	function	<code>app/mcp/dynamics/tools.py:126</code>	 Internal helper for the opportunity summary step.
_product_summary	function	<code>app/mcp/dynamics/tools.py:152</code>	 Internal helper for the product summary step.
_order_summary	function	<code>app/mcp/dynamics/tools.py:170</code>	 Internal helper for the order summary step.
_activity_summary	function	<code>app/mcp/dynamics/tools.py:211</code>	 Internal helper for the activity summary step.
_owner_summary	function	<code>app/mcp/dynamics/tools.py:231</code>	 Internal helper for the owner summary step.
_quote_summary	function		

Symbol	Kind	Location	Signature / summary
		app/mcp/dynamics/tools.py:249	Internal helper for the quote summary step.
<code>_shipment_summary</code>	function	app/mcp/dynamics/tools.py:269	Internal helper for the shipment summary step.
<code>_service_case_summary</code>	function	app/mcp/dynamics/tools.py:290	Internal helper for the service case summary step.
<code>_collection</code>	function	app/mcp/dynamics/tools.py:315	<code>item_schema: dict[str, Any], key: str</code> Every collection-returning tool shares this envelope.
<code>_prepare_source_checkout</code>	function	app/mcp/paper_search_server.py:49	Internal helper for the prepare source checkout step.
<code>inject_openalex_api_key</code>	function	app/mcp/paper_search_server.py:59	<code>searcher: Any, api_key: str</code> Inject the key into a searcher session without logging the secret.
<code>_normalize_openaire_legacy_json</code>	function	app/mcp/paper_search_server.py:71	<code>payload: Any</code> Guard against <code>"response": null / "results": null</code> in the OpenAIRE legacy search payload, which otherwise crashes the caller's <code>`data.get('response', {}).get('results', {}).get('result', [])`</code> chain (a present <code>`None`</code> value bypasses <code>`dict.get`</code> 's default).
<code>_JsonNullSafeOpenAireResponse</code>	class	app/mcp/paper_search_server.py:87	Wraps a <code>requests.Response</code> so <code>`json()`</code> never nests <code>`None`</code> under <code>`response`/`results`</code> for the OpenAIRE legacy search endpoint.
<code>_JsonNullSafeOpenAireResponse.json</code>	method	app/mcp/paper_search_server.py:110	<code>self, *args: Any, **kwargs: Any</code> Compute the json.
<code>patch_openaire_query_param</code>	function	app/mcp/paper_search_server.py:123	<code>module_name: str='paper_search_mcp.academic_platforms.openaire'</code> Rename the legacy fallback's <code>`query`</code> param to the supported <code>`keywords`</code> , and normalize its response so a null <code>`response`/`results`</code> doesn't raise in the caller's <code>`get()`</code> chain.
<code>_JsonDefaultsDataKey</code>	class	app/mcp/paper_search_server.py:157	Wraps a <code>requests.Response</code> so <code>`json()`</code> always has a <code>`data`</code> key.
<code>_JsonDefaultsDataKey.json</code>	method	app/mcp/paper_search_server.py:179	<code>self, *args: Any, **kwargs: Any</code> Compute the json.
<code>_wait_for_semantic_scholar_rate_limit</code>	function	app/mcp/paper_search_server.py:199	<code>min_interval_seconds: float</code> Block until at least <code>`min_interval_seconds`</code> has passed since the last call.
<code>patch_semantic_scholar_empty_results</code>	function	app/mcp/paper_search_server.py:214	<code>module_name: str='paper_search_mcp.academic_platforms.semantic', min_interval_seconds: float=1.1</code> Make zero-result Semantic Scholar responses parse instead of <code>KeyError</code> , and pace requests to stay under the 1 req/s cumulative keyed rate limit.
<code>main</code>	function	app/mcp/paper_search_server.py:246	Compute the main.
<code>MCPPolicyError</code>	class	app/mcp/policy.py:63	<code>RuntimeError</code> A tool call was refused by policy.

Symbol	Kind	Location	Signature / summary
MCPApprovalRequired	class	app/mcp/policy.py:78	MCPPolicyError A write was refused because no human review is guaranteed before it.
_words	function	app/mcp/policy.py:95	tool_name: str Split a tool name into words across the naming styles servers use: `get-associated-opportunities`, `get_associated_opportunities`, `getAssociatedOpportunities`.
classify_by_name	function	app/mcp/policy.py:103	tool_name: str Best-effort classification from the tool's name alone.
classify_by_annotations	function	app/mcp/policy.py:123	annotations: Any Read the server's own hints, if it declared any.
classify_tool	function	app/mcp/policy.py:151	tool_name: str, *, connection: MCPServerConnection None=None, annotations: Any=None The platform's classification of one tool.
is_write	function	app/mcp/policy.py:177	operation: OperationClass Anything not provably a read is treated as a write.
PolicyDecision	class	app/mcp/policy.py:187	 The outcome of the gate, kept as data so it can be logged, shown in the Builder, and asserted on in tests.
PolicyDecision.raise_if_denied	method	app/mcp/policy.py:197	self Raise the if denied.
evaluate	function	app/mcp/policy.py:206	*, connection: MCPServerConnection, tool_name: str, annotations: Any=None, user_role: str None=None, approval_satisfied: bool=False Run the gate for one tool call.
MCPToolPolicy	class	app/mcp/registry.py:53	BaseModel Per-tool overrides, set by the deployment rather than by the server.
MCPServerConnection	class	app/mcp/registry.py:74	BaseModel One configured MCP server.
MCPServerConnection.id_is_safe	method	app/mcp/registry.py:122	cls, value: str Compute the id is safe.
MCPServerConnection.stdio_needs_a_command	method	app/mcp/registry.py:138	self Compute the stdio needs a command.
MCPServerConnection.secrets_are_references	method	app/mcp/registry.py:149	self Reject a literal secret pasted where a reference belongs.
MCPServerConnection.label	method	app/mcp/registry.py:166	self The label.
MCPServerConnection.resolve_environment	method	app/mcp/registry.py:170	self Build the subprocess environment, reading secrets at launch time.
MCPServerConnection.describe	method	app/mcp/registry.py:192	self Builder-facing description.
MCPServerConnection.permits_tool	method	app/mcp/registry.py:215	self, tool_name: str Compute the permits tool.

Symbol	Kind	Location	Signature / summary
<code>MCPServerConnection.policy_for</code>	method	<code>app/mcp/registry.py:230</code>	<code>self, tool_name: str</code> Compute the policy for.
<code>MCPServerRegistry</code>	class	<code>app/mcp/registry.py:242</code>	<code></code> The set of configured servers, keyed by id.
<code>MCPServerRegistry.ids</code>	method	<code>app/mcp/registry.py:274</code>	<code>self</code> The ids.
<code>MCPServerRegistry.add</code>	method	<code>app/mcp/registry.py:278</code>	<code>self, connection: MCPServerConnection</code> Add the result.
<code>MCPServerRegistry.get</code>	method	<code>app/mcp/registry.py:286</code>	<code>self, server_id: str</code> Return the result.
<code>MCPServerRegistry.require</code>	method	<code>app/mcp/registry.py:297</code>	<code>self, server_id: str</code> Compute the require.
<code>MCPServerRegistry.all</code>	method	<code>app/mcp/registry.py:314</code>	<code>self</code> Compute the all.
<code>MCPServerRegistry.record_health</code>	method	<code>app/mcp/registry.py:322</code>	<code>self, server_id: str, *, healthy: bool, tool_count: int=0, error: str None=None</code> Record the health.
<code>MCPServerRegistry.health</code>	method	<code>app/mcp/registry.py:345</code>	<code>self, server_id: str</code> Compute the health.
<code>MCPServerRegistry.describe_all</code>	method	<code>app/mcp/registry.py:359</code>	<code>self</code> Compute the describe all.
<code>load_servers</code>	function	<code>app/mcp/registry.py:371</code>	<code>raw: str None=None</code> Parse configured servers from JSON.
<code>MCPToolResult</code>	class	<code>app/mcp/results.py:37</code>	<code>BaseModel</code> The normalised result every MCP tool call produces.
<code>_depth</code>	function	<code>app/mcp/results.py:58</code>	<code>value: Any, current: int=0</code> Internal helper for the depth step.
<code>_as_object</code>	function	<code>app/mcp/results.py:80</code>	<code>value: Any</code> Normalise a parsed payload into an object, or reject it.
<code>parse_json_text</code>	function	<code>app/mcp/results.py:95</code>	<code>text: str, *, max_bytes: int</code> Parse a text payload that looks like JSON, safely.
<code>normalise_result</code>	function	<code>app/mcp/results.py:126</code>	<code>response: Any, *, max_bytes: int=256000, tool_name: str=''</code> Normalise an MCP `CallToolResult` into a workflow output.
<code>_collect_text</code>	function	<code>app/mcp/results.py:202</code>	<code>content: Any</code> Internal helper for the collect text step.
<code>_trim</code>	function	<code>app/mcp/results.py:219</code>	<code>text: str, max_bytes: int</code> Internal helper for the trim step.
<code>field_paths_from_schema</code>	function	<code>app/mcp/results.py:234</code>	<code>schema: dict[str, Any] None, prefix: str='', depth: int=0</code> Flatten a JSON Schema into typed dotted paths.
<code>_json_type</code>	function	<code>app/mcp/results.py:291</code>	<code>schema: dict[str, Any]</code> Internal helper for the json type step.
<code>_services</code>	function	<code>app/mcp/server.py:45</code>	<code></code> Internal helper for the services step.

Symbol	Kind	Location	Signature / summary
<code>list_tools</code>	async function	<code>app/mcp/server.py:59</code>	List the tools.
<code>call_tool</code>	async function	<code>app/mcp/server.py:137</code>	<code>name: str, arguments: dict</code> Compute the call tool.
<code>main</code>	async function	<code>app/mcp/server.py:260</code>	Compute the main.
<code>MCPToolError</code>	class	<code>app/mcp/service.py:40</code>	<code>RuntimeError</code> A tool call failed in a way worth reporting to a workflow author.
<code>MCPToolError.as_payload</code>	method	<code>app/mcp/service.py:75</code>	<code>self</code> Compute the as payload.
<code>ToolDescriptor</code>	class	<code>app/mcp/service.py:91</code>	<code>dict</code> A discovered tool, described for the Builder.
<code>MCPIntegrationService</code>	class	<code>app/mcp/service.py:99</code>	Provides the MCPIntegrationService behaviour.
<code>MCPIntegrationService.describe_servers</code>	method	<code>app/mcp/service.py:122</code>	<code>self</code> Connections for the Builder's server picker.
<code>MCPIntegrationService.discover_tools</code>	method	<code>app/mcp/service.py:131</code>	<code>self, server_id: str, *, refresh: bool=False</code> Ask the server what it can do.
<code>MCPIntegrationService.describe_tool</code>	method	<code>app/mcp/service.py:172</code>	<code>self, connection: MCPServerConnection, tool: Any</code> Internal helper for the describe tool step.
<code>MCPIntegrationService.find_tool</code>	method	<code>app/mcp/service.py:227</code>	<code>self, server_id: str, tool_name: str</code> Find the tool.
<code>MCPIntegrationService.health_check</code>	method	<code>app/mcp/service.py:244</code>	<code>self, server_id: str</code> Compute the health check.
<code>MCPIntegrationService.call</code>	method	<code>app/mcp/service.py:271</code>	<code>self, *, server_id: str, tool_name: str, arguments: dict[str, Any], run_id: str='', node_id: str='', session_id: str='', user_role: str None=None, approval_satisfied: bool=False, audit_db: Any=None, timeout_override: float None=None</code> Run one tool through the full gate.
<code>MCPIntegrationService.require_connection</code>	method	<code>app/mcp/service.py:480</code>	<code>self, server_id: str, tool_name: str</code> Internal helper for the require connection step.
<code>MCPIntegrationService.replay_or_refuse</code>	method	<code>app/mcp/service.py:508</code>	<code>self, existing: dict[str, Any], key: str, connection: MCPServerConnection, tool_name: str, decision: PolicyDecision</code> Internal helper for the replay or refuse step.
<code>MCPIntegrationService.audit</code>	method	<code>app/mcp/service.py:551</code>	<code>self, audit_db: Any, *, run_id: str, node_id: str, session_id: str, server_id: str, tool_name: str, operation: str, status: str, started: float, detail: dict[str, Any]</code> Record the invocation (§27).
<code>_tool_error_payload</code>	function	<code>app/mcp/service.py:594</code>	

Symbol	Kind	Location	Signature / summary
			result: MCPToolResult Recognise a structured error a server returned in place of a result.
_humanise	function	app/mcp/service.py:610	name: str Internal helper for the humanise step.
discover_nodes	function	app/nodes/__init__.py:23	*, force: bool=False Import all built-in node modules and return module import failures.
node_discovery_errors	function	app/nodes/__init__.py:46	 Return a copy so callers cannot mutate discovery state.
_Empty	class	app/nodes/_stubs.py:9	BaseModel Pydantic model defining the Empty shape.
_LiteralConfig	class	app/nodes/_stubs.py:14	BaseModel Pydantic model defining the LiteralConfig shape.
_LiteralOutput	class	app/nodes/_stubs.py:23	BaseModel Pydantic model defining the LiteralOutput shape.
LiteralNode	class	app/nodes/_stubs.py:33	NodeType Emits whatever literal is in its config.
LiteralNode.run	method	app/nodes/_stubs.py:41	self, state, resolved_config Run the result.
_EchoConfig	class	app/nodes/_stubs.py:51	BaseModel Pydantic model defining the EchoConfig shape.
_EchoOutput	class	app/nodes/_stubs.py:60	BaseModel Pydantic model defining the EchoOutput shape.
EchoNode	class	app/nodes/_stubs.py:70	NodeType Renders a template string against state.
EchoNode.run	method	app/nodes/_stubs.py:78	self, state, resolved_config Run the result.
_repo_root	function	app/nodes/about_synthesis.py:45	 Internal helper for the repo root step.
_edge_targets	function	app/nodes/about_synthesis.py:55	edge: dict[str, Any] Internal helper for the edge targets step.
_adjacency	function	app/nodes/about_synthesis.py:77	 type_name -> {"upstream": [...], "downstream": [...], "example": str None}, mined once from every workflow YAML on disk.
_schema_fields_summary	function	app/nodes/about_synthesis.py:153	schema: Type[Any] None Internal helper for the schema fields summary step.
_important_config_fields	function	app/nodes/about_synthesis.py:172	config_schema: Type[Any] None Internal helper for the important config fields step.
synthesize_about	function	app/nodes/about_synthesis.py:192	klass: 'Type[NodeType]' Best-effort `about` dict for a node type that didn't author its own.
LanguagePolicy	class	app/nodes/ai_task.py:190	BaseModel Multilingual behaviour, configured on the AI Task rather than modelled as separate translation nodes (§8).

Symbol	Kind	Location	Signature / summary
<code>LanguagePolicy.directive</code>	method	<code>app/nodes/ai_task.py:207</code>	<code>self</code> Compute the directive.
<code>AITaskExample</code>	class	<code>app/nodes/ai_task.py:243</code>	<code>BaseModel</code> A few-shot example.
<code>AITaskConfig</code>	class	<code>app/nodes/ai_task.py:254</code>	<code>BaseModel</code> Pydantic model defining the AITaskConfig shape.
<code>AITaskConfig.structured_tasks_declare_a_schema</code>	method	<code>app/nodes/ai_task.py:330</code>	<code>self</code> Compute the structured tasks declare a schema.
<code>AITaskInput</code>	class	<code>app/nodes/ai_task.py:344</code>	<code>BaseModel</code> Pydantic model defining the AITaskInput shape.
<code>AITaskOutput</code>	class	<code>app/nodes/ai_task.py:349</code>	<code>BaseModel</code> The typed contract every downstream node sees.
<code>_confidence_fields</code>	function	<code>app/nodes/ai_task.py:368</code>	<code>fields: list[FieldSpec]</code> Internal helper for the confidence fields step.
<code>effective_fields</code>	function	<code>app/nodes/ai_task.py:409</code>	<code>config: dict[str, Any]</code> The schema the model is actually held to, generated fields included.
<code>AITaskAgent</code>	class	<code>app/nodes/ai_task.py:424</code>	<code>NodeType</code> Workflow node type implementing the AITaskAgent capability.
<code>AITaskAgent.required_services</code>	method	<code>app/nodes/ai_task.py:469</code>	<code>cls, config: dict[str, Any]</code> Compute the required services.
<code>AITaskAgent.preflight_output_fields</code>	method	<code>app/nodes/ai_task.py:481</code>	<code>cls, config: dict[str, Any]</code> Authorise `result.<path>` references for the author's own schema.
<code>AITaskAgent.preflight_static_output_values</code>	method	<code>app/nodes/ai_task.py:504</code>	<code>cls, config: dict[str, Any]</code> Compute the preflight static output values.
<code>AITaskAgent.run</code>	method	<code>app/nodes/ai_task.py:524</code>	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
<code>AITaskAgent._system_prompt</code>	method	<code>app/nodes/ai_task.py:549</code>	<code>self, cfg: AITaskConfig, fields: list[FieldSpec]</code> Internal helper for the system prompt step.
<code>AITaskAgent._user_prompt</code>	method	<code>app/nodes/ai_task.py:579</code>	<code>self, cfg: AITaskConfig, fields: list[FieldSpec]</code> Internal helper for the user prompt step.
<code>AITaskAgent._examples_block</code>	method	<code>app/nodes/ai_task.py:606</code>	<code>cfg: AITaskConfig</code> Internal helper for the examples block step.
<code>AITaskAgent._run_text</code>	method	<code>app/nodes/ai_task.py:631</code>	<code>self, cfg: AITaskConfig, llm: Any, system: str, user: str</code> Run the text.
<code>AITaskAgent._run_structured</code>	method	<code>app/nodes/ai_task.py:672</code>	<code>self, cfg: AITaskConfig, llm: Any, system: str, user: str, fields: list[FieldSpec]</code> Run the structured.
<code>AITaskAgent._success</code>	method	<code>app/nodes/ai_task.py:761</code>	<code>self, cfg: AITaskConfig, parsed: dict[str, Any], attempt: int</code> Internal helper for the success step.

Symbol	Kind	Location	Signature / summary
AITaskAgent._failure	method	app/nodes/ai_task.py:793	<code>self, cfg: AITaskConfig, status: str, message: str, *, attempts: int</code> Internal helper for the failure step.
_detected_language	function	app/nodes/ai_task.py:828	<code>parsed: dict[str, Any]</code> Surface a language field from the author's schema as a runtime field.
_looks_like_refusal	function	app/nodes/ai_task.py:842	<code>error: Exception</code> Distinguish a model declining the task from a malformed response.
human_approved	function	app/nodes/approval.py:13	<code>state: dict[str, Any], approved_by: str None=None</code> Did a human review approve something earlier on this run's path?
NodeType	class	app/nodes/base.py:16	<code>ABC</code> Abstract base defining the NodeType contract.
NodeType.definition	method	app/nodes/base.py:73	<code>cls, config: dict[str, Any] None=None</code> Return the complete standardized contract for this node type.
NodeType.run	method	app/nodes/base.py:113	<code>self, state: 'WorkflowState', resolved_config: dict[str, Any]</code> Execute the node.
NodeType.required_services	method	app/nodes/base.py:133	<code>cls, config: dict[str, Any]</code> Service names this node needs at runtime, given its resolved config.
NodeType.preflight_output_fields	method	app/nodes/base.py:140	<code>cls, config: dict[str, Any]</code> Valid dotted template-reference suffixes for this node's output, beyond the static output_schema field names — e.g.
NodeType.preflight_static_output_values	method	app/nodes/base.py:150	<code>cls, config: dict[str, Any]</code> Output fields whose value preflight can already prove, without running the node — e.g.
BoundedDeepResearchInput	class	app/nodes/bounded_deep_research_agent.py:27	<code>BaseModel</code> Pydantic model defining the BoundedDeepResearchInput shape.
BoundedDeepResearchConfig	class	app/nodes/bounded_deep_research_agent.py:32	<code>BaseModel</code> Pydantic model defining the BoundedDeepResearchConfig shape.
BoundedDeepResearchConfig.coerce_briefs	method	app/nodes/bounded_deep_research_agent.py:72	<code>cls, value: Any</code> Internal helper for the coerce briefs step.
BoundedDeepResearchOutput	class	app/nodes/bounded_deep_research_agent.py:90	<code>BaseModel</code> Pydantic model defining the BoundedDeepResearchOutput shape.
BoundedDeepResearchAgent	class	app/nodes/bounded_deep_research_agent.py:117	<code>NodeType</code> Workflow node type implementing the BoundedDeepResearchAgent capability.
BoundedDeepResearchAgent.required_services	method	app/nodes/bounded_deep_research_agent.py:130	<code>cls, config: dict[str, Any]</code> Compute the required services.
	method		

Symbol	Kind	Location	Signature / summary
BoundedDeepResearchAgent.run		app/nodes/bounded_deep_research_agent.py:141	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
BoundedDeepResearchAgent.record_costs	method	app/nodes/bounded_deep_research_agent.py:301	<code>self, state: dict[str, Any], dossiers: list[ResearchDossier]</code> Record the costs.
_candidate_records	function	app/nodes/bounded_deep_research_agent.py:339	<code>dossiers: list[ResearchDossier], *, max_citations_per_brief: int, max_candidates_per_claim: int</code> Internal helper for the candidate records step.
_search_audit	function	app/nodes/bounded_deep_research_agent.py:452	<code>briefs: list[ResearchBrief], dossiers: list[ResearchDossier], failures: list[dict[str, str]]</code> Search the audit.
CallCoverageInput	class	app/nodes/call_coverage.py:19	<code>BaseModel</code> Pydantic model defining the CallCoverageInput shape.
CallCoverageConfig	class	app/nodes/call_coverage.py:24	<code>BaseModel</code> Pydantic model defining the CallCoverageConfig shape.
CallCoverageMatrixAgent	class	app/nodes/call_coverage.py:30	<code>NodeType</code> Workflow node type implementing the CallCoverageMatrixAgent capability.
CallCoverageMatrixAgent.run	method	app/nodes/call_coverage.py:41	<code>self, state: dict, resolved_config: dict</code> Run the result.
category_for	function	app/nodes/categories.py:92	<code>type_name: str</code> Compute the category for.
icon_for	function	app/nodes/categories.py:121	<code>type_name: str</code> Compute the icon for.
family_for	function	app/nodes/categories.py:189	<code>type_name: str</code> Compute the family for.
execution_kind_for	function	app/nodes/categories.py:201	<code>type_name: str, *, uses_llm: bool</code> Compute the execution kind for.
CitationRegistryBuilderInput	class	app/nodes/citation_registry_builder.py:47	<code>BaseModel</code> Pydantic model defining the CitationRegistryBuilderInput shape.
CitationRegistryBuilderConfig	class	app/nodes/citation_registry_builder.py:52	<code>BaseModel</code> Pydantic model defining the CitationRegistryBuilderConfig shape.
CitationRegistryBuilderConfig.coerce_documents	method	app/nodes/citation_registry_builder.py:69	<code>cls, value: Any</code> Internal helper for the coerce documents step.
CitationEntry	class	app/nodes/citation_registry_builder.py:85	<code>BaseModel</code> Pydantic model defining the CitationEntry shape.
CitationRegistryBuilderOutput	class	app/nodes/citation_registry_builder.py:109	<code>BaseModel</code> Pydantic model defining the CitationRegistryBuilderOutput shape.
CitationRegistryBuilder	class		

Symbol	Kind	Location	Signature / summary
		app/nodes/ citation_registry_builder.py:134	NodeType Workflow node type implementing the CitationRegistryBuilder capability.
CitationRegistryBuilder.run	method	app/nodes/ citation_registry_builder.py:152	self, state: dict[str, Any], resolved_config: dict[str, Any] Run the result.
ClaimEvidenceVerifierInput	class	app/nodes/ claim_evidence_verifier.py:24	BaseModel Pydantic model defining the ClaimEvidenceVerifierInput shape.
ClaimEvidenceVerifierConfig	class	app/nodes/ claim_evidence_verifier.py:29	BaseModel Pydantic model defining the ClaimEvidenceVerifierConfig shape.
ClaimEvidenceVerifierOutput	class	app/nodes/ claim_evidence_verifier.py:42	BaseModel Pydantic model defining the ClaimEvidenceVerifierOutput shape.
ClaimEvidenceVerifier	class	app/nodes/ claim_evidence_verifier.py:62	NodeType Workflow node type implementing the ClaimEvidenceVerifier capability.
ClaimEvidenceVerifier.required_services	method	app/nodes/ claim_evidence_verifier.py:74	cls, config: dict[str, Any] Compute the required services.
ClaimEvidenceVerifier._source_text	method	app/nodes/ claim_evidence_verifier.py:85	self, source, cap: int Internal helper for the source text step.
ClaimEvidenceVerifier.run	method	app/nodes/ claim_evidence_verifier.py:110	self, state: dict, resolved_config: dict[str, Any] Run the result.
ConceptAlternativesInput	class	app/nodes/ concept_alternatives.py:21	BaseModel Pydantic model defining the ConceptAlternativesInput shape.
ConceptAlternativesConfig	class	app/nodes/ concept_alternatives.py:26	BaseModel Pydantic model defining the ConceptAlternativesConfig shape.
ConceptAlternativesOutput	class	app/nodes/ concept_alternatives.py:39	ConceptAlternativeSet Provides the ConceptAlternativesOutput behaviour.
ConceptAlternativesAgent	class	app/nodes/ concept_alternatives.py:53	NodeType Workflow node type implementing the ConceptAlternativesAgent capability.
ConceptAlternativesAgent.required_services	method	app/nodes/ concept_alternatives.py:65	cls, config: dict[str, Any] Compute the required services.
ConceptAlternativesAgent.run	method	app/nodes/ concept_alternatives.py:76	self, state: dict, resolved_config: dict Run the result.
ConceptFreezeInput	class	app/nodes/ concept_freeze.py:23	BaseModel Pydantic model defining the ConceptFreezeInput shape.
ConceptFreezeConfig	class	app/nodes/ concept_freeze.py:28	BaseModel Pydantic model defining the ConceptFreezeConfig shape.

Symbol	Kind	Location	Signature / summary
ConceptFreezeConfig_coerce_alternatives	method	app/nodes/concept_freeze.py:40	<code>cls, value: Any</code> Internal helper for the coerce alternatives step.
ConceptFreezeOutput	class	app/nodes/concept_freeze.py:52	<code>BaseModel</code> Pydantic model defining the ConceptFreezeOutput shape.
ConceptFreezeAgent	class	app/nodes/concept_freeze.py:62	<code>NodeType</code> Workflow node type implementing the ConceptFreezeAgent capability.
ConceptFreezeAgent.run	method	app/nodes/concept_freeze.py:74	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
ConsistencyCheckerOutput	class	app/nodes/consistency_checker.py:39	<code>BaseModel</code> Validated output for ConsistencyChecker.run() (output_schema ClassVar).
ConsistencyCheckerInput	class	app/nodes/consistency_checker.py:46	<code>BaseModel</code> Pydantic model defining the ConsistencyCheckerInput shape.
ConsistencyCheckerConfig	class	app/nodes/consistency_checker.py:51	<code>BaseModel</code> Node config — base.__init__ does config_schema(**raw_config).
_check	function	app/nodes/consistency_checker.py:57	<code>graph: ProposalGraph</code> Check the result.
ConsistencyChecker	class	app/nodes/consistency_checker.py:149	<code>NodeType</code> Deterministic gate.
ConsistencyChecker.run	method	app/nodes/consistency_checker.py:159	<code>self, state: dict, config: dict</code> Run the result.
DataType	class	app/nodes/contracts.py:17	<code>str, Enum</code> Closed vocabulary used by generic edge compatibility policies.
NodeConstraint	class	app/nodes/contracts.py:32	<code>BaseModel</code> One named, reusable compatibility policy attached declaratively.
NodeDefinition	class	app/nodes/contracts.py:41	<code>BaseModel</code> Complete generic contract for one registered node type.
NodeDefinition.validate_contract	method	app/nodes/contracts.py:65	<code>self</code> Reject metadata gaps and contradictory declarations early.
CompatibilityIssue	class	app/nodes/contracts.py:79	<code>BaseModel</code> Stable, structured reason an edge is invalid or needs attention.
CompatibilityResult	class	app/nodes/contracts.py:90	<code>BaseModel</code> Result of applying every universal policy to one directed edge.
data_types_from_schema	function	app/nodes/contracts.py:98	<code>schema: type[BaseModel]</code> Derive broad transport types from a node's existing Pydantic schema.
_collect_json_types	function	app/nodes/contracts.py:108	

Symbol	Kind	Location	Signature / summary
			field: dict[str, Any], found: set[DataType] No summary declared.
check_compatibility	function	app/nodes/contracts.py:129	source: NodeDefinition, target: NodeDefinition, *, available_capabilities: set[str] None=None, available_permissions: set[str] None=None Apply universal policies; never branches on a source/target pair.
TransformOperation	class	app/nodes/data_transform.py:46	BaseModel One deterministic assignment into the output object.
TransformOperation.operation_has_its_inputs	method	app/nodes/data_transform.py:67	self Compute the operation has its inputs.
DataTransformConfig	class	app/nodes/data_transform.py:113	BaseModel Pydantic model defining the DataTransformConfig shape.
DataTransformConfig.targets_are_unique	method	app/nodes/data_transform.py:134	self Compute the targets are unique.
DataTransformInput	class	app/nodes/data_transform.py:150	BaseModel Pydantic model defining the DataTransformInput shape.
DataTransformOutput	class	app/nodes/data_transform.py:155	BaseModel Pydantic model defining the DataTransformOutput shape.
DataTransformAgent	class	app/nodes/data_transform.py:173	NodeType Workflow node type implementing the DataTransformAgent capability.
DataTransformAgent.required_services	method	app/nodes/data_transform.py:212	cls, config: dict[str, Any] Compute the required services.
DataTransformAgent.preflight_output_fields	method	app/nodes/data_transform.py:224	cls, config: dict[str, Any] Compute the preflight output fields.
DataTransformAgent.preflight_static_output_values	method	app/nodes/data_transform.py:241	cls, config: dict[str, Any] Compute the preflight static output values.
DataTransformAgent.run	method	app/nodes/data_transform.py:254	self, state, resolved_config: dict[str, Any] Run the result.
_is_blank	function	app/nodes/data_transform.py:281	value: Any Return whether blank.
_apply	function	app/nodes/data_transform.py:299	op: TransformOperation, context: dict[str, Any] Apply the result.
_as_text	function	app/nodes/data_transform.py:379	value: Any Internal helper for the as text step.
_as_number	function	app/nodes/data_transform.py:399	value: Any Internal helper for the as number step.
_as_boolean	function	app/nodes/data_transform.py:424	value: Any Internal helper for the as boolean step.

Symbol	Kind	Location	Signature / summary
DecisionConfig	class	app/nodes/decision.py:119	<code>BaseModel</code> Pydantic model defining the DecisionConfig shape.
DecisionConfig.output_field_names	method	app/nodes/decision.py:150	<code>self</code> Compute the output field names.
DecisionInput	class	app/nodes/decision.py:162	<code>BaseModel</code> Pydantic model defining the DecisionInput shape.
DecisionOutput	class	app/nodes/decision.py:167	<code>BaseModel</code> Facts the rules established, plus the reasoning behind them.
DecisionAgent	class	app/nodes/decision.py:179	<code>NodeType</code> Workflow node type implementing the DecisionAgent capability.
DecisionAgent.required_services	method	app/nodes/decision.py:216	<code>cls, config: dict[str, Any]</code> Compute the required services.
DecisionAgent.preflight_output_fields	method	app/nodes/decision.py:228	<code>cls, config: dict[str, Any]</code> Authorise `decisions.<field>` for every field the rules can set.
DecisionAgent.preflight_static_output_values	method	app/nodes/decision.py:238	<code>cls, config: dict[str, Any]</code> Compute the preflight static output values.
DecisionAgent.run	method	app/nodes/decision.py:251	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
DOCXRenderInput	class	app/nodes/docx_renderer.py:84	<code>BaseModel</code> Pydantic model defining the DOCXRenderInput shape.
DOCXRenderConfig	class	app/nodes/docx_renderer.py:89	<code>BaseModel</code> Pydantic model defining the DOCXRenderConfig shape.
DOCXRenderOutput	class	app/nodes/docx_renderer.py:106	<code>BaseModel</code> Pydantic model defining the DOCXRenderOutput shape.
DOCXProposalRenderer	class	app/nodes/docx_renderer.py:120	<code>NodeType</code> Workflow node type implementing the DOCXProposalRenderer capability.
DOCXProposalRenderer.required_services	method	app/nodes/docx_renderer.py:129	<code>cls, config: dict[str, Any]</code> Compute the required services.
DOCXProposalRenderer.run	method	app/nodes/docx_renderer.py:140	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
DOCXProposalRenderer.render_docx_bytes	method	app/nodes/docx_renderer.py:187	<code>self, sections: dict[str, str], template_name: str, client: str, title: str</code> Render the docx bytes.
DOCXProposalRenderer.add_title_block	method	app/nodes/docx_renderer.py:231	<code>self, doc, title, client, tpl</code> Add the title block.
DOCXProposalRenderer.style_heading	method	app/nodes/docx_renderer.py:257	<code>self, paragraph, tpl, top_level: bool</code> Internal helper for the style heading step.
DOCXProposalRenderer.render_markdown	method	app/nodes/docx_renderer.py:279	<code>self, doc, md: str, tpl</code> Render the markdown.

Symbol	Kind	Location	Signature / summary
DOCXProposalRender.render_table	method	app/nodes/docx_renderer.py:335	<code>self, doc, lines, start, tpl</code> Render the table.
DOCXProposalRender.add_inline	method	app/nodes/docx_renderer.py:388	<code>self, paragraph, text: str, bold: bool=False</code> Add the inline.
DynamicFigureAgentInput	class	app/nodes/dynamic_figure_agent.py:62	<code>BaseModel</code> Pydantic model defining the DynamicFigureAgentInput shape.
DynamicFigureAgentConfig	class	app/nodes/dynamic_figure_agent.py:67	<code>BaseModel</code> Pydantic model defining the DynamicFigureAgentConfig shape.
GeneratedFigure	class	app/nodes/dynamic_figure_agent.py:90	<code>BaseModel</code> Pydantic model defining the GeneratedFigure shape.
DynamicFigureAgentOutput	class	app/nodes/dynamic_figure_agent.py:109	<code>BaseModel</code> Pydantic model defining the DynamicFigureAgentOutput shape.
DynamicFigureAgent	class	app/nodes/dynamic_figure_agent.py:127	<code>NodeType</code> Workflow node type implementing the DynamicFigureAgent capability.
DynamicFigureAgent.run	method	app/nodes/dynamic_figure_agent.py:145	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
_sub_captioned	function	app/nodes/dynamic_figure_agent.py:297	<code>content: str</code> Internal helper for the sub captioned step.
_escape	function	app/nodes/dynamic_figure_agent.py:323	<code>text: str</code> Internal helper for the escape step.
EmailRecipient	class	app/nodes/email_integration.py:74	<code>BaseModel</code> Pydantic model defining the EmailRecipient shape.
EmailNodeConfig	class	app/nodes/email_integration.py:87	<code>BaseModel</code> Pydantic model defining the EmailNodeConfig shape.
EmailNodeConfig.operation_has_what_it_needs	method	app/nodes/email_integration.py:146	<code>self</code> Compute the operation has what it needs.
EmailNodeInput	class	app/nodes/email_integration.py:176	<code>BaseModel</code> Pydantic model defining the EmailNodeInput shape.
EmailMessageSummary	class	app/nodes/email_integration.py:181	<code>BaseModel</code> The message shape downstream nodes address.
EmailNodeOutput	class	app/nodes/email_integration.py:202	<code>BaseModel</code> Pydantic model defining the EmailNodeOutput shape.
EmailAgent	class	app/nodes/email_integration.py:229	<code>NodeType</code> Workflow node type implementing the EmailAgent capability.
EmailAgent.required_services	method	app/nodes/email_integration.py:272	<code>cls, config: dict[str, Any]</code> Compute the required services.
EmailAgent.preflight_output_fields	method	app/nodes/email_integration.py:284	<code>cls, config: dict[str, Any]</code> Authorise the message shape as dotted paths.
EmailAgent.run	method	app/nodes/email_integration.py:299	

Symbol	Kind	Location	Signature / summary
			<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
<code>_criteria</code>	function	<code>app/nodes/email_integration.py:358</code>	<code>cfg: EmailNodeConfig</code> Internal helper for the criteria step.
<code>_draft</code>	function	<code>app/nodes/email_integration.py:379</code>	<code>cfg: EmailNodeConfig</code> Draft the result.
<code>_summarise</code>	function	<code>app/nodes/email_integration.py:400</code>	<code>message: Any</code> Internal helper for the summarise step.
<code>is_side_effect</code>	function	<code>app/nodes/email_integration.py:426</code>	<code>operation: str</code> Whether this operation changes something outside the platform.
<code>EndOutputField</code>	class	<code>app/nodes/end.py:33</code>	<code>BaseModel</code> Pydantic model defining the EndOutputField shape.
<code>EndConfig</code>	class	<code>app/nodes/end.py:48</code>	<code>BaseModel</code> Pydantic model defining the EndConfig shape.
<code>EndInput</code>	class	<code>app/nodes/end.py:82</code>	<code>BaseModel</code> Pydantic model defining the EndInput shape.
<code>EndOutput</code>	class	<code>app/nodes/end.py:87</code>	<code>BaseModel</code> Pydantic model defining the EndOutput shape.
<code>_humanize</code>	function	<code>app/nodes/end.py:96</code>	<code>value: str</code> Internal helper for the humanize step.
<code>EndAgent</code>	class	<code>app/nodes/end.py:109</code>	<code>NodeType</code> Workflow node type implementing the EndAgent capability.
<code>EndAgent.required_services</code>	method	<code>app/nodes/end.py:155</code>	<code>cls, config: dict[str, Any]</code> Compute the required services.
<code>EndAgent.preflight_output_fields</code>	method	<code>app/nodes/end.py:167</code>	<code>cls, config: dict[str, Any]</code> Compute the preflight output fields.
<code>EndAgent.run</code>	method	<code>app/nodes/end.py:195</code>	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
<code>ClaimSearchPlan</code>	class	<code>app/nodes/evidence_agent.py:46</code>	<code>BaseModel</code> Pydantic model defining the ClaimSearchPlan shape.
<code>_sanitize_query</code>	function	<code>app/nodes/evidence_agent.py:60</code>	<code>text: str</code> Strip boolean/advanced-search syntax from a generated query.
<code>ScholarlyCandidateDiscoveryInput</code>	class	<code>app/nodes/evidence_agent.py:75</code>	<code>BaseModel</code> Pydantic model defining the ScholarlyCandidateDiscoveryInput shape.
<code>ScholarlyCandidateDiscoveryConfig</code>	class	<code>app/nodes/evidence_agent.py:80</code>	<code>BaseModel</code> Pydantic model defining the ScholarlyCandidateDiscoveryConfig shape.
<code>ScholarlyCandidateDiscoveryOutput</code>	class	<code>app/nodes/evidence_agent.py:133</code>	<code>BaseModel</code> Pydantic model defining the ScholarlyCandidateDiscoveryOutput shape.
<code>_ScholarlyCandidateDiscovery</code>	class	<code>app/nodes/evidence_agent.py:161</code>	

Symbol	Kind	Location	Signature / summary
			NodeType Workflow node type implementing the ScholarlyCandidateDiscovery capability.
_ScholarlyCandidateDiscovery.required_services	method	app/nodes/evidence_agent.py:168	cls, config: dict[str, Any] Compute the required services.
_ScholarlyCandidateDiscovery.run	method	app/nodes/evidence_agent.py:179	self, state: dict[str, Any], resolved_config: dict[str, Any] Run the result.
_ScholarlyCandidateDiscovery.query_planning_guidance	method	app/nodes/evidence_agent.py:377	self, cfg: ScholarlyCandidateDiscoveryConfig Search methodology text from the configured Scientific Agent Skill.
_ScholarlyCandidateDiscovery.search_plan	method	app/nodes/evidence_agent.py:405	self, llm: Any, claim: Claim, model: str None, *, skill_guidance: str='' Search the plan.
ScholarlyCandidateDiscoveryAgent	class	app/nodes/evidence_agent.py:496	_ScholarlyCandidateDiscovery Provides the ScholarlyCandidateDiscoveryAgent behaviour.
ExcelInput	class	app/nodes/excel_tool.py:16	BaseModel Pydantic model defining the ExcelInput shape.
ExcelConfig	class	app/nodes/excel_tool.py:21	BaseModel Pydantic model defining the ExcelConfig shape.
ExcelOutput	class	app/nodes/excel_tool.py:30	BaseModel Pydantic model defining the ExcelOutput shape.
ExcelTableExtractor	class	app/nodes/excel_tool.py:44	NodeType Workflow node type implementing the ExcelTableExtractor capability.
ExcelTableExtractor.required_services	method	app/nodes/excel_tool.py:53	cls, config: dict[str, Any] Compute the required services.
ExcelTableExtractor.run	method	app/nodes/excel_tool.py:64	self, state, resolved_config: dict[str, Any] Run the result.
ExternalActionConfig	class	app/nodes/external_action.py:45	BaseModel Pydantic model defining the ExternalActionConfig shape.
ExternalActionInput	class	app/nodes/external_action.py:108	BaseModel Pydantic model defining the ExternalActionInput shape.
ExternalActionOutput	class	app/nodes/external_action.py:113	BaseModel Pydantic model defining the ExternalActionOutput shape.
ExternalActionAgent	class	app/nodes/external_action.py:138	NodeType Workflow node type implementing the ExternalActionAgent capability.
ExternalActionAgent.required_services	method	app/nodes/external_action.py:182	cls, config: dict[str, Any] Compute the required services.
ExternalActionAgent.run	method	app/nodes/external_action.py:193	self, state, resolved_config: dict[str, Any] Run the result.

Symbol	Kind	Location	Signature / summary
ExternalActionAgent._failure	method	app/nodes/ external_action.py:245	<code>self, cfg: ExternalActionConfig, error: Exception</code> Internal helper for the failure step.
FigureSpec	class	app/nodes/ figure_embedder.py:28	<code>BaseModel</code> Pydantic model defining the FigureSpec shape.
FigureEmbedderInput	class	app/nodes/ figure_embedder.py:57	<code>BaseModel</code> Pydantic model defining the FigureEmbedderInput shape.
FigureEmbedderConfig	class	app/nodes/ figure_embedder.py:62	<code>BaseModel</code> Pydantic model defining the FigureEmbedderConfig shape.
FigureEmbedderOutput	class	app/nodes/ figure_embedder.py:73	<code>BaseModel</code> Pydantic model defining the FigureEmbedderOutput shape.
_marker_pattern	function	app/nodes/ figure_embedder.py:94	<code>marker: str</code> Internal helper for the marker pattern step.
FigureEmbedder	class	app/nodes/ figure_embedder.py:110	<code>NodeType</code> Workflow node type implementing the FigureEmbedder capability.
FigureEmbedder.run	method	app/nodes/ figure_embedder.py:121	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
GraphNormalizerOutput	class	app/nodes/ graph_normalizer.py:55	<code>BaseModel</code> What GraphNormalizer.run() writes back.
GraphNormalizerInput	class	app/nodes/ graph_normalizer.py:63	<code>BaseModel</code> Pydantic model defining the GraphNormalizerInput shape.
GraphNormalizerConfig	class	app/nodes/ graph_normalizer.py:68	<code>BaseModel</code> Node config — base.__init__ does config_schema(**raw_config).
GraphExtraction	class	app/nodes/ graph_normalizer.py:79	<code>BaseModel</code> Provider-enforced extraction envelope.
GraphNormalizer	class	app/nodes/ graph_normalizer.py:211	<code>NodeType</code> LLM extracts structure from source text; pydantic validates it; valid objects are written to proposal_graph.
GraphNormalizer.required_services	method	app/nodes/ graph_normalizer.py:225	<code>cls, config: dict[str, Any]</code> Compute the required services.
GraphNormalizer.run	method	app/nodes/ graph_normalizer.py:236	<code>self, state: dict, config: dict</code> Run the result.
GraphNormalizer._parse_json	method	app/nodes/ graph_normalizer.py:321	<code>raw: str</code> Parse the LLM's JSON and fail loudly when extraction is malformed.
HorizonDOCXProposalRendererInput	class	app/nodes/ horizon_docx_renderer.py:23	<code>BaseModel</code> Pydantic model defining the HorizonDOCXProposalRendererInput shape.
HorizonDOCXProposalRendererConfig	class	app/nodes/ horizon_docx_renderer.py:28	<code>BaseModel</code> Pydantic model defining the HorizonDOCXProposalRendererConfig shape.

Symbol	Kind	Location	Signature / summary
HorizonDOCXProposalRendererOutput	class	app/nodes/ horizon_docx_renderer.py:65	BaseModel Pydantic model defining the HorizonDOCXProposalRendererOutput shape.
_safe_component	function	app/nodes/ horizon_docx_renderer.py:98	value: str Internal helper for the safe component step.
HorizonDOCXProposalRenderer	class	app/nodes/ horizon_docx_renderer.py:111	NodeType Workflow node type implementing the HorizonDOCXProposalRenderer capability.
HorizonDOCXProposalRenderer.required_services	method	app/nodes/ horizon_docx_renderer.py:124	cls, config: dict[str, Any] Compute the required services.
HorizonDOCXProposalRenderer.run	method	app/nodes/ horizon_docx_renderer.py:135	self, state: dict[str, Any], resolved_config: dict[str, Any] Run the result.
HorizonEvaluationInput	class	app/nodes/ horizon_evaluation.py:17	BaseModel Pydantic model defining the HorizonEvaluationInput shape.
HorizonEvaluationConfig	class	app/nodes/ horizon_evaluation.py:22	BaseModel Pydantic model defining the HorizonEvaluationConfig shape.
HorizonEvaluationAgent	class	app/nodes/ horizon_evaluation.py:42	NodeType Workflow node type implementing the HorizonEvaluationAgent capability.
HorizonEvaluationAgent.required_services	method	app/nodes/ horizon_evaluation.py:54	cls, config: dict[str, Any] Compute the required services.
HorizonEvaluationAgent.run	method	app/nodes/ horizon_evaluation.py:65	self, state: dict, resolved_config: dict Run the result.
HorizonHTMLProposalRendererInput	class	app/nodes/ html_proposal_renderer.py:20	BaseModel Pydantic model defining the HorizonHTMLProposalRendererInput shape.
HorizonHTMLProposalRendererConfig	class	app/nodes/ html_proposal_renderer.py:25	BaseModel Pydantic model defining the HorizonHTMLProposalRendererConfig shape.
HorizonHTMLProposalRendererOutput	class	app/nodes/ html_proposal_renderer.py:56	BaseModel Pydantic model defining the HorizonHTMLProposalRendererOutput shape.
_safe_component	function	app/nodes/ html_proposal_renderer.py:85	value: str Internal helper for the safe component step.
HorizonHTMLProposalRenderer	class	app/nodes/ html_proposal_renderer.py:98	NodeType Workflow node type implementing the HorizonHTMLProposalRenderer capability.
HorizonHTMLProposalRenderer.required_services	method	app/nodes/ html_proposal_renderer.py:110	cls, config: dict[str, Any] Compute the required services.
HorizonHTMLProposalRenderer.run	method	app/nodes/ html_proposal_renderer.py:121	self, state: dict[str, Any], resolved_config: dict[str, Any] Run the result.

Symbol	Kind	Location	Signature / summary
ReviewPanel	class	app/nodes/ human_in_loop.py:24	<code>BaseModel</code> One labelled block the reviewer sees (§29).
HITLConfig	class	app/nodes/ human_in_loop.py:38	<code>BaseModel</code> Pydantic model defining the HITLConfig shape.
HITLInput	class	app/nodes/ human_in_loop.py:97	<code>BaseModel</code> Pydantic model defining the HITLInput shape.
HITLReviewContent	class	app/nodes/ human_in_loop.py:102	<code>BaseModel</code> Canonical review document passed between the UI and workflow runtime.
HITLOutput	class	app/nodes/ human_in_loop.py:113	<code>BaseModel</code> Pydantic model defining the HITLOutput shape.
ReviewPanelValue	class	app/nodes/ human_in_loop.py:132	<code>BaseModel</code> A resolved review panel, ready to render.
HITLInterruptPayload	class	app/nodes/ human_in_loop.py:143	<code>BaseModel</code> What the Cockpit sees when this node pauses.
HumanInLoopAgent	class	app/nodes/ human_in_loop.py:157	<code>NodeType</code> Workflow node type implementing the HumanInLoopAgent capability.
HumanInLoopAgent.required_services	method	app/nodes/ human_in_loop.py:198	<code>cls, config: dict[str, Any]</code> Compute the required services.
HumanInLoopAgent.run	method	app/nodes/ human_in_loop.py:211	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
_resolve_path	function	app/nodes/ human_in_loop.py:329	<code>path: str, state: dict</code> Resolve the path.
_review_panels	function	app/nodes/ human_in_loop.py:350	<code>cfg: HITLConfig, state: dict[str, Any]</code> Resolve each configured panel against state.
_review_content	function	app/nodes/ human_in_loop.py:379	<code>cfg: HITLConfig, state: dict[str, Any], context: dict[str, Any]</code> Internal helper for the review content step.
_content_format	function	app/nodes/ human_in_loop.py:459	<code>value: Any</code> Internal helper for the content format step.
_content_as_text	function	app/nodes/ human_in_loop.py:471	<code>value: Any</code> Internal helper for the content as text step.
_edited_value	function	app/nodes/ human_in_loop.py:487	<code>content: HITLReviewContent</code> Restore structured editor content before patching workflow state.
_patch_reviewed_content	function	app/nodes/ human_in_loop.py:501	<code>state: dict[str, Any], path: str, value: Any</code> Replace only the configured editable path for downstream nodes.
_set_nested	function	app/nodes/ human_in_loop.py:535	<code>container: Any, parts: list[str], value: Any</code> Set the nested.
_RichTextSanitizer	class	app/nodes/ human_in_loop.py:557	<code>HTMLParser</code> Small allow-list sanitizer for browser-generated editor markup.

Symbol	Kind	Location	Signature / summary
<code>_RichTextSanitizer.handle_starttag</code>	method	<code>app/nodes/human_in_loop.py:571</code>	<code>self, tag: str, attrs: list[tuple[str, str None]]</code> Handle the starttag.
<code>_RichTextSanitizer.handle_startendtag</code>	method	<code>app/nodes/human_in_loop.py:586</code>	<code>self, tag: str, attrs: list[tuple[str, str None]]</code> Handle the startendtag.
<code>_RichTextSanitizer.handle_endtag</code>	method	<code>app/nodes/human_in_loop.py:601</code>	<code>self, tag: str</code> Handle the endtag.
<code>_RichTextSanitizer.handle_data</code>	method	<code>app/nodes/human_in_loop.py:610</code>	<code>self, data: str</code> Handle the data.
<code>sanitize_rich_html</code>	function	<code>app/nodes/human_in_loop.py:619</code>	<code>value: str None</code> Sanitize the rich html.
<code>IntegrationNodeConfig</code>	class	<code>app/nodes/integration.py:83</code>	<code>BaseModel</code> Pydantic model defining the IntegrationNodeConfig shape.
<code>IntegrationNodeConfig.operation_has_what_it_needs</code>	method	<code>app/nodes/integration.py:124</code>	<code>self</code> Compute the operation has what it needs.
<code>IntegrationNodeInput</code>	class	<code>app/nodes/integration.py:147</code>	<code>BaseModel</code> Pydantic model defining the IntegrationNodeInput shape.
<code>CloudFileMetaOut</code>	class	<code>app/nodes/integration.py:152</code>	<code>BaseModel</code> The file shape downstream nodes address — flattened so a mapping reads `files.items.name` rather than a nested provider object.
<code>IntegrationNodeOutput</code>	class	<code>app/nodes/integration.py:166</code>	<code>BaseModel</code> Pydantic model defining the IntegrationNodeOutput shape.
<code>IntegrationAgent</code>	class	<code>app/nodes/integration.py:206</code>	<code>NodeType</code> Workflow node type implementing the IntegrationAgent capability.
<code>IntegrationAgent.required_services</code>	method	<code>app/nodes/integration.py:243</code>	<code>cls, config: dict[str, Any]</code> Compute the required services.
<code>IntegrationAgent.preflight_output_fields</code>	method	<code>app/nodes/integration.py:255</code>	<code>cls, config: dict[str, Any]</code> Compute the preflight output fields.
<code>IntegrationAgent.run</code>	method	<code>app/nodes/integration.py:276</code>	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
<code>_meta_out</code>	function	<code>app/nodes/integration.py:375</code>	<code>meta: CloudFileMeta</code> Internal helper for the meta out step.
<code>_to_workflow_file_ref</code>	function	<code>app/nodes/integration.py:396</code>	<code>downloaded: DownloadedFile</code> Turn a downloaded cloud file into the platform's own file contract.
<code>_error_output</code>	function	<code>app/nodes/integration.py:423</code>	<code>cfg: IntegrationNodeConfig, *, status: str, error_code: str, error: str, retryable: bool=False</code> Internal helper for the error output step.
<code>_InternalFactDraft</code>	class	<code>app/nodes/internal_project_evidence_retriever.py:34</code>	<code>BaseModel</code> Pydantic model defining the InternalFactDraft shape.

Symbol	Kind	Location	Signature / summary
<code>_InternalExtraction</code>	class	<code>app/nodes/internal_project_evidence_retriever.py:57</code>	<code>BaseModel</code> Pydantic model defining the InternalExtraction shape.
<code>InternalProjectEvidenceRetrieverInput</code>	class	<code>app/nodes/internal_project_evidence_retriever.py:68</code>	<code>BaseModel</code> Pydantic model defining the InternalProjectEvidenceRetrieverInput shape.
<code>InternalProjectEvidenceRetrieverConfig</code>	class	<code>app/nodes/internal_project_evidence_retriever.py:73</code>	<code>BaseModel</code> Pydantic model defining the InternalProjectEvidenceRetrieverConfig shape.
<code>InternalProjectEvidenceRetrieverOutput</code>	class	<code>app/nodes/internal_project_evidence_retriever.py:100</code>	<code>BaseModel</code> Pydantic model defining the InternalProjectEvidenceRetrieverOutput shape.
<code>InternalProjectEvidenceRetrieverAgent</code>	class	<code>app/nodes/internal_project_evidence_retriever.py:126</code>	<code>NodeType</code> Workflow node type implementing the InternalProjectEvidenceRetrieverAgent capability.
<code>InternalProjectEvidenceRetrieverAgent.required_services</code>	method	<code>app/nodes/internal_project_evidence_retriever.py:139</code>	<code>cls, config: dict[str, Any]</code> Compute the required services.
<code>InternalProjectEvidenceRetrieverAgent.run</code>	method	<code>app/nodes/internal_project_evidence_retriever.py:153</code>	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
<code>_research_questions</code>	function	<code>app/nodes/internal_project_evidence_retriever.py:417</code>	<code>*, graph: Any, research_briefs: Any, limit: int</code> Internal helper for the research questions step.
<code>_normalise_registry</code>	function	<code>app/nodes/internal_project_evidence_retriever.py:466</code>	<code>value: Any</code> Internal helper for the normalise registry step.
<code>_source_blocks</code>	function	<code>app/nodes/internal_project_evidence_retriever.py:505</code>	<code>text: str</code> Internal helper for the source blocks step.
<code>_source_class</code>	function	<code>app/nodes/internal_project_evidence_retriever.py:525</code>	<code>metadata: dict[str, Any]</code> Internal helper for the source class step.
<code>_explicitly_approved</code>	function	<code>app/nodes/internal_project_evidence_retriever.py:543</code>	<code>metadata: dict[str, Any]</code> Internal helper for the explicitly approved step.
<code>_resolve_source_name</code>	function	<code>app/nodes/internal_project_evidence_retriever.py:561</code>	<code>requested: str, blocks: dict[str, str]</code> Resolve the source name.
<code>KimiVisionInput</code>	class	<code>app/nodes/kimi_vision_agent.py:14</code>	<code>BaseModel</code> Pydantic model defining the KimiVisionInput shape.
<code>KimiVisionConfig</code>	class	<code>app/nodes/kimi_vision_agent.py:19</code>	<code>BaseModel</code> Pydantic model defining the KimiVisionConfig shape.
<code>KimiVisionOutput</code>	class	<code>app/nodes/kimi_vision_agent.py:44</code>	<code>BaseModel</code> Pydantic model defining the KimiVisionOutput shape.
<code>KimiVisionAgent</code>	class	<code>app/nodes/kimi_vision_agent.py:70</code>	<code>NodeType</code> Workflow node type implementing the KimiVisionAgent capability.
	method		

Symbol	Kind	Location	Signature / summary
KimiVisionAgent.required_services		app/nodes/ kimi_vision_agent.py:82	<code>cls, config: dict[str, Any]</code> Compute the required services.
KimiVisionAgent.run	method	app/nodes/ kimi_vision_agent.py:93	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
KnowledgeRetrievalError	class	app/nodes/ knowledge_retrieval.py:31	<code>RuntimeError</code> Stable node error safe for workflow/UI handling and operator logs.
KnowledgeRetrievalConfig	class	app/nodes/ knowledge_retrieval.py:39	<code>BaseModel</code> Pydantic model defining the KnowledgeRetrievalConfig shape.
KnowledgeRetrievalInput	class	app/nodes/ knowledge_retrieval.py:57	<code>BaseModel</code> Pydantic model defining the KnowledgeRetrievalInput shape.
KnowledgeRetrievalOutput	class	app/nodes/ knowledge_retrieval.py:62	<code>BaseModel</code> Pydantic model defining the KnowledgeRetrievalOutput shape.
KnowledgeRetrieval	class	app/nodes/ knowledge_retrieval.py:90	<code>NodeType</code> Workflow node type implementing the KnowledgeRetrieval capability.
KnowledgeRetrieval.required_services	method	app/nodes/ knowledge_retrieval.py:102	<code>cls, config: dict[str, Any]</code> Compute the required services.
KnowledgeRetrieval.run	method	app/nodes/ knowledge_retrieval.py:113	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
KnowledgeRetrieval._validate_resources	method	app/nodes/ knowledge_retrieval.py:203	<code>self, cfg: KnowledgeRetrievalConfig, scope: str</code> Revalidate saved references immediately before every execution.
KnowledgeRetrieval._retrieve_with_policy	method	app/nodes/ knowledge_retrieval.py:243	<code>self, request, scope: str, llm: Any</code> Bound retrieval and retry only recognized transient failures once.
MCPAgentConfig	class	app/nodes/mcp_agent.py:22	<code>BaseModel</code> Pydantic model defining the MCPAgentConfig shape.
MCPAgentInput	class	app/nodes/mcp_agent.py:50	<code>BaseModel</code> Pydantic model defining the MCPAgentInput shape.
ToolCallRecord	class	app/nodes/mcp_agent.py:55	<code>BaseModel</code> Pydantic model defining the ToolCallRecord shape.
MCPAgentOutput	class	app/nodes/mcp_agent.py:70	<code>BaseModel</code> Pydantic model defining the MCPAgentOutput shape.
MCPAgent	class	app/nodes/mcp_agent.py:86	<code>NodeType</code> Workflow node type implementing the MCPAgent capability.
MCPAgent.required_services	method	app/nodes/mcp_agent.py:95	<code>cls, config: dict[str, Any]</code> Compute the required services.
MCPAgent.run	method	app/nodes/mcp_agent.py:106	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
MCPToolConfig	class	app/nodes/mcp_tool.py:43	

Symbol	Kind	Location	Signature / summary
			BaseModel Pydantic model defining the MCPToolConfig shape.
MCPToolInput	class	app/nodes/mcp_tool.py:113	BaseModel Pydantic model defining the MCPToolInput shape.
MCPToolOutput	class	app/nodes/mcp_tool.py:118	BaseModel One output shape for every tool on every server.
MCPToolAgent	class	app/nodes/mcp_tool.py:154	NodeType Workflow node type implementing the MCPToolAgent capability.
MCPToolAgent.required_services	method	app/nodes/mcp_tool.py:196	cls, config: dict[str, Any] Compute the required services.
MCPToolAgent.preflight_output_fields	method	app/nodes/mcp_tool.py:208	cls, config: dict[str, Any] Authorise the fixed envelope.
MCPToolAgent.run	method	app/nodes/mcp_tool.py:224	self, state, resolved_config: dict[str, Any] Run the result.
MCPToolAgent._required_arguments	method	app/nodes/mcp_tool.py:326	self, service: Any, cfg: MCPToolConfig The argument names the server itself declares as required.
MCPToolAgent._skipped	method	app/nodes/mcp_tool.py:372	self, cfg: MCPToolConfig, missing: list[str] Internal helper for the skipped step.
MCPToolAgent._failure	method	app/nodes/mcp_tool.py:408	self, cfg: MCPToolConfig, error: Exception Internal helper for the failure step.
_empty_arguments	function	app/nodes/mcp_tool.py:463	arguments: dict[str, Any] Argument names that were supplied but resolved to nothing.
_summarise	function	app/nodes/mcp_tool.py:479	data: dict[str, Any] Lift 'count', 'found' and 'first' out of whatever shape the tool returned.
MethodologyEngineeringInput	class	app/nodes/methodology_engineering.py:27	BaseModel Pydantic model defining the MethodologyEngineeringInput shape.
MethodologyEngineeringConfig	class	app/nodes/methodology_engineering.py:32	BaseModel Pydantic model defining the MethodologyEngineeringConfig shape.
_MethodCardDraft	class	app/nodes/methodology_engineering.py:52	BaseModel Pydantic model defining the MethodCardDraft shape.
MethodCard	class	app/nodes/methodology_engineering.py:81	BaseModel Pydantic model defining the MethodCard shape.
MethodologyEngineeringOutput	class	app/nodes/methodology_engineering.py:113	BaseModel Pydantic model defining the MethodologyEngineeringOutput shape.
MethodologyEngineeringAgent	class	app/nodes/methodology_engineering.py:129	NodeType Workflow node type implementing the MethodologyEngineeringAgent capability.

Symbol	Kind	Location	Signature / summary
MethodologyEngineeringAgent.required_services	method	app/nodes/ methodology_engineering.py: 143	<code>cls, config: dict[str, Any]</code> Compute the required services.
MethodologyEngineeringAgent.run	method	app/nodes/ methodology_engineering.py: 154	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
MethodologyEngineeringAgent.draft_one_card	method	app/nodes/ methodology_engineering.py: 215	<code>self, *, llm: Any, catalog: Any, objective_id: str, objective_text: str, model: str, max_skills: int, known_brief_ids: set[str], known_claim_ids: set[str], selected_concept: Any=None</code> Draft the one card.
MinIOEvidenceIngestionInput	class	app/nodes/ minio_evidence_ingestion.py: 43	<code>BaseModel</code> Pydantic model defining the MinIOEvidenceIngestionInput shape.
MinIOEvidenceIngestionConfig	class	app/nodes/ minio_evidence_ingestion.py: 48	<code>BaseModel</code> Pydantic model defining the MinIOEvidenceIngestionConfig shape.
MinIOEvidenceIngestionConfig.reject_unresolved	method	app/nodes/ minio_evidence_ingestion.py: 74	<code>cls, value: Any</code> Internal helper for the reject unresolved step.
MinIOEvidenceIngestionOutput	class	app/nodes/ minio_evidence_ingestion.py: 88	<code>BaseModel</code> Pydantic model defining the MinIOEvidenceIngestionOutput shape.
MinIOEvidenceIngestion	class	app/nodes/ minio_evidence_ingestion.py: 108	<code>NodeType</code> Workflow node type implementing the MinIOEvidenceIngestion capability.
MinIOEvidenceIngestion.run	method	app/nodes/ minio_evidence_ingestion.py: 126	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
_load_pages	function	app/nodes/ minio_evidence_ingestion.py: 291	<code>raw: bytes</code> Load the pages.
_source_path	function	app/nodes/ minio_evidence_ingestion.py: 305	<code>doc: dict[str, Any], number: int</code> Internal helper for the source path step.
_chunk_pages	function	app/nodes/ minio_evidence_ingestion.py: 321	<code>pages: list[dict[str, Any]], *, chunk_chars: int, overlap: int, max_chunks: int</code> Chunk the pages.
_rfc3339_now	function	app/nodes/ minio_evidence_ingestion.py: 359	<code></code> Internal helper for the rfc3339 now step.
ImageGenerationInput	class	app/nodes/ openai_image_generation_agent.py:15	<code>BaseModel</code> Pydantic model defining the ImageGenerationInput shape.
ImageGenerationConfig	class	app/nodes/ openai_image_generation_agent.py:20	<code>BaseModel</code> Pydantic model defining the ImageGenerationConfig shape.

Symbol	Kind	Location	Signature / summary
ImageGenerationOutput	class	app/nodes/openai_image_generation_agent.py:67	<code>BaseModel</code> Pydantic model defining the ImageGenerationOutput shape.
OpenAllImageGenerationAgent	class	app/nodes/openai_image_generation_agent.py:89	<code>NodeType</code> Workflow node type implementing the OpenAllImageGenerationAgent capability.
OpenAllImageGenerationAgent.required_services	method	app/nodes/openai_image_generation_agent.py:101	<code>cls, config: dict[str, Any]</code> Compute the required services.
OpenAllImageGenerationAgent.run	method	app/nodes/openai_image_generation_agent.py:114	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
PaperQAEvidenceSynthesizerInput	class	app/nodes/paperqa_evidence_synthesizer.py:29	<code>BaseModel</code> Pydantic model defining the PaperQAEvidenceSynthesizerInput shape.
PaperQAEvidenceSynthesizerConfig	class	app/nodes/paperqa_evidence_synthesizer.py:34	<code>BaseModel</code> Pydantic model defining the PaperQAEvidenceSynthesizerConfig shape.
PaperQAEvidenceSynthesizerConfig.coerce_documents	method	app/nodes/paperqa_evidence_synthesizer.py:56	<code>cls, value: Any</code> Internal helper for the coerce documents step.
DocumentCoverage	class	app/nodes/paperqa_evidence_synthesizer.py:68	<code>BaseModel</code> Pydantic model defining the DocumentCoverage shape.
ClaimSynthesis	class	app/nodes/paperqa_evidence_synthesizer.py:83	<code>BaseModel</code> Pydantic model defining the ClaimSynthesis shape.
PaperQAEvidenceSynthesizerOutput	class	app/nodes/paperqa_evidence_synthesizer.py:107	<code>BaseModel</code> Pydantic model defining the PaperQAEvidenceSynthesizerOutput shape.
PaperQAEvidenceSynthesizerAgent	class	app/nodes/paperqa_evidence_synthesizer.py:131	<code>NodeType</code> Workflow node type implementing the PaperQAEvidenceSynthesizerAgent capability.
PaperQAEvidenceSynthesizerAgent.required_services	method	app/nodes/paperqa_evidence_synthesizer.py:144	<code>cls, config: dict[str, Any]</code> Compute the required services.
PaperQAEvidenceSynthesizerAgent.run	method	app/nodes/paperqa_evidence_synthesizer.py:155	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
PaperQAEvidenceSynthesizerAgent.synthesize_claim	method	app/nodes/paperqa_evidence_synthesizer.py:221	<code>self, *, claim_id: str, question: str, documents: list[FullTextDocument], store: Any, settings: Any, paperqa: Any</code> Internal helper for the synthesize claim step.
PDFExtractInput	class	app/nodes/pdf_tool.py:24	<code>BaseModel</code> Pydantic model defining the PDFExtractInput shape.
PDFExtractConfig	class	app/nodes/pdf_tool.py:29	<code>BaseModel</code> Pydantic model defining the PDFExtractConfig shape.
PDFExtractOutput	class	app/nodes/pdf_tool.py:38	<code>BaseModel</code> Pydantic model defining the PDFExtractOutput shape.

Symbol	Kind	Location	Signature / summary
PDFTextExtractor	class	app/nodes/pdf_tool.py:50	NodeType Workflow node type implementing the PDFTextExtractor capability.
PDFTextExtractor.required_services	method	app/nodes/pdf_tool.py:59	cls, config: dict[str, Any] Compute the required services.
PDFTextExtractor.run	method	app/nodes/pdf_tool.py:70	self, state, resolved_config: dict[str, Any] Run the result.
PDFRenderInput	class	app/nodes/pdf_tool.py:96	BaseModel Pydantic model defining the PDFRenderInput shape.
PDFRenderConfig	class	app/nodes/pdf_tool.py:101	BaseModel Pydantic model defining the PDFRenderConfig shape.
PDFRenderOutput	class	app/nodes/pdf_tool.py:118	BaseModel Pydantic model defining the PDFRenderOutput shape.
PDFProposalRenderrer	class	app/nodes/pdf_tool.py:132	NodeType Workflow node type implementing the PDFProposalRenderrer capability.
PDFProposalRenderrer.required_services	method	app/nodes/pdf_tool.py:141	cls, config: dict[str, Any] Compute the required services.
PDFProposalRenderrer.run	method	app/nodes/pdf_tool.py:152	self, state, resolved_config: dict[str, Any] Run the result.
PPTInput	class	app/nodes/powerpoint_tool.py:17	BaseModel Pydantic model defining the PPTInput shape.
PPTConfig	class	app/nodes/powerpoint_tool.py:22	BaseModel Pydantic model defining the PPTConfig shape.
PPTOutput	class	app/nodes/powerpoint_tool.py:35	BaseModel Pydantic model defining the PPTOutput shape.
PowerPointProposalSlides	class	app/nodes/powerpoint_tool.py:49	NodeType Workflow node type implementing the PowerPointProposalSlides capability.
PowerPointProposalSlides.required_services	method	app/nodes/powerpoint_tool.py:58	cls, config: dict[str, Any] Compute the required services.
PowerPointProposalSlides.run	method	app/nodes/powerpoint_tool.py:69	self, state, resolved_config: dict[str, Any] Run the result.
PriorProjectRetrieverInput	class	app/nodes/prior_project_retriever.py:55	BaseModel Pydantic model defining the PriorProjectRetrieverInput shape.
PriorProjectRetrieverConfig	class	app/nodes/prior_project_retriever.py:60	BaseModel Pydantic model defining the PriorProjectRetrieverConfig shape.
PriorProjectRetrieverConfig.coerce_briefs	method	app/nodes/prior_project_retriever.py:87	cls, value: Any Internal helper for the coerce briefs step.
PriorProjectRetrieverOutput	class	app/nodes/prior_project_retriever.py:111	BaseModel Pydantic model defining the PriorProjectRetrieverOutput shape.

Symbol	Kind	Location	Signature / summary
PriorProjectRetrieverAgent	class	app/nodes/ prior_project_retriever.py: 139	NodeType Workflow node type implementing the PriorProjectRetrieverAgent capability.
PriorProjectRetrieverAgent .required_services	method	app/nodes/ prior_project_retriever.py: 152	cls, config: dict[str, Any] Compute the required services.
PriorProjectRetrieverAgent .run	method	app/nodes/ prior_project_retriever.py: 163	self, state: dict[str, Any], resolved_config: dict[str, Any] Run the result.
_allowed_official_url	function	app/nodes/ prior_project_retriever.py: 346	url: str, hosts: tuple[str, ...] Internal helper for the allowed official url step.
PairVerdict	class	app/nodes/ proposal_evidence_factory.p y:61	BaseModel Pydantic model defining the PairVerdict shape.
ProposalEvidenceFactoryI nput	class	app/nodes/ proposal_evidence_factory.p y:80	BaseModel Pydantic model defining the ProposalEvidenceFactoryInput shape.
ProposalEvidenceFactory Config	class	app/nodes/ proposal_evidence_factory.p y:85	BaseModel Pydantic model defining the ProposalEvidenceFactoryConfig shape.
ProposalEvidenceFactory Config._coerce_candidat s	method	app/nodes/ proposal_evidence_factory.p y:113	cls, value: Any Internal helper for the coerce candidates step.
ProposalEvidenceFactory Config._coerce_document s	method	app/nodes/ proposal_evidence_factory.p y:126	cls, value: Any Internal helper for the coerce documents step.
ProposalEvidenceFactory Config._coerce_search_au dit	method	app/nodes/ proposal_evidence_factory.p y:139	cls, value: Any Internal helper for the coerce search audit step.
ProposalEvidenceFactory Config._coerce_rejected_c andidates	method	app/nodes/ proposal_evidence_factory.p y:152	cls, value: Any Internal helper for the coerce rejected candidates step.
ProposalEvidenceFactory Output	class	app/nodes/ proposal_evidence_factory.p y:166	ProposalEvidencePackage Provides the ProposalEvidenceFactoryOutput behaviour.
_materiality	function	app/nodes/ proposal_evidence_factory.p y:179	claim_type: str Internal helper for the materiality step.
_authority	function	app/nodes/ proposal_evidence_factory.p y:195	value: str Internal helper for the authority step.
_strength	function	app/nodes/ proposal_evidence_factory.p y:210	score: int Internal helper for the strength step.
_evidence_score	function	app/nodes/ proposal_evidence_factory.p y:230	*, document: FullTextDocument, stance: str, confidence: float, exact_locator: bool, corroborated: bool Internal helper for the evidence score step.

Symbol	Kind	Location	Signature / summary
<code>_bibtex_key</code>	function	<code>app/nodes/proposal_evidence_factory.py:333</code>	<code>entry: CitationRegistryEntry</code> Internal helper for the bibtex key step.
<code>ProposalEvidenceFactoryAgent</code>	class	<code>app/nodes/proposal_evidence_factory.py:351</code>	<code>NodeType</code> Workflow node type implementing the <code>ProposalEvidenceFactoryAgent</code> capability.
<code>ProposalEvidenceFactoryAgent.required_services</code>	method	<code>app/nodes/proposal_evidence_factory.py:363</code>	<code>cls, config: dict[str, Any]</code> Compute the required services.
<code>ProposalEvidenceFactoryAgent.run</code>	method	<code>app/nodes/proposal_evidence_factory.py:374</code>	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
<code>ProposalEvidenceFactoryAgent.verify_document</code>	method	<code>app/nodes/proposal_evidence_factory.py:747</code>	<code>self, *, llm: Any, store: Any, claim_text: str, document: FullTextDocument, candidate: CandidateSource None, model: str, passage_limit: int</code> Verify the document.
<code>ProposalEvidenceFactoryAgent.document</code>	method	<code>app/nodes/proposal_evidence_factory.py:907</code>	<code>document_id: str, documents: list[FullTextDocument]</code> Internal helper for the document step.
<code>ProposalEvidenceFactoryAgent.source_id_for_document</code>	method	<code>app/nodes/proposal_evidence_factory.py:926</code>	<code>document: FullTextDocument</code> Internal helper for the source id for document step.
<code>ProposalEvidenceFactoryAgent.source_from_document</code>	method	<code>app/nodes/proposal_evidence_factory.py:942</code>	<code>self, document: FullTextDocument, candidate: CandidateSource None, *, passage: str</code> Internal helper for the source from document step.
<code>ProposalEvidenceFactoryAgent.relation_from_link</code>	method	<code>app/nodes/proposal_evidence_factory.py:974</code>	<code>link: ClaimEvidenceLink, document: FullTextDocument</code> Internal helper for the relation from link step.
<code>ProposalEvidenceFactoryAgent.gap_for_claim</code>	method	<code>app/nodes/proposal_evidence_factory.py:1013</code>	<code>claim_id: str, final_status: str, materiality: str, *, independent_count: int, required_independent: int, contradiction_searched: bool, metadata_ok: bool, retraction_ok: bool, policy: EvidencePolicy</code> Internal helper for the gap for claim step.
<code>ProposalEvidenceFactoryAgent.citation_registry</code>	method	<code>app/nodes/proposal_evidence_factory.py:1080</code>	<code>self, retained: dict[str, FullTextDocument], candidates: dict[str, CandidateSource], claims: list[VerifiedClaim]</code> Internal helper for the citation registry step.
<code>ProposalEvidenceFactoryAgent.cited_markdown</code>	method	<code>app/nodes/proposal_evidence_factory.py:1130</code>	<code>claims: list[VerifiedClaim]</code> Internal helper for the cited markdown step.
<code>ProposalEvidenceFactoryAgent.bibtex</code>	method	<code>app/nodes/proposal_evidence_factory.py:1158</code>	<code>entry: CitationRegistryEntry</code> Internal helper for the bibtex step.
<code>ProposalEvidenceFactoryAgent.quantitative_registry</code>	method	<code>app/nodes/proposal_evidence_factory.py:1182</code>	<code>claims: list[VerifiedClaim], links: list[ClaimEvidenceLink]</code> Internal helper for the quantitative registry step.

Symbol	Kind	Location	Signature / summary
ProposalEvidenceFactoryAgent_quality_report	method	app/nodes/ proposal_evidence_factory.py:1235	<code>*, policy: EvidencePolicy, verified_claims: list[VerifiedClaim], citation_registry: list[CitationRegistryEntry], links: list[ClaimEvidenceLink], audit: list[SearchAuditRecord], rejected: list[RejectedCandidate], quantitative_registry: list[dict[str, Any]]</code> Internal helper for the quality report step.
ProposalSubmissionGateInput	class	app/nodes/ proposal_submission_gate.py:13	<code>BaseModel</code> Pydantic model defining the ProposalSubmissionGateInput shape.
ProposalSubmissionGateConfig	class	app/nodes/ proposal_submission_gate.py:18	<code>BaseModel</code> Pydantic model defining the ProposalSubmissionGateConfig shape.
ProposalSubmissionGateOutput	class	app/nodes/ proposal_submission_gate.py:54	<code>BaseModel</code> Pydantic model defining the ProposalSubmissionGateOutput shape.
_messages	function	app/nodes/ proposal_submission_gate.py:78	<code>value: str list[Any]</code> Internal helper for the messages step.
ProposalSubmissionGate	class	app/nodes/ proposal_submission_gate.py:108	<code>NodeType</code> Combine hard proposal checks into one auditable release decision.
ProposalSubmissionGate.run	method	app/nodes/ proposal_submission_gate.py:125	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
ProposalTruthGraphInput	class	app/nodes/ proposal_truth_graph.py:24	<code>BaseModel</code> Pydantic model defining the ProposalTruthGraphInput shape.
ProposalTruthGraphConfig	class	app/nodes/ proposal_truth_graph.py:29	<code>BaseModel</code> Pydantic model defining the ProposalTruthGraphConfig shape.
ProposalTruthGraphConfig._coerce_verified_claims	method	app/nodes/ proposal_truth_graph.py:55	<code>cls, value: Any</code> Internal helper for the coerce verified claims step.
ProposalTruthGraphConfig._coerce_evidence_gaps	method	app/nodes/ proposal_truth_graph.py:72	<code>cls, value: Any</code> Internal helper for the coerce evidence gaps step.
ProposalTruthGraphConfig._coerce_structured_data_records	method	app/nodes/ proposal_truth_graph.py:89	<code>cls, value: Any</code> Internal helper for the coerce structured data records step.
ProposalTruthGraphConfig._coerce_internal_evidence_records	method	app/nodes/ proposal_truth_graph.py:106	<code>cls, value: Any</code> Internal helper for the coerce internal evidence records step.
ProposalTruthGraphOutput	class	app/nodes/ proposal_truth_graph.py:122	<code>BaseModel</code> Pydantic model defining the ProposalTruthGraphOutput shape.
ProposalTruthGraphAgent	class	app/nodes/ proposal_truth_graph.py:149	<code>NodeType</code> Workflow node type implementing the ProposalTruthGraphAgent capability.

Symbol	Kind	Location	Signature / summary
ProposalTruthGraphAgent.run	method	app/nodes/proposal_truth_graph.py:160	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
_review_queue	function	app/nodes/proposal_truth_graph.py:325	<code>claims: list[VerifiedClaim], gaps: list[EvidenceGap], graph: Any</code> Internal helper for the review queue step.
_string_list	function	app/nodes/proposal_truth_graph.py:383	<code>value: Any</code> Internal helper for the string list step.
PythonSnippetConfig	class	app/nodes/python_snippet.py:36	<code>BaseModel</code> Pydantic model defining the PythonSnippetConfig shape.
PythonSnippetInput	class	app/nodes/python_snippet.py:75	<code>BaseModel</code> Pydantic model defining the PythonSnippetInput shape.
PythonSnippetOutput	class	app/nodes/python_snippet.py:80	<code>BaseModel</code> Pydantic model defining the PythonSnippetOutput shape.
PythonSnippetAgent	class	app/nodes/python_snippet.py:100	<code>NodeType</code> Workflow node type implementing the PythonSnippetAgent capability.
PythonSnippetAgent.required_services	method	app/nodes/python_snippet.py:141	<code>cls, config: dict[str, Any]</code> Compute the required services.
PythonSnippetAgent.preflight_static_output_values	method	app/nodes/python_snippet.py:153	<code>cls, config: dict[str, Any]</code> Compute the preflight static output values.
PythonSnippetAgent.run	method	app/nodes/python_snippet.py:170	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
PythonSnippetAgent._failure	method	app/nodes/python_snippet.py:230	<code>self, cfg: PythonSnippetConfig, code: str, message: str, *, stdout: str='', stderr: str='', duration_s: float=0.0</code> Internal helper for the failure step.
scan_snippet_for_warnings	function	app/nodes/python_snippet.py:269	<code>code: str</code> AST-level authoring hints — never a security control.
assigns_output	function	app/nodes/python_snippet.py:298	<code>code: str</code> Best-effort: does this snippet's top level ever touch `output` at all?
RAGConfig	class	app/nodes/rag.py:19	<code>BaseModel</code> Pydantic model defining the RAGConfig shape.
RAGConfig.reject_ambiguous_saved_agent_config	method	app/nodes/rag.py:71	<code>self</code> Compute the reject ambiguous saved agent config.
RAGInput	class	app/nodes/rag.py:94	<code>BaseModel</code> Pydantic model defining the RAGInput shape.
Citation	class	app/nodes/rag.py:99	<code>BaseModel</code> Pydantic model defining the Citation shape.
RAGOutput	class	app/nodes/rag.py:116	<code>BaseModel</code> Pydantic model defining the RAGOutput shape.
RAGAgent	class	app/nodes/rag.py:156	

Symbol	Kind	Location	Signature / summary
			NodeType Workflow node type implementing the RAGAgent capability.
RAGAgent.required_services	method	app/nodes/rag.py:165	cls, config: dict[str, Any] Compute the required services.
RAGAgent.run	method	app/nodes/rag.py:178	self, state, resolved_config: dict[str, Any] Run the result.
_EmptySchema	class	app/nodes/registry.py:25	BaseModel Placeholder for nodes without a declared schema.
NodeRegistryError	class	app/nodes/registry.py:29	KeyError Exception raised for the NodeRegistryError case.
NodeRegistry	class	app/nodes/registry.py:34	Provides the NodeRegistry behaviour.
NodeRegistry.register	method	app/nodes/registry.py:43	cls, node_class: Type[NodeType] Register a node class.
NodeRegistry.get	method	app/nodes/registry.py:66	cls, type_name: str Return the result.
NodeRegistry.manifest	method	app/nodes/registry.py:83	cls Return the JSON schemas and presentation metadata used by the Builder.
_manifest_entry	function	app/nodes/registry.py:97	klass: Type[NodeType] Internal helper for the manifest entry step.
_uses_llm	function	app/nodes/registry.py:158	klass: Type[NodeType] Whether this node type needs a model, per its own preflight declaration.
_with_model_catalog	function	app/nodes/registry.py:173	schema: dict Expose the model catalog as a Builder dropdown.
ResearchSourceAcquirerInput	class	app/nodes/research_source_acquirer.py:35	BaseModel Pydantic model defining the ResearchSourceAcquirerInput shape.
ResearchSourceAcquirerConfig	class	app/nodes/research_source_acquirer.py:40	BaseModel Pydantic model defining the ResearchSourceAcquirerConfig shape.
ResearchSourceAcquirerConfig.coerce_candidates	method	app/nodes/research_source_acquirer.py:70	cls, value: Any Internal helper for the coerce candidates step.
ResearchSourceAcquirerOutput	class	app/nodes/research_source_acquirer.py:82	BaseModel Pydantic model defining the ResearchSourceAcquirerOutput shape.
ResearchSourceAcquirer	class	app/nodes/research_source_acquirer.py:100	NodeType Workflow node type implementing the ResearchSourceAcquirer capability.
ResearchSourceAcquirer.required_services	method	app/nodes/research_source_acquirer.py:112	cls, config: dict[str, Any] Compute the required services.
ResearchSourceAcquirer.run	method		

Symbol	Kind	Location	Signature / summary
		app/nodes/ research_source_acquirer.py :123	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
<code>_acquire_candidate</code>	async function	app/nodes/ research_source_acquirer.py :228	<code>candidate: CandidateSource, *, client: httpx.AsyncClient, store: Any, run_id: str, policy: EvidencePolicy, max_redirects: int</code> Acquire the candidate.
<code>_bounded_candidates</code>	function	app/nodes/ research_source_acquirer.py :344	<code>candidates: list[CandidateSource], *, per_claim: int, total: int</code> Internal helper for the bounded candidates step.
<code>_fetch_public_source</code>	async function	app/nodes/ research_source_acquirer.py :388	<code>client: httpx.AsyncClient, url: str, *, max_bytes: int, max_redirects: int</code> Fetch the public source.
<code>_require_public_url</code>	async function	app/nodes/ research_source_acquirer.py :437	<code>url: str</code> Internal helper for the require public url step.
<code>_html_pages</code>	function	app/nodes/ research_source_acquirer.py :478	<code>raw: bytes, *, chars_per_page: int=9000</code> Internal helper for the html pages step.
<code>_metadata_status</code>	async function	app/nodes/ research_source_acquirer.py :507	<code>candidate: CandidateSource, *, final_url: str, client: httpx.AsyncClient</code> Internal helper for the metadata status step.
<code>RouteRule</code>	class	app/nodes/router.py:58	<code>BaseModel</code> One legacy rule.
<code>RouteCase</code>	class	app/nodes/router.py:65	<code>BaseModel</code> One deterministic branch in `conditions` mode.
<code>RouteCase.case_is_decidable</code>	method	app/nodes/router.py:79	<code>self</code> Compute the case is decidable.
<code>RouterConfig</code>	class	app/nodes/router.py:93	<code>BaseModel</code> Pydantic model defining the RouterConfig shape.
<code>RouterConfig.mode_has_what_it_needs</code>	method	app/nodes/router.py:155	<code>self</code> Compute the mode has what it needs.
<code>RouterConfig.multi_selection_not_supported_in_rule_mode</code>	method	app/nodes/router.py:174	<code>self</code> Compute the multi selection not supported in rule mode.
<code>RouterConfig.route_names</code>	method	app/nodes/router.py:191	<code>self</code> Every route this router can emit — read by preflight to compare against the edge's declared branches, and by the Builder to draw them.
<code>RouterOutput</code>	class	app/nodes/router.py:206	<code>BaseModel</code> Pydantic model defining the RouterOutput shape.
<code>RouterInput</code>	class	app/nodes/router.py:241	<code>BaseModel</code> Pydantic model defining the RouterInput shape.
<code>RouterAgent</code>	class	app/nodes/router.py:254	<code>NodeType</code> Workflow node type implementing the RouterAgent capability.
	method	app/nodes/router.py:317	

Symbol	Kind	Location	Signature / summary
RouterAgent.required_services			<code>cls, config: dict[str, Any]</code> Compute the required services.
RouterAgent.run	method	<code>app/nodes/router.py:330</code>	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
RouterAgent._route_by_field	method	<code>app/nodes/router.py:364</code>	<code>self, cfg: RouterConfig, state: WorkflowState</code> Internal helper for the route by field step.
RouterAgent._route_by_field_multi	method	<code>app/nodes/router.py:430</code>	<code>self, cfg: RouterConfig, state: WorkflowState</code> Multi-Route, field mode: 'route_field' resolves to a 'list' of values (e.g.
RouterAgent._route_by_conditions	method	<code>app/nodes/router.py:501</code>	<code>self, cfg: RouterConfig, state: WorkflowState</code> Internal helper for the route by conditions step.
RouterAgent._route_by_conditions_multi	method	<code>app/nodes/router.py:541</code>	<code>self, cfg: RouterConfig, state: WorkflowState</code> Multi-Route, conditions mode: every case is evaluated (never stop at the first match) and every matching route is collected — business policy here is additive, the same reasoning DecisionAgent's rule engine already applies (<code>app/runtime/rules.py</code>).
RouterAgent._route_by_rule	method	<code>app/nodes/router.py:592</code>	<code>self, cfg, state</code> Internal helper for the route by rule step.
RouterAgent._eval_condition	method	<code>app/nodes/router.py:618</code>	<code>self, condition: str None, state: dict</code> Safe expression eval — supports only 'path OP literal'.
RouterAgent._parse_literal	method	<code>app/nodes/router.py:649</code>	<code>s: str</code> Parse the literal.
RouterAgent._route_by_llm	method	<code>app/nodes/router.py:670</code>	<code>self, cfg</code> Internal helper for the route by llm step.
RouterAgent._route_by_llm_multi	method	<code>app/nodes/router.py:707</code>	<code>self, cfg: RouterConfig</code> Multi-Route, llm mode: ask the model to choose all routes that apply and parse a comma-separated list back — for genuinely fuzzy "which of these departments does this enquiry touch" judgment.
_branch_key	function	<code>app/nodes/router.py:752</code>	<code>value: Any</code> Normalise a routed value into a branch key.
_matched_summaries	function	<code>app/nodes/router.py:766</code>	<code>trace: GroupTrace</code> Internal helper for the matched summaries step.
_ResearchBriefDraft	class	<code>app/nodes/scientific_research_planner.py:16</code>	<code>BaseModel</code> Pydantic model defining the ResearchBriefDraft shape.
_ResearchPlanDraft	class	<code>app/nodes/scientific_research_planner.py:41</code>	<code>BaseModel</code> Pydantic model defining the ResearchPlanDraft shape.
ScientificResearchPlannerInput	class	<code>app/nodes/scientific_research_planner.py:52</code>	<code>BaseModel</code> Pydantic model defining the ScientificResearchPlannerInput shape.
	class		

Symbol	Kind	Location	Signature / summary
ScientificResearchPlannerConfig		app/nodes/scientific_research_planner.py:57	<code>BaseModel</code> Pydantic model defining the ScientificResearchPlannerConfig shape.
ScientificResearchPlannerOutput	class	app/nodes/scientific_research_planner.py:88	<code>BaseModel</code> Pydantic model defining the ScientificResearchPlannerOutput shape.
ScientificResearchPlannerAgent	class	app/nodes/scientific_research_planner.py:134	<code>NodeType</code> Workflow node type implementing the ScientificResearchPlannerAgent capability.
ScientificResearchPlannerAgent.required_services	method	app/nodes/scientific_research_planner.py:146	<code>cls, config: dict[str, Any]</code> Compute the required services.
ScientificResearchPlannerAgent.run	method	app/nodes/scientific_research_planner.py:157	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
_json_text	function	app/nodes/scientific_research_planner.py:350	<code>value: Any</code> Internal helper for the json text step.
ScientificSkillConfig	class	app/nodes/scientific_skill_agent.py:12	<code>BaseModel</code> Pydantic model defining the ScientificSkillConfig shape.
ScientificSkillInput	class	app/nodes/scientific_skill_agent.py:46	<code>BaseModel</code> Pydantic model defining the ScientificSkillInput shape.
ScientificSkillOutput	class	app/nodes/scientific_skill_agent.py:51	<code>BaseModel</code> Pydantic model defining the ScientificSkillOutput shape.
ScientificSkillAgent	class	app/nodes/scientific_skill_agent.py:67	<code>NodeType</code> Workflow node type implementing the ScientificSkillAgent capability.
ScientificSkillAgent.required_services	method	app/nodes/scientific_skill_agent.py:79	<code>cls, config: dict[str, Any]</code> Compute the required services.
ScientificSkillAgent.run	method	app/nodes/scientific_skill_agent.py:90	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
SQLQueryConfig	class	app/nodes/sql_query.py:36	<code>BaseModel</code> Pydantic model defining the SQLQueryConfig shape.
SQLQueryInput	class	app/nodes/sql_query.py:78	<code>BaseModel</code> Pydantic model defining the SQLQueryInput shape.
SQLQueryOutput	class	app/nodes/sql_query.py:83	<code>BaseModel</code> Pydantic model defining the SQLQueryOutput shape.
SQLQueryAgent	class	app/nodes/sql_query.py:108	<code>NodeType</code> Workflow node type implementing the SQLQueryAgent capability.
SQLQueryAgent.required_services	method	app/nodes/sql_query.py:147	<code>cls, config: dict[str, Any]</code> Compute the required services.
SQLQueryAgent.preflight_output_fields	method	app/nodes/sql_query.py:159	<code>cls, config: dict[str, Any]</code> Compute the preflight output fields.

Symbol	Kind	Location	Signature / summary
SQLQueryAgent.run	method	app/nodes/sql_query.py:173	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
SQLQueryAgent._failure	method	app/nodes/sql_query.py:231	<code>self, cfg: SQLQueryConfig, error: MCPToolError</code> Internal helper for the failure step.
StartFileField	class	app/nodes/start.py:39	<code>BaseModel</code> A file-typed form field.
StartFileField.resolved_source	method	app/nodes/start.py:54	<code>self</code> Compute the resolved source.
StartConfig	class	app/nodes/start.py:63	<code>BaseModel</code> Pydantic model defining the StartConfig shape.
StartInput	class	app/nodes/start.py:93	<code>BaseModel</code> Pydantic model defining the StartInput shape.
StartOutput	class	app/nodes/start.py:98	<code>BaseModel</code> Pydantic model defining the StartOutput shape.
StartAgent	class	app/nodes/start.py:117	<code>NodeType</code> Workflow node type implementing the StartAgent capability.
StartAgent.required_services	method	app/nodes/start.py:160	<code>cls, config: dict[str, Any]</code> Compute the required services.
StartAgent.preflight_output_fields	method	app/nodes/start.py:172	<code>cls, config: dict[str, Any]</code> Compute the preflight output fields.
StartAgent.preflight_static_output_values	method	app/nodes/start.py:197	<code>cls, config: dict[str, Any]</code> Compute the preflight static output values.
StartAgent.run	method	app/nodes/start.py:212	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
_as_field_specs	function	app/nodes/start.py:264	<code>fields: list[InputFieldBinding]</code> Internal helper for the as field specs step.
_StructuredDatasetPlan	class	app/nodes/structured_dataset_retriever.py:31	<code>BaseModel</code> Pydantic model defining the StructuredDatasetPlan shape.
StructuredDatasetRetrieverInput	class	app/nodes/structured_dataset_retriever.py:42	<code>BaseModel</code> Pydantic model defining the StructuredDatasetRetrieverInput shape.
StructuredDatasetRetrieverConfig	class	app/nodes/structured_dataset_retriever.py:47	<code>BaseModel</code> Pydantic model defining the StructuredDatasetRetrieverConfig shape.
StructuredDatasetRetrieverConfig.coerce_queries	method	app/nodes/structured_dataset_retriever.py:78	<code>cls, value: Any</code> Internal helper for the coerce queries step.
StructuredDatasetRetrieverOutput	class	app/nodes/structured_dataset_retriever.py:109	<code>BaseModel</code> Pydantic model defining the StructuredDatasetRetrieverOutput shape.
StructuredDatasetRetrieverAgent	class	app/nodes/structured_dataset_retriever.py:139	<code>NodeType</code> Workflow node type implementing the StructuredDatasetRetrieverAgent capability.

Symbol	Kind	Location	Signature / summary
StructuredDatasetRetrieverAgent.required_services	method	app/nodes/structured_dataset_retriever.py:152	<code>cls, config: dict[str, Any]</code> Compute the required services.
StructuredDatasetRetrieverAgent.run	method	app/nodes/structured_dataset_retriever.py:166	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
StructuredDatasetRetrieverAgent.plan_queries	method	app/nodes/structured_dataset_retriever.py:350	<code>self, cfg: StructuredDatasetRetrieverConfig, graph: Any</code> Internal helper for the plan queries step.
_json_text	function	app/nodes/structured_dataset_retriever.py:405	<code>value: Any</code> Internal helper for the json text step.
_deduplicate_queries	function	app/nodes/structured_dataset_retriever.py:419	<code>queries: list[StructuredDatasetQuery]</code> Internal helper for the deduplicate queries step.
_decode_jsonstat	function	app/nodes/structured_dataset_retriever.py:436	<code>*, query: StructuredDatasetQuery, payload: dict[str, Any], endpoint: str, parameters: dict[str, Any], accessed_at: str, response_sha256: str, snapshot_object_key: str, limit: int, source_version: str</code> Decode the jsonstat.
_coordinates	function	app/nodes/structured_dataset_retriever.py:558	<code>flat_index: int, sizes: list[int]</code> Internal helper for the coordinates step.
_dimension_code	function	app/nodes/structured_dataset_retriever.py:578	<code>dimension: dict[str, Any], position: int</code> Internal helper for the dimension code step.
_indexed_value	function	app/nodes/structured_dataset_retriever.py:599	<code>values: Any, index: int</code> Internal helper for the indexed value step.
SubprocessConfig	class	app/nodes/subprocess.py:54	<code>BaseModel</code> Pydantic model defining the SubprocessConfig shape.
SubprocessInput	class	app/nodes/subprocess.py:94	<code>BaseModel</code> Pydantic model defining the SubprocessInput shape.
SubprocessOutput	class	app/nodes/subprocess.py:99	<code>BaseModel</code> Pydantic model defining the SubprocessOutput shape.
SubprocessAgent	class	app/nodes/subprocess.py:115	<code>NodeType</code> Workflow node type implementing the SubprocessAgent capability.
SubprocessAgent.required_services	method	app/nodes/subprocess.py:158	<code>cls, config: dict[str, Any]</code> Compute the required services.
SubprocessAgent.run	method	app/nodes/subprocess.py:169	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
SubprocessAgent.get_or_create_launch	method	app/nodes/subprocess.py:199	<code>self, cfg: SubprocessConfig, state: WorkflowState</code> Return the or create launch.
	method		

Symbol	Kind	Location	Signature / summary
SubprocessAgent._finalize		app/nodes/subprocess.py:319	<code>self, cfg: SubprocessConfig, decision: dict[str, Any]</code> Finalize the result.
_load_child_workflow	function	app/nodes/subprocess.py:364	<code>name: str</code> Load the child workflow.
_resolve_child_inputs	function	app/nodes/subprocess.py:385	<code>cfg: SubprocessConfig, child_spec: Any, state: WorkflowState</code> Explicit mapping -> same-named parent workflow input -> same-named parent node's whole output -> None.
_coerce_child_input	function	app/nodes/subprocess.py:414	<code>value: Any, input_spec: Any</code> No summary declared.
TextAssemblerConfig	class	app/nodes/text_assembler.py:28	<code>BaseModel</code> Pydantic model defining the TextAssemblerConfig shape.
TextAssemblerInput	class	app/nodes/text_assembler.py:39	<code>BaseModel</code> Pydantic model defining the TextAssemblerInput shape.
TextAssemblerOutput	class	app/nodes/text_assembler.py:44	<code>BaseModel</code> Pydantic model defining the TextAssemblerOutput shape.
TextAssemblerAgent	class	app/nodes/text_assembler.py:64	<code>NodeType</code> Workflow node type implementing the TextAssemblerAgent capability.
TextAssemblerAgent.run	method	app/nodes/text_assembler.py:103	<code>self, state: Any, resolved_config: dict[str, Any]</code> Run the result.
TransformInputField	class	app/nodes/transform.py:25	<code>BaseModel</code> One named input the Prompt Template editor's Inputs section declares.
TransformConfig	class	app/nodes/transform.py:57	<code>BaseModel</code> Pydantic model defining the TransformConfig shape.
TransformConfig.has_a_prompt_source	method	app/nodes/transform.py:113	<code>self</code> Return whether a prompt source.
TransformConfig.deterministic_mode_has_operations	method	app/nodes/transform.py:129	<code>self</code> Compute the deterministic mode has operations.
is_new_style	function	app/nodes/transform.py:152	<code>cfg: TransformConfig</code> Whether this node was authored through the Inputs/Instructions/Outputs editor rather than with a hand-written 'prompt_template'.
_render_input_value	function	app/nodes/transform.py:163	<code>value: Any</code> Render one resolved input field's value for embedding in the prompt.
_is_semantically_empty	function	app/nodes/transform.py:179	<code>value: Any</code> No summary declared.
TransformInput	class	app/nodes/transform.py:198	<code>BaseModel</code> Pydantic model defining the TransformInput shape.
TransformOutput	class	app/nodes/transform.py:203	<code>BaseModel</code> Pydantic model defining the TransformOutput shape.

Symbol	Kind	Location	Signature / summary
<code>_decode_json_container</code>	function	<code>app/nodes/transform.py:228</code>	<code>value: Any</code> Decode a JSON object or array returned inside a string.
<code>_build_response_model</code>	function	<code>app/nodes/transform.py:275</code>	<code>name: str, schema: dict[str, str]</code> Build the response model.
<code>TransformAgent</code>	class	<code>app/nodes/transform.py:301</code>	<code>NodeType</code> Workflow node type implementing the TransformAgent capability.
<code>TransformAgent.required_services</code>	method	<code>app/nodes/transform.py:385</code>	<code>cls, config: dict[str, Any]</code> Compute the required services.
<code>TransformAgent.preflight_output_fields</code>	method	<code>app/nodes/transform.py:399</code>	<code>cls, config: dict[str, Any]</code> Compute the preflight output fields.
<code>TransformAgent.preflight_static_output_values</code>	method	<code>app/nodes/transform.py:435</code>	<code>cls, config: dict[str, Any]</code> Compute the preflight static output values.
<code>TransformAgent.run</code>	method	<code>app/nodes/transform.py:460</code>	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
<code>TransformAgent.run_deterministic</code>	method	<code>app/nodes/transform.py:484</code>	<code>self, cfg: TransformConfig, state: Any</code> Run the deterministic.
<code>TransformAgent.run_legacy</code>	method	<code>app/nodes/transform.py:518</code>	<code>self, cfg: TransformConfig, llm: Any</code> Run the legacy.
<code>TransformAgent.run_new_style</code>	method	<code>app/nodes/transform.py:608</code>	<code>self, cfg: TransformConfig, llm: Any</code> Run the new style.
<code>TransformAgent.new_style_system_prompt</code>	method	<code>app/nodes/transform.py:650</code>	<code>self, cfg: TransformConfig</code> Internal helper for the new style system prompt step.
<code>TransformAgent.new_style_user_prompt</code>	method	<code>app/nodes/transform.py:681</code>	<code>self, cfg: TransformConfig</code> Internal helper for the new style user prompt step.
<code>TransformAgent.failure_result</code>	method	<code>app/nodes/transform.py:700</code>	<code>self, error: Exception</code> The non-raising counterpart to the two `RuntimeError`s above — only reachable when `fail_on_error=False`.
<code>TransformAgent.complete_structured_with_retries</code>	method	<code>app/nodes/transform.py:713</code>	<code>self, llm: Any, *, model: str, system: str, user: str, response_model: Type[BaseModel], temperature: float, max_tokens: int, max_retries: int, fail_on_error: bool=True, reject_empty_fields: list[str] None=None</code> Complete the structured with retries.
<code>WebSearchAgentInput</code>	class	<code>app/nodes/web_search_agent.py:13</code>	<code>BaseModel</code> Pydantic model defining the WebSearchAgentInput shape.
<code>WebSearchAgentConfig</code>	class	<code>app/nodes/web_search_agent.py:18</code>	<code>BaseModel</code> Pydantic model defining the WebSearchAgentConfig shape.
<code>WebSearchHit</code>	class	<code>app/nodes/web_search_agent.py:44</code>	<code>BaseModel</code> Pydantic model defining the WebSearchHit shape.
<code>WebSearchAgentOutput</code>	class	<code>app/nodes/web_search_agent.py:61</code>	<code>BaseModel</code> Pydantic model defining the WebSearchAgentOutput shape.

Symbol	Kind	Location	Signature / summary
WebSearchAgent	class	app/nodes/ web_search_agent.py:85	<code>NodeType</code> Workflow node type implementing the WebSearchAgent capability.
WebSearchAgent.required_services	method	app/nodes/ web_search_agent.py:97	<code>cls, config: dict[str, Any]</code> Compute the required services.
WebSearchAgent.run	method	app/nodes/ web_search_agent.py:108	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
WorkflowFileLoaderInput	class	app/nodes/ workflow_file_loader.py:18	<code>BaseModel</code> Pydantic model defining the WorkflowFileLoaderInput shape.
WorkflowFileLoaderConfig	class	app/nodes/ workflow_file_loader.py:23	<code>BaseModel</code> Pydantic model defining the WorkflowFileLoaderConfig shape.
LoadedWorkflowFile	class	app/nodes/ workflow_file_loader.py:39	<code>BaseModel</code> Pydantic model defining the LoadedWorkflowFile shape.
WorkflowFileLoaderOutput	class	app/nodes/ workflow_file_loader.py:62	<code>BaseModel</code> Pydantic model defining the WorkflowFileLoaderOutput shape.
WorkflowFileLoader	class	app/nodes/ workflow_file_loader.py:82	<code>NodeType</code> Read PDF/Office/Markdown/code inputs while retaining images as refs.
WorkflowFileLoader.required_services	method	app/nodes/ workflow_file_loader.py:96	<code>cls, config: dict[str, Any]</code> Compute the required services.
WorkflowFileLoader.run	method	app/nodes/ workflow_file_loader.py:107	<code>self, state: dict[str, Any], resolved_config: dict[str, Any]</code> Run the result.
_friendly_number	function	app/nodes/ workflow_input.py:41	<code>value: float</code> "1.0" reads as a technical artifact of a float field; a business- friendly error message says "1", not "1.0".
InputFieldBinding	class	app/nodes/ workflow_input.py:78	<code>FieldSpec</code> One declared field and where its value comes from.
InputFieldBinding._default_incomplete_legacy_shape	method	app/nodes/ workflow_input.py:176	<code>cls, data: Any</code> A binding saved before 'item_type'/'fields'/'enum_values' existed on this model (when 'type' was an unvalidated free-text string) may be missing the piece its type now requires.
InputFieldBinding.resolved_source	method	app/nodes/ workflow_input.py:194	<code>self</code> Compute the resolved source.
WorkflowInputConfig	class	app/nodes/ workflow_input.py:203	<code>BaseModel</code> Pydantic model defining the WorkflowInputConfig shape.
WorkflowInputConfig.as_field_specs	method	app/nodes/ workflow_input.py:225	<code>self</code> The declared shape, as the same FieldSpec rows every other node uses.
WorkflowInputInput	class	app/nodes/ workflow_input.py:243	<code>BaseModel</code> Pydantic model defining the WorkflowInputInput shape.

Symbol	Kind	Location	Signature / summary
WorkflowInputOutput	class	app/nodes/ workflow_input.py:248	<code>BaseModel</code> Pydantic model defining the WorkflowInputOutput shape.
WorkflowInputAgent	class	app/nodes/ workflow_input.py:264	<code>NodeType</code> Workflow node type implementing the WorkflowInputAgent capability.
WorkflowInputAgent.required_services	method	app/nodes/ workflow_input.py:308	<code>cls, config: dict[str, Any]</code> Compute the required services.
WorkflowInputAgent.preflight_output_fields	method	app/nodes/ workflow_input.py:320	<code>cls, config: dict[str, Any]</code> Compute the preflight output fields.
WorkflowInputAgent.preflight_static_output_values	method	app/nodes/ workflow_input.py:341	<code>cls, config: dict[str, Any]</code> Compute the preflight static output values.
WorkflowInputAgent.run	method	app/nodes/ workflow_input.py:354	<code>self, state, resolved_config: dict[str, Any]</code> Run the result.
resolve_field_bindings	function	app/nodes/ workflow_input.py:377	<code>bindings: list[InputFieldBinding], sample: dict[str, Any], state: dict[str, Any]</code> Resolve each declared field's source path against workflow state.
apply_conditional_fields	function	app/nodes/ workflow_input.py:408	<code>bindings: list[InputFieldBinding], data: dict[str, Any], missing: list[str]</code> Evaluate visible_when/required_when against this form's OWN already-resolved field values — never workflow graph state, so this reuses app/runtime/rules.py's evaluate_group with zero new evaluation logic, just a context shaped so a bare field name resolves (the same "name matches a node_outputs key" shorthand resolve_path already supports elsewhere).
validate_field_constraints	function	app/nodes/ workflow_input.py:454	<code>bindings: list[InputFieldBinding], data: dict[str, Any]</code> Business-friendly validation beyond shape (§20/21 — never a raw Pydantic path like VALIDATION_SCHEMA_FIELD_002): text length/pattern, email/url/phone format, percentage range, and the semantic checks a compound preset's own object shape can't express on its own (date_range ordering, an allowed unit/
reject_values_outside_declared_shape	function	app/nodes/ workflow_input.py:519	<code>specs: list[FieldSpec], data: dict[str, Any]</code> Validate what actually arrived against the declared shape before any downstream node can read it — e.g.
LedgerEntry	class	app/observability/ cost_ledger.py:68	 Provides the LedgerEntry behaviour.
configure_pricing_db	function	app/observability/ cost_ledger.py:136	<code>db: Any</code> Called once from app/main.py's lifespan, right after Mongo is available.
_refresh_pricing_overrides_if_stale	function	app/observability/ cost_ledger.py:142	 Refresh the pricing overrides if stale.
CostLedger	class	app/observability/ cost_ledger.py:161	 Provides the CostLedger behaviour.

Symbol	Kind	Location	Signature / summary
CostLedger.calculate	method	app/observability/ cost_ledger.py:172	<code>model: str, input_tokens: int, output_tokens: int, *, cache_creation_input_tokens: int=0, cache_read_input_tokens: int=0</code> Compute the result.
CostLedger.record	method	app/observability/ cost_ledger.py:206	<code>self, entry: LedgerEntry</code> Record the result.
CostLedger.record_no_charge	method	app/observability/ cost_ledger.py:256	<code>self, *, run_id: str, session_id: str, node_id: str, stage: str, task_type: str='retrieval', note: str=''</code> Log a pipeline stage that genuinely has no model charge.
CostLedger.group_by	method	app/observability/ cost_ledger.py:291	<code>entries: list[dict], key: str</code> Group the by.
CostLedger.run_summary	method	app/observability/ cost_ledger.py:339	<code>self, run_id: str, session_id: str None=None</code> Run the summary.
CostLedger.session_summary	method	app/observability/ cost_ledger.py:373	<code>self, session_id: str</code> Compute the session summary.
CostLedger.query	method	app/observability/ cost_ledger.py:388	<code>self, *, since: datetime None=None, until: datetime None=None, collection_id: str None=None</code> Raw entries across the whole ledger (not scoped to one run/session), for account-wide reporting (app/api/cost_admin.py).
configure_logging	function	app/observability/ logging.py:11	<code>environment: str='development'</code> Configure the logging.
get_logger	function	app/observability/ logging.py:51	<code>name: str=__name__</code> Return the logger.
track_node	function	app/observability/ metrics.py:197	<code>node_type: str</code> Time a node, record its outcome, and track in-flight count.
record_paused	function	app/observability/ metrics.py:222	<code>node_type: str</code> HITL pause is not an error and not a success — its own outcome.
record_llm_usage	function	app/observability/ metrics.py:230	<code>model: str, prompt_tokens: int, completion_tokens: int, ok: bool</code> Record token usage and call outcome.
configure_tracing	function	app/observability/ tracing.py:19	<code>app</code> Configure the tracing.
ConceptAlternativeSet	class	app/proposal_graph/ concepts.py:60	<code>BaseModel</code> Pydantic model defining the ConceptAlternativeSet shape.
ConceptAlternativeSet.one_of_each_posture	method	app/proposal_graph/ concepts.py:69	<code>self</code> Compute the one of each posture.
_ConceptDraft	class	app/proposal_graph/ concepts.py:85	<code>BaseModel</code> Pydantic model defining the ConceptDraft shape.
_ConceptJudgment	class	app/proposal_graph/ concepts.py:110	<code>BaseModel</code> Pydantic model defining the ConceptJudgment shape.

Symbol	Kind	Location	Signature / summary
<code>_valid_ids</code>	function	<code>app/proposal_graph/concepts.py:129</code>	<code>values: list[str], known: set[str]</code> Internal helper for the valid ids step.
<code>score_concept</code>	function	<code>app/proposal_graph/concepts.py:142</code>	<code>concept: ConceptAlternative, graph: ProposalGraph</code> Evidence-weighted score based only on IDs that exist in the graph.
<code>_composite_score</code>	function	<code>app/proposal_graph/concepts.py:196</code>	<code>concept: ConceptAlternative, graph: ProposalGraph</code> Internal helper for the composite score step.
<code>_draft_one_concept</code>	async function	<code>app/proposal_graph/concepts.py:260</code>	<code>llm: Any, *, posture: ConceptPosture, graph_payload: dict[str, Any], concept_note: str, model: str</code> Draft the one concept.
<code>judge_concept</code>	async function	<code>app/proposal_graph/concepts.py:302</code>	<code>llm: Any, *, concept: ConceptAlternative, model: str, skill_guidance: str=''</code> Independently score the 5 qualitative dimensions and attack the concept.
<code>generate_concept_alternatives</code>	async function	<code>app/proposal_graph/concepts.py:341</code>	<code>llm: Any, *, graph: ProposalGraph, model: str='claude-opus-5', judge_model: str None=None, concept_note: str='', skill_catalog: Any None=None</code> Create conservative, balanced, and ambitious options from graph facts.
<code>CallCoverageRow</code>	class	<code>app/proposal_graph/coverage.py:13</code>	<code>BaseModel</code> Pydantic model defining the CallCoverageRow shape.
<code>CallCoverageMatrix</code>	class	<code>app/proposal_graph/coverage.py:39</code>	<code>BaseModel</code> Pydantic model defining the CallCoverageMatrix shape.
<code>_known_object_ids</code>	function	<code>app/proposal_graph/coverage.py:60</code>	<code>graph: ProposalGraph</code> Internal helper for the known object ids step.
<code>build_call_coverage_matrix</code>	function	<code>app/proposal_graph/coverage.py:88</code>	<code>graph: ProposalGraph</code> Build the call coverage matrix.
<code>ClaimSupportVerdict</code>	class	<code>app/proposal_graph/evidence_verification.py:20</code>	<code>BaseModel</code> Pydantic model defining the ClaimSupportVerdict shape.
<code>_normalise_space</code>	function	<code>app/proposal_graph/evidence_verification.py:35</code>	<code>value: str</code> Internal helper for the normalise space step.
<code>quote_exists_in_source</code>	function	<code>app/proposal_graph/evidence_verification.py:47</code>	<code>quote: str, source_text: str</code> Whitespace-tolerant, otherwise verbatim passage validation.
<code>verify_claim_against_text</code>	async function	<code>app/proposal_graph/evidence_verification.py:54</code>	<code>llm: Any, *, claim: str, source_text: str, model: str='claude-sonnet-4-5'</code> Return a typed verdict and fail closed when the quote is not traceable.
<code>relation_from_verdict</code>	function	<code>app/proposal_graph/evidence_verification.py:109</code>	<code>*, relation_id: str, claim_id: str, source_id: str, source_version_id: str None, locator: str, verifier_model: str, verdict: ClaimSupportVerdict</code> Compute the relation from verdict.

Symbol	Kind	Location	Signature / summary
ProposalGraph	class	app/proposal_graph/graph.py:55	<code>BaseModel</code> All 15 object types, each as an id-keyed dict.
_merge_object	function	app/proposal_graph/graph.py:76	<code>left: BaseModel, right: BaseModel</code> Field-by-field merge; right wins only where it has a non-null/non-empty value.
merge_graph	function	app/proposal_graph/graph.py:87	<code>left: Any, right: Any</code> Reducer for the `proposal_graph` state key.
empty_graph	function	app/proposal_graph/graph.py:108	<code></code> Compute the empty graph.
HorizonJudgeVerdict	class	app/proposal_graph/horizon_evaluator.py:38	<code>BaseModel</code> Pydantic model defining the HorizonJudgeVerdict shape.
HorizonJudgeResult	class	app/proposal_graph/horizon_evaluator.py:55	<code>HorizonJudgeVerdict</code> Provides the HorizonJudgeResult behaviour.
HorizonCriterionPanel	class	app/proposal_graph/horizon_evaluator.py:66	<code>BaseModel</code> Pydantic model defining the HorizonCriterionPanel shape.
HorizonEvaluationReport	class	app/proposal_graph/horizon_evaluator.py:81	<code>BaseModel</code> Pydantic model defining the HorizonEvaluationReport shape.
_provider	function	app/proposal_graph/horizon_evaluator.py:105	<code>model: str</code> Internal helper for the provider step.
validate_independent_models	function	app/proposal_graph/horizon_evaluator.py:120	<code>evaluator_models: list[str], generator_model: str None</code> Validate the independent models.
deterministic_horizon_blockers	function	app/proposal_graph/horizon_evaluator.py:142	<code>graph: ProposalGraph</code> Compute the deterministic horizon blockers.
evaluate_horizon_proposal	async function	app/proposal_graph/horizon_evaluator.py:191	<code>llm: Any, *, proposal_text: str, graph: ProposalGraph, generator_model: str None, evaluator_models: list[str], criterion_threshold: float=3.0, total_threshold: float=10.0</code> Compute the evaluate horizon proposal.
Status	class	app/proposal_graph/models.py:30	<code>str, Enum</code> Enumeration of Status values.
Authority	class	app/proposal_graph/models.py:37	<code>str, Enum</code> Provenance strength for evidence — used by the evidence layer later.
EvidenceStance	class	app/proposal_graph/models.py:56	<code>str, Enum</code> Enumeration of EvidenceStance values.
ConceptPosture	class	app/proposal_graph/models.py:63	<code>str, Enum</code> Enumeration of ConceptPosture values.
CallRequirement	class	app/proposal_graph/models.py:73	<code>BaseModel</code> Pydantic model defining the CallRequirement shape.
EvidenceSource	class	app/proposal_graph/models.py:101	<code>BaseModel</code> Pydantic model defining the EvidenceSource shape.
Claim	class	app/proposal_graph/models.py:127	<code>BaseModel</code> Pydantic model defining the Claim shape.

Symbol	Kind	Location	Signature / summary
EvidenceRelation	class	app/proposal_graph/models.py:150	<code>BaseModel</code> Auditable claim -> exact passage -> immutable source-version relation.
Objective	class	app/proposal_graph/models.py:171	<code>BaseModel</code> Pydantic model defining the Objective shape.
Innovation	class	app/proposal_graph/models.py:190	<code>BaseModel</code> Pydantic model defining the Innovation shape.
Result	class	app/proposal_graph/models.py:214	<code>BaseModel</code> Pydantic model defining the Result shape.
Outcome	class	app/proposal_graph/models.py:229	<code>BaseModel</code> Pydantic model defining the Outcome shape.
Impact	class	app/proposal_graph/models.py:246	<code>BaseModel</code> Pydantic model defining the Impact shape.
Task	class	app/proposal_graph/models.py:264	<code>BaseModel</code> Pydantic model defining the Task shape.
WorkPackage	class	app/proposal_graph/models.py:281	<code>BaseModel</code> Pydantic model defining the WorkPackage shape.
Partner	class	app/proposal_graph/models.py:306	<code>BaseModel</code> Pydantic model defining the Partner shape.
KPI	class	app/proposal_graph/models.py:332	<code>BaseModel</code> Pydantic model defining the KPI shape.
Risk	class	app/proposal_graph/models.py:358	<code>BaseModel</code> Pydantic model defining the Risk shape.
ComplianceObject	class	app/proposal_graph/models.py:383	<code>BaseModel</code> Pydantic model defining the ComplianceObject shape.
OpenQuestion	class	app/proposal_graph/models.py:403	<code>BaseModel</code> Pydantic model defining the OpenQuestion shape.
ConceptAlternative	class	app/proposal_graph/models.py:421	<code>BaseModel</code> Pydantic model defining the ConceptAlternative shape.
proposal_graph_from_state	function	app/proposal_graph/state.py:10	<code>state: dict[str, Any]</code> Compute the proposal graph from state.
proposal_graph_state_update	function	app/proposal_graph/state.py:24	<code>graph: ProposalGraph</code> Compute the proposal graph state update.
_now	function	app/proposal_graph/workspace_store.py:21	 Internal helper for the now step.
_canonical_hash	function	app/proposal_graph/workspace_store.py:30	<code>payload: Any</code> Internal helper for the canonical hash step.
SourceVersionRecord	class	app/proposal_graph/workspace_store.py:49	<code>BaseModel</code> Pydantic model defining the SourceVersionRecord shape.
ProposalSnapshot	class	app/proposal_graph/workspace_store.py:77	<code>BaseModel</code> Pydantic model defining the ProposalSnapshot shape.
ApprovalRecord	class	app/proposal_graph/workspace_store.py:100	<code>BaseModel</code> Pydantic model defining the ApprovalRecord shape.
ProposalWorkspaceStore	class		

Symbol	Kind	Location	Signature / summary
		app/proposal_graph/workspace_store.py:129	Provides the ProposalWorkspaceStore behaviour.
ProposalWorkspaceStore.ensure_indexes	method	app/proposal_graph/workspace_store.py:141	<code>self</code> Ensure the indexes.
ProposalWorkspaceStore.register_source_version	method	app/proposal_graph/workspace_store.py:175	<code>self, *, session_id: str, proposal_id: str, source_id: str, content: str, title: str, created_by: str, identifier: str None=None, authority: str='unverified', metadata: dict[str, Any] None=None</code> Register the source version.
ProposalWorkspaceStore.list_source_versions	method	app/proposal_graph/workspace_store.py:266	<code>self, *, session_id: str, proposal_id: str, source_id: str None=None</code> List the source versions.
ProposalWorkspaceStore.source_text	method	app/proposal_graph/workspace_store.py:295	<code>self, *, session_id: str, proposal_id: str, source_id: str, version_id: str None=None</code> Compute the source text.
ProposalWorkspaceStore.save_snapshot	method	app/proposal_graph/workspace_store.py:338	<code>self, *, session_id: str, proposal_id: str, graph: ProposalGraph, created_by: str, reason: str</code> Save the snapshot.
ProposalWorkspaceStore.request_approval	method	app/proposal_graph/workspace_store.py:387	<code>self, *, session_id: str, proposal_id: str, graph: ProposalGraph, stage: str, requested_by: str, selected_concept_id: str None=None</code> Compute the request approval.
ProposalWorkspaceStore.decide_approval	method	app/proposal_graph/workspace_store.py:454	<code>self, *, session_id: str, proposal_id: str, approval_id: str, decision: Literal['approved', 'rejected', 'changes_requested'], decided_by: str, comment: str None</code> Compute the decide approval.
ProposalWorkspaceStore.list_approvals	method	app/proposal_graph/workspace_store.py:503	<code>self, *, session_id: str, proposal_id: str</code> List the approvals.
RAGCitation	class	app/rag/models.py:9	<code>BaseModel</code> Pydantic model defining the RAGCitation shape.
RAGQueryResponse	class	app/rag/models.py:34	<code>BaseModel</code> Pydantic model defining the RAGQueryResponse shape.
_build_sources	function	app/rag/service.py:27	<code>citations: list[RAGCitation], chunk_by_id: dict[str, Any]</code> Group citations into one entry per source document (§37 dedup shape).
_build_relevant_context	function	app/rag/service.py:57	<code>chunks: list[Any]</code> One entry per retrieved chunk actually used to ground the answer.
RAGService	class	app/rag/service.py:75	<code></code> Provides the RAGService behaviour.
RAGService._select_route	method	app/rag/service.py:89	<code>self, config: RetrievalRoutingProfileConfig, query:</code>

Symbol	Kind	Location	Signature / summary
			<code>str, llm: Any</code> Select the route.
RAGService.query	method	<code>app/rag/service.py:147</code>	<code>self, *, owner_scope_id: str, rag_agent_id: str, query: str, runtime_filters: dict[str, Any] MetadataFilterGroup None=None, document_ids: list[str] None=None, runtime_context: dict[str, Any] None=None, llm: Any None=None</code> Query the result.
_projected_call_cost	function	<code>app/research/deep_research.py:39</code>	<code>model: str, prompt_text: str, max_tokens: int</code> Worst-case pre-call cost: rough input-token estimate, full max_tokens output.
ResearchBrief	class	<code>app/research/deep_research.py:59</code>	<code>BaseModel</code> Pydantic model defining the ResearchBrief shape.
ResearchCitation	class	<code>app/research/deep_research.py:88</code>	<code>BaseModel</code> Pydantic model defining the ResearchCitation shape.
ResearchToolTrace	class	<code>app/research/deep_research.py:125</code>	<code>BaseModel</code> Pydantic model defining the ResearchToolTrace shape.
ResearchUsage	class	<code>app/research/deep_research.py:136</code>	<code>BaseModel</code> Pydantic model defining the ResearchUsage shape.
ResearchDossier	class	<code>app/research/deep_research.py:151</code>	<code>BaseModel</code> Pydantic model defining the ResearchDossier shape.
BoundedToolResearchService	class	<code>app/research/deep_research.py:199</code>	<code></code> Bounded, in-process replacement for the OpenAI Deep Research job API.
BoundedToolResearchService.available	method	<code>app/research/deep_research.py:233</code>	<code>self</code> Compute the available.
BoundedToolResearchService.research	method	<code>app/research/deep_research.py:241</code>	<code>self, *, brief: ResearchBrief, instructions: str, max_duration_seconds: float=1800.0, max_iterations: int=15, max_cost_per_call_usd: float=15.0</code> Compute the research.
BoundedToolResearchService.attribute_citations	method	<code>app/research/deep_research.py:513</code>	<code>self, brief: ResearchBrief, citations: list[ResearchCitation], max_cost_per_call_usd: float</code> Map each gathered citation to ONE claim, in place.
_CitationAttribution	class	<code>app/research/deep_research.py:588</code>	<code>BaseModel</code> Pydantic model defining the CitationAttribution shape.
_CitationAttributionSet	class	<code>app/research/deep_research.py:607</code>	<code>BaseModel</code> Pydantic model defining the CitationAttributionSet shape.
_brief_prompt	function	<code>app/research/deep_research.py:616</code>	<code>brief: ResearchBrief</code> Internal helper for the brief prompt step.
get_deep_research_service	function	<code>app/research/deep_research.py:649</code>	<code>*, llm: Any, web_search: Any</code> Return the deep research service.
SkillDocument	class	<code>app/research/skills.py:163</code>	<code></code> Provides the SkillDocument behaviour.

Symbol	Kind	Location	Signature / summary
SkillDocument.metadata	method	app/research/skills.py:179	<code>self</code> Compute the metadata.
SkillSelection	class	app/research/skills.py:194	<code></code> Provides the SkillSelection behaviour.
SkillSelection.names	method	app/research/skills.py:205	<code>self</code> The names.
SkillSelection.versions	method	app/research/skills.py:210	<code>self</code> The versions.
ScientificSkillCatalog	class	app/research/skills.py:215	<code></code> Read and select only configured, locally installed skill documents.
ScientificSkillCatalog.refresh	method	app/research/skills.py:241	<code>self</code> Refresh the result.
ScientificSkillCatalog.probe	method	app/research/skills.py:274	<code>self</code> Probe the result.
ScientificSkillCatalog.metadata	method	app/research/skills.py:286	<code>self</code> Compute the metadata.
ScientificSkillCatalog.load_errors	method	app/research/skills.py:299	<code>self</code> The load errors.
ScientificSkillCatalog.loaded_skill_names	method	app/research/skills.py:304	<code>self</code> Allowlisted skills whose SKILL.md actually parsed successfully.
ScientificSkillCatalog.select	method	app/research/skills.py:314	<code>self, *, objective: str, requested: Iterable[str]=(), auto_select: bool=True, max_skills: int=3</code> Select the result.
ScientificSkillCatalog.prompt_bundle	method	app/research/skills.py:389	<code>self, selection: SkillSelection</code> Build a bounded prompt that represents every selected skill.
_parse_skill	function	app/research/skills.py:424	<code>text: str</code> Parse the skill.
_sanitize_instructions	function	app/research/skills.py:468	<code>text: str</code> Keep research guidance while removing execution/install directions.
_skill_score	function	app/research/skills.py:495	<code>objective: str, skill: SkillDocument</code> Internal helper for the skill score step.
_compress_one	async function	app/retrieval/compressor.py:47	<code>query: str, chunk: RetrievedChunk, llm: LLMGateway, model: str</code> Compress a single chunk.
compress_chunks	async function	app/retrieval/compressor.py:83	<code>query: str, chunks: list[RetrievedChunk], llm: LLMGateway, model: str='claude-haiku-4-5'</code> Compress all chunks in parallel.
GraphContextProvider	class	app/retrieval/context.py:20	<code>Protocol</code> Provides the GraphContextProvider behaviour.
GraphContextProvider.expand	method	app/retrieval/context.py:22	<code>self, chunks: list[RetrievedChunk], *, query: str</code> No summary declared.

Symbol	Kind	Location	Signature / summary
expand_context	async function	app/retrieval/context.py:27	<code>chunks: list[RetrievedChunk], *, strategy: str, client: Any, collection_name: str, filters: RetrievalFilters, index_id: str None</code> Apply the requested context-expansion strategy to already-kept chunks.
assemble_context	function	app/retrieval/context.py:67	<code>chunks: list[RetrievedChunk]</code> Build the final "[N] (source: ...)" block passed to generation.
RetrievalStrategy	class	app/retrieval/contracts.py:16	<code>Protocol</code> Provides the RetrievalStrategy behaviour.
FusionStrategy	class	app/retrieval/contracts.py:30	<code>Protocol</code> Provides the FusionStrategy behaviour.
Reranker	class	app/retrieval/contracts.py:37	<code>Protocol</code> Provides the Reranker behaviour.
ContextExpander	class	app/retrieval/contracts.py:44	<code>Protocol</code> Provides the ContextExpander behaviour.
metadata_property_name	function	app/retrieval/filters.py:49	<code>key: str</code> Sanitize a user-supplied metadata key into a safe Weaviate property name.
MetadataFilterValidationError	class	app/retrieval/filters.py:58	<code>ValueError</code> Exception raised for the MetadataFilterValidationError case.
_field_type	function	app/retrieval/filters.py:63	<code>field: str, schema: dict[str, Any]</code> Internal helper for the field type step.
_check_field	function	app/retrieval/filters.py:82	<code>field: str, schema: dict[str, Any]</code> Check the field.
_check_value	function	app/retrieval/filters.py:111	<code>field: str, value: Any, schema: dict[str, Any]</code> Reject a value whose type contradicts the collection's declared schema.
validate_metadata_document	function	app/retrieval/filters.py:134	<code>metadata: dict[str, Any], schema: dict[str, Any]</code> Validate an ingestion-time metadata document against a Collection schema.
_validate_group	function	app/retrieval/filters.py:142	<code>group: MetadataFilterGroup, schema: dict[str, Any]</code> Validate the group.
validate_metadata_filters	function	app/retrieval/filters.py:159	<code>metadata: MetadataFilterGroup None, schema: dict[str, Any]</code> Validate a caller-supplied (or self-query-generated) filter tree.
coerce_metadata_filter_group	function	app/retrieval/filters.py:174	<code>value: dict[str, Any] MetadataFilterGroup None</code> Normalize workflow "runtime_filters"/API filter payloads to one shape.
_predicate_to_filter	function	app/retrieval/filters.py:202	<code>predicate: MetadataFilterPredicate</code> Internal helper for the predicate to filter step.
_group_to_filter	function	app/retrieval/filters.py:235	<code>group: MetadataFilterGroup</code> Group the to filter.

Symbol	Kind	Location	Signature / summary
build_secure_where_filter	function	app/retrieval/filters.py:254	<code>filters: RetrievalFilters, *, index_id: str None=None, exclude_parent_chunks: bool=False</code> Compile the non-removable security clause, ANDed with user metadata.
reciprocal_rank_fusion	function	app/retrieval/fusion.py:9	<code>result_sets: list[list[RetrievedChunk]], *, limit: int</code> Fuse independently-ranked result sets (dense+sparse, or per-query multi-query results) into one ranking via Reciprocal Rank Fusion.
deduplicate	function	app/retrieval/fusion.py:36	<code>candidates: list[RetrievedChunk]</code> Drop repeat "chunk_id"s, keeping the highest-scoring occurrence.
_build_where_filter	function	app/retrieval/hybrid_search.py:22	<code>f: RetrievalFilters</code> Translate filters into a Weaviate Filter expression.
hybrid_search	async function	app/retrieval/hybrid_search.py:44	<code>client: Any, embedder: Embedder, collection_name: str, query: str, filters: RetrievalFilters, top_k: int, alpha: float</code> Hybrid BM25 + vector search with metadata pre-filter.
MetadataFilterPredicate	class	app/retrieval/models.py:30	<code>BaseModel</code> Pydantic model defining the MetadataFilterPredicate shape.
MetadataFilterGroup	class	app/retrieval/models.py:46	<code>BaseModel</code> Pydantic model defining the MetadataFilterGroup shape.
RetrievalFilters	class	app/retrieval/models.py:64	<code>BaseModel</code> Metadata pre-filter applied BEFORE hybrid search runs in Weaviate.
RetrievalFilters.user_filter_dump	method	app/retrieval/models.py:90	<code>self</code> Trace-safe view of caller-supplied filters (no security clauses).
RetrievalQuery	class	app/retrieval/models.py:96	<code>BaseModel</code> Single retrieval request.
RetrievedChunk	class	app/retrieval/models.py:131	<code>BaseModel</code> One chunk that survived the pipeline.
RetrievalStage	class	app/retrieval/models.py:176	<code>BaseModel</code> One named step in the staged retrieval pipeline, for the trace.
RetrievalResult	class	app/retrieval/models.py:186	<code>BaseModel</code> Result of one retrieve() call.
_safe_complete	async function	app/retrieval/query_transform.py:21	<code>llm: Any, *, system: str, user: str</code> Internal helper for the safe complete step.
_split_lines	function	app/retrieval/query_transform.py:41	<code>text: str, *, limit: int</code> Split the lines.
transform_query	async function	app/retrieval/query_transform.py:55	<code>request: RetrievalQuery, *, llm: Any, metadata_schema: dict[str, Any]</code> Return "(semantic_query, transformed_queries, generated_filters)".
rewrite_query			

Symbol	Kind	Location	Signature / summary
	async function	app/retrieval/query_understanding.py:22	<code>query: str, llm: LLMGateway</code> Return a search-optimized version of the query.
<code>_RerankScore</code>	class	app/retrieval/reranker.py:33	<code>BaseModel</code> One score for one chunk.
<code>_RerankResponse</code>	class	app/retrieval/reranker.py:40	<code>BaseModel</code> Whole response: a list of scores, one per input chunk.
<code>_format_candidates</code>	function	app/retrieval/reranker.py:65	<code>candidates: list[RetrievedChunk]</code> Build the numbered candidate block for the user prompt.
<code>rerank</code>	async function	app/retrieval/reranker.py:81	<code>query: str, candidates: list[RetrievedChunk], top_n: int, llm: LLMGateway, model: str='gpt-5'</code> Score all candidates in one LLM call, return top_n sorted by score.
<code>retrieve</code>	async function	app/retrieval/retriever.py:30	<code>q: RetrievalQuery, *, weaviate_client: object, llm: LLMGateway, embedder: Embedder, collection_registry=None</code> Run the four-stage retrieval pipeline.
<code>RetrievalAuthorizationError</code>	class	app/retrieval/service.py:39	<code>PermissionError</code> Exception raised for the RetrievalAuthorizationError case.
<code>RetrievalCompatibilityError</code>	class	app/retrieval/service.py:44	<code>ValueError</code> Exception raised for the RetrievalCompatibilityError case.
<code>_safe_candidate</code>	function	app/retrieval/service.py:49	<code>chunk: RetrievedChunk</code> Trace score/provenance, not duplicate raw source text per stage.
<code>RetrievalService</code>	class	app/retrieval/service.py:69	 Provides the RetrievalService behaviour.
<code>RetrievalService.retrieve</code>	method	app/retrieval/service.py:115	<code>self, request: RetrievalQuery, *, owner_scope_id: str, llm: Any None=None</code> Retrieve the result.
<code>RetrievalService._record_failed_trace</code>	method	app/retrieval/service.py:154	<code>self, *, request: RetrievalQuery, request_id: str, owner_scope_id: str, error: Exception, total_ms: float</code> Record the failed trace.
<code>RetrievalService._retrieve_impl</code>	method	app/retrieval/service.py:211	<code>self, request: RetrievalQuery, *, owner_scope_id: str, request_id: str, llm: Any None=None</code> Retrieve the impl.
<code>RetrievalService.compare</code>	method	app/retrieval/service.py:644	<code>self, requests: list[RetrievalQuery], *, owner_scope_id: str, llm: Any None=None</code> Compare the result.
<code>_chunk_from_object</code>	function	app/retrieval/strategies.py:16	<code>obj: Any, *, score_kind: str, rank: int</code> Chunk the from object.
<code>dense_search</code>	async function	app/retrieval/strategies.py:63	<code>*, client: Any, collection_name: str, embedder: Embedder, query: str, filters: RetrievalFilters, index_id: str None, top_k: int, exclude_parent_chunks:</code>

Symbol	Kind	Location	Signature / summary
			<code>bool=False</code> Compute the dense search.
<code>sparse_search</code>	async function	<code>app/retrieval/strategies.py:106</code>	<code>*, client: Any, collection_name: str, query: str, filters: RetrievalFilters, index_id: str None, top_k: int, exclude_parent_chunks: bool=False</code> Compute the sparse search.
<code>native_hybrid_search</code>	async function	<code>app/retrieval/strategies.py:146</code>	<code>*, client: Any, collection_name: str, embedder: Embedder, query: str, filters: RetrievalFilters, index_id: str None, top_k: int, alpha: float, exclude_parent_chunks: bool=False</code> Compute the native hybrid search.
<code>fetch_chunks_by_id</code>	async function	<code>app/retrieval/strategies.py:197</code>	<code>*, client: Any, collection_name: str, chunk_ids: list[str], filters: RetrievalFilters, index_id: str None=None</code> Fetch the chunks by id.
<code>ensure_collection_schema_on</code>	function	<code>app/retrieval/weaviate_client.py:75</code>	<code>client: weaviate.WeaviateClient, name: str</code> Create "name" with the standard chunk schema if it doesn't exist yet.
<code>upsert_objects_on</code>	function	<code>app/retrieval/weaviate_client.py:100</code>	<code>client: weaviate.WeaviateClient, collection_name: str, objects: list[dict[str, Any]], vectors: list[list[float]]</code> Batch-insert "objects"/"vectors" into "collection_name".
<code>WeaviateClient</code>	class	<code>app/retrieval/weaviate_client.py:141</code>	<code></code> Thin wrapper around weaviate-client v4.
<code>WeaviateClient.connect</code>	method	<code>app/retrieval/weaviate_client.py:157</code>	<code>self</code> Open the gRPC + REST connection.
<code>WeaviateClient.close</code>	method	<code>app/retrieval/weaviate_client.py:191</code>	<code>self</code> Close the underlying gRPC channel.
<code>WeaviateClient.ensure_schema</code>	method	<code>app/retrieval/weaviate_client.py:200</code>	<code>self</code> Create the DocumentChunk collection if it doesn't exist.
<code>WeaviateClient.ensure_collection_schema</code>	method	<code>app/retrieval/weaviate_client.py:226</code>	<code>self, name: str</code> Like :meth:`ensure_schema`, but for an arbitrary physical collection.
<code>WeaviateClient.upsert_objects</code>	method	<code>app/retrieval/weaviate_client.py:231</code>	<code>self, collection_name: str, objects: list[dict[str, Any]], vectors: list[list[float]]</code> Like :meth:`upsert_chunks`, but targeting an arbitrary physical collection.
<code>WeaviateClient.reset_schema</code>	method	<code>app/retrieval/weaviate_client.py:238</code>	<code>self</code> DESTRUCTIVE: delete the collection and recreate it.
<code>WeaviateClient.upsert_chunks</code>	method	<code>app/retrieval/weaviate_client.py:251</code>	<code>self, chunks: list[dict[str, Any]], vectors: list[list[float]]</code> Insert or replace chunks.
<code>WeaviateClient.count_chunks</code>	method	<code>app/retrieval/weaviate_client.py:303</code>	<code>self</code> Total chunks in the collection.

Symbol	Kind	Location	Signature / summary
WeaviateClient.add_properly	method	app/retrieval/weaviate_client.py:309	<code>self, prop: Property</code> Non-destructive schema evolution.
get_weaviate_client	function	app/retrieval/weaviate_client.py:319	<code></code> Lazily-constructed module-level client.
DeterministicFixResult	class	app/runtime/autofix.py:62	<code></code> Provides the DeterministicFixResult behaviour.
_get_path	function	app/runtime/autofix.py:75	<code>raw: Any, path: str</code> Return the path.
_set_path	function	app/runtime/autofix.py:91	<code>raw: Any, path: str, value: Any</code> Set the path.
_replace_template_reference	function	app/runtime/autofix.py:108	<code>text: str, node_id: str, old_remainder: str, new_remainder: str</code> Rewrite the field portion of a <code>`{{node_id.old_remainder}}`</code> (or <code>`{{outputs.node_id.old_remainder}}`</code>) reference to <code>`new_remainder`</code> , leaving everything else in the string untouched.
_replace_template_node_id	function	app/runtime/autofix.py:142	<code>text: str, old_node_id: str, new_node_id: str</code> Rewrite the node-id portion of a template reference, keeping whatever field path followed it.
_node_by_id	function	app/runtime/autofix.py:170	<code>raw: dict, node_id: str</code> Internal helper for the node by id step.
_fix_template_unknown_output_field	function	app/runtime/autofix.py:186	<code>raw: dict, issue: PreflightIssue</code> Internal helper for the fix template unknown output field step.
_fix_template_unknown_structured_field	function	app/runtime/autofix.py:233	<code>raw: dict, issue: PreflightIssue</code> Internal helper for the fix template unknown structured field step.
_fix_template_unknown_node	function	app/runtime/autofix.py:297	<code>raw: dict, issue: PreflightIssue</code> Internal helper for the fix template unknown node step.
_replace_template_named_root_segment	function	app/runtime/autofix.py:338	<code>text: str, root: str, old_name: str, new_name: str</code> Rewrite a <code>`{{root.old_name...}}`</code> reference (root is <code>`inputs`</code> or <code>`variables`</code>) to address <code>`new_name`</code> instead, keeping any remainder.
_fix_template_unknown_named_root	function	app/runtime/autofix.py:365	<code>raw: dict, issue: PreflightIssue, *, root: str, noun: str</code> Internal helper for the fix template unknown named root step.
_fix_template_unknown_input	function	app/runtime/autofix.py:405	<code>raw: dict, issue: PreflightIssue</code> Internal helper for the fix template unknown input step.
_fix_template_unknown_variable	function	app/runtime/autofix.py:418	<code>raw: dict, issue: PreflightIssue</code> Internal helper for the fix template unknown variable step.
_fix_router_multiple_defaults	function	app/runtime/autofix.py:431	<code>raw: dict, issue: PreflightIssue</code> A legacy rule-mode Router with more than one <code>`default: true`</code> rule — keep the first and clear the flag on the rest.
_fix_router_duplicate_rule	function	app/runtime/autofix.py:463	<code>raw: dict, issue: PreflightIssue</code> Duplicate rule names in a legacy rule-mode Router — keep the first occurrence of each name.

Symbol	Kind	Location	Signature / summary
<code>_fix_unknown_node_type</code>	function	<code>app/runtime/autofix.py:496</code>	<code>raw: dict, issue: PreflightIssue</code> Internal helper for the fix unknown node type step.
<code>_fix_model_not_in_catalog</code>	function	<code>app/runtime/autofix.py:522</code>	<code>raw: dict, issue: PreflightIssue</code> Internal helper for the fix model not in catalog step.
<code>_fix_unknown_node_config_field</code>	function	<code>app/runtime/autofix.py:560</code>	<code>raw: dict, issue: PreflightIssue</code> Internal helper for the fix unknown node config field step.
<code>_accepts_plain_string</code>	function	<code>app/runtime/autofix.py:597</code>	<code>annotation: Any</code> Whether a pydantic field annotation permits an ordinary <code>`str`</code> — true for <code>`str`</code> itself and for <code>`str   None`</code> / <code>`Optional[str]`</code> .
<code>_fix_dict_where_string_expected</code>	function	<code>app/runtime/autofix.py:607</code>	<code>raw: dict, issue: PreflightIssue</code> A recurring, purely mechanical generation mistake: giving a field that only accepts one string a dict of several labelled values instead — e.g.
<code>apply_deterministic_fixes</code>	function	<code>app/runtime/autofix.py:664</code>	<code>yaml_text: str, report: WorkflowPreflightReport</code> Apply the deterministic fixes.
<code>_workflow_identity</code>	function	<code>app/runtime/autofix.py:736</code>	<code>yaml_text: str</code> (name, node ids) for a parseable workflow dict, or (None, set()) if <code>`yaml_text`</code> isn't one — used to sanity-check an LLM repair actually revised the given workflow rather than returning an unrelated one.
<code>_preserves_identity</code>	function	<code>app/runtime/autofix.py:755</code>	<code>original_yaml: str, candidate_yaml: str</code> Guards against a repaired workflow that is well-formed but is not actually a fix of the input — the observed failure mode was an LLM repair call returning a different, generic workflow (one whose own job was "repair workflow YAML") instead of the user's real workflow with its validation errors fixed.
<code>format_preflight_feedback</code>	function	<code>app/runtime/autofix.py:774</code>	<code>report: WorkflowPreflightReport</code> Format the preflight feedback.
<code>LlmRepairAttempt</code>	class	<code>app/runtime/autofix.py:790</code>	 Provides the LlmRepairAttempt behaviour.
<code>repair_with_llm</code>	async function	<code>app/runtime/autofix.py:801</code>	<code>yaml_text: str, report: WorkflowPreflightReport, *, static_check: Callable[[str], Awaitable[WorkflowPreflightReport]], generate_yaml: Callable[[str, str None, str None], Awaitable[str]]</code> Feed remaining preflight errors back to the LLM up to MAX_LLM_REPAIR_ATTEMPTS times.
<code>_make_runtime_fn</code>	function	<code>app/runtime/compiler.py:63</code>	<code>instance, bus: RunEventBus None, services: dict</code> Wrap a NodeType instance so it conforms to LangGraph's 'state -> partial state' contract.
<code>_wire_edges</code>	function	<code>app/runtime/compiler.py:428</code>	<code>graph: StateGraph, edges: list[EdgeSpec], hitl_ids: set[str], multi_router_ids: set[str] None=None</code> Add edges, return the set of source node ids (used to compute terminals).
<code>compile_workflow</code>	function	<code>app/runtime/compiler.py:670</code>	

Symbol	Kind	Location	Signature / summary
			<code>spec: WorkflowSpec, checkpointer=None, services=None</code> Compute the compile workflow.
RedisLease	class	<code>app/runtime/coordination.py:29</code>	A renewable, compare-and-release distributed lease.
RedisLease.acquire	method	<code>app/runtime/coordination.py:46</code>	<code>self</code> Acquire the result.
RedisLease.renew	method	<code>app/runtime/coordination.py:62</code>	<code>self</code> Renew the result.
RedisLease.release	method	<code>app/runtime/coordination.py:80</code>	<code>self</code> Release the result.
RedisLease.keep_alive	method	<code>app/runtime/coordination.py:94</code>	<code>self</code> Return only when the lease is lost; cancellation stops the loop.
_merge_mapping	function	<code>app/runtime/domain_state.py:16</code>	<code>left: Any, right: Any</code> Safe default for simple dict-shaped domain state.
DomainStateRegistry	class	<code>app/runtime/domain_state.py:35</code>	Reducer registry keyed by a stable use-case namespace.
DomainStateRegistry.register	method	<code>app/runtime/domain_state.py:41</code>	<code>cls, namespace: str, reducer: DomainReducer</code> Register the result.
DomainStateRegistry.reducer_for	method	<code>app/runtime/domain_state.py:58</code>	<code>cls, namespace: str</code> Compute the reducer for.
merge_domain_state	function	<code>app/runtime/domain_state.py:70</code>	<code>left: dict[str, Any] None, right: dict[str, Any] None</code> LangGraph reducer for state written by independent use-case packs.
RunEvent	class	<code>app/runtime/events.py:34</code>	Provides the RunEvent behaviour.
RunEvent.terminal	method	<code>app/runtime/events.py:63</code>	<code>self</code> The terminal.
RunEvent.to_json	method	<code>app/runtime/events.py:67</code>	<code>self</code> Compute the to json.
RunEventBus	class	<code>app/runtime/events.py:82</code>	Provides the RunEventBus behaviour.
RunEventBus._key	method	<code>app/runtime/events.py:120</code>	<code>run_id: str, session_id: str None</code> Internal helper for the key step.
RunEventBus.publish	method	<code>app/runtime/events.py:132</code>	<code>self, evt: RunEvent</code> Publish the result.
RunEventBus._redis_names	method	<code>app/runtime/events.py:169</code>	<code>run_id: str, session_id: str None</code> Return opaque, tenant-scoped Redis keys without leaking identifiers.
RunEventBus._publish_redis	method	<code>app/runtime/events.py:178</code>	<code>self, evt: RunEvent</code> Publish the redis.
RunEventBus.subscribe	method	<code>app/runtime/events.py:202</code>	<code>self, run_id: str, session_id: str None=None, *, after_event_id: int </code>

Symbol	Kind	Location	Signature / summary
			None=None Subscribe the result.
RunEventBus._subscribe_redis	method	app/runtime/events.py:242	self, run_id: str, session_id: str None, *, after_event_id: int None Subscribe the redis.
RunEventBus._decode_redis_event	method	app/runtime/events.py:288	fields: dict[Any, Any] Decode the redis event.
RunEventBus._offer	method	app/runtime/events.py:307	queue: asyncio.Queue[RunEvent], event: RunEvent Internal helper for the offer step.
RunEventBus._pump_redis	method	app/runtime/events.py:319	self, pubsub: Any, queue: asyncio.Queue[RunEvent], last_seen: int Internal helper for the pump redis step.
RunEventBus.unsubscribe	method	app/runtime/events.py:353	self, run_id: str, queue: asyncio.Queue[RunEvent], session_id: str None=None Unsubscribe the result.
RunEventBus.close	method	app/runtime/events.py:393	self Stop Redis subscription pumps before the shared client is closed.
_find_rejection	function	app/runtime/executor.py:29	state: dict[str, Any] Find the rejection.
_project_output	function	app/runtime/executor.py:44	spec: WorkflowSpec, state: dict[str, Any], original_inputs: dict[str, Any] Build the caller-facing output declared by the workflow contract.
run_workflow	async function	app/runtime/executor.py:83	spec: WorkflowSpec, inputs: dict[str, Any], session_id: str None=None, collection_id: str='default', services: dict[str, Any] None=None, run_id: str None=None, reused_node_results: dict[str, dict[str, Any]] None=None, retry_source_run_id: str None=None, hitl_resume_decisions: dict[str, dict[str, Any]] None=None, resume_replay: bool=False Run the workflow.
FieldSpec	class	app/runtime/field_schema.py:67	BaseModel One row in the visual schema builder.
FieldSpec._migrate_legacy_item_enum_values	method	app/runtime/field_schema.py:96	cls, data: Any Fold list-of-enum values stored under the field-level `enum_values` key into `item_enum_values`.
FieldSpec.name_is_an_identifier	method	app/runtime/field_schema.py:122	cls, value: str Compute the name is an identifier.
FieldSpec.shape_is_coherent	method	app/runtime/field_schema.py:143	self Compute the shape is coherent.
validate_fields	function	app/runtime/field_schema.py:192	fields: list[FieldSpec], *, where: str='output' Reject duplicate names per level and runaway nesting depth.
_scalar_annotation	function		

Symbol	Kind	Location	Signature / summary
		app/runtime/ field_schema.py:240	<code>kind: str, enum_values: list[str]</code> Internal helper for the scalar annotation step.
<code>_annotation_for</code>	function	app/runtime/ field_schema.py:258	<code>spec: FieldSpec, model_name: str, *, permit_none: bool</code> Internal helper for the annotation for step.
<code>_build_model</code>	function	app/runtime/ field_schema.py:297	<code>name: str, fields: list[FieldSpec]</code> Build the model.
<code>_safe_model_name</code>	function	app/runtime/ field_schema.py:336	<code>name: str</code> Internal helper for the safe model name step.
<code>build_response_model</code>	function	app/runtime/ field_schema.py:351	<code>fields: list[FieldSpec], *, model_name: str='StructuredOutput'</code> Compile the visual schema into a Pydantic model.
<code>json_schema</code>	function	app/runtime/ field_schema.py:361	<code>fields: list[FieldSpec], *, model_name: str='StructuredOutput'</code> JSON Schema for the visual schema, via the compiled Pydantic model.
<code>FieldPath</code>	class	app/runtime/ field_schema.py:383	<code>BaseModel</code> One addressable value inside a compiled schema.
<code>field_paths</code>	function	app/runtime/ field_schema.py:403	<code>fields: list[FieldSpec], prefix: str=''</code> Flatten a visual schema into typed dotted paths, parents included.
<code>_field_paths</code>	function	app/runtime/ field_schema.py:413	<code>fields: list[FieldSpec], prefix: str, *, parent_unavailable: bool</code> Internal helper for the field paths step.
<code>parse_fields</code>	function	app/runtime/ field_schema.py:469	<code>raw: Any</code> Coerce YAML/JSON rows into FieldSpec, with a clear error on bad shape.
<code>describe_schema</code>	function	app/runtime/ field_schema.py:481	<code>fields: list[FieldSpec]</code> A compact, human-readable contract used in prompts and the About tab.
<code>HITLResumeError</code>	class	app/runtime/hitl.py:25	<code>KeyError</code> Exception raised for the HITLResumeError case.
<code>HITLResumeConflict</code>	class	app/runtime/hitl.py:30	<code>RuntimeError</code> Another worker already claimed this paused checkpoint.
<code>_find_rejection</code>	function	app/runtime/hitl.py:34	<code>state: dict</code> Any node output carrying decision == 'reject' means a HITL gate was rejected.
<code>_resume_output</code>	function	app/runtime/hitl.py:42	<code>checkpoint: dict[str, Any] None, final_state: dict[str, Any]</code> The same projection a fresh <code>`run_workflow`</code> completion computes (<code>executor._project_output`</code> ) — a resumed completion must produce it too, or every caller reading <code>result["output"]`</code> (subprocess delivery chief among them — see <code>`app/workflow/subprocess_callback.py`'s <code>_select_result`</code>, which is exactly how a SubprocessAgent parent learns its child's result) silently gets <code>`None`</code> for any workflow that paused at least once before finishing.</code>
<code>resume_workflow</code>	async function	app/runtime/hitl.py:62	

Symbol	Kind	Location	Signature / summary
			<code>run_id: str, decision: dict[str, Any]</code> Resume a paused workflow with the user's decision payload.
<code>_validate_saved_decision</code>	function	<code>app/runtime/hitl.py:70</code>	<code>checkpoint: dict[str, Any], decision: dict[str, Any]</code> Validate the saved decision.
<code>resume_workflow_durable</code>	async function	<code>app/runtime/hitl.py:109</code>	<code>run_id: str, decision: dict[str, Any], *, services: dict[str, Any] None=None, session_id: str None=None, actor: str='system'</code> Resume in memory when possible, otherwise rebuild from Mongo.
<code>load_workflow</code>	function	<code>app/runtime/loader.py:12</code>	<code>path: str Path</code> Load the workflow.
<code>load_workflow_from_string</code>	function	<code>app/runtime/loader.py:26</code>	<code>yaml_text: str</code> Load the workflow from string.
<code>_register_builders</code>	function	<code>app/runtime/logic_preflight.py:52</code>	Populate the typed-output builders lazily.
<code>_declared_value</code>	function	<code>app/runtime/logic_preflight.py:299</code>	<code>parsed: Any, name: str</code> Internal helper for the declared value step.
<code>_infer_kind</code>	function	<code>app/runtime/logic_preflight.py:318</code>	<code>value: Any</code> Internal helper for the infer kind step.
<code>_json_schema_kind</code>	function	<code>app/runtime/logic_preflight.py:353</code>	<code>schema: dict[str, Any]</code> Internal helper for the json schema kind step.
<code>_json_schema_paths</code>	function	<code>app/runtime/logic_preflight.py:370</code>	<code>schema: dict[str, Any], prefix: str</code> Flatten a JSON-Schema object's `properties` into typed dotted paths.
<code>_static_mcp_tool_registries</code>	function	<code>app/runtime/logic_preflight.py:413</code>	Per-server `TOOLS_BY_NAME` registries for the built-in MCP connectors.
<code>_static_mcp_tool_definition</code>	function	<code>app/runtime/logic_preflight.py:436</code>	<code>server_id: Any, tool_name: Any</code> Internal helper for the static mcp tool definition step.
<code>_static_mcp_output_schema</code>	function	<code>app/runtime/logic_preflight.py:451</code>	<code>server_id: Any, tool_name: Any</code> The declared output_schema for a tool this platform ships itself.
<code>_static_mcp_required_arguments</code>	function	<code>app/runtime/logic_preflight.py:460</code>	<code>server_id: Any, tool_name: Any, arguments: dict[str, Any]</code> Argument names the tool's own static schema requires but `arguments` never mentions — or `None` when this tool isn't one preflight can see statically (a live/third-party server), in which case there is nothing to report.
<code>_static_subprocess_child_spec</code>	function	<code>app/runtime/logic_preflight.py:496</code>	<code>config: dict[str, Any]</code> Load the referenced workflow's spec purely to read its shape (declared output contract) — None for anything not statically resolvable (missing name, bad charset, file not found, fails to parse).
<code>_shallow_child_node_paths</code>	function	<code>app/runtime/logic_preflight.py:520</code>	<code>child_spec: WorkflowSpec, node_id: str</code> One node's typed output paths, one level deep, deliberately NOT a recursive call into <code>build_field_index(child_spec)</code> : a SubprocessAgent output node's own further-nested child is left untyped

Symbol	Kind	Location	Signature / summary
			rather than resolved, so a long — or cyclic — subprocess reference chain can never blow the Python call stack just to compute field paths.
<code>_transform_kind</code>	function	<code>app/runtime/logic_preflight.py:563</code>	<code>operation: str</code> Internal helper for the transform kind step.
<code>_kind_from_annotation</code>	function	<code>app/runtime/logic_preflight.py:604</code>	<code>annotation: Any</code> Best-effort (kind, nullable) for a declared output_schema field.
<code>_declared_output_paths</code>	function	<code>app/runtime/logic_preflight.py:640</code>	<code>klass: Any</code> Typed paths for a node's own output_schema fields.
<code>FieldIndex</code>	class	<code>app/runtime/logic_preflight.py:676</code>	<code></code> Every value the workflow can produce, keyed by its reference path.
<code>FieldIndex.add</code>	method	<code>app/runtime/logic_preflight.py:687</code>	<code>self, node_id: str, path: FieldPath</code> Add the result.
<code>FieldIndex.get</code>	method	<code>app/runtime/logic_preflight.py:697</code>	<code>self, reference: str</code> Return the result.
<code>FieldIndex.node_of</code>	method	<code>app/runtime/logic_preflight.py:708</code>	<code>self, reference: str</code> Compute the node of.
<code>FieldIndex.paths_for</code>	method	<code>app/runtime/logic_preflight.py:726</code>	<code>self, node_id: str</code> Compute the paths for.
<code>build_field_index</code>	function	<code>app/runtime/logic_preflight.py:743</code>	<code>spec: WorkflowSpec</code> Typed paths for every node in the workflow.
<code>validate_business_logic</code>	function	<code>app/runtime/logic_preflight.py:840</code>	<code>spec: WorkflowSpec, *, issue: Callable[..., None], guaranteed_before: Callable[[str, str], bool] None=None</code> Run every logic check.
<code>_check_conditional_join_reachability</code>	function	<code>app/runtime/logic_preflight.py:873</code>	<code>spec: WorkflowSpec, issue: Callable[..., None]</code> Two conditional dispatches into one shared step form an unsatisfiable join.
<code>_reaching_branches</code>	function	<code>app/runtime/logic_preflight.py:969</code>	<code>edge: Any, node_id: str, forward: dict[str, set[str]]</code> Which of this router's branches can reach `node_id`.
<code>_reachable_from</code>	function	<code>app/runtime/logic_preflight.py:980</code>	<code>start: str, forward: dict[str, set[str]]</code> Internal helper for the reachable from step.
<code>_edge_targets</code>	function	<code>app/runtime/logic_preflight.py:1001</code>	<code>edge: Any</code> Internal helper for the edge targets step.
<code>_check_decision</code>	function	<code>app/runtime/logic_preflight.py:1019</code>	<code>node: NodeSpec, config: dict[str, Any], index: FieldIndex, nodes: dict[str, NodeSpec], issue: Callable[..., None], guaranteed_before: Callable[[str, str], bool] None</code> Check the decision.
<code>_check_router</code>	function	<code>app/runtime/logic_preflight.py:1059</code>	<code>node: NodeSpec, config: dict[str, Any], index: FieldIndex, nodes: dict[str, NodeSpec], issue: Callable[..., None], guaranteed_before: Callable[[str, str],</code>

Symbol	Kind	Location	Signature / summary
			<code>bool] None</code> Check the router.
<code>_check_condition</code>	function	<code>app/runtime/logic_preflight.py:1176</code>	<code>condition: Condition, *, node: NodeSpec, index: FieldIndex, nodes: dict[str, NodeSpec], path: str, issue: Callable[..., None], guaranteed_before: Callable[[str, str], bool] None, label: str</code> Check the condition.
<code>_check_threshold</code>	function	<code>app/runtime/logic_preflight.py:1283</code>	<code>condition: Condition, described: FieldPath, value: float, node: NodeSpec, path: str, issue: Callable[..., None], label: str</code> Catch a threshold that can never do anything.
<code>_numeric_bounds</code>	function	<code>app/runtime/logic_preflight.py:1328</code>	<code>described: FieldPath</code> Bounds for a numeric field, when the schema declares them.
<code>_resolve_reference</code>	function	<code>app/runtime/logic_preflight.py:1340</code>	<code>reference: str, *, node: NodeSpec, index: FieldIndex, nodes: dict[str, NodeSpec], path: str, issue: Callable[..., None], guaranteed_before: Callable[[str, str], bool] None, label: str</code> Resolve a rule/route reference, reporting what is wrong with it.
<code>_check_ai_task</code>	function	<code>app/runtime/logic_preflight.py:1455</code>	<code>node: NodeSpec, config: dict[str, Any], issue: Callable[..., None]</code> Contract sanity on a visually built schema.
<code>_check_transform_agent</code>	function	<code>app/runtime/logic_preflight.py:1471</code>	<code>node: NodeSpec, config: dict[str, Any], issue: Callable[..., None]</code> Same contract sanity as <code>_check_ai_task</code>, for a new-style TransformAgent node's <code>output_fields</code>.
<code>_check_output_field_enum</code> <code>s</code>	function	<code>app/runtime/logic_preflight.py:1495</code>	<code>node: NodeSpec, fields: list[FieldSpec], issue: Callable[..., None]</code> Check the output field enums.
<code>_check_external_actions</code>	function	<code>app/runtime/logic_preflight.py:1533</code>	<code>spec: WorkflowSpec, issue: Callable[..., None], guaranteed_before: Callable[[str, str], bool] None</code> An outward-facing action with nothing between it and the model.
<code>_check_external_action</code>	function	<code>app/runtime/logic_preflight.py:1606</code>	<code>node: NodeSpec, issue: Callable[..., None], reviewed_before: Callable[[str], bool]</code> A write or external_action call with nothing between it and the model.
<code>_check_subprocess_agent</code> <code>s</code>	function	<code>app/runtime/logic_preflight.py:1642</code>	<code>spec: WorkflowSpec, issue: Callable[..., None], guaranteed_before: Callable[[str, str], bool] None</code> Everything about a Subprocess step preflight can prove without actually running the child: the referenced workflow exists and parses, every explicit input mapping targets a real declared input of it, every <i>*required*</i> input of it resolves to something, and the whole reference graph this workflow reac

Symbol	Kind	Location	Signature / summary
<code>_load_subprocess_refs</code>	function	<code>app/runtime/logic_preflight.py:1764</code>	<code>name: str</code> Workflow names a *saved* workflow's own SubprocessAgent nodes reference.
<code>_find_subprocess_cycle</code>	function	<code>app/runtime/logic_preflight.py:1786</code>	<code>entry_refs: set[str]</code> Iterative cycle detection over the on-disk subprocess call graph.
<code>_check_sql_query_agents</code>	function	<code>app/runtime/logic_preflight.py:1840</code>	<code>spec: WorkflowSpec, issue: Callable[..., None]</code> A literal <code>'{...}'</code> inside the <code>'sql'</code> field text would be substituted by the compiler's generic templating pass before <code>run()</code> ever sees it — landing a mapped value directly in SQL text instead of going through <code>'params'</code> as a bound <code>'%(name)s'</code> placeholder.
<code>_check_python_snippet_agents</code>	function	<code>app/runtime/logic_preflight.py:1883</code>	<code>spec: WorkflowSpec, issue: Callable[..., None]</code> Everything decidable about a Python snippet without running it: a syntax error (would fail every run identically, so it is an error, not a warning), plus authoring hints — an import with no effect inside the sandbox, a dunder-attribute access, or a snippet that never touches <code>'output'</code> at all.
<code>_check_mcp_agent</code>	function	<code>app/runtime/logic_preflight.py:1935</code>	<code>node: NodeSpec, issue: Callable[..., None], reviewed_before: Callable[[str], bool]</code> MCPAgent chooses its own tools at runtime, so — unlike MCPToolAgent — there is no single static tool name preflight can classify read/write against.
<code>_check_mcp_tool</code>	function	<code>app/runtime/logic_preflight.py:1968</code>	<code>node: NodeSpec, issue: Callable[..., None], reviewed_before: Callable[[str], bool]</code> Validate an MCP tool step without contacting the server.
<code>unreachable_branch_targets</code>	function	<code>app/runtime/logic_preflight.py:2100</code>	<code>spec: WorkflowSpec, reachable: Iterable[str]</code> Router branch targets no run can reach.
<code>sanitize_preview</code>	function	<code>app/runtime/node_events.py:15</code>	<code>value: Any</code> Short, safe preview for the Cockpit card.
<code>is_graph_interrupt</code>	function	<code>app/runtime/node_events.py:39</code>	<code>exc: BaseException</code> LangGraph's interrupt class has moved between versions.
<code>interrupt_payload</code>	function	<code>app/runtime/node_events.py:48</code>	<code>exc: BaseException</code> Recover the actual value passed to <code>"langgraph.types.interrupt(...)"</code> .
<code>DuplicateYamlKeyError</code>	class	<code>app/runtime/preflight.py:40</code>	<code>ValueError</code> Raised when YAML silently would have overwritten a duplicate key.
<code>_UniqueKeyLoader</code>	class	<code>app/runtime/preflight.py:44</code>	<code>yaml.SafeLoader</code> Provides the UniqueKeyLoader behaviour.
<code>_construct_unique_mapping</code>	function	<code>app/runtime/preflight.py:49</code>	<code>loader: _UniqueKeyLoader, node: yaml.MappingNode, deep: bool=False</code> Internal helper for the construct unique mapping step.
<code>PreflightSeverity</code>	class	<code>app/runtime/preflight.py:83</code>	

Symbol	Kind	Location	Signature / summary
			<code>str, Enum</code> Enumeration of PreflightSeverity values.
PreflightIssue	class	app/runtime/preflight.py:89	<code>BaseModel</code> Pydantic model defining the PreflightIssue shape.
PreflightCheck	class	app/runtime/preflight.py:108	<code>BaseModel</code> Pydantic model defining the PreflightCheck shape.
WorkflowPreflightReport	class	app/runtime/preflight.py:121	<code>BaseModel</code> Pydantic model defining the WorkflowPreflightReport shape.
WorkflowPreflightReport.errors	method	app/runtime/preflight.py:144	<code>self</code> The errors.
WorkflowPreflightReport.warnings	method	app/runtime/preflight.py:153	<code>self</code> The warnings.
WorkflowPreflightReport.refresh	method	app/runtime/preflight.py:161	<code>self</code> Refresh the result.
WorkflowPreflightError	class	app/runtime/preflight.py:171	<code>ValueError</code> Carries the complete report when execution is blocked.
_issue	function	app/runtime/preflight.py:188	<code>report: WorkflowPreflightReport, code: str, message: str, *, severity: PreflightSeverity=PreflightSeverity.ERROR, path: str None=None, node_id: str None=None, suggestion: str None=None</code> Internal helper for the issue step.
_add_check	function	app/runtime/preflight.py:221	<code>report: WorkflowPreflightReport, name: str, before: int, detail: str</code> Add the check.
_load_unique_yaml	function	app/runtime/preflight.py:249	<code>yaml_text: str</code> Load the unique yaml.
_parse_spec	function	app/runtime/preflight.py:261	<code>yaml_text: str, report: WorkflowPreflightReport</code> Parse the spec.
_validate_registry	function	app/runtime/preflight.py:339	<code>report: WorkflowPreflightReport</code> Validate the registry.
_unknown_config_fields	function	app/runtime/preflight.py:404	<code>node_spec: NodeSpec, node_class: type[NodeType]</code> Internal helper for the unknown config fields step.
_validated_node_config	function	app/runtime/preflight.py:424	<code>node_spec: NodeSpec</code> Internal helper for the validated node config step.
_iter_model_values	function	app/runtime/preflight.py:445	<code>config: Any, path: str='config'</code> Internal helper for the iter model values step.
_required_services_for_node	function	app/runtime/preflight.py:470	<code>node: NodeSpec</code> Delegates to the node type's own `required_services()` override (see app/nodes/base.py) — every node-type-specific service dependency lives beside the node that declares it, not in a central if/elif chain here.
_validate_nodes	function	app/runtime/preflight.py:488	<code>spec: WorkflowSpec, report: WorkflowPreflightReport</code> Validate the nodes.

Symbol	Kind	Location	Signature / summary
<code>_edge_targets</code>	function	<code>app/runtime/preflight.py:667</code>	<code>edge: EdgeSpec</code> Internal helper for the edge targets step.
<code>_adjacency</code>	function	<code>app/runtime/preflight.py:685</code>	<code>spec: WorkflowSpec</code> Internal helper for the adjacency step.
<code>_reachable</code>	function	<code>app/runtime/preflight.py:705</code>	<code>start: str, graph: dict[str, set[str]]</code> Internal helper for the reachable step.
<code>_topological_order</code>	function	<code>app/runtime/preflight.py:726</code>	<code>reachable: set[str], forward: dict[str, set[str]], reverse: dict[str, set[str]]</code> Kahn's-algorithm topological sort restricted to `reachable`.
<code>_guaranteed_before</code>	function	<code>app/runtime/preflight.py:755</code>	<code>entry: str, forward: dict[str, set[str]], reverse: dict[str, set[str]]</code> For each node reachable from entry, the set of nodes GUARANTEED to have already run by the time it fires.
<code>_has_cycle</code>	function	<code>app/runtime/preflight.py:793</code>	<code>graph: dict[str, set[str]]</code> Return whether cycle.
<code>_validate_router_edges</code>	function	<code>app/runtime/preflight.py:828</code>	<code>spec: WorkflowSpec, report: WorkflowPreflightReport</code> Validate the router edges.
<code>_validate_typed_router</code>	function	<code>app/runtime/preflight.py:967</code>	<code>source: NodeSpec, config: dict[str, Any], mode: str, edge: EdgeSpec, edge_path: str, report: WorkflowPreflightReport</code> Branch/route agreement for the Builder's visual router modes.
<code>_multi_router_ids</code>	function	<code>app/runtime/preflight.py:1045</code>	<code>spec: WorkflowSpec</code> RouterAgent nodes configured for Multi-Route (selection="multi") — their branches are OPTIONAL (any subset may fire), not EXCLUSIVE (exactly one fires), so they must never enter the mutual-exclusivity analysis below (that would either miss a real deadlock or misreport its cause).
<code>_plain_targets</code>	function	<code>app/runtime/preflight.py:1061</code>	<code>spec: WorkflowSpec</code> Every target reached by a plain (non-HITL, non-router) edge, mapped to its declared predecessors — mirrors <code>app/runtime/compiler.py</code> 's <code>_wire_edges</code> step 3 exactly, which is what actually decides which predecessors get combined into one <code>add_edge(..., target)</code> AND-join.
<code>_exclusive_branch_groups</code>	function	<code>app/runtime/preflight.py:1082</code>	<code>spec: WorkflowSpec, forward: dict[str, set[str]]</code> For each SINGLE-selection router (condition+branches) edge with 2+ distinct targets, the node sets exclusively reachable via each individual branch target (i.e.
<code>_mutually_exclusive</code>	function	<code>app/runtime/preflight.py:1120</code>	<code>a: str, b: str, exclusive_groups: list[list[set[str]]]</code> True if `a` and `b` are exclusively reachable via two DIFFERENT branches of the SAME router — i.e.
<code>_validate_fanin_reachability</code>	function	<code>app/runtime/preflight.py:1142</code>	<code>spec: WorkflowSpec, report: WorkflowPreflightReport, forward: dict[str, set[str]]</code> A plain-edge fan-in target (2+ declared predecessors combined into one <code>add_edge(..., target)</code> AND-join —

Symbol	Kind	Location	Signature / summary
			see app/runtime/compiler.py's <code>_wire_edges</code>) can only ever fire if ALL of its predecessors actually run in the same invocation.
<code>_multi_route_branch_sets</code>	function	app/runtime/preflight.py:1190	<code>spec: WorkflowSpec, forward: dict[str, set[str]]</code> Per Multi-Route router edge: branch label -> nodes reachable ONLY via that branch's own target (not via any of the router's OTHER branch targets).
<code>_selection_deps</code>	function	app/runtime/preflight.py:1219	<code>node_id: str, branch_sets: list[tuple[str, dict[str, set[str]]]]</code> Every (multi_router_id, branch_label) this node's execution is conditioned on — a node exclusively reachable via one branch of a Multi-Route router depends on that branch having been selected.
<code>_validate_multiroute_andjoin</code>	function	app/runtime/preflight.py:1238	<code>spec: WorkflowSpec, report: WorkflowPreflightReport, forward: dict[str, set[str]]</code> A fan-in target whose declared predecessors depend on DIFFERENT Multi-Route selections can silently never fire, the same failure shape FANIN_UNREACHABLE_ANDJOIN already catches for ordinary (single-select) routers' mutually-exclusive branches — confirmed empirically against real langgraph 1.2.9: a target dispatched to only a SUBSET of a multi-router's own branches is never revisited for the branches that weren't selected, so any node whose AND-join lists all of them hangs forever on the ones that never ran.
<code>_validate_start_end_edges</code>	function	app/runtime/preflight.py:1317	<code>spec: WorkflowSpec, report: WorkflowPreflightReport</code> Start represents the beginning of execution and End its terminus — enforce that structurally rather than treating them as decorative.
<code>_validate_start_fields</code>	function	app/runtime/preflight.py:1365	<code>spec: WorkflowSpec, report: WorkflowPreflightReport</code> Validates one Start node's own field/file_field list.
<code>_validate_graph</code>	function	app/runtime/preflight.py:1460	<code>spec: WorkflowSpec, report: WorkflowPreflightReport</code> Validate the graph.
<code>_iter_strings</code>	function	app/runtime/preflight.py:1599	<code>value: Any, path: str</code> Internal helper for the iter strings step.
<code>_annotation_permits_none</code>	function	app/runtime/preflight.py:1622	<code>annotation: Any</code> True if a declared field type allows None ("<code>X None</code>" / <code>Optional[X]</code>).
<code>_validate_template_output_path</code>	function	app/runtime/preflight.py:1635	<code>spec: WorkflowSpec, current_node: NodeSpec, reference: str, report: WorkflowPreflightReport, path: str, forward: dict[str, set[str]], guaranteed: dict[str, set[str]], optional: bool=False</code> Validate the template output path.
<code>_matches_declared_field</code>	function	app/runtime/preflight.py:1925	<code>reference: str, declared: set[str]</code> Does a dotted reference match one of a node's declared output paths?

Symbol	Kind	Location	Signature / summary
<code>_validate_templates</code>	function	<code>app/runtime/preflight.py:1965</code>	<code>spec: WorkflowSpec, report: WorkflowPreflightReport, forward: dict[str, set[str]], reverse: dict[str, set[str]]</code> Validate the templates.
<code>_validate_inputs</code>	function	<code>app/runtime/preflight.py:2014</code>	<code>spec: WorkflowSpec, provided_inputs: dict[str, Any] None, report: WorkflowPreflightReport</code> Validate the inputs.
<code>_compile_dry_run</code>	function	<code>app/runtime/preflight.py:2095</code>	<code>spec: WorkflowSpec, services: dict[str, Any] None, report: WorkflowPreflightReport</code> Internal helper for the compile dry run step.
<code>_validate_guided_experience</code>	function	<code>app/runtime/preflight.py:2136</code>	<code>spec: WorkflowSpec, report: WorkflowPreflightReport</code> Validate authored Guided Run copy without penalising legacy workflows.
<code>preflight_workflow_yaml</code>	function	<code>app/runtime/preflight.py:2208</code>	<code>yaml_text: str, *, provided_inputs: dict[str, Any] None=None, services: dict[str, Any] None=None, compile_graph: bool=True</code> Run all deterministic checks.
<code>_validate_business_logic</code>	function	<code>app/runtime/preflight.py:2234</code>	<code>spec: WorkflowSpec, report: WorkflowPreflightReport, forward: dict[str, set[str]], reverse: dict[str, set[str]]</code> Check visually authored rules, routes and schemas (zero tokens).
<code>preflight_workflow_spec</code>	function	<code>app/runtime/preflight.py:2335</code>	<code>spec: WorkflowSpec, *, provided_inputs: dict[str, Any] None=None, services: dict[str, Any] None=None, compile_graph: bool=False</code> Validate an already parsed spec without changing or running it.
<code>_probe_services</code>	async function	<code>app/runtime/preflight.py:2356</code>	<code>spec: WorkflowSpec, services: dict[str, Any], report: WorkflowPreflightReport, *, require_run_history: bool, owner_scope_id: str None=None</code> Probe the services.
<code>_probe_integration_nodes</code>	function	<code>app/runtime/preflight.py:2572</code>	<code>spec: WorkflowSpec, service: Any, report: WorkflowPreflightReport</code> Zero-token: checks the referenced connection exists on the service — no provider HTTP call, just a dict lookup.
<code>_probe_web_search_nodes</code>	function	<code>app/runtime/preflight.py:2609</code>	<code>spec: WorkflowSpec, service: Any, report: WorkflowPreflightReport</code> Zero-token: <code>WebSearchService.resolve_provider</code> does a settings check only, no HTTP request — see <code>app/tools/web_io.py</code> .
<code>_probe_image_generation_nodes</code>	function	<code>app/runtime/preflight.py:2645</code>	<code>spec: WorkflowSpec, service: Any, report: WorkflowPreflightReport</code> Zero-token: <code>OpenAllImageGenerationService.available()</code> checks settings only — see <code>app/tools/image_io.py</code> .
<code>_probe_kimi_vision_nodes</code>	function		

Symbol	Kind	Location	Signature / summary
		app/runtime/ preflight.py:2682	<code>spec: WorkflowSpec, service: Any, report: WorkflowPreflightReport</code> Zero-token: <code>KimiVisionService.available()</code> checks settings only — see <code>app/tools/vision_io.py</code> .
<code>_probe_rag_resource_nodes</code>	async function	app/runtime/ preflight.py:2711	<code>spec: WorkflowSpec, repository: Any, report: WorkflowPreflightReport, *, owner_scope_id: str</code> Confirm every referenced Knowledge resource exists and is usable.
<code>_probe_workflow_model_access</code>	async function	app/runtime/ preflight.py:2860	<code>spec: WorkflowSpec, llm: Any, report: WorkflowPreflightReport</code> Confirm provider/project access without making a generation request.
<code>preflight_workflow_for_run</code>	async function	app/runtime/ preflight.py:3009	<code>yaml_text: str, *, provided_inputs: dict[str, Any] None, services: dict[str, Any], probe_services: bool=True, require_run_history: bool=True, owner_scope_id: str None=None</code> Strict API gate used immediately before a new or retried run.
<code>require_preflight</code>	function	app/runtime/ preflight.py:3048	<code>report: WorkflowPreflightReport</code> Compute the require preflight.
<code>operators_for_type</code>	function	app/runtime/rules.py:129	<code>field_type: str</code> Compute the operators for type.
<code>resolve_path</code>	function	app/runtime/rules.py:168	<code>context: dict[str, Any], path: str</code> Read a dotted path out of workflow state, returning “_MISSING” sentinel semantics as “None” for callers that don't care about the distinction.
<code>path_exists</code>	function	app/runtime/rules.py:182	<code>context: dict[str, Any], path: str</code> Compute the path exists.
<code>_resolve</code>	function	app/runtime/rules.py:195	<code>context: dict[str, Any], path: str</code> Resolve the result.
<code>Condition</code>	class	app/runtime/rules.py:257	<code>BaseModel</code> One leaf test: “field operator value”.
<code>Condition.value_present_when_needed</code>	method	app/runtime/rules.py:267	<code>self</code> Compute the value present when needed.
<code>ConditionGroup</code>	class	app/runtime/rules.py:289	<code>BaseModel</code> AND / OR / NOT over conditions and nested groups (§10).
<code>ConditionGroup.group_is_usable</code>	method	app/runtime/rules.py:300	<code>self</code> Group the is usable.
<code>Action</code>	class	app/runtime/rules.py:322	<code>BaseModel</code> What a matched rule does.
<code>Action.rename_legacy_append</code>	method	app/runtime/rules.py:338	<code>cls, value: Any</code> `merge` is `append`'s new name — the accumulate-into-a-list behavior is unchanged, only the label.
<code>Action.value_matches_operation</code>	method	app/runtime/rules.py:347	<code>self</code> Compute the value matches operation.
<code>Rule</code>	class	app/runtime/rules.py:362	

Symbol	Kind	Location	Signature / summary
			<code>BaseModel</code> One IF/THEN rule.
<code>Rule.rule_is_decidable</code>	method	<code>app/runtime/rules.py:378</code>	<code>self</code> Compute the rule is decidable.
<code>ConditionTrace</code>	class	<code>app/runtime/rules.py:403</code>	<code>BaseModel</code> Why one condition passed or failed — rendered verbatim in the UI.
<code>GroupTrace</code>	class	<code>app/runtime/rules.py:419</code>	<code>BaseModel</code> Pydantic model defining the GroupTrace shape.
<code>RuleTrace</code>	class	<code>app/runtime/rules.py:439</code>	<code>BaseModel</code> Pydantic model defining the RuleTrace shape.
<code>RuleEvaluation</code>	class	<code>app/runtime/rules.py:460</code>	<code>BaseModel</code> Result of evaluating a rule set: the facts, plus the full reasoning.
<code>RuleEvaluation.explanation_lines</code>	method	<code>app/runtime/rules.py:469</code>	<code>self</code> Flat, human-readable "matched rules" list for the run trace (§24).
<code>_leaf_traces</code>	function	<code>app/runtime/rules.py:488</code>	<code>node: GroupTrace ConditionTrace None</code> Internal helper for the leaf traces step.
<code>_as_number</code>	function	<code>app/runtime/rules.py:509</code>	<code>value: Any</code> Internal helper for the as number step.
<code>_is_empty</code>	function	<code>app/runtime/rules.py:530</code>	<code>value: Any</code> Return whether empty.
<code>_contains</code>	function	<code>app/runtime/rules.py:548</code>	<code>haystack: Any, needle: Any</code> Internal helper for the contains step.
<code>_boolish</code>	function	<code>app/runtime/rules.py:576</code>	<code>value: Any</code> Read a value as a boolean, or None if it isn't one.
<code>_equals</code>	function	<code>app/runtime/rules.py:594</code>	<code>actual: Any, expected: Any</code> Internal helper for the equals step.
<code>_truthy</code>	function	<code>app/runtime/rules.py:619</code>	<code>value: Any</code> Internal helper for the truthy step.
<code>evaluate_condition</code>	function	<code>app/runtime/rules.py:634</code>	<code>condition: Condition, context: dict[str, Any]</code> Compute the evaluate condition.
<code>_previewable</code>	function	<code>app/runtime/rules.py:708</code>	<code>value: Any</code> Keep traces small — they are stored in run history and sent over SSE.
<code>_summarise</code>	function	<code>app/runtime/rules.py:719</code>	<code>condition: Condition, actual: Any, missing: bool, matched: bool</code> Internal helper for the summarise step.
<code>evaluate_group</code>	function	<code>app/runtime/rules.py:747</code>	<code>group: ConditionGroup, context: dict[str, Any]</code> Compute the evaluate group.
<code>evaluate_rules</code>	function	<code>app/runtime/rules.py:776</code>	<code>rules: list[Rule], context: dict[str, Any], *, initial: dict[str, Any] None=None</code> Evaluate every rule in order and accumulate the facts they set.

Symbol	Kind	Location	Signature / summary
<code>_apply_action</code>	function	<code>app/runtime/rules.py:827</code>	<code>action: Action, values: dict[str, Any], context: dict[str, Any]</code> Write one fact.
<code>collect_condition_fields</code>	function	<code>app/runtime/rules.py:863</code>	<code>group: ConditionGroup None</code> Every field path a group reads — used by preflight to check references.
<code>collect_conditions</code>	function	<code>app/runtime/rules.py:876</code>	<code>group: ConditionGroup None</code> Every leaf condition in a group, for type checking in preflight.
<code>WorkflowFileRef</code>	class	<code>app/runtime/schema.py:32</code>	<code>BaseModel</code> Stable object-storage reference passed through workflow state.
<code>WorkflowInputSpec</code>	class	<code>app/runtime/schema.py:54</code>	<code>BaseModel</code> Pydantic model defining the <code>WorkflowInputSpec</code> shape.
<code>WorkflowInputSpec.validate_file_options</code>	method	<code>app/runtime/schema.py:75</code>	<code>self</code> Validate the file options.
<code>WorkflowInputSpec.effective_max_files</code>	method	<code>app/runtime/schema.py:91</code>	<code>self, platform_limit: int</code> Compute the effective max files.
<code>ModelRoutingPolicy</code>	class	<code>app/runtime/schema.py:104</code>	<code>BaseModel</code> Automatic-selection policy consulted by the deterministic <code>ModelRouter</code> when a node's model is the <code>AUTO_MODEL</code> sentinel.
<code>GuidedStageSpec</code>	class	<code>app/runtime/schema.py:124</code>	<code>BaseModel</code> Optional business-language grouping used by <code>GuidedRun</code> .
<code>WorkflowExperienceSpec</code>	class	<code>app/runtime/schema.py:146</code>	<code>BaseModel</code> Workflow-level settings for the non-technical runtime surface.
<code>NodeExperienceSpec</code>	class	<code>app/runtime/schema.py:156</code>	<code>BaseModel</code> Optional user-facing copy and handoff metadata for one node.
<code>LibraryDurationRange</code>	class	<code>app/runtime/schema.py:181</code>	<code>BaseModel</code> Pydantic model defining the <code>LibraryDurationRange</code> shape.
<code>LibraryHumanReviews</code>	class	<code>app/runtime/schema.py:194</code>	<code>BaseModel</code> Pydantic model defining the <code>LibraryHumanReviews</code> shape.
<code>LibraryEvidencePolicy</code>	class	<code>app/runtime/schema.py:207</code>	<code>BaseModel</code> Pydantic model defining the <code>LibraryEvidencePolicy</code> shape.
<code>LibraryMetadataSpec</code>	class	<code>app/runtime/schema.py:220</code>	<code>BaseModel</code> Optional catalog/presentation metadata for the <code>Workflow Library</code> .
<code>NodeSpec</code>	class	<code>app/runtime/schema.py:248</code>	<code>BaseModel</code> One node instance in a workflow.
<code>NodeSpec.selected_model_must_be_allowed</code>	method	<code>app/runtime/schema.py:284</code>	<code>self</code> Compute the selected model must be allowed.
	method	<code>app/runtime/schema.py:308</code>	

Symbol	Kind	Location	Signature / summary
NodeSpec.data_protection_mode_must_be_valid			<code>self</code> Compute the data protection mode must be valid.
NodeSpec.effective_config	method	<code>app/runtime/schema.py:325</code>	<code>self</code> Compute the effective config.
StaticVariable	class	<code>app/runtime/schema.py:337</code>	<code>BaseModel</code> Pydantic model defining the StaticVariable shape.
WorkflowOutputNode	class	<code>app/runtime/schema.py:350</code>	<code>BaseModel</code> Pydantic model defining the WorkflowOutputNode shape.
WorkflowOutputSpec	class	<code>app/runtime/schema.py:361</code>	<code>BaseModel</code> Pydantic model defining the WorkflowOutputSpec shape.
EdgeSpec	class	<code>app/runtime/schema.py:372</code>	<code>BaseModel</code> Edges support simple, fan-out, and conditional routing.
WorkflowSpec	class	<code>app/runtime/schema.py:383</code>	<code>BaseModel</code> The stable, use-case-neutral workflow contract.
WorkflowSpec.validate_graph_references	method	<code>app/runtime/schema.py:406</code>	<code>self</code> Validate the graph references.
WorkflowSpec.derive_inputs_from_start_node	method	<code>app/runtime/schema.py:516</code>	<code>self</code> A Start node (<code>app/nodes/start.py</code>) is the single place a workflow author declares what this workflow needs — this projects those declarations into the existing <code>`inputs:`</code> contract so file-upload validation and the "required input is missing" pre-execution check (<code>app/workflow/file_inputs.py</code> 's <code>validate_workflow_inputs</code>) apply to them for free, with no new validator (§37/§38).
_enabled	function	<code>app/runtime/sleep_guard.py:35</code>	<code></code> Internal helper for the enabled step.
acquire	async function	<code>app/runtime/sleep_guard.py:46</code>	<code></code> Block idle sleep for the duration of one workflow run.
release	async function	<code>app/runtime/sleep_guard.py:66</code>	<code></code> Release this run's hold; sleep is allowed again once none remain.
SnippetRunnerUnavailable	class	<code>app/runtime/snippet_client.py:24</code>	<code>RuntimeError</code> The sidecar could not be reached at all — refused connection, no socket file, or it never responded.
SnippetRunnerClient	class	<code>app/runtime/snippet_client.py:30</code>	<code></code> Provides the SnippetRunnerClient behaviour.
SnippetRunnerClient.run	method	<code>app/runtime/snippet_client.py:40</code>	<code>self, code: str, inputs: dict[str, Any], *, timeout_seconds: float, memory_mb: int</code> Run the result.
SnippetRunnerClient.probe	method	<code>app/runtime/snippet_client.py:67</code>	<code>self</code> Preflight health check (SNIPPET_RUNNER_UNAVAILABLE) — a real, trivial round trip, not just "did connect() succeed", since a listening-but-wedged daemon is just as unusable.
SnippetRunnerClient.call	method	<code>app/runtime/snippet_client.py:80</code>	<code>self, request: dict[str, Any], timeout_seconds: float</code> Internal helper for the call step.

Symbol	Kind	Location	Signature / summary
<code>_socket_path</code>	function	<code>app/runtime/snippet_daemon.py:47</code>	Internal helper for the socket path step.
<code>_clamped</code>	function	<code>app/runtime/snippet_daemon.py:56</code>	<code>request: dict[str, Any]</code> Never trust the client's own limits past the sidecar's own ceiling — a compromised or buggy app process must not be able to ask this container to run something for an hour.
<code>_rlimit_preexec</code>	function	<code>app/runtime/snippet_daemon.py:73</code>	<code>memory_mb: int, timeout_seconds: float</code> Runs in the child, after fork, before exec — this is real isolation, not the request's own JSON asking nicely.
<code>_execute</code>	async function	<code>app/runtime/snippet_daemon.py:102</code>	<code>request: SnippetRequest</code> Execute the result.
<code>_handle</code>	async function	<code>app/runtime/snippet_daemon.py:172</code>	<code>reader: asyncio.StreamReader, writer: asyncio.StreamWriter</code> Handle the result.
<code>main</code>	async function	<code>app/runtime/snippet_daemon.py:208</code>	Compute the main.
<code>read_message</code>	async function	<code>app/runtime/snippet_protocol.py:21</code>	<code>reader: asyncio.StreamReader, limit: int=MAX_MESSAGE_BYTES</code> Read one EOF-terminated message, accumulating across as many socket reads as it takes.
<code>SnippetRequest</code>	class	<code>app/runtime/snippet_protocol.py:47</code>	<code>TypedDict</code> Provides the SnippetRequest behaviour.
<code>SnippetResponse</code>	class	<code>app/runtime/snippet_protocol.py:67</code>	<code>TypedDict</code> Provides the SnippetResponse behaviour.
<code>merge_node_outputs</code>	function	<code>app/runtime/state.py:12</code>	<code>a: dict[str, Any], b: dict[str, Any]</code> Reducer for node_outputs.
<code>WorkflowState</code>	class	<code>app/runtime/state.py:25</code>	<code>TypedDict</code> Provides the WorkflowState behaviour.
<code>_lookup</code>	function	<code>app/runtime/templating.py:29</code>	<code>path: str, state: dict</code> Walk a dotted path against workflow state.
<code>_resolve_reference</code>	function	<code>app/runtime/templating.py:90</code>	<code>match: 're.Match[str]', state: dict</code> Resolve one matched reference, honouring the optional marker.
<code>_as_text</code>	function	<code>app/runtime/templating.py:101</code>	<code>value: Any</code> Render a substituted value inside a larger string.
<code>resolve</code>	function	<code>app/runtime/templating.py:110</code>	<code>value: Any, state: dict</code> Recursively walk a config value and replace template strings.
<code>_union_members</code>	function	<code>app/runtime/templating.py:155</code>	<code>annotation: Any</code> Internal helper for the union members step.
<code>_accepts_none</code>	function	<code>app/runtime/templating.py:169</code>	<code>annotation: Any</code> Whether a value of None satisfies this annotation.
<code>_element_annotation</code>	function	<code>app/runtime/templating.py:178</code>	<code>annotation: Any, container: type, position: int</code> The declared type of a dict value / list item, or <code>_UNKNOWN</code> .
<code>prune_absent</code>	function		

Symbol	Kind	Location	Signature / summary
		app/runtime/templating.py:193	<code>config: dict[str, Any], schema: Any</code> Drop config values that resolved to None but cannot be None.
<code>_require_session</code>	function	app/security/audit.py:48	<code>session_id: str</code> The isolation boundary, enforced in ONE place.
<code>summarize_payload</code>	function	app/security/audit.py:59	<code>data: dict[str, Any] None</code> Record SHAPE, not CONTENT.
<code>write_audit_event</code>	async function	app/security/audit.py:75	<code>db, run_id: str, session_id: str, node_id: str, event_type: str, actor: str='system', payload: dict[str, Any] None=None</code> Write the audit event.
<code>read_audit_events</code>	async function	app/security/audit.py:112	<code>db, session_id: str, run_id: str None=None, limit: int=200</code> The ONLY read path.
<code>CurrentUser</code>	class	app/security/dependencies.py:15	 Provides the CurrentUser behaviour.
<code>_user_from_token</code>	function	app/security/dependencies.py:29	<code>token: str None</code> Internal helper for the user from token step.
<code>get_current_user</code>	async function	app/security/dependencies.py:59	<code>token: str None=Depends(oauth2_scheme), access_token: str None=Cookie(default=None)</code> Return the current user.
<code>get_current_user_cookie_or_header</code>	async function	app/security/dependencies.py:75	<code>token: str None=Depends(oauth2_scheme), access_token: str None=Cookie(default=None)</code> Header first (API clients), then HttpOnly cookie (browser EventSource).
<code>require_permission</code>	function	app/security/dependencies.py:82	<code>permission: str</code> Compute the require permission.
<code>EntityMatch</code>	class	app/security/entity_ner.py:27	 Provides the EntityMatch behaviour.
<code>_load_pipeline</code>	function	app/security/entity_ner.py:42	 Load the pipeline.
<code>extract_entities</code>	function	app/security/entity_ner.py:63	<code>text: str</code> PERSON/ORG spans via spaCy.
<code>EntityProtectionError</code>	class	app/security/entity_protection_errors.py:13	<code>Exception</code> Base class for every fail-closed entity-protection error.
<code>EntityVaultUnavailableError</code>	class	app/security/entity_protection_errors.py:17	<code>EntityProtectionError</code> The Mongo-backed mapping vault could not be reached (down/timeout).
<code>EntityScanUnavailableError</code>	class	app/security/entity_protection_errors.py:21	<code>EntityProtectionError</code> The spaCy NER safety-net pass crashed or could not run.
<code>VaultKeyMisconfiguredError</code>	class	app/security/entity_protection_errors.py:25	<code>EntityProtectionError</code> entity_vault_master_key is missing, a placeholder, or too short.

Symbol	Kind	Location	Signature / summary
UnresolvedEntityRiskError	class	app/security/ entity_protection_errors.py:29	<code>EntityProtectionError</code> A residual entity candidate could not be confidently tokenized.
ResponseLeakDetectedError	class	app/security/ entity_protection_errors.py:33	<code>EntityProtectionError</code> A registered real entity value appeared verbatim in a model response.
EntityRegistry	class	app/security/ entity_registry.py:38	 Provides the EntityRegistry behaviour.
EntityRegistry.register	method	app/security/ entity_registry.py:48	<code>self, *, session_id: str, collection_id: str, entity_type: str, value: str, source: str='manual'</code> Register the result.
EntityRegistry.list_entities	method	app/security/ entity_registry.py:84	<code>self, *, session_id: str, collection_id: str</code> List the entities.
EntityRegistry.delete	method	app/security/ entity_registry.py:100	<code>self, *, session_id: str, collection_id: str, entity_type: str, value: str</code> Delete the result.
EntityRegistry.sync_from_partners	method	app/security/ entity_registry.py:121	<code>self, *, session_id: str, collection_id: str, graph: Any</code> Register every partner on a finalized ProposalGraph.
ProcessingMode	class	app/security/ entity_tokenizer.py:47	<code>str, Enum</code> Enumeration of ProcessingMode values.
TokenizeResult	class	app/security/ entity_tokenizer.py:55	 Provides the TokenizeResult behaviour.
DetokenizeResult	class	app/security/ entity_tokenizer.py:67	 Provides the DetokenizeResult behaviour.
_Span	class	app/security/ entity_tokenizer.py:79	 Provides the Span behaviour.
_overlaps	function	app/security/ entity_tokenizer.py:96	<code>start: int, end: int, claimed: list[tuple[int, int]]</code> Internal helper for the overlaps step.
_boundary_pattern	function	app/security/ entity_tokenizer.py:110	<code>value: str</code> Internal helper for the boundary pattern step.
_ambiguous_short_ner_match	function	app/security/ entity_tokenizer.py:125	<code>value: str</code> Return True for short acronym-like NER guesses that are too ambiguous.
EntityTokenizerService	class	app/security/ entity_tokenizer.py:137	 Provides the EntityTokenizerService behaviour.
EntityTokenizerService.registry	method	app/security/ entity_tokenizer.py:149	<code>self</code> Exposed for the manual-registration API and proposal-graph sync.
EntityTokenizerService.ensure_indexes	method	app/security/ entity_tokenizer.py:153	<code>self</code> Ensure the indexes.
EntityTokenizerService.tokenize	method	app/security/ entity_tokenizer.py:157	<code>self, text: str, *, session_id: str, collection_id: str='default', mode: ProcessingMode=ProcessingMode.PSEUDONYMISED</code> Tokenize the result.

Symbol	Kind	Location	Signature / summary
EntityTokenizerService.detect_spans	method	app/security/entity_tokenizer.py:211	<code>self, text: str, *, session_id: str, collection_id: str</code> Detect the spans.
EntityTokenizerService.tokenize	method	app/security/entity_tokenizer.py:311	<code>self, value: Any, *, session_id: str, collection_id: str='default'</code> Recursively walks str/dict/list, restoring placeholders in every leaf string.
EntityTokenizerService.detokenize_text	method	app/security/entity_tokenizer.py:350	<code>self, text: str, *, session_id: str, collection_id: str</code> Detokenize the text.
_load_kek	function	app/security/entity_vault.py:47	<code></code> Derive the vault's Key Encryption Key from settings, failing closed.
_normalize	function	app/security/entity_vault.py:76	<code>value: str</code> Normalize the result.
entity_hash	function	app/security/entity_vault.py:88	<code>entity_type: str, value: str</code> Deterministic match key for one (type, normalized value) pair.
_mapping_id	function	app/security/entity_vault.py:94	<code>session_id: str, collection_id: str, ehash: str</code> Internal helper for the mapping id step.
_counter_id	function	app/security/entity_vault.py:108	<code>session_id: str, collection_id: str, entity_type: str</code> Internal helper for the counter id step.
_expiry	function	app/security/entity_vault.py:122	<code></code> Internal helper for the expiry step.
_seal	function	app/security/entity_vault.py:133	<code>real_value: str, *, session_id: str, collection_id: str, placeholder: str</code> Internal helper for the seal step.
_unseal	function	app/security/entity_vault.py:161	<code>doc: dict[str, Any], *, session_id: str, collection_id: str, placeholder: str</code> Internal helper for the unseal step.
EntityVault	class	app/security/entity_vault.py:183	<code></code> Mongo-backed envelope-encrypted mapping vault, scoped per (session, collection).
EntityVault._mappings	method	app/security/entity_vault.py:195	<code>self</code> The mappings.
EntityVault._counters	method	app/security/entity_vault.py:200	<code>self</code> The counters.
EntityVault.ensure_indexes	method	app/security/entity_vault.py:204	<code>self</code> Ensure the indexes.
EntityVault.get_or_create_placeholder	method	app/security/entity_vault.py:225	<code>self, *, session_id: str, collection_id: str, entity_type: str, real_value: str, source: str</code> Stable placeholder for one entity within a scope.
EntityVault.list_scope_entities	method	app/security/entity_vault.py:296	<code>self, *, session_id: str, collection_id: str</code> Decrypted known entities for a scope — feeds the registry-first literal-match detection pass and the manual-registration listing API.

Symbol	Kind	Location	Signature / summary
EntityVault.resolve_placeholders	method	app/security/entity_vault.py:331	<code>self, *, session_id: str, collection_id: str, placeholders: set[str]</code> Batch placeholder -> real value lookup, for detokenizing a response.
EntityVault.delete_entity	method	app/security/entity_vault.py:363	<code>self, *, session_id: str, collection_id: str, entity_type: str, value: str</code> Delete the entity.
ensure_indexes	async function	app/security/entity_vault.py:386	<code>db: Any</code> Module-level convenience matching this codebase's "ensure_*_indexes" convention.
GuardrailViolation	class	app/security/guardrails.py:25	<code>ValueError</code> Raised when a request or generated output violates a hard guardrail.
GuardrailFinding	class	app/security/guardrails.py:30	<code></code> Provides the GuardrailFinding behaviour.
GuardrailResult	class	app/security/guardrails.py:42	<code></code> Provides the GuardrailResult behaviour.
_redact_matches	function	app/security/guardrails.py:94	<code>text: str, patterns: tuple[tuple[str, re.Pattern[str]], ...], *, path: str</code> Redact the matches.
_walk	function	app/security/guardrails.py:119	<code>value: Any, *, path: str, transform</code> Internal helper for the walk step.
check_workflow_inputs	function	app/security/guardrails.py:164	<code>value: Any</code> Block injection and apply the configured PII policy to direct inputs.
check_generated_output	function	app/security/guardrails.py:219	<code>value: Any</code> Remove credentials/private keys from model and tool output.
create_access_token	function	app/security/jwt_handler.py:16	<code>data: dict[str, Any]</code> Create the access token.
decode_token	function	app/security/jwt_handler.py:43	<code>token: str</code> Decode the token.
processing_mode_to_data_class	function	app/security/llm_policy.py:44	<code>processing_mode: str None</code> Maps Eurskem's per-node ProcessingMode onto the classification lattice.
effective_data_class	function	app/security/llm_policy.py:56	<code>declared: DataClass, *, entities_detected: bool</code> max(declared, detected) — detecting a real entity in the text implies at least CONFIDENTIAL sensitivity, regardless of what the workflow declared.
provider_and_backend_for_gateway	function	app/security/llm_policy.py:76	<code>gateway_class_name: str</code> Compute the provider and backend for gateway.
ProviderComplianceMetadata	class	app/security/llm_policy.py:89	<code></code> Provides the ProviderComplianceMetadata behaviour.
_parse_csv_env	function	app/security/llm_policy.py:100	<code>name: str</code> Parse the csv env.
get_provider_compliance_metadata	function	app/security/llm_policy.py:113	<code>provider: str None</code> Operator-configured ZDR/data-collection facts — a

Symbol	Kind	Location	Signature / summary
			business/legal fact about Eurskem's actual vendor agreements, not derivable from code.
PolicyDecision	class	app/security/llm_policy.py:131	Provides the PolicyDecision behaviour.
OpaClient	class	app/security/llm_policy.py:146	Minimal OPA REST client — mirrors omniroute/src/lib/policy/opa/client.ts exactly (same endpoint shape, same fail-closed contract).
OpaClient.evaluate	method	app/security/llm_policy.py:174	<code>self, policy_input: dict</code> Compute the evaluate.
get_default_opa_client	function	app/security/llm_policy.py:217	Return the default opa client.
enforce_policy	async function	app/security/llm_policy.py:229	<code>*, workspace_id: str, data_class: DataClass, gateway_class_name: str, model: str, capabilities: list[str] None=None, opa_client: OpaClient None=None</code> Evaluates the routing policy for one candidate (provider, model) pair.
RequestContextMiddleware	class	app/security/middleware.py:25	<code>BaseHTTPMiddleware</code> Provides the RequestContextMiddleware behaviour.
RequestContextMiddleware.dispatch	method	app/security/middleware.py:27	<code>self, request: Request, call_next: Callable</code> Dispatch the result.
RequestSizeLimitMiddleware	class	app/security/middleware.py:87	<code>BaseHTTPMiddleware</code> Provides the RequestSizeLimitMiddleware behaviour.
RequestSizeLimitMiddleware.dispatch	method	app/security/middleware.py:89	<code>self, request: Request, call_next: Callable</code> Dispatch the result.
RedisRateLimitMiddleware	class	app/security/middleware.py:125	<code>BaseHTTPMiddleware</code> Provides the RedisRateLimitMiddleware behaviour.
RedisRateLimitMiddleware.dispatch	method	app/security/middleware.py:129	<code>self, request: Request, call_next: Callable</code> Dispatch the result.
_rate_limit_subject	function	app/security/middleware.py:209	<code>request: Request</code> Internal helper for the rate limit subject step.
_route_label	function	app/security/middleware.py:229	<code>request: Request</code> Internal helper for the route label step.
hash_password	function	app/security/passwords.py:11	<code>password: str</code> Hash the password.
verify_password	function	app/security/passwords.py:27	<code>password: str, encoded: str None</code> Verify the password.
PresidioMatch	class	app/security/presidio_client.py:36	Provides the PresidioMatch behaviour.
_map_entity_type	function	app/security/presidio_client.py:53	<code>presidio_type: str</code> Map the entity type.
analyze	async function	app/security/presidio_client.py:65	<code>text: str, *, language: str='en', score_threshold: float=0.5, client:</code>

Symbol	Kind	Location	Signature / summary
			<code>httpx.AsyncClient None=None</code> Calls POST {presidio_analyzer_url}/analyze.
Role	class	<code>app/security/rbac.py:9</code>	<code>str, Enum</code> Enumeration of Role values.
has_permission	function	<code>app/security/rbac.py:24</code>	<code>role: Role, permission: str</code> Return whether permission.
stub_login	function	<code>app/security/sso_stub.py:20</code>	<code>username: str, role: str, session_id: str None=None</code> Issue a JWT as if Azure AD had authenticated the user.
ensure_user_indexes	async function	<code>app/security/users.py:14</code>	<code>db: Any</code> Ensure the user indexes.
validate_username	function	<code>app/security/users.py:24</code>	<code>username: str</code> Validate the username.
upsert_local_user	async function	<code>app/security/users.py:41</code>	<code>db: Any, *, username: str, password: str, role: Role, active: bool=True</code> Upsert the local user.
authenticate_local_user	async function	<code>app/security/users.py:75</code>	<code>db: Any, *, username: str, password: str</code> Authenticate the local user.
content_hash	function	<code>app/storage/minio_client.py:33</code>	<code>path: Path, chunk_size: int=65536</code> Compute SHA256 hash of file contents.
key_for_path	function	<code>app/storage/minio_client.py:49</code>	<code>path: Path</code> The object key a file should be stored under: 'sha256:<hash>.<ext>'.
knowledge_key_for_path	function	<code>app/storage/minio_client.py:60</code>	<code>path: Path, owner_scope_id: str</code> Content-addressed key for a Knowledge Studio upload, scoped by owner.
ObjectStore	class	<code>app/storage/minio_client.py:75</code>	 S3-compatible object store wrapper.
ObjectStore.ensure_bucket	method	<code>app/storage/minio_client.py:116</code>	<code>self, bucket: str None=None</code> Create the bucket if it doesn't exist.
ObjectStore.put_file	method	<code>app/storage/minio_client.py:134</code>	<code>self, local_path: Path, key: str None=None, content_type: str None=None, extra_metadata: dict[str, str] None=None</code> Upload a local file.
ObjectStore.put_bytes	method	<code>app/storage/minio_client.py:173</code>	<code>self, data: bytes, key: str, content_type: str None=None</code> Upload raw bytes under a given key.
ObjectStore.get_bytes	method	<code>app/storage/minio_client.py:196</code>	<code>self, key: str</code> Download an object's contents.
ObjectStore.object_exists	method	<code>app/storage/minio_client.py:203</code>	<code>self, key: str</code> Cheap existence check using HEAD.
ObjectStore.list_objects	method	<code>app/storage/minio_client.py:215</code>	<code>self, prefix: str</code> List objects under a key prefix.
ObjectStore.presigned_url	method	<code>app/storage/minio_client.py:239</code>	<code>self, key: str, expires_seconds: int=3600</code>

Symbol	Kind	Location	Signature / summary
			Generate a temporary URL that grants read access to one object.
<code>get_object_store</code>	function	<code>app/storage/minio_client.py:267</code>	Lazily-constructed module-level store.
<code>narrative_char_count</code>	function	<code>app/tools/content_length.py:20</code>	<code>content: str</code> Return `content`'s length with embedded base64 image data URIs excluded.
<code>DatabaseResponse</code>	class	<code>app/tools/database_lookup.py:25</code>	Provides the DatabaseResponse behaviour.
<code>PublicDatabaseLookupService</code>	class	<code>app/tools/database_lookup.py:42</code>	Rate-limited, allow-listed public database transport.
<code>PublicDatabaseLookupService.close</code>	method	<code>app/tools/database_lookup.py:66</code>	<code>self</code> Close the result.
<code>PublicDatabaseLookupService.query_eurostat</code>	method	<code>app/tools/database_lookup.py:72</code>	<code>self, query: StructuredDatasetQuery, *, timeout_seconds: float=45.0, max_response_bytes: int=20 * 1024 * 1024</code> Query the eurostat.
<code>PublicDatabaseLookupService._client</code>	method	<code>app/tools/database_lookup.py:163</code>	<code>self, timeout_seconds: float</code> Internal helper for the client step.
<code>PublicDatabaseLookupService._rate_limit</code>	method	<code>app/tools/database_lookup.py:179</code>	<code>self</code> Internal helper for the rate limit step.
<code>_pairs_for_audit</code>	function	<code>app/tools/database_lookup.py:191</code>	<code>pairs: list[tuple[str, str]]</code> Internal helper for the pairs for audit step.
<code>get_database_lookup_service</code>	function	<code>app/tools/database_lookup.py:212</code>	Return the database lookup service.
<code>ProposalDocxRenderResult</code>	class	<code>app/tools/docx_proposal_rendering.py:71</code>	Provides the ProposalDocxRenderResult behaviour.
<code>_InlineStyle</code>	class	<code>app/tools/docx_proposal_rendering.py:97</code>	Provides the InlineStyle behaviour.
<code>_TablePlacement</code>	class	<code>app/tools/docx_proposal_rendering.py:122</code>	Provides the TablePlacement behaviour.
<code>_BuildState</code>	class	<code>app/tools/docx_proposal_rendering.py:142</code>	Provides the BuildState behaviour.
<code>_set_run_font</code>	function	<code>app/tools/docx_proposal_rendering.py:164</code>	<code>run, *, name: str='Arial', size: float None=None, color: str None=None, bold: bool None=None, italic: bool None=None</code> Set the run font.
<code>_set_shading</code>	function	<code>app/tools/docx_proposal_rendering.py:201</code>	<code>element, fill: str</code> Set the shading.
<code>_set_run_shading</code>	function	<code>app/tools/docx_proposal_rendering.py:223</code>	<code>run, fill: str</code> Set the run shading.

Symbol	Kind	Location	Signature / summary
<code>_append_styled_text</code>	function	<code>app/tools/docx_proposal_rendering.py: 241</code>	<code>paragraph, text: str, style: '_InlineStyle'</code> Append the styled text.
<code>_set_paragraph_borders</code>	function	<code>app/tools/docx_proposal_rendering.py: 276</code>	<code>paragraph, *, left: tuple[str, int, int] None=None, bottom: tuple[str, int, int] None=None</code> Set the paragraph borders.
<code>_set_cell_shading</code>	function	<code>app/tools/docx_proposal_rendering.py: 308</code>	<code>cell, fill: str</code> Set the cell shading.
<code>_set_cell_margins</code>	function	<code>app/tools/docx_proposal_rendering.py: 318</code>	<code>cell, margins: dict[str, int] None=None</code> Set the cell margins.
<code>_set_cell_width</code>	function	<code>app/tools/docx_proposal_rendering.py: 340</code>	<code>cell, width_dxa: int</code> Set the cell width.
<code>_set_repeat_header</code>	function	<code>app/tools/docx_proposal_rendering.py: 356</code>	<code>row</code> Set the repeat header.
<code>_set_table_borders</code>	function	<code>app/tools/docx_proposal_rendering.py: 370</code>	<code>table, *, style: str='single', color: str=_BORDER, size: int=4</code> Set the table borders.
<code>_set_table_geometry</code>	function	<code>app/tools/docx_proposal_rendering.py: 401</code>	<code>table, widths_dxa: list[int], *, indent_dxa: int=_TABLE_INDENT_DXA</code> Apply fixed Word table geometry with matching grid and cell widths.
<code>_column_widths</code>	function	<code>app/tools/docx_proposal_rendering.py: 450</code>	<code>rows: list[list[str]], column_count: int, *, total_width: int=_TABLE_WIDTH_DXA</code> Internal helper for the column widths step.
<code>_set_update_fields</code>	function	<code>app/tools/docx_proposal_rendering.py: 491</code>	<code>doc: DocxDocument</code> Set the update fields.
<code>_add_field</code>	function	<code>app/tools/docx_proposal_rendering.py: 505</code>	<code>paragraph, instruction: str, fallback: str=''</code> Add the field.
<code>_add_toc_field</code>	function	<code>app/tools/docx_proposal_rendering.py: 528</code>	<code>paragraph</code> Add the toc field.
<code>_add_external_hyperlink</code>	function	<code>app/tools/docx_proposal_rendering.py: 541</code>	<code>paragraph, text: str, url: str, *, bold: bool=False, italic: bool=False</code> Add the external hyperlink.
<code>_bookmark_name</code>	function	<code>app/tools/docx_proposal_rendering.py: 584</code>	<code>name: str, bookmark_id: int</code> Internal helper for the bookmark name step.
<code>_add_internal_hyperlink</code>	function	<code>app/tools/docx_proposal_rendering.py: 602</code>	<code>paragraph, text: str, anchor: str, *, bookmark_id: int, bold: bool=False, size: float=10</code> Add the internal hyperlink.
<code>_add_bookmark</code>	function		

Symbol	Kind	Location	Signature / summary
		app/tools/ docx_proposal_rendering.py: 642	<code>paragraph, name: str, bookmark_id: int</code> Add the bookmark.
<code>_add_footnote_reference</code>	function	app/tools/ docx_proposal_rendering.py: 660	<code>paragraph, footnote_id: int</code> Add the footnote reference.
<code>_footnote_body_text</code>	function	app/tools/ docx_proposal_rendering.py: 678	<code>number: int, citation: dict[str, Any]</code> Internal helper for the footnote body text step.
<code>_build_footnotes_part_xml</code>	function	app/tools/ docx_proposal_rendering.py: 697	<code>footnote_bodies: dict[int, str]</code> Build the footnotes part xml.
<code>_attach_footnotes_part</code>	function	app/tools/ docx_proposal_rendering.py: 765	<code>doc: DocxDocument, footnote_bodies: dict[int, str]</code> Internal helper for the attach footnotes part step.
<code>_toc_with_pdf_pages</code>	function	app/tools/ docx_proposal_rendering.py: 788	<code>entries: list[dict[str, Any]], pdf: bytes</code> Attach deterministic page numbers from the matching PDF layout.
<code>_set_alt_text</code>	function	app/tools/ docx_proposal_rendering.py: 823	<code>inline_shape, alt_text: str</code> Set the alt text.
<code>_normalise_metadata</code>	function	app/tools/ docx_proposal_rendering.py: 835	<code>metadata: dict[str, Any]</code> Internal helper for the normalise metadata step.
<code>_HorizonDocxBuilder</code>	class	app/tools/ docx_proposal_rendering.py: 860	<code></code> Provides the HorizonDocxBuilder behaviour.
<code>_HorizonDocxBuilder.configure_document</code>	method	app/tools/ docx_proposal_rendering.py: 893	<code>self</code> Configure the document.
<code>_HorizonDocxBuilder.configure_styles</code>	method	app/tools/ docx_proposal_rendering.py: 920	<code>self</code> Configure the styles.
<code>_HorizonDocxBuilder.configure_footnote_styles</code>	method	app/tools/ docx_proposal_rendering.py: 995	<code>self, styles, normal</code> Configure the footnote styles.
<code>_HorizonDocxBuilder.configure_header_footer</code>	method	app/tools/ docx_proposal_rendering.py: 1027	<code>self, section</code> Configure the header footer.
<code>_HorizonDocxBuilder.add_cover</code>	method	app/tools/ docx_proposal_rendering.py: 1090	<code>self, evidence_blockers: list[str]</code> Add the cover.
<code>_HorizonDocxBuilder.add_toc</code>	method	app/tools/ docx_proposal_rendering.py: 1187	<code>self, entries: list[dict[str, Any]]</code> Add the toc.
<code>_HorizonDocxBuilder.add_fragment</code>	method	app/tools/ docx_proposal_rendering.py: 1241	<code>self, fragment: str</code> Add the fragment.
	method		

Symbol	Kind	Location	Signature / summary
<code>_HorizonDocxBUILDER.render_block</code>		<code>app/tools/docx_proposal_rendering.py: 1251</code>	<code>self, node, *, list_level: int=0</code> Render the block.
<code>_HorizonDocxBUILDER.append_text_with_footnotes</code>	method	<code>app/tools/docx_proposal_rendering.py: 1377</code>	<code>self, paragraph, text: str, style: _InlineStyle</code> Append the text with footnotes.
<code>_HorizonDocxBUILDER.append_inline</code>	method	<code>app/tools/docx_proposal_rendering.py: 1413</code>	<code>self, paragraph, container, style: _InlineStyle None=None</code> Append the inline.
<code>_HorizonDocxBUILDER.render_list</code>	method	<code>app/tools/docx_proposal_rendering.py: 1485</code>	<code>self, list_tag: Tag, *, list_level: int</code> Render the list.
<code>_HorizonDocxBUILDER.table_placements</code>	method	<code>app/tools/docx_proposal_rendering.py: 1505</code>	<code>self, table_tag: Tag</code> Internal helper for the table placements step.
<code>_HorizonDocxBUILDER.render_table</code>	method	<code>app/tools/docx_proposal_rendering.py: 1550</code>	<code>self, table_tag: Tag</code> Render the table.
<code>_HorizonDocxBUILDER.placeholder_text</code>	method	<code>app/tools/docx_proposal_rendering.py: 1631</code>	<code>self, figure: Tag</code> Internal helper for the placeholder text step.
<code>_HorizonDocxBUILDER.render_placeholder</code>	method	<code>app/tools/docx_proposal_rendering.py: 1651</code>	<code>self, prompt: str</code> Render the placeholder.
<code>_HorizonDocxBUILDER.decode_data_image</code>	method	<code>app/tools/docx_proposal_rendering.py: 1684</code>	<code>self, source: str</code> Decode the data image.
<code>_HorizonDocxBUILDER.render_image</code>	method	<code>app/tools/docx_proposal_rendering.py: 1712</code>	<code>self, image: Tag, caption: Tag None=None</code> Render the image.
<code>_HorizonDocxBUILDER.render_figure</code>	method	<code>app/tools/docx_proposal_rendering.py: 1748</code>	<code>self, figure: Tag</code> Render the figure.
<code>_HorizonDocxBUILDER.add_bibliography</code>	method	<code>app/tools/docx_proposal_rendering.py: 1771</code>	<code>self, citation_registry: list[dict[str, Any]]</code> Add the bibliography.
<code>_HorizonDocxBUILDER.add_evidence_annex</code>	method	<code>app/tools/docx_proposal_rendering.py: 1816</code>	<code>self, evidence_qa: dict[str, Any], evidence_blockers: list[str]</code> Add the evidence annex.
<code>_HorizonDocxBUILDER.save</code>	method	<code>app/tools/docx_proposal_rendering.py: 1874</code>	<code>self</code> Save the result.
<code>render_horizon_proposal_docx</code>	function	<code>app/tools/docx_proposal_rendering.py: 1887</code>	<code>*, content: str, content_format: Literal['markdown', 'html'], metadata: dict[str, Any], citation_registry: list[dict[str, Any]] None=None, evidence_qa: dict[str, Any] None=None, evidence_blockers: list[str] None=None, include_toc: bool=True, include_bibliography: bool=True, include_evidence_annex: bool=False, page_limit: int None=45,</code>

Symbol	Kind	Location	Signature / summary
			<code>max_embedded_image_bytes: int=10000000,</code> <code>enable_footnotes: bool=False</code> Create an editable Horizon Part B DOCX from sanitised HTML/Markdown.
<code>read_tables_from_excel</code>	function	<code>app/tools/excel_io.py:9</code>	<code>file_bytes: bytes</code> Return a dict of {sheet_name: rows} where rows is a list of lists.
<code>GeneratedImage</code>	class	<code>app/tools/image_io.py:45</code>	Provides the GeneratedImage behaviour.
<code>OpenAllImageGenerationService</code>	class	<code>app/tools/image_io.py:64</code>	Provides the OpenAllImageGenerationService behaviour.
<code>OpenAllImageGenerationService.available</code>	method	<code>app/tools/image_io.py:83</code>	<code>self, provider: str='openai'</code> Compute the available.
<code>OpenAllImageGenerationService.generate</code>	method	<code>app/tools/image_io.py:93</code>	<code>self, prompt: str, *, size: str='auto',</code> <code>quality: str='auto', output_format:</code> <code>str='png', model: str None=None,</code> <code>provider: ImageProvider='openai'</code> Generate the result.
<code>OpenAllImageGenerationService._generate_openrouter</code>	method	<code>app/tools/image_io.py:178</code>	<code>self, prompt: str, *, size: str,</code> <code>quality: str, output_format: str, model:</code> <code>str</code> No summary declared.
<code>_image_request_kwargs</code>	function	<code>app/tools/image_io.py:240</code>	<code>*, model: str, prompt: str, size: str,</code> <code>quality: str, output_format: str</code> Internal helper for the image request kwargs step.
<code>_model_unavailable</code>	function	<code>app/tools/image_io.py:286</code>	<code>exc: BaseException</code> Internal helper for the model unavailable step.
<code>get_image_generation_service</code>	function	<code>app/tools/image_io.py:315</code>	Return the image generation service.
<code>extract_text_from_pdf</code>	function	<code>app/tools/pdf_io.py:17</code>	<code>file_bytes: bytes</code> Return a list of {page_num, text} for each page.
<code>render_proposal_pdf</code>	function	<code>app/tools/pdf_io.py:29</code>	<code>sections: dict[str, str], template: str,</code> <code>client_name: str, proposal_title: str</code> Render an HTML proposal, then convert to PDF bytes via WeasyPrint.
<code>build_proposal_pptx</code>	function	<code>app/tools/powerpoint_io.py:9</code>	<code>sections: dict[str, str], title: str,</code> <code>client_name: str</code> Title slide + one content slide per section.
<code>ProposalRenderResult</code>	class	<code>app/tools/proposal_rendering.py:57</code>	Provides the ProposalRenderResult behaviour.
<code>_markdown</code>	function	<code>app/tools/proposal_rendering.py:78</code>	Internal helper for the markdown step.
<code>_slug</code>	function	<code>app/tools/proposal_rendering.py:97</code>	<code>value: str</code> Internal helper for the slug step.
<code>_unique_id</code>	function	<code>app/tools/proposal_rendering.py:110</code>	<code>base: str, used: set[str]</code> Internal helper for the unique id step.
<code>_sanitise_fragment</code>	function	<code>app/tools/proposal_rendering.py:129</code>	<code>value: str, content_format: str</code> Internal helper for the sanitise fragment step.

Symbol	Kind	Location	Signature / summary
<code>_toc</code>	function	<code>app/tools/proposal_rendering.py:210</code>	<code>fragment: str</code> Internal helper for the toc step.
<code>_safe_url_fetcher</code>	function	<code>app/tools/proposal_rendering.py:232</code>	<code>url: str, *args: Any, **kwargs: Any</code> Allow embedded images and renderer-owned local assets only.
<code>render_horizon_proposal</code>	function	<code>app/tools/proposal_rendering.py:248</code>	<code>*, content: str, content_format: Literal['markdown', 'html'], metadata: dict[str, Any], citation_registry: list[dict[str, Any]] None=None, evidence_qa: dict[str, Any] None=None, evidence_blockers: list[str] None=None, include_toc: bool=True, include_bibliography: bool=True, include_evidence_annex: bool=False, page_limit: int None=45</code> Render a Horizon Part B proposal and return immutable output metadata.
<code>VisionAnalysis</code>	class	<code>app/tools/vision_io.py:28</code>	 Provides the VisionAnalysis behaviour.
<code>KimiVisionService</code>	class	<code>app/tools/vision_io.py:43</code>	 Send an object-storage image to Kimi as a base64 content part.
<code>KimiVisionService.available</code>	method	<code>app/tools/vision_io.py:61</code>	<code>self</code> Compute the available.
<code>KimiVisionService.analyze</code>	method	<code>app/tools/vision_io.py:69</code>	<code>self, image_bytes: bytes, *, content_type: str, prompt: str, model: KimiVisionModel='kimi-k3', max_completion_tokens: int=8192</code> Compute the analyze.
<code>get_kimi_vision_service</code>	function	<code>app/tools/vision_io.py:159</code>	 Return the kimi vision service.
<code>WebResult</code>	class	<code>app/tools/web_io.py:22</code>	 Provides the WebResult behaviour.
<code>WebSearchResponse</code>	class	<code>app/tools/web_io.py:40</code>	 Provides the WebSearchResponse behaviour.
<code>TavilyQuotaExhausted</code>	class	<code>app/tools/web_io.py:61</code>	<code>RuntimeError</code> Tavily cannot serve the request because quota/rate is exhausted.
<code>WebSearchService</code>	class	<code>app/tools/web_io.py:65</code>	 Provides the WebSearchService behaviour.
<code>WebSearchService.available_providers</code>	method	<code>app/tools/web_io.py:88</code>	<code>self</code> Compute the available providers.
<code>WebSearchService.resolve_provider</code>	method	<code>app/tools/web_io.py:105</code>	<code>self, requested: SearchProvider</code> Resolve Auto without making a provider request.
<code>WebSearchService.search</code>	method	<code>app/tools/web_io.py:110</code>	<code>self, query: str, *, provider: SearchProvider='auto', top_k: int=8, fallback_to_openai: bool=True</code> Search the result.
<code>WebSearchService._select_provider</code>	method	<code>app/tools/web_io.py:193</code>	<code>self, requested: SearchProvider</code> Select the provider.

Symbol	Kind	Location	Signature / summary
WebSearchService._search_tavily	method	app/tools/web_io.py:216	<code>self, query: str, top_k: int</code> Search the tavily.
WebSearchService._search_openai	method	app/tools/web_io.py:283	<code>self, query: str, top_k: int</code> Search the openai.
WebSearchService._search_kimi	method	app/tools/web_io.py:349	<code>self, query: str, top_k: int</code> Search the kimi.
WebSearchService._stub_results	method	app/tools/web_io.py:460	<code>query: str, top_k: int</code> Internal helper for the stub results step.
_rank_score	function	app/tools/web_io.py:484	<code>index: int, count: int</code> Rank the score.
_url_citations	function	app/tools/web_io.py:497	<code>value: Any</code> Internal helper for the url citations step.
_response_usage	function	app/tools/web_io.py:537	<code>response: Any</code> Internal helper for the response usage step.
get_web_search_service	function	app/tools/web_io.py:564	<code></code> Return the web search service.
search_web	async function	app/tools/web_io.py:576	<code>query: str, top_k: int=8, provider: SearchProvider='auto'</code> MCP-compatible entry point returning JSON-safe candidate records.
_utc_now	function	app/workflow/builder_store.py:26	<code></code> Internal helper for the utc now step.
_digest	function	app/workflow/builder_store.py:35	<code>text: str</code> Internal helper for the digest step.
_atomic_write	function	app/workflow/builder_store.py:47	<code>path: Path, text: str</code> Internal helper for the atomic write step.
WorkflowBuilderStore	class	app/workflow/builder_store.py:60	<code></code> Own autosave drafts and immutable versions for one workflows root.
WorkflowBuilderStore.validate_name	method	app/workflow/builder_store.py:73	<code>name: str</code> Validate the name.
WorkflowBuilderStore.validate_version_id	method	app/workflow/builder_store.py:87	<code>version_id: str</code> Validate the version id.
WorkflowBuilderStore.workflow_path	method	app/workflow/builder_store.py:100	<code>self, name: str</code> Compute the workflow path.
WorkflowBuilderStore.draft_path	method	app/workflow/builder_store.py:111	<code>self, name: str</code> Draft the path.
WorkflowBuilderStore.versions_dir	method	app/workflow/builder_store.py:122	<code>self, name: str</code> Compute the versions dir.
WorkflowBuilderStore.save_draft	method	app/workflow/builder_store.py:133	<code>self, name: str, yaml_text: str, *, canvas: dict[str, Any] None=None</code> Save the draft.
WorkflowBuilderStore.read_draft	method	app/workflow/builder_store.py:172	<code>self, name: str</code> Read the draft.
WorkflowBuilderStore.delete_draft	method	app/workflow/builder_store.py:199	<code>self, name: str</code> Delete the draft.

Symbol	Kind	Location	Signature / summary
WorkflowBuilderStore.record_version	method	app/workflow/builder_store.py:214	<code>self, name: str, yaml_text: str</code> Record the version.
WorkflowBuilderStore.save_workflow	method	app/workflow/builder_store.py:234	<code>self, name: str, yaml_text: str</code> Save the workflow.
WorkflowBuilderStore.get_version	method	app/workflow/builder_store.py:254	<code>self, name: str, version_id: str</code> Return the version.
WorkflowBuilderStore.list_versions	method	app/workflow/builder_store.py:270	<code>self, name: str</code> List the versions.
WorkflowBuilderStore.restore_version	method	app/workflow/builder_store.py:319	<code>self, name: str, version_id: str</code> Restore the version.
WorkflowBuilderStore.delete_workflow	method	app/workflow/builder_store.py:333	<code>self, name: str</code> Remove the executable YAML together with every Builder-owned trace of it — an autosave draft and the whole immutable version history — so nothing orphaned lingers under ``.builder`` once the workflow itself is gone.
_split_pascal_case	function	app/workflow/capability_selection.py:55	<code>name: str</code> "RouterAgent" -> "router agent"; "MCPToolAgent" -> "mcp tool agent" — without this, a type name is one opaque token and never matches a request's ordinary-language words.
_tokens	function	app/workflow/capability_selection.py:62	<code>text: str</code> Internal helper for the tokens step.
_entry_text	function	app/workflow/capability_selection.py:74	<code>entry: dict[str, Any]</code> Internal helper for the entry text step.
select_candidate_node_types	function	app/workflow/capability_selection.py:93	<code>prompt: str, manifest: list[dict[str, Any]], *, max_candidates: int=12</code> Return the type names the generator's system prompt should describe: every core node type, Literal/Echo, and up to `max_candidates` specialized types ranked by keyword overlap with `prompt`.
select_generation_model	function	app/workflow/capability_selection.py:163	<code>prompt: str, manifest: list[dict[str, Any]]</code> Deterministic complexity → one of two model tiers, no LLM call.
ChatConversationNotFoundError	class	app/workflow/chat_conversation_store.py:18	<code>LookupError</code> The conversation is not accessible in the authenticated owner scope.
ChatConversationRecord	class	app/workflow/chat_conversation_store.py:22	<code>BaseModel</code> No summary declared.
ChatConversationRecord.public	method	app/workflow/chat_conversation_store.py:31	<code>self</code> No summary declared.
ChatConversationMessageRecord	class	app/workflow/chat_conversation_store.py:41	<code>BaseModel</code> No summary declared.
ChatConversationMessageRecord.public	method	app/workflow/chat_conversation_store.py:52	<code>self</code> No summary declared.

Symbol	Kind	Location	Signature / summary
<code>ensure_chat_conversation_indexes</code>	async function	<code>app/workflow/chat_conversation_store.py:63</code>	<code>db: Any</code> No summary declared.
<code>ChatConversationStore</code>	class	<code>app/workflow/chat_conversation_store.py:80</code>	Conversation persistence where every operation is owner-scoped.
<code>ChatConversationStore.get_or_create</code>	method	<code>app/workflow/chat_conversation_store.py:87</code>	<code>self, owner_scope_id: str, *, workflow_source: ChatWorkflowSource, workflow_id: str</code> No summary declared.
<code>ChatConversationStore.get</code>	method	<code>app/workflow/chat_conversation_store.py:118</code>	<code>self, owner_scope_id: str, conversation_id: str</code> No summary declared.
<code>ChatConversationStore.list_messages</code>	method	<code>app/workflow/chat_conversation_store.py:129</code>	<code>self, owner_scope_id: str, conversation_id: str</code> No summary declared.
<code>ChatConversationStore.append_message</code>	method	<code>app/workflow/chat_conversation_store.py:139</code>	<code>self, owner_scope_id: str, conversation_id: str, *, message_id: str, role: ChatMessageRole, content: dict[str, Any], run_id: str None</code> No summary declared.
<code>ChatConversationStore.replace_message</code>	method	<code>app/workflow/chat_conversation_store.py:179</code>	<code>self, owner_scope_id: str, conversation_id: str, message_id: str, *, role: ChatMessageRole, content: dict[str, Any], run_id: str None</code> No summary declared.
<code>ChatOutputCompatibility</code>	class	<code>app/workflow/chat_output_compatibility.py:14</code>	<code>BaseModel</code> No summary declared.
<code>_terminal_ids</code>	function	<code>app/workflow/chat_output_compatibility.py:31</code>	<code>spec: WorkflowSpec</code> No summary declared.
<code>analyze_chat_output</code>	function	<code>app/workflow/chat_output_compatibility.py:40</code>	<code>spec: WorkflowSpec</code> Describe whether a workflow has a meaningful normalized Chat result.
<code>_default_output_compatibility</code>	function	<code>app/workflow/chat_workflow_store.py:15</code>	Safe response shape for records created before output analysis existed.
<code>ChatWorkflowConflictError</code>	class	<code>app/workflow/chat_workflow_store.py:27</code>	<code>ValueError</code> A private workflow slug already exists for this owner.
<code>ChatWorkflowNotFoundError</code>	class	<code>app/workflow/chat_workflow_store.py:31</code>	<code>LookupError</code> A private workflow was not found in the authenticated owner scope.
<code>ChatWorkflowRecord</code>	class	<code>app/workflow/chat_workflow_store.py:35</code>	<code>BaseModel</code> No summary declared.
<code>ChatWorkflowRecord.public_summary</code>	method	<code>app/workflow/chat_workflow_store.py:55</code>	<code>self</code> Return card metadata without owner identity or workflow YAML.
<code>ensure_chat_workflow_indexes</code>	async function	<code>app/workflow/chat_workflow_store.py:73</code>	<code>db: Any</code> No summary declared.

Symbol	Kind	Location	Signature / summary
ChatWorkflowStore	class	app/workflow/ chat_workflow_store.py:89	 CRUD where every query is pinned to an authenticated owner scope.
ChatWorkflowStore.create	method	app/workflow/ chat_workflow_store.py:95	<pre>self, *, owner_scope_id: str, slug: str, display_name: str, description: str, yaml_text: str, source: Literal['generated', 'imported', 'existing'], source_workflow_name: str None=None, output_compatibility: dict[str, Any] None=None, managed: bool=False, adapter_key: str None=None, adapter_fingerprint: str None=None</pre> No summary declared.
ChatWorkflowStore.list_private	method	app/workflow/ chat_workflow_store.py:144	<pre>self, owner_scope_id: str</pre> No summary declared.
ChatWorkflowStore.get	method	app/workflow/ chat_workflow_store.py:151	<pre>self, owner_scope_id: str, chat_workflow_id: str</pre> No summary declared.
ChatWorkflowStore.get_by_slug	method	app/workflow/ chat_workflow_store.py:161	<pre>self, owner_scope_id: str, slug: str</pre> No summary declared.
ChatWorkflowStore.get_by_adapter_key	method	app/workflow/ chat_workflow_store.py:169	<pre>self, owner_scope_id: str, adapter_key: str</pre> No summary declared.
ChatWorkflowStore.replace_managed_adapter	method	app/workflow/ chat_workflow_store.py:177	<pre>self, owner_scope_id: str, chat_workflow_id: str, *, display_name: str, yaml_text: str, description: str, source: Literal['generated', 'imported', 'existing'], source_workflow_name: str None, output_compatibility: dict[str, Any], adapter_key: str, adapter_fingerprint: str</pre> Claim or refresh a system-managed adapter without changing its stable ID.
ChatWorkflowStore.replace_executable	method	app/workflow/ chat_workflow_store.py:217	<pre>self, owner_scope_id: str, chat_workflow_id: str, *, yaml_text: str, description: str, output_compatibility: dict[str, Any]</pre> Replace only executable metadata for an owner-scoped private workflow.
ChatWorkflowStore.archive	method	app/workflow/ chat_workflow_store.py:249	<pre>self, owner_scope_id: str, chat_workflow_id: str</pre> No summary declared.
ChatWorkflowStore.request_publication	method	app/workflow/ chat_workflow_store.py:267	<pre>self, owner_scope_id: str, chat_workflow_id: str</pre> No summary declared.
WorkspaceExperience	class	app/workflow/ chat_workspace_planner.py:26	 No summary declared.
WorkspaceExperience.publish	method	app/workflow/ chat_workspace_planner.py:34	<pre>self</pre> No summary declared.
WorkspacePlan	class		

Symbol	Kind	Location	Signature / summary
		app/workflow/ chat_workspace_planner.py:9 6	No summary declared.
WorkspacePlan.public	method	app/workflow/ chat_workspace_planner.py:1 06	<code>self</code> No summary declared.
_experience	function	app/workflow/ chat_workspace_planner.py:1 19	<code>value: str None</code> No summary declared.
_base	function	app/workflow/ chat_workspace_planner.py:1 23	<code>title: str, welcome: str, allow_attachments: bool=True</code> No summary declared.
_experience_block	function	app/workflow/ chat_workspace_planner.py:1 54	<code>name: str, purpose: str, output: str</code> No summary declared.
_answer_node	function	app/workflow/ chat_workspace_planner.py:1 62	<code>instructions: str, input_fields: list[dict[str, Any]]</code> No summary declared.
_reply	function	app/workflow/ chat_workspace_planner.py:1 82	<code>message: str, *, sources: str None=None, handoff: dict[str, Any] None=None</code> No summary declared.
build_existing_workflow_c hat_adapter	function	app/workflow/ chat_workspace_planner.py:1 94	<code>workflow_name: str, workflow_spec: WorkflowSpec, *, runtime_metadata: dict[str, Any] None=None</code> Wrap any saved workflow with natural-language input/ output LLM calls.
build_llm_adapter	function	app/workflow/ chat_workspace_planner.py:3 21	<code>title: str='AI Workspace', previous_result: Any=None</code> No summary declared.
build_scientific_skill_adapt er	function	app/workflow/ chat_workspace_planner.py:3 37	<code>skill_name: str</code> Build a conversational workflow that applies one approved skill.
build_file_adapter	function	app/workflow/ chat_workspace_planner.py:3 68	<code>title: str, instructions: str</code> No summary declared.
build_vision_adapter	function	app/workflow/ chat_workspace_planner.py:4 02	<code>title: str</code> No summary declared.
build_multimodal_file_ada pter	function	app/workflow/ chat_workspace_planner.py:4 29	<code>title: str, instructions: str</code> Analyze mixed uploaded sources, including one image plus text files.
build_web_adapter	function	app/workflow/ chat_workspace_planner.py:4 75	<code>title: str</code> No summary declared.
build_mcp_adapter	function	app/workflow/ chat_workspace_planner.py:4 99	<code>title: str, server_id: str, tool: str</code> No summary declared.
build_retrieval_adapter	function	app/workflow/ chat_workspace_planner.py:5 28	<code>title: str, collection_id: str, retrieval_profile_id: str, instructions: str, document_ids: list[str] </code>

Symbol	Kind	Location	Signature / summary
			None=None No summary declared.
build_rag_adapter	function	app/workflow/ chat_workspace_planner.py:5 57	title: str, rag_agent_id: str, document_ids: list[str] None=None No summary declared.
build_artifact_adapter	function	app/workflow/ chat_workspace_planner.py:6 08	title: str, output: Literal['pdf', 'docx', 'pptx'], *, with_files: bool, previous_result: Any=None No summary declared.
build_image_generation_adapter	function	app/workflow/ chat_workspace_planner.py:6 75	title: str='Image Creator' No summary declared.
build_grounded_diagram_adapter	function	app/workflow/ chat_workspace_planner.py:7 05	title: str='Architecture Diagram' Read attached sources, explain them, and generate a grounded visual image.
plan_workspace	function	app/workflow/ chat_workspace_planner.py:7 73	objective: str, *, experience_id: str None=None, skill_name: str None=None, selected_workflow: str None=None, preferred_output: OutputKind='auto', has_attachments: bool=False, attachment_categories: list[str] None=None, collection_id: str None=None, retrieval_profile_id: str None=None, rag_agent_id: str None=None, document_ids: list[str] None=None, integration_connection: str None=None, integration_tool: str None=None, previous_run_id: str None=None, previous_result: Any=None No summary declared.
_key	function	app/workflow/ claim_verifications.py:14	run_id: str, record_id: str Internal helper for the key step.
ensure_indexes	async function	app/workflow/ claim_verifications.py:27	db Ensure the indexes.
get_verifications	async function	app/workflow/ claim_verifications.py:38	db, run_id: str Return the verifications.
save_verification	async function	app/workflow/ claim_verifications.py:52	db, run_id: str, record_id: str, result: dict[str, Any] Save the verification.
WorkflowFileInputError	class	app/workflow/ file_inputs.py:55	ValueError Raised when an upload or workflow file reference is invalid.
scope_token	function	app/workflow/ file_inputs.py:59	session_id: str Return a non-reversible, path-safe tenant/session token.
workflow_input_prefix	function	app/workflow/ file_inputs.py:65	session_id: str Compute the workflow input prefix.
safe_filename	function	app/workflow/ file_inputs.py:77	filename: str None Strip paths/control characters while retaining a readable name.
extension_for	function		

Symbol	Kind	Location	Signature / summary
		app/workflow/ file_inputs.py:85	<code>filename: str</code> Compute the extension for.
<code>category_for_extension</code>	function	app/workflow/ file_inputs.py:97	<code>extension: str</code> Compute the category for extension.
<code>content_type_for</code>	function	app/workflow/ file_inputs.py:115	<code>filename: str, extension: str</code> Compute the content type for.
<code>accepted_by_spec</code>	function	app/workflow/ file_inputs.py:133	<code>ref: WorkflowFileRef, spec: WorkflowInputSpec</code> Accept category names or literal extensions in workflow YAML.
<code>store_upload</code>	async function	app/workflow/ file_inputs.py:140	<code>upload: UploadFile, *, session_id: str, object_store: Any, remaining_total_bytes: int None=None</code> Stream one multipart upload to disk, enforce limits, then store it.
<code>validate_workflow_inputs</code>	async function	app/workflow/ file_inputs.py:224	<code>specs: dict[str, WorkflowInputSpec], values: dict[str, Any], *, session_id: str, object_store: Any None</code> Validate and normalize declared workflow file inputs before execution.
<code>validate_workflow_file_reference</code>	async function	app/workflow/ file_inputs.py:303	<code>raw_ref: WorkflowFileRef dict[str, Any], *, session_id: str, object_store: Any None, require_parseable_text: bool=False</code> Validate one file reference before a download, extraction, or HITL edit.
<code>extract_workflow_file_text</code>	async function	app/workflow/ file_inputs.py:339	<code>raw_ref: WorkflowFileRef dict[str, Any], *, session_id: str, object_store: Any None, max_chars: int=1000000</code> Extract one scoped upload into plain text for the HITL editor.
<code>record_uploaded_files</code>	async function	app/workflow/ file_inputs.py:396	<code>db: Any None, *, session_id: str, refs: list[WorkflowFileRef]</code> Persist upload metadata for audit/cleanup without storing file bytes.
<code>_humanize</code>	function	app/workflow/ library_metadata.py:35	<code>identifier: str</code> Internal helper for the humanize step.
<code>_terminal_node_types</code>	function	app/workflow/ library_metadata.py:48	<code>spec: WorkflowSpec</code> Internal helper for the terminal node types step.
<code>infer_output_types</code>	function	app/workflow/ library_metadata.py:62	<code>spec: WorkflowSpec</code> Compute the infer output types.
<code>_default_human_review_count</code>	function	app/workflow/ library_metadata.py:80	<code>spec: WorkflowSpec</code> Internal helper for the default human review count step.
<code>library_summary</code>	function	app/workflow/ library_metadata.py:92	<code>spec: WorkflowSpec</code> The Library card/list-facing metadata dict for one workflow.
<code>readiness_summary</code>	function	app/workflow/ library_metadata.py:153	<code>report: WorkflowPreflightReport</code> Maps a structural preflight report to the Library's three-state readiness model (ready / ready_with_warnings / blocked), reusing the report's own plain-language issue text ('message'/'suggestion') instead of inventing new copy or surfacing only a technical code.

Symbol	Kind	Location	Signature / summary
start_new_run_record	async function	app/workflow/ orchestration.py:28	db: Any, *, run_id: str, session: str, spec: WorkflowSpec, workflow_yaml: str, inputs: dict[str, Any], collection_id: str, retry_of_run_id: str None=None, attempt: int=1, origin: str='direct', history_visibility: str='global', workflow_id: str None=None, workflow_version_id: str None=None, conversation_id: str None=None, message_id: str None=None Create the durable "running" record and checkpoint before execution.
record_run_failure	async function	app/workflow/ orchestration.py:90	db: Any, *, run_id: str, session: str, error: BaseException str, services: dict[str, Any] None=None Persist an exception raised while executing/resuming a run.
finalize_run_result	async function	app/workflow/ orchestration.py:126	result: dict[str, Any], *, db: Any, run_id: str, session: str, record_rejection_reason: bool=False, services: dict[str, Any] None=None Persist a terminal or re-paused outcome.
run_and_finalize	async function	app/workflow/ orchestration.py:179	coro: Awaitable[dict[str, Any]], *, db: Any, run_id: str, session: str, record_rejection_reason: bool=False, services: dict[str, Any] None=None Await one execution coroutine and persist its outcome end to end.
BackgroundRunManager	class	app/workflow/ orchestration.py:223	Own detached run tasks with a Redis lease shared by all workers.
BackgroundRunManager.l aunch	method	app/workflow/ orchestration.py:249	self, coro: Awaitable[dict[str, Any]], *, db: Any, run_id: str, session: str, record_rejection_reason: bool=False, services: dict[str, Any] None=None Launch the result.
BackgroundRunManager._ run_owned	method	app/workflow/ orchestration.py:301	self, coro: Awaitable[dict[str, Any]], *, db: Any, run_id: str, session: str, record_rejection_reason: bool, services: dict[str, Any] None=None Run the owned.
BackgroundRunManager.c lose	method	app/workflow/ orchestration.py:399	self Cancel owned work and let run_and_finalize persist interruption.
_reconcile_subprocess_cal lback	async function	app/workflow/ orchestration.py:409	db: Any, run_id: str, session: str, services: dict[str, Any] None=*, status: str, output: Any, node_outputs: dict[str, Any], error: str None Best-effort: if this run_id is a Subprocess node's child, deliver its result to the waiting parent (app.workflow.subprocess_callback).
ensure_indexes	async function	app/workflow/ preflight_stats.py:27	db: Any Ensure the indexes.
record_attempt			db: Any, *, source: AttemptSource, workflow_name: str None, success:

Symbol	Kind	Location	Signature / summary
	async function	app/workflow/preflight_stats.py:39	<code>bool, error_codes: list[str], initial_error_codes: list[str] None=None, deterministic_fixes_applied: int=0, llm_attempts: int=0, total_attempts: int=1</code> Record the attempt.
preflight_stats	async function	app/workflow/preflight_stats.py:84	<code>db: Any, *, since: datetime None=None, sample_limit: int=2000</code> Compute the preflight stats.
template_variables	function	app/workflow/prompt_template_store.py:19	<code>content: str</code> Return unique placeholders in first-appearance order.
PromptTemplateRecord	class	app/workflow/prompt_template_store.py:24	<code>BaseModel</code> No summary declared.
PromptTemplateRecord.public	method	app/workflow/prompt_template_store.py:36	<code>self</code> No summary declared.
built_in_templates	function	app/workflow/prompt_template_store.py:62	<code></code> No summary declared.
ensure_prompt_template_indexes	async function	app/workflow/prompt_template_store.py:74	<code>db: Any</code> No summary declared.
PromptTemplateStore	class	app/workflow/prompt_template_store.py:80	<code></code> No summary declared.
PromptTemplateStore.create	method	app/workflow/prompt_template_store.py:84	<code>self, owner_scope_id: str, *, title: str, description: str, category: PromptCategory, content: str</code> No summary declared.
PromptTemplateStore.list	method	app/workflow/prompt_template_store.py:93	<code>self, owner_scope_id: str</code> No summary declared.
PromptTemplateStore.get	method	app/workflow/prompt_template_store.py:97	<code>self, owner_scope_id: str, template_id: str</code> No summary declared.
PromptTemplateStore.update	method	app/workflow/prompt_template_store.py:101	<code>self, owner_scope_id: str, template_id: str, **changes: Any</code> No summary declared.
PromptTemplateStore.delete	method	app/workflow/prompt_template_store.py:114	<code>self, owner_scope_id: str, template_id: str</code> No summary declared.
_require_session	function	app/workflow/run_chat_store.py:17	<code>session_id: str</code> Internal helper for the require session step.
ensure_run_chat_indexes	async function	app/workflow/run_chat_store.py:31	<code>db</code> Ensure the run chat indexes.
get_run_chat_turns	async function	app/workflow/run_chat_store.py:42	<code>db, *, session_id: str, run_id: str</code> Return the run chat turns.
append_run_chat_turn	async function	app/workflow/run_chat_store.py:60	<code>db, *, session_id: str, run_id: str, question: str, answer: str, model: str</code> Append one Q&A turn and return the full updated turn list.
_is_document_size_error	function	app/workflow/run_history.py:44	<code>exc: Exception</code> Return whether document size error.

Symbol	Kind	Location	Signature / summary
<code>_externalize_if_large</code>	async function	app/workflow/run_history.py:57	<code>db, *, run_id: str, node_id: str, field: str, value: Any, force: bool=False</code> Return `value` unchanged, or a small GridFS pointer if it's too big.
<code>_inflate_value</code>	async function	app/workflow/run_history.py:94	<code>db, value: Any</code> Reverse `_externalize_if_large`: fetch and decode a GridFS pointer.
<code>_inflate_run_document</code>	async function	app/workflow/run_history.py:113	<code>db, doc: dict[str, Any] None</code> Transparently resolve GridFS pointers so callers see the original shape.
<code>_require_session</code>	function	app/workflow/run_history.py:151	<code>session_id: str</code> Enforce the same isolation boundary on every history operation.
<code>_node_key</code>	function	app/workflow/run_history.py:159	<code>node_id: str</code> Return a Mongo-safe map key while preserving node_id in the value.
<code>redact_for_history</code>	function	app/workflow/run_history.py:165	<code>value: Any</code> Recursively remove prompts and credentials from operator-visible data.
<code>build_node_input</code>	function	app/workflow/run_history.py:193	<code>state: dict[str, Any], resolved_config: dict[str, Any]</code> Build the inspectable input for one node without duplicating all outputs.
<code>ensure_indexes</code>	async function	app/workflow/run_history.py:218	<code>db</code> Create indexes used by run detail and newest-first history views.
<code>upsert_run</code>	async function	app/workflow/run_history.py:243	<code>db, run_id: str, session_id: str, workflow_name: str None=None, status: str None=None, node_types: dict[str, str] None=None, inputs: dict[str, Any] None=None, variables: dict[str, Any] None=None, outputs: dict[str, Any] None=None, workflow_yaml: str None=None, started_at: float None=None, ended_at: float None=None, node_count: int None=None, completed_node_count: int None=None, retry_of_run_id: str None=None, attempt: int None=None, reused_node_count: int None=None, reused_nodes: list[str] None=None, error: str None=None, origin: str None=None, history_visibility: str None=None, workflow_id: str None=None, workflow_version_id: str None=None, conversation_id: str None=None, message_id: str None=None</code> Create or patch one run without erasing fields omitted by the caller.
<code>record_node_started</code>	async function	app/workflow/run_history.py:390	<code>db, *, run_id: str, session_id: str, node_id: str, type_name: str, node_input: dict[str, Any], started_at: float</code> Mark one node active and persist its effective, redacted input.

Symbol	Kind	Location	Signature / summary
record_node_completed	async function	app/workflow/ run_history.py:464	<code>db, *, run_id: str, session_id: str, node_id: str, output: dict[str, Any], model_selections: list[dict[str, Any]] None=None, ended_at: float, duration_s: float</code> Persist a full node output as soon as that node completes.
record_node_reused	async function	app/workflow/ run_history.py:545	<code>db, *, run_id: str, session_id: str, node_id: str, type_name: str, output: dict[str, Any], source_run_id: str, ended_at: float, duration_s: float</code> Persist a node result replayed from a previous failed attempt.
record_node_paused	async function	app/workflow/ run_history.py:637	<code>db, *, run_id: str, session_id: str, node_id: str, paused_at: float, pause_kind: str='hitl_gate'</code> Mark a node and the overall run as paused.
request_pause	async function	app/workflow/ run_history.py:682	<code>db, *, run_id: str, session_id: str</code> Ask a running workflow to pause at its next node boundary.
clear_pause_request	async function	app/workflow/ run_history.py:704	<code>db, *, run_id: str, session_id: str</code> Clear the pause request.
is_pause_requested	async function	app/workflow/ run_history.py:724	<code>db, *, run_id: str, session_id: str</code> Cheap per-node check — the executor calls this before every node.
record_node_failed	async function	app/workflow/ run_history.py:734	<code>db, *, run_id: str, session_id: str, node_id: str, type_name: str, error: str, error_type: str None=None, error_traceback: str None=None, model_selections: list[dict[str, Any]] None=None, ended_at: float, duration_s: float</code> Persist node attribution and partial run state on failure.
initialize_run_checkpoint	async function	app/workflow/ run_history.py:785	<code>db, *, run_id: str, session_id: str, workflow_yaml: str, inputs: dict[str, Any], collection_id: str, retry_of_run_id: str None=None</code> Create the private checkpoint used for zero-token node replay.
record_checkpoint_node_completed	async function	app/workflow/ run_history.py:834	<code>db, *, run_id: str, session_id: str, node_id: str, output: dict[str, Any], extra_state: dict[str, Any]</code> Save the exact replayable result immediately after node completion.
record_checkpoint_paused	async function	app/workflow/ run_history.py:912	<code>db, *, run_id: str, session_id: str, node_id: str, pause_context: Any, pause_kind: str='hitl_gate'</code> Persist enough information to resume a paused run after process restart.
record_checkpoint_approval	async function	app/workflow/ run_history.py:950	<code>db, *, run_id: str, session_id: str, node_id: str, decision: dict[str, Any], actor: str</code> Append an immutable human decision before resuming execution.

Symbol	Kind	Location	Signature / summary
mark_checkpoint_status	async function	app/workflow/ run_history.py:986	<code>db, *, run_id: str, session_id: str, status: str</code> Mark the checkpoint status.
get_retry_checkpoint	async function	app/workflow/ run_history.py:1021	<code>db, session_id: str, run_id: str</code> Return a session-scoped private checkpoint for the retry endpoint.
get_resume_checkpoint	async function	app/workflow/ run_history.py:1049	<code>db, session_id: str, run_id: str</code> Return a restart-safe checkpoint only when it is currently paused.
_as_aware_utc	function	app/workflow/ run_history.py:1064	<code>value: datetime</code> Internal helper for the as aware utc step.
_process_is_alive	function	app/workflow/ run_history.py:1076	<code>pid: int None</code> Best-effort same-host liveness check for a run's owning process.
_reconcile_if_stale	async function	app/workflow/ run_history.py:1106	<code>db, doc: dict[str, Any] None</code> Flip an orphaned "running" run to "failed" the moment it is read.
get_run	async function	app/workflow/ run_history.py:1193	<code>db, session_id: str, run_id: str</code> Return one full run if it belongs to the caller's session.
list_runs	async function	app/workflow/ run_history.py:1205	<code>db, session_id: str, limit: int=50, *, include_conversation_only: bool=False</code> Return light run summaries, newest first, including active runs.
workflow_stats	async function	app/workflow/ run_history.py:1246	<code>db, session_id: str, workflow_name: str, *, sample_limit: int=200</code> Library "Runs and performance" tab: success rate, median duration, and the most common failure, computed from this session's own run history for one workflow.
_delete_run_blobs	async function	app/workflow/ run_history.py:1345	<code>db, *, run_id: str</code> Remove every GridFS blob externalized for this run (see ``_externalize_if_large``).
delete_run	async function	app/workflow/ run_history.py:1374	<code>db, *, run_id: str, session_id: str</code> Permanently remove a run's history, checkpoint, and externalized blobs.
cleanup_stale_runs	async function	app/workflow/ run_history.py:1391	<code>db</code> Periodic sweep (see the background loop started in app/main.py) that hard-deletes runs stuck in "running" or "paused" for at least ``settings.run_auto_cleanup_after_seconds``.
_select_result	function	app/workflow/ subprocess_callback.py:52	<code>launch: dict[str, Any], *, output: Any, node_outputs: dict[str, Any]</code> Project the child's result the way the SubprocessConfig author asked for (result_from, recorded on the launch doc at reserve time — see app/nodes/subprocess.py's SubprocessConfig): workflow_output the child's own declared `output` contract (what every other caller of that workflow would get back) n
deliver_by_token	async function	app/workflow/ subprocess_callback.py:76	<code>db: Any, services: dict[str, Any], token: str, *, status: str, output: Any, node_outputs: dict[str, Any], error: str</code>

Symbol	Kind	Location	Signature / summary
			None Deliver the by token.
deliver_by_child_run_id	async function	app/workflow/ subprocess_callback.py:178	db: Any, services: dict[str, Any], child_run_id: str, *, status: str, output: Any, node_outputs: dict[str, Any], error: str None Deliver completion when the run belongs to a Subprocess launch.
ensure_indexes	async function	app/workflow/ subprocess_launches.py:42	db: Any Ensure the indexes.
find_by_launch_key	async function	app/workflow/ subprocess_launches.py:70	db: Any, launch_key: str Find the by launch key.
reserve_launch	async function	app/workflow/ subprocess_launches.py:83	db: Any, *, launch_key: str, parent_run_id: str, parent_node_id: str, parent_session_id: str, child_run_id: str, child_workflow: str, result_from: str, result_node: str None, timeout_seconds: float Claim 'launch_key' for one child launch.
find_by_child_run_id	async function	app/workflow/ subprocess_launches.py:132	db: Any, child_run_id: str Find the by child run id.
find_by_token	async function	app/workflow/ subprocess_launches.py:145	db: Any, token: str Find the by token.
deliver	async function	app/workflow/ subprocess_launches.py:158	db: Any, *, token: str, decision: dict[str, Any] Atomically mark the launch "delivered", once.